LEARNING MADE EASY

Microsoft 365®

Excel Office Scripts

for dummies®

A Wiley Brand

Modernize your Excel reporting for the cloud

Take full control of your online spreadsheets

Create programs that run in the browser

Michael Alexander

Author of *Microsoft Excel 365 Bible*

Microsoft 365® Excel Office Scripts

for dummies®
A Wiley Brand

Microsoft 365® Excel Office Scripts

by Michael Alexander

for dummies®
A Wiley Brand

Microsoft 365® Excel Office Scripts For Dummies®

Published by: **John Wiley & Sons, Inc.,** 111 River Street, Hoboken, NJ 07030-5774, www.wiley.com

Copyright © 2026 by John Wiley & Sons, Inc. All rights reserved, including rights for text and data mining and training of artificial technologies or similar technologies.

Media and software compilation copyright © 2026 by John Wiley & Sons, Inc. All rights reserved, including rights for text and data mining and training of artificial technologies or similar technologies.

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without the prior written permission of the Publisher or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 750-4470, or on the web at www.copyright.com. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at http://www.wiley.com/go/permissions.

The manufacturer's authorized representative according to the EU General Product Safety Regulation is Wiley-VCH GmbH, Boschstr. 12, 69469 Weinheim, Germany, e-mail: Product_Safety@wiley.com.

Trademarks: Wiley, For Dummies, the Dummies Man logo, Dummies.com, Making Everything Easier, and related trade dress are trademarks or registered trademarks of John Wiley & Sons, Inc. and may not be used without written permission. Microsoft 365 and Excel are trademarks or registered trademarks of Microsoft Corporation. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc. is not associated with any product or vendor mentioned in this book. *Microsoft 365® Excel Office Scripts For Dummies®* is an independent publication and is neither affiliated with, nor authorized, sponsored, or approved, by Microsoft Corporation.

LIMIT OF LIABILITY/DISCLAIMER OF WARRANTY: THE PUBLISHER AND THE AUTHOR MAKE NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE ACCURACY OR COMPLETENESS OF THE CONTENTS OF THIS WORK AND SPECIFICALLY DISCLAIM ALL WARRANTIES, INCLUDING WITHOUT LIMITATION WARRANTIES OF FITNESS FOR A PARTICULAR PURPOSE. NO WARRANTY MAY BE CREATED OR EXTENDED BY SALES OR PROMOTIONAL MATERIALS. THE ADVICE AND STRATEGIES CONTAINED HEREIN MAY NOT BE SUITABLE FOR EVERY SITUATION. THIS WORK IS SOLD WITH THE UNDERSTANDING THAT THE PUBLISHER IS NOT ENGAGED IN RENDERING LEGAL, ACCOUNTING, OR OTHER PROFESSIONAL SERVICES. IF PROFESSIONAL ASSISTANCE IS REQUIRED, THE SERVICES OF A COMPETENT PROFESSIONAL PERSON SHOULD BE SOUGHT. NEITHER THE PUBLISHER NOR THE AUTHOR SHALL BE LIABLE FOR DAMAGES ARISING HEREFROM. THE FACT THAT AN ORGANIZATION OR WEBSITE IS REFERRED TO IN THIS WORK AS A CITATION AND/OR A POTENTIAL SOURCE OF FURTHER INFORMATION DOES NOT MEAN THAT THE AUTHOR OR THE PUBLISHER ENDORSES THE INFORMATION THE ORGANIZATION OR WEBSITE MAY PROVIDE OR RECOMMENDATIONS IT MAY MAKE. FURTHER, READERS SHOULD BE AWARE THAT INTERNET WEBSITES LISTED IN THIS WORK MAY HAVE CHANGED OR DISAPPEARED BETWEEN WHEN THIS WORK WAS WRITTEN AND WHEN IT IS READ.

For general information on our other products and services, please contact our Customer Care Department within the U.S. at 877-762-2974, outside the U.S. at 317-572-3993, or fax 317-572-4002. For technical support, please visit https://hub.wiley.com/community/support/dummies.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media that is not included in the version you purchased, you may download this material at http://booksupport.wiley.com. For more information about Wiley products, visit www.wiley.com.

Library of Congress Control Number: 2026942666

ISBN 978-1-394-41269-3 (pbk); ISBN 978-1-394-41271-6 (ebk); ISBN 978-1-394-41270-9 (ebk)

Contents at a Glance

Introduction 1

Part 1: The Fundamentals of Excel Office Scripts 5

CHAPTER 1: Introducing Office Scripts for Excel 7

CHAPTER 2: Exploring the Office Scripts Environment. 15

Part 2: Basic Programming Concepts in Office Scripts 33

CHAPTER 3: Understanding Scripting Fundamentals 35

CHAPTER 4: Handling Errors with Style 63

CHAPTER 5: Controlling Program Flow and Conditions 81

Part 3: Scripting Common Excel Objects 103

CHAPTER 6: Starting with Worksheets 105

CHAPTER 7: Working with Ranges and Tables 131

CHAPTER 8: Pivoting with Pivot Tables 163

CHAPTER 9: Scripting Visual Elements. 185

Part 4: Going Beyond Excel with Power Automate 221

CHAPTER 10: Introducing Power Automate 223

CHAPTER 11: Integrating Excel with Email 247

CHAPTER 12: Integrating Excel with SharePoint 267

Part 5: The Part of Tens 283

CHAPTER 13: Ten Best Practices When Writing Scripts 285

CHAPTER 14: Ten Tips for Improving Script Performance 299

Index 309

Table of Contents

INTRODUCTION 1
About This Book 2
Foolish Assumptions 2
Icons Used in This Book 2
Beyond the Book 3
Where to Go from Here 3

PART 1: THE FUNDAMENTALS OF EXCEL OFFICE SCRIPTS 5

CHAPTER 1: Introducing Office Scripts for Excel 7
Seeing How VBA and Office Scripts Differ 8
Offline versus online 8
Security 9
Events 10
Language 10
Types of procedures 10
Code storage 11
Script triggers 11
Understanding the Requirements and Limitations of Office Scripts 11
Subscription requirement 12
Technical and organizational limitations 12
Data limits 12
Meager documentation 13
Benefits of Office Scripts 13

CHAPTER 2: Exploring the Office Scripts Environment 15
Diving into the Automate Tab 15
Writing Your First Script 17
Sharing Scripts 21
Emailing the .osts file 21
Moving or copying the .osts file to a shared location 21
Attaching scripts to your workbook and adding a button 22
Detaching scripts from your workbook 23
Creating Office Scripts with Action Recorder 23
Recording your actions 24
Renaming your script 25
Running your script 26
Editing a script 26

Deleting scripts 27
Limitations of Action Recorder 28
Using the Insert Action Command 28
Comparing Absolute Mode and Relative Mode Recording 30

PART 2: BASIC PROGRAMMING CONCEPTS IN OFFICE SCRIPTS 33

CHAPTER 3: Understanding Scripting Fundamentals 35
Understanding the Object Model 35
Understanding Methods 38
Enhancing Code with Variables 40
Using let versus const 42
Understanding data types 45
Adding Clarity with Comments 52
Adding Flexibility with Functions 53
Defining a function 54
Understanding function return values 57
Using function parameters 57

CHAPTER 4: Handling Errors with Style 63
Exploring Office Scripts Debugging Tools 64
Using the console.log method 64
Understanding try . . . catch statements 67
Examining Error-Handling Patterns 72
The try . . . catch . . . finally pattern 72
The validation-first pattern 72
The guard clause pattern 74
The fallback pattern 74
Contending with Common Error Types 75
Syntax errors 76
Reference errors 77
Object not found errors 78
Invalid range and out-of-bounds errors 78
Invalid arguments 79

CHAPTER 5: Controlling Program Flow and Conditions 81
Getting Logical with Conditions 82
Using the basic if statement 82
A word about equality operators 83
Planning for false conditions with if . . . else statements 84
Extending conditional checks with if . . . else if statements 85
Simplifying if statements with switch statements 87

Using Logical Operators in Conditional Analyses 88
The AND operator (&&) 89
The OR operator (||) 90
The NOT operator (!) 90
The nullish coalescing operator (??) 91
The ternary operator 93
Implementing Loops 94
Understanding for loops 94
Understanding prefixing versus postfixing 97
Leveraging for. . .of loops 98
Understanding while loops 100

PART 3: SCRIPTING COMMON EXCEL OBJECTS 103

CHAPTER 6: Starting with Worksheets 105
Working with Worksheets 106
Adding a new worksheet 106
Adding a new worksheet for each month 107
Deleting all but a specified sheet 108
Unhiding all worksheets in a workbook 110
Hiding all but a specified worksheet 112
Moving Worksheets Around 114
Moving a sheet to the first position 114
Moving a sheet to the last position 114
Moving a specific sheet before another sheet 115
Duplicating worksheets 116
Understanding Worksheet Protection 118
Checking if a worksheet is protected 119
Protecting a worksheet 119
Unprotecting a worksheet 120
Protecting all worksheets in a workbook 120
Unprotecting all worksheets in a workbook 123
Extending the Worksheets Collection 124
Sorting worksheets alphabetically 125
Creating a table of contents for your workbook 127

CHAPTER 7: Working with Ranges and Tables 131
Navigating Ranges 132
Using methods that return metadata 133
Leveraging range shortcut methods 135
Getting flexible with the getCell method 141
Creating custom range functions 144

Manipulating Data in Ranges 147
Copying data from one range to another 147
Converting all formulas in a range to values 149
Clearing cell contents 151
Deleting rows and columns 152
Working with Excel Tables 155
Referencing parts of a table 156
Creating a table from a range 157
Applying table filters 158

CHAPTER 8: **Pivoting with Pivot Tables** 163
Understanding Pivot Table Hierarchy 164
Starting a pivot table 165
Adding data items to the pivot table 168
Customizing data field items 169
Avoiding errors when adding data items 172
Putting it all together 173
Refreshing All Pivot Tables in a Workbook 176
Changing Pivot Report Layouts 177
Setting the report layout 178
Adjusting grand totals and subtotals 178
Working with Slicers 179
Creating a slicer 179
Filtering a slicer 181

CHAPTER 9: **Scripting Visual Elements** 185
Understanding Custom Number Formatting 186
Hiding and suppressing zeroes 188
Formatting numbers in thousands and millions 188
Formatting dates and times 189
Enhancing Data with Conditional Formatting 190
Capturing conditional formatting code with Action Recorder . . . 192
Conditional formatting that fails to record properly 194
Automating Charts 199
Understanding the chart object model 199
Working the chart elements 202
Overcoming chart recording challenges 207
Converting Visualizations into Images 207
Creating Custom Visuals 211
Creating a bullet graph 211
Creating a waffle chart with conditional formatting 215

PART 4: GOING BEYOND EXCEL WITH POWER AUTOMATE 221

CHAPTER 10: Introducing Power Automate 223
Going with the Flow. 224
Exploring the benefits of flows 224
Understanding Power Automate requirements 224
Exploring the Power Automate Interface. 226
The Create option 226
The Templates option 227
The My Flows option 228
Creating Your First Flow 229
Understanding triggers. 229
Gathering your thoughts and files 230
Building the flow 231
Passing Parameters to a Script from Power Automate 240
Understanding the Limitations of Power Automate 242
Excel data and API limitations 242
Power Automate platform limits. 243
Best practices to avoid Office Scripts failures in flows. 243
Discovering the Classic Flow Designer 244

CHAPTER 11: Integrating Excel with Email 247
Integrating Excel and Outlook 247
Emailing an attachment 248
Sending emails to a dynamic list of recipients 252
Embedding images in emails. 257
Distributing Excel Data as PDF Files 261
Introducing OfficeScript.convertToPdf. 262
Leveraging OfficeScript.sendMail 263

CHAPTER 12: Integrating Excel with SharePoint 267
Preparing the Files. 268
Reviewing the Scripts 269
Building the Flow 274
Reading data arrays from the SharePoint file 274
Writing data arrays to new workbooks 277

PART 5: THE PART OF TENS 283

CHAPTER 13: Ten Best Practices When Writing Scripts 285
Apply Descriptive Variable and Function Names 285
Use Established JavaScript Conventions 287
Apply Indentation Standards 288

Write Effective Comments 289
Leverage Comments to Partition Code 290
Centralize Object References. 291
Break Long Scripts into Custom Functions 292
Build Modular Scripts 294
Organize a Script Library 294
Manage Script Versions 296

CHAPTER 14: **Ten Tips for Improving Script Performance** 299
Read and Write Data in Batches 299
Using the tiny boat 300
Using the cargo ship 300
Avoid Repeated Lookups 301
Sort and Filter in Arrays, Not Cells 302
Apply Formatting to Ranges, Not Cells 303
Use Table Objects Whenever Possible 304
Leverage the getUsedRange Method 304
Postpone Calculations Until the End 305
Minimize Formula Usage in Scripts. 306
Minimize Actions that Change Display 307
Create Modular Scripts 308

INDEX 309

Introduction

For decades, VBA (Visual Basic for Applications) has been the go-to programming language for developers, analysts, and power users who need to automate repetitive tasks, build custom workflows, and push Microsoft Office beyond its default capabilities. If you've ever clicked a button in Excel and watched a report format itself or automatically send an email, VBA was almost certainly doing the heavy lifting behind the scenes.

Then came Microsoft's shift to a cloud-first world. With Office 365, Excel workbooks stopped being isolated files on a single hard drive and became living, collaborative documents hosted online. This transformation unlocked enormous benefits but also exposed a major limitation. VBA wasn't built for the cloud.

VBA requires direct access to the local file system, registry, and other PC resources. Those tight local dependencies make VBA incompatible with browser-based Excel. As Excel moved online, VBA stayed behind. This is where Office Scripts stepped in.

Office Scripts is Microsoft's modern automation platform designed for Excel on the web. It runs safely inside the browser, works seamlessly with cloud-hosted workbooks, and brings automation back to the online version of Excel. With Office Scripts, you can record actions, write TypeScript-based scripts, and automate your workflows directly in Office 365.

Because you picked up this book, you're probably feeling the gap that VBA left behind as Excel transitioned to the cloud. If you've ever written a macro and wished it worked online, Office Scripts and the chapters ahead will show you how to bring back that power.

About This Book

The goal of this book is to give you a solid foundation in Office Scripts for Excel. Each chapter guides you through practical examples that demonstrate the techniques you'll need to write your own scripts. After reading this book, you will be able to:

- Use Action Recorder to record your own scripts.
- Create scripts that manipulate worksheets and ranges.
- Automate the creation of pivot tables and charts.
- Effectively debug and optimize your scripts.
- Integrate your scripts with Power Automate.

Over the last few years, Microsoft has adopted an agile release cycle, allowing them to release updates to Microsoft Office and the Excel API scripting package practically monthly. This news is great for those who love seeing new features. It's not so great if you're trying to document the features of these tools in a book.

Microsoft will continue to add new features after this book's publication, which means you may encounter new functionality not covered here. However, Office Scripts is a mature programming platform, so any changes that come your way won't be so drastic as to turn this book into a doorstop. The core functionality covered in these chapters will remain relevant — even if the mechanics change a bit.

Foolish Assumptions

When it comes to you, dear reader, my assumption is that you come to this book with some programming experience. Specifically, I assume you're coming from VBA and need to make a transition to Office Scripts. In that light, this book doesn't dwell on fundamental programming concepts. You dive right in with Office Script syntax.

Icons Used in This Book

As you read this book, you'll see icons in the margins that indicate material of interest (or not, as the case may be). This section briefly describes each icon in the book.

TIP

Tips are nice because they help you save time or perform some task without a lot of extra work. The tips in this book are timesaving techniques or pointers to resources that you should try if you want to get the maximum benefit from Excel and Office Scripts.

WARNING

Try to avoid doing anything marked with a warning icon, which (as you might expect) represents a danger of one sort or another.

TECHNICAL STUFF

Whenever you see this icon, think advanced tip or technique. You might find these tidbits of useful information just too boring for words, or they could contain the solution you need to get a script running. Skip these bits of information whenever you like.

REMEMBER

If you don't get anything else out of a particular chapter or section, remember the material marked by this icon. This text contains an essential process or a bit of information you ought to remember.

Beyond the Book

If you want to follow along with the exercises in this book, download the sample files at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

As a bonus, you can also download a cheat sheet containing TypeScript coding patterns to use while you develop your scripts as well as some tips for leveraging LLMs (large language models) and AI to help write scripts. Go to `www.dummies.com` and type *Microsoft 365 Excel Office Scripts For Dummies cheat sheet* in the Search field.

Where to Go from Here

It's time to start your Office Scripts journey. If you're new to Office Scripts, I recommend you start with Chapter 1 to get the lay of the land before moving into scripting.

If you've already started working with Office Scripts (recorded a few scripts or run a few scripts by other authors) and you want to start writing your own code, start with Chapter 3 to dive right into the fundamentals of TypeScript.

If you're a seasoned TypeScript developer and looking to learn more about automating Excel with Office Scripts, feel free to jump right to Part 3, where you get into the Excel object model.

1 The Fundamentals of Excel Office Scripts

IN THIS PART . . .

Discover the changing landscape of Excel programming and explore the role Office Scripts plays in Excel automation.

Dive into the Office Scripts environment and record your first automated process.

IN THIS CHAPTER

- Recognizing the differences between Office Scripts and VBA
- Understanding the requirements and limitations of Office Scripts
- Seeing what Office Scripts can do for you

Chapter 1
Introducing Office Scripts for Excel

Whether you're new to Excel programming or a seasoned veteran, you've likely heard about Excel's native programming language, *VBA (Visual Basic for Applications)*. Every Office application has its own version of the VBA programming language. For decades, developers and business analysts have used VBA to automate the Microsoft suite of applications.

Then along came Office 365, a set of online business applications allowing for seamless sharing and collaboration of Excel, Word, and PowerPoint documents. With Office 365, you can create an online Excel document that can be used by multiple people simultaneously, can be used on tablets as well as PCs, and can be integrated with the broader suite of Microsoft cloud-based platforms such as Power Automate.

Ah, but there's a catch. Because online Excel documents are not stored on your local PC, they don't have access to your local operating system. More importantly, they don't have access to the fundamental requirements needed to run VBA. Therefore, it's impossible to run VBA macros and programs with Excel online documents.

Enter Office Scripts, a language based on TypeScript, an open-source superset of JavaScript. (JavaScript is a lightweight programming language that runs directly in the browser, allowing developers to manipulate web page content, respond to

user events, and communicate with servers without reloading the page.) Because Office Scripts has TypeScript at its core, it compiles to JavaScript, which means it can be used in online cloud environments. In short, with Office Scripts, unlike VBA, we can automate Excel online workbooks.

Before you start your journey into using Office Scripts, it's important to understand some of its requirements and the fundamental programming differences you'll encounter when working in this new realm of Excel automation. This chapter paves the way for everything else that follows and gives you a feel for how Office Scripts fit into the big picture of Excel development. Be patient and resist the urge to jump to Chapter 2 instead.

Seeing How VBA and Office Scripts Differ

In this section, you explore the fundamental differences between Office Scripts and VBA.

Offline versus online

Office Scripts will not replace VBA anytime soon. Office Scripts is designed for the automation of online cloud-based Office documents, while VBA still holds a fundamental role in the development of offline desktop applications.

If your application needs to write to your file system, respond to keyboard or mouse clicks, or interact with your operating system, VBA is the language you would use. VBA would also need to be used if your solution is installed as an Excel add-in (a supplemental program that integrates directly into the Excel ribbon interface).

If your organization runs on a modern cloud environment — meaning your Excel workbooks are saved online, available from any device, and connected to other Microsoft services — you need Office Scripts for online development. Online development with Office Scripts allows your automated processes to be managed in a controlled cloud environment, to be accessed by multiple users, and to be used across various platforms such as tablets and handheld devices.

Table 1-1 is a quick reference to help you choose between VBA and Office Scripts when tackling a new project.

TABLE 1-1 Using VBA versus Office Scripts

Project Requirement	VBA	Office Scripts
Write to the local PC file system	Yes	
Respond to keyboard or mouse events	Yes	
Provide a custom ribbon interface	Yes	
Respond to workbook, worksheet, or other events	Yes	
Install as an add-in	Yes	
Automate shared Excel online workbooks		Yes
Function in a controlled cloud environment		Yes
Integrate with SharePoint lists and files		Yes
Integrate with Power Automate cloud workflows		Yes

Security

Because VBA runs on your local PC, VBA macros, especially from untrusted sources, can pose significant security risks. The unfettered access to the operating system that VBA provides can open the possibility for malicious code to manipulate files, access system resources, and install malware. To mitigate this threat, Excel requires users to explicitly enable macros for each file. If macros are disabled, VBA code won't run.

In contrast, Office Scripts run in the cloud, so they don't access your local machine or file system. Because they don't access local resources, the risk of file-based malware is reduced, making scripts inherently more secure.

A FEW WORDS ABOUT TERMINOLOGY

Excel online programming terminology can be confusing. Search for information about this topic online, and you may see the term *Office Scripts* or *Excel Scripts*. What's the difference? Just like Microsoft Office has an overarching programming language called VBA, you can think of *Office Scripts* as the overarching programming language for online Office development. *Excel Scripts* is often used as a shorthand for referring to the ExcelScript Package, which is the core API (application programming interface) in Office Scripts and provides all the objects and methods necessary to interact with Excel. For our purposes, however, Office Scripts for Excel and Excel Scripts mean the same thing.

Events

VBA supports a wide range of event triggers. For example, VBA can trigger a macro to run when a workbook is opened, a worksheet is selected, or a specific cell is changed. Event triggers give VBA developers great flexibility when designing the user experience for their Excel application.

As of this writing, Office Scripts has no native support for events. You can't trigger a script to run when a workbook opens or a cell changes. A script can run only when a user explicitly starts it. In Part 4, however, you discover how to simulate events by using Power Automate flows to trigger scripts to run.

Language

VBA uses the proprietary Microsoft scripting language with application-specific objects and methods and rich support for Windows API calls. VBA provides a wide array of reference libraries that let you interact with practically every aspect of the local environment in which it runs.

Office Scripts uses TypeScript and application-specific API packages, which compile down to cloud-compatible JavaScript. Because the objects and methods come from managed cloud resources and API packages, you'll always have the latest versions of Excel and Office Scripts without needing to install anything.

Types of procedures

In VBA, you can write code in subprocedures, functions, or events. A subprocedure executes one line of code at a time until the procedure finishes. Functions are like subprocedures in that they execute each line until complete, except they can return a value back to the user. For example, you can create a function that adds two numbers and returns the answer to a specific cell. Finally, VBA can be written as events that trigger when an action is taken (a workbook is opened, a worksheet is selected, a specific cell is changed, and so on).

With Office Scripts, you're limited to using only functions. Every Office Script for Excel requires a minimum of one function called `main` with a parameter that represents the current active Excel workbook. If you record a script, the first line of the script will always be the following:

```
Function main(workbook: ExcelScript.Workbook)
```

When you run a script, it looks for this function and passes the active workbook into the function. You can add functions as needed, but the `main` function is required.

REMEMBER

A script has access to only the workbook in which it is running. Even something as simple as copying a value from one workbook to another can't be done with a single script.

Code storage

Code written in VBA is embedded in the `.xlsm` workbook in which it's saved. This embedded code travels with the workbook when it's emailed or saved to another location. Users have immediate access to VBA code in an .xlsm file when they enable macros for that file.

In contrast, Office Scripts offers centralized, cloud-based script management. Scripts are not embedded in the workbook. Instead, they're stored as separate `.osts` (Office Scripts TypeScript) files on your OneDrive account under Documents/Office Scripts/. You can run these scripts from any workbook as long as you're logged in to the same OneDrive account and have permission to access and edit the workbook.

Scripts can also be saved on SharePoint, where you can access them if you have access to that site. You can view, manage, and share scripts through the Excel Office Scripts interface.

Script triggers

You can trigger a VBA script in a variety of ways. You can start the code manually from the Macros dialog box, assign your macro to a shape or button, use an event to trigger your code, or use a worksheet function in a cell.

Office Scripts can be started manually via the Automate tab or from the Code Editor task pane. Additionally, your script can be assigned to a button. In Part 4, you discover how to trigger a script by using Power Automate flows.

Understanding the Requirements and Limitations of Office Scripts

Now that you understand where Office Scripts fits into the world of Excel development, let's take a moment to review some of the requirements and limitations of Office Scripts.

TIP

The list of requirements and limitations may change as Microsoft continues to develop Office Scripts. For updates, visit `https://learn.microsoft.com/en-us/office/dev/scripts/testing/platform-limits`.

Subscription requirement

Office Scripts is available only with a Microsoft 365 subscription. To get started, you need one of the following licenses:

- Microsoft 365 Personal or Family
- Office 365 Business or Business Premium
- Office 365 ProPlus or ProPlus for Devices
- Office 365 A3 or A5
- Office 365 Enterprise E1, Enterprise E3, or Enterprise E5
- Office 365 F3

REMEMBER

If you're using a Personal or Family plan, the features that allow Office Scripts to integrate Power Automate are not available. You can check your current subscription by signing into `https://account.microsoft.com`.

Technical and organizational limitations

Some organizations turn off automation features by default for security reasons. If you find that you can't run Office Scripts, check with your Microsoft 365 administrator to ensure that Office Scripts for Excel is enabled and that no group policy settings are limiting its use.

To see the Automate tab when using Excel online, third-party cookies must be enabled on your browser. If you don't see the Automate tab, check your browser settings or speak with your IT department to ensure that third-party cookies have not been disabled.

Data limits

If you work with a range of cells in an Excel worksheet and need to store that range object in memory, it can't have more than 5 million cells. In addition, data transferred between the script and the Excel workbook can't exceed 5MB. If you exceed one of these Excel data limits, you'll receive an error message that reads

"The response payload size has exceeded the limit." In these rare cases, you'll need to split your ranges or data into smaller, more manageable chunks.

You are limited to 1,600 calls to the Run script action per day. This limit is usually an issue only with scheduled Power Automate flows that run Office Scripts.

Finally, all scripts have a 120-second timeout. A long-running script will fail if all the code is not resolved within this time limit. In this case, you would need to optimize or split your script into separate scripts.

Meager documentation

Documentation for Office Scripts is sparse. Unlike VBA, which has been around for decades, Office Scripts code samples and object model diagrams will be scarce until more people adopt the program. Fortunately, Microsoft's built-in Action Recorder can assist you in writing Office Scripts based on your keyboard entries and mouse clicks. You explore Action Recorder in the next chapter.

Benefits of Office Scripts

With it robust features, Office Scripts enable countless ways to improve data processing and gain efficiencies. I end this chapter with five key benefits of leveraging Office Scripts:

- **Automate a recurring task.** Are you producing the same report week after week using the same tedious steps? You can turn those steps into a script that can be run on demand. And because the steps are codified, you'll achieve the same results every time with no chance for human error.
- **Automate a repetitive task.** Excel work is often repetitive. When you need to do the same set of tasks for every worksheet in a workbook, use Office Scripts instead.
- **Optimize work with functions.** Use Office Scripts to create functions that perform custom calculations, run custom transformations on text, or apply custom formatting based on certain conditions. You can extend the capabilities of your workbooks beyond Excel's built-in features.
- **Simplify the user experience.** Oftentimes, your workbooks will be used by people who know little to nothing about Excel. You can employ Office Scripts to build an interface that provides users with friendly buttons that perform actions for them. You'll be surprised at how much you can cut down on

human error by relying on your scripts to guide users through a planned workflow.

- **Interact with other applications.** One of the more exciting aspects of Office Scripts is how it can be integrated in your cloud workflows. With the help of Power Automate flows, you can integrate your Excel workbooks with SharePoint, automatically send emails through Outlook, and save parts of your Excel workbook to a PDF stored in the cloud. The best part is that these Power Automate flows can be configured to run on a schedule, freeing you up to work on other things.

Are you excited yet? Good. *Now* you can jump to Chapter 2.

IN THIS CHAPTER

- Surveying the Automate tab
- Creating your first script
- Discovering how to share your scripts
- Using Action Recorder to create a script
- Introducing the Insert Action command
- Using absolute versus relative recording

Chapter 2

Exploring the Office Scripts Environment

In this chapter, you meet the Office Script editor by using Excel's Automate tab. By tradition, the first step in learning a new programming language is to write a simple procedure that prints *Hello, World!* You do that in this chapter! You also discover how to leverage the nifty Action Recorder to help write code for you.

Diving into the Automate Tab

To get your first look at the Automate tab, you need to log into your Microsoft 365 account and start a new online Excel workbook:

1. **Open your favorite internet browser and navigate to `https://onedrive.live.com`.**

   The Home screen for your OneDrive account appears.

2. **Click the Create or Upload button and then select Excel Workbook, as shown in Figure 2-1.**

   A new workbook is created and opened.

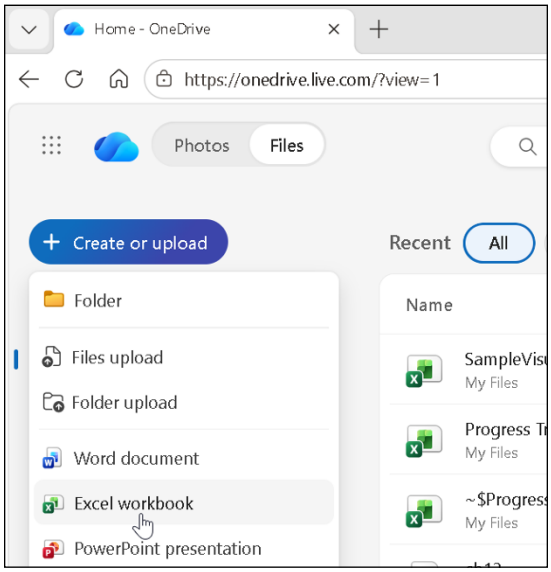


FIGURE 2-1: Create an online Excel workbook.

3. **On the Excel ribbon, click the Automate tab to see the controls shown in Figure 2-2.**

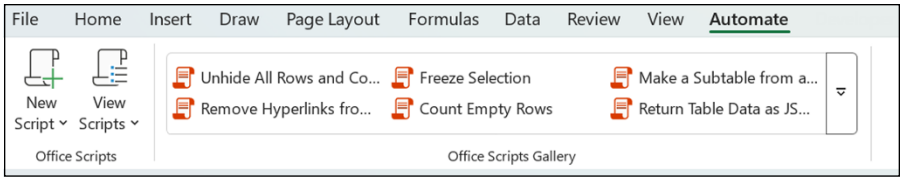


FIGURE 2-2: The Automate tab controls.

TIP

As long as you are working with Excel online, the Automate tab will be available. If you're working with an Excel workbook on your local desktop, you may need to take extra steps to see the Automate tab. If the Automate tab is missing, make sure you're connected to a supported Microsoft 365 plan (see Chapter 1 for requirements). Next, enable the Automate tab by going to the ribbon and clicking File ➪ Options ➪ Customize Ribbon and then selecting the Automate option under Main tabs. If you're doing everything correctly but the Automate tab is still missing, contact your IT administrator to ensure that Office Scripts has been enabled for your organization.

The Automate tab (refer to Figure 2-2) is your entry point to the Office Script editor. Here, you'll find the following icons:

- **New Script:** Displays options to create a script from scratch using the Office Scripts task pane, or open the Record Actions task pane, where actions you take are recorded and translated into code. This pane is the Office Scripts version of a macro recorder.
- **View Scripts:** Displays options to view a list of recent scripts you've worked with or a list of sample scripts provided by Microsoft. Choosing View Scripts and then Recent Scripts opens the Office Scripts task pane (see Figure 2-3), where you can start a new script, open a recently modified script, or see all the scripts stored in your OneDrive account in the `Documents/Office Scripts` folder.
- **Office Scripts Gallery:** Lists scripts to run. Click the down arrow to reveal six sample scripts that come with Excel. As you create and edit scripts, the gallery lists the scripts you most recently modified.

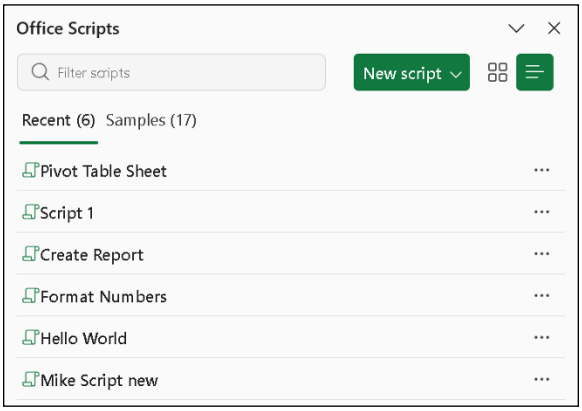


FIGURE 2-3: You can see a list of recent scripts.

Writing Your First Script

For your first script, let's throw originality out the window and keep with the tradition of having Excel print *Hello World!* Let the fun begin:

1. **Click the Automate tab, click the New Script icon, and then choose the Create in Code Editor option.**

   The Office Scripts task pane opens with the basic starter code shown in Figure 2-4. You'll see the same code every time you start a new script. The code simply gives you the basic structure you need to get started.

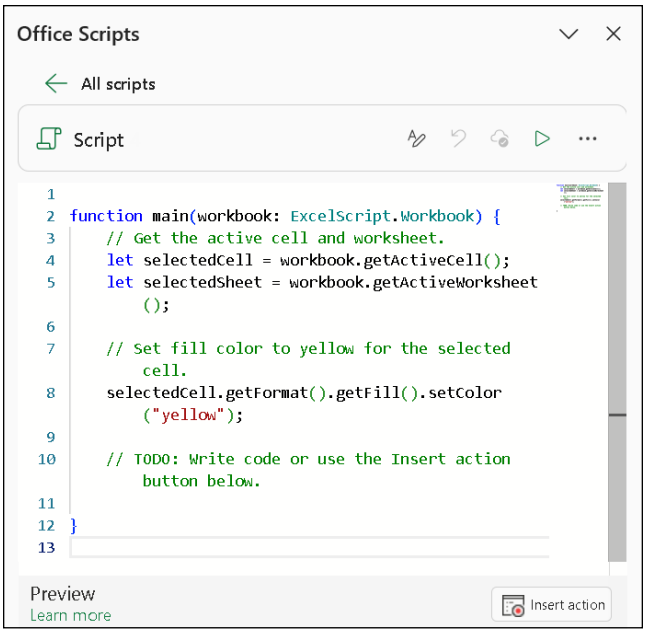


FIGURE 2-4: The default starting code after selecting New Script.

2. Delete all the code between the curly brackets, and pay attention to what is left, as shown in Figure 2-5.

Every Office Scripts requires, at a minimum, the syntax shown in Figure 2-5. This is called the `main` function block. Any code you type in the block is executed when the script runs.

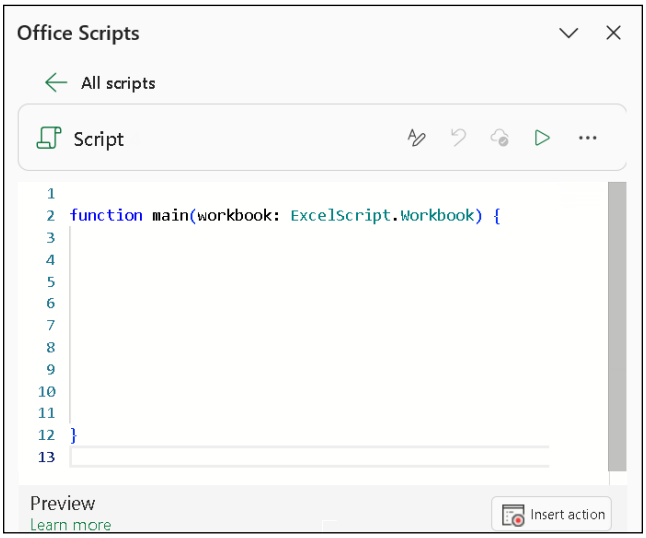


FIGURE 2-5: The main function block is the minimum required syntax.

3. **Enter the following code in the block (between the curly brackets):**

```
function main(workbook: ExcelScript.Workbook) {

workbook.getActiveWorksheet()
.getRange("A1")
.setValue("Hello World!")

workbook.getActiveWorksheet()
.getRange("A1")
.getFormat()
.setColumnWidth(75)

}
```

This code tells Excel to go to cell A1, write `Hello World!`, and then expand the column width. Don't get intimidated by the syntax. Upcoming chapters shed light on what this all means.

4. **Test your script by clicking the run icon above the code editor, as shown in Figure 2-6.**

Excel runs each set of instructions sequentially, first filling cell A1 with `Hello World!` and then adjusting the column width of A1.

FIGURE 2-6: This code writes `Hello World!` in cell A1.

TIP

You can run any script from its Office Scripts task pane. The easiest way to run a script is to go to the Automate tab, click View Scripts➪Recent Scripts, and then find and click your target script. It opens in the Office Scripts task pane, where you can then run and edit the script as needed.

When you create a script, Excel automatically gives it a default name, such as Script 1 or Script 2. You can rename the script by clicking the rename icon (see Figure 2-7) and entering a new name.

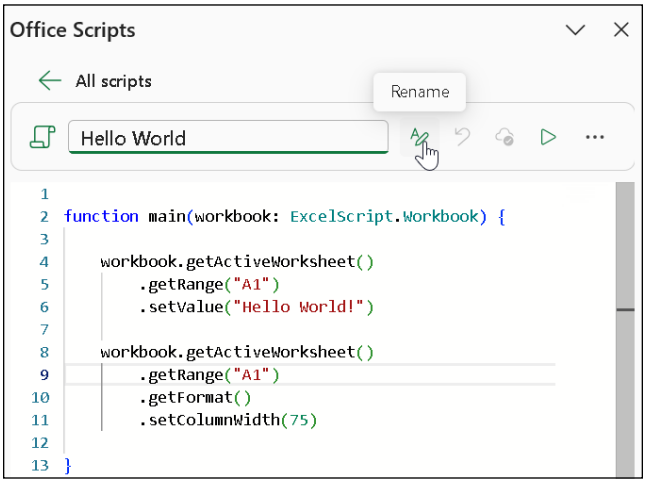


FIGURE 2-7: Rename the script.

Unlike VBA, where code is embedded in the workbook, saving an Office Scripts does not add code to the workbook. The script is saved to your OneDrive account as an independent `.osts` file in the `Documents/Office Scripts` folder (see Figure 2-8).

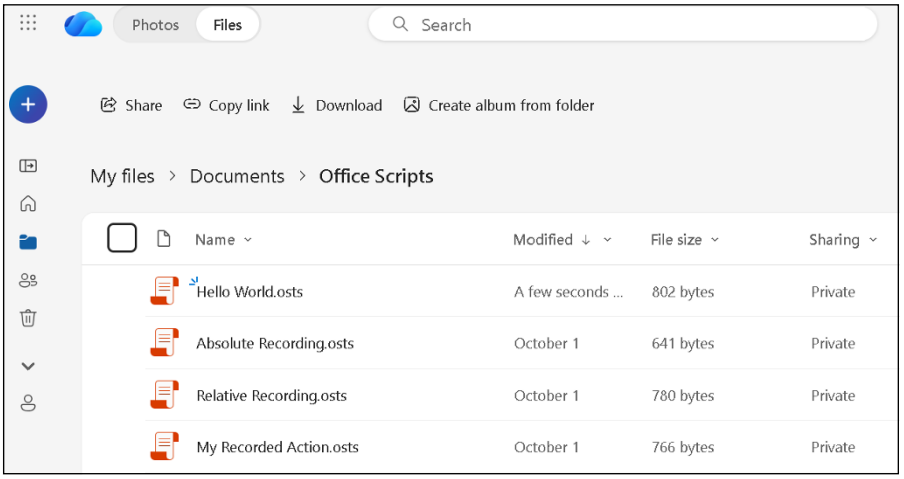


FIGURE 2-8: Your new script is in your OneDrive account under `Documents/Office Scripts`.

If you open an Office Scripts `.osts` file directly, you'll be surprised to find what looks like gibberish, as shown in Figure 2-9. The syntax you're seeing is the raw

JSON format of the script. JSON (JavaScript Object Notation) is a text-based format used for storing and transporting data between web applications. It's not user friendly, but Excel reads it just fine.

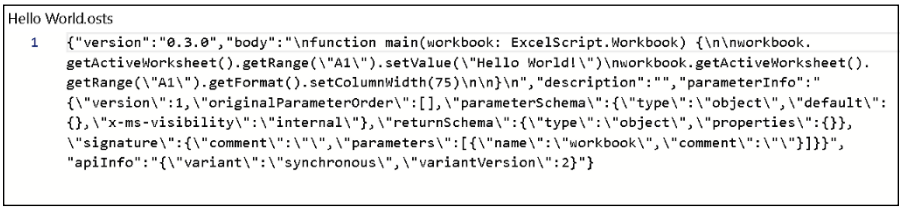


FIGURE 2-9: Office Scripts are stored as raw JSON.

Sharing Scripts

Any script you create can be used by only you unless you share it. You can share the code for other developers to leverage in their own way or share access to the script so multiple users can run it.

Emailing the .osts file

If you want to share your script with another analyst or developer, the most straightforward option is to email the `.osts` file for the script. The receiver can simply place the `.osts` file in the `Documents/Office Scripts` folder on their own OneDrive, and then review, edit, and run the script by using the Automate tab in Excel.

Moving or copying the .osts file to a shared location

You can right-click your `.osts` script file, just like any other file in OneDrive, to display a menu of commands, including Move To and Copy To. You use the Move To command to move your script from your `Documents/Office Scripts` folder to a new location, such as a SharePoint folder or a shared team folder. Anyone who has access to the new folder will be able to review, edit, and run the script by using the Automate tab in Excel. The Copy To command is similar, except your script stays in your `Documents/Office Scripts` folder and a copy is sent to the chosen location.

TIP

You can use the Copy To command to save backups of your important scripts to a safe location.

TIP

Instead of creating a script from scratch every time, consider using the Copy To command to create a duplicate of existing code. You'll save time by repurposing work you've already done.

Attaching scripts to your workbook and adding a button

If your script will be used by many users across your organization, consider adding a button to the workbook to trigger the script. Then anyone who has permission to access the workbook can simply click the button — without requiring permission to access your OneDrive or SharePoint folder.

Do the following to add a button for your script:

1. **On the Automate tab, click View Scripts and then click Recent Scripts.**
2. **Find your target script and click it.**

   The Office Scripts task pane appears.
3. **Click the ellipsis (. . .) and then click the Add in Workbook button.**

   You see a button with the same name as the script, as shown in Figure 2-10. The code from your script is now embedded in the workbook, and users can run the attached script without needing permissions to the script document library on OneDrive or SharePoint.

   You can format, rename, and move the button, just as you would any other shape in Excel.

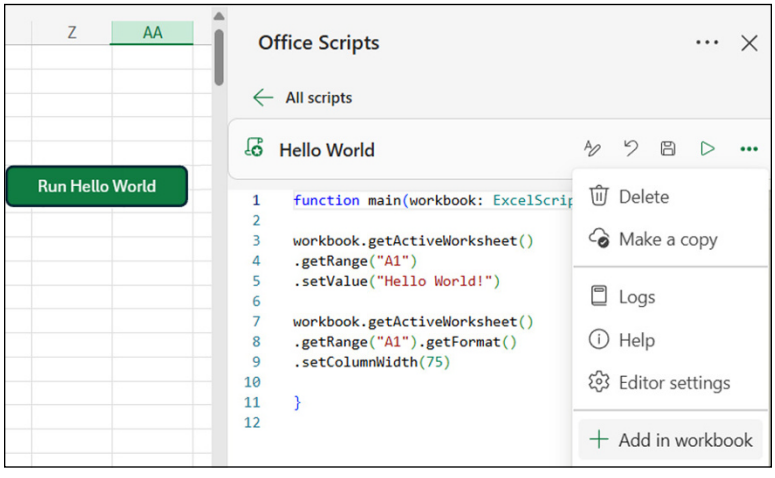


FIGURE 2-10: Attach your script to the workbook so users can run the script without specific folder permissions.

REMEMBER

The Add in Workbook option is a business- and enterprise-level feature, so if you have a Microsoft 365 Personal or Family plan subscription, you will not have the option of attaching Office Scripts to a button in the workbook.

Detaching scripts from your workbook

Deleting the button tied to an Office Scripts does not detach the script from the workbook. To remove the script, you need to do the following:

1. **On the Automate tab, click View Scripts and then click Scripts from this Workbook.**
2. **Select the script you want to edit.**
3. **Scroll down to the Associate with Workbook toggle, as shown in Figure 2-11, and click it to the off position.**

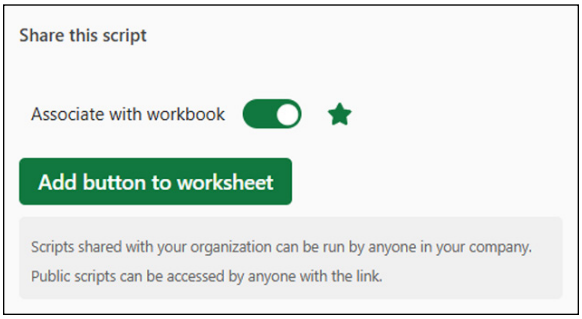


FIGURE 2-11: Toggle the Associate with Workbook option off to remove a script from the workbook.

Creating Office Scripts with Action Recorder

Learning a new programming language from the ground up can be daunting. Luckily, Excel is here to assist with *Action Recorder*, which translates the actions you take into code that you can play back.

Think of recording your actions in Excel like programming a phone number into your cell phone. First, you manually dial and save a number. Then you can redial that number with the touch of a button. Similarly in Excel, you dial in your keystrokes and mouse clicks while Action Recorder translates your actions to written TypeScript code that can be played back anytime you want.

Recording your actions

Follow these steps to create a simple script using Action Recorder:

1. **Open a new workbook.**
2. **On the Automate tab, click New Script and then click Create from Recording.**

   The Record Actions task pane appears, as shown in Figure 2-12. The small circle next to the word *Recording* is slowly blinking to let you know that Excel is recording.

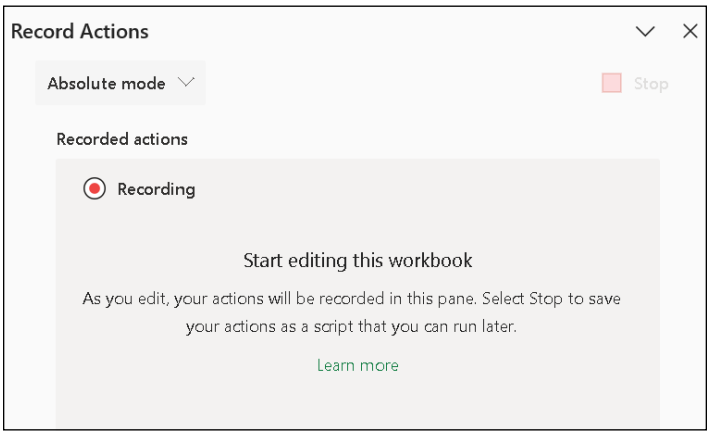


FIGURE 2-12: The Record Actions task pane is active while recording a script.

3. **Take the actions you want recorded.**

   To follow along with the example, select cell B3, type your name in the selected cell, and then press Enter.

4. **Return to the Record Actions task pane and click the Stop button.**

   You see the Script Details task pane for the newly created script, as shown in Figure 2-13.

Excel gave the new script a default name of Script 3. (Your script name may be different.) The Script Details pane also displays the steps taken, in this case setting the value of cell B3 to `Mike Alexander`.

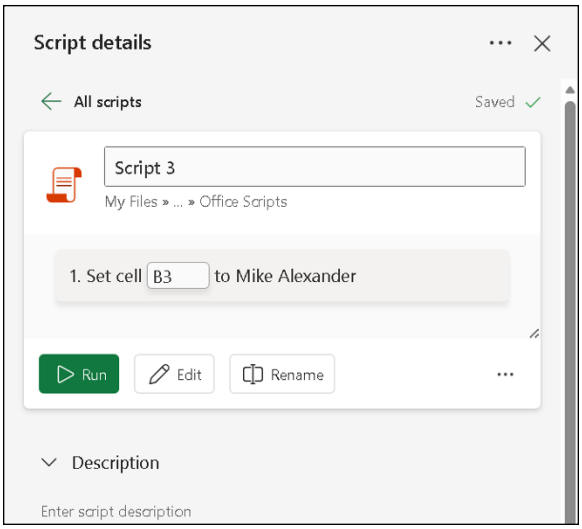


FIGURE 2-13: The Script Details task pane opens after you stop recording an action.

Renaming your script

To rename the script, click the rename icon, type the new name, and then press Enter. To follow along with the example, rename your script **My Recorded Action**, as shown in Figure 2-14.

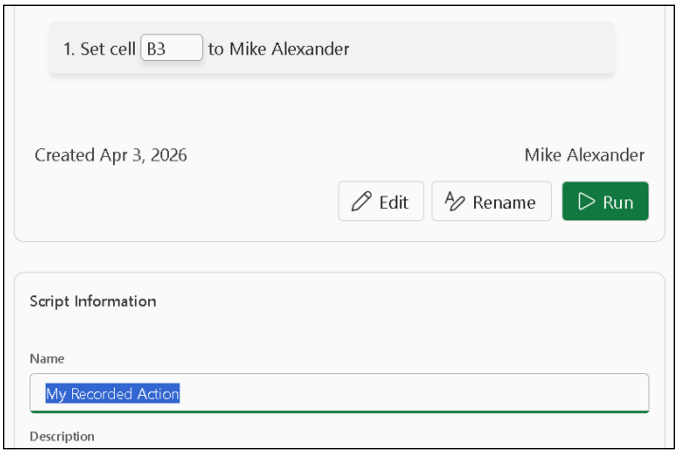


FIGURE 2-14: You can rename your script directly in the Script Details task pane.

The name you give your script is essentially the file name for its associated `.osts` file on OneDrive. For example, renaming a script `MyCoolScript` results in a file called `MyCoolScript.osts` on OneDrive.

REMEMBER

Script names can't include characters that are unsupported in OneDrive. These characters include the double quote ("), asterisk (*), colon (:), less than symbol (<), greater than symbol (>), question mark (?), backslash (/), forward slash (\), and pipe (|).

Running your script

In general, you can run any script in its Office Scripts task pane. Simply do the following:

1. **Go to the Automate tab, click View Scripts, and then click Recent Scripts.**
2. **Find your script and then click the Run script icon, as shown in Figure 2-15.**

   Excel runs each set of instructions in the script sequentially.

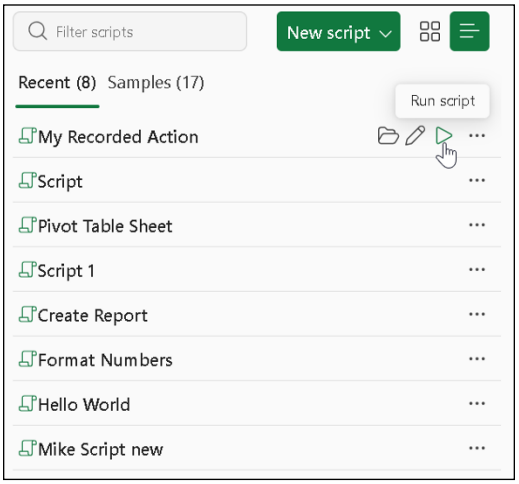


FIGURE 2-15: You can run a script by clicking its associated Run command.

TIP

You can run a script also by adding a button to your workbook. Refer to the "Attaching scripts to your workbook and adding a button" section.

Editing a script

To edit a script, you need to activate the Office Scripts task pane and then choose edit mode. Do the following:

1. **Go to the Automate tab, click View Scripts, and then click Recent Scripts.**
2. **Find and click the script you want to edit.**

   The Office Scripts task pane opens.
3. **Click the edit icon (pencil) to open the editor screen.**

   Figure 2-16 displays the editor screen for the My Recorded Action script you created a moment ago. This editor is the primary interface you use to edit and add code.

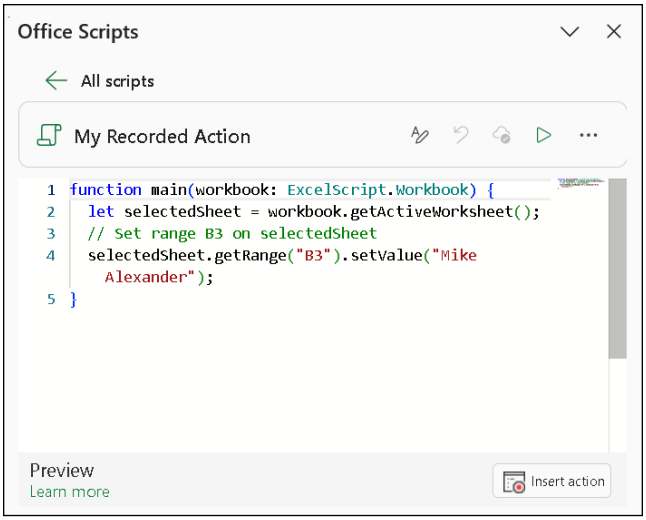


FIGURE 2-16: Office Scripts editor is the interface you use to edit scripts.

Deleting scripts

You'll no doubt have to delete some of the scripts you create. You can remove old scripts directly in your OneDrive account. Simply log into OneDrive, navigate to `Documents/Office Scripts`, and then delete the target script.

Alternatively, you can delete a script from Excel by taking the following steps:

1. **Go to the Automate tab, click View Scripts, and then click Recent Scripts.**
2. **Find your script and then click the ellipsis (. . .) to expand the context menu.**
3. **Click Delete.**

Limitations of Action Recorder

Although Action Recorder is an excellent tool for ramping up your coding skills and quickly pulling together an automated solution, it has limitations. Here are a few gaps in functionality you'll need to keep in mind when using Action Recorder:

- **Limited recording of UI actions:** Action Recorder is brilliant at recording cell edits, formatting, table creation, and basic structural edits. However, it often fails to capture actions that involve interaction with the Excel interface. For example, it won't accurately capture Power Query actions or advanced PivotTable interactions such as filtering or pivot slicer configurations. If the action you need to record requires a lot of Excel dialog boxes, review the recorded script carefully to ensure that Action Recorder didn't miss anything.
- **One worksheet at a time:** Action Recorder can record actions only in a single worksheet at a time, making it unsuitable for recording tasks that span multiple worksheets.
- **No programming logic:** Action Recorder captures the actions you take, not why you took them. It isn't designed to inherently build logic or infer conditional branches. In other words, you must add logic like "if X then do Y" after Action Recorder gives you the basic raw steps.
- **What you see is what you get:** Action Recorder can create code based only on what is on the worksheet at the time of recording. It can't anticipate changes in layout or create dynamic code that can handle changing data situations.
- **Superfluous code:** Action Recorder is not going to win programmer of the year anytime soon. The scripts it generates are rarely optimized and often include overly verbose code and unnecessary steps that can affect performance. As your programming skills improve, you will recognize areas where you can streamline generated code.

Using the Insert Action Command

The Insert Action command enables you to record a new action and add it to existing code. It's different from Action Recorder in that it records just a single action. The Insert Action command provides an easy way to incrementally add actions to your scripts as you develop your solution.

To illustrate this feature, open the My Recorded Action script and add an action. You'll add bold font formatting to the cell where your name is written:

1. **Go to the Automate tab, click View Scripts, and then click Recent Scripts.**
2. **Find and click the My Recorded Action script.**

   The Office Scripts task pane opens.
3. **Click the edit icon (pencil) to display the editor.**
4. **Click the run icon (right-facing triangle) to trigger the script.**

   Your name is written in cell B3.
5. **Click Insert Action, at the bottom of the editor.**

   You'll know the recorder is active when you see a *Waiting for your action . . .* message at the bottom of the screen.
6. **On the worksheet, right-click cell B3 and make the font bold.**

   The bottom of the Office Scripts pane now displays the Insert button.
7. **In the code window, put your cursor after the last semicolon and before the last curly bracket.**

   This is an important step because the Insert command inserts code wherever your cursor sits. You want the new code correctly placed in the block (between the curly brackets).
8. **Click the Insert button, as shown in Figure 2-17.**

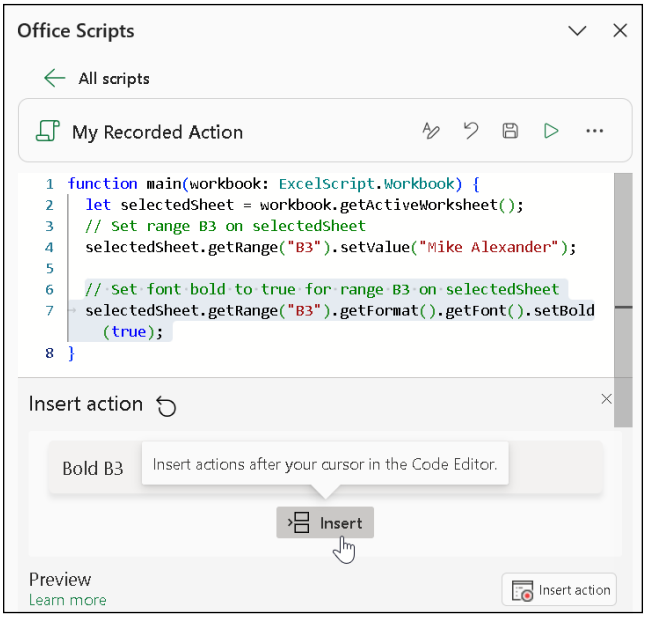


FIGURE 2-17: Click the Insert button to add your newly recorded action to the script.

This is pretty cool! Not only did you add new code to your script, but you did it without knowing any of the syntax.

The Insert Action command is a real timesaver. But more than that, you can leverage this feature as a script tutor. Want to get the syntax for adding a formula to a cell? Record yourself adding the formula and let Excel give you the syntax! With Action Recorder and the Insert Action commands by your side, learning Office Scripts becomes a lot less scary.

Comparing Absolute Mode and Relative Mode Recording

I round out this chapter on the Office Scripts environment with a quick review of Action Recorder's two recording modes: absolute mode and relative mode. When writing scripts that navigate and manipulate cell ranges, the way you reference ranges in your code determines how Excel interprets cell locations.

An *absolute reference* points to a fixed cell range. For example, `workbook.getActiveWorksheet().getrange("B3")` is an absolute reference. No matter where you are in the workbook, this reference will always point to cell B3 on the active worksheet.

A *relative reference* points to a cell relative to the currently active cell. For instance, `selectedCell.getOffsetRange(-1, 0)` is a relative reference that selects the cell above the currently selected cell. This reference points not to a specific cell but to a cell relative to the position of the currently selected cell.

By default, all actions in Action Recorder are recorded in absolute mode. You can change the mode by using the drop-down list at the top of the Record Actions task pane, as shown in Figure 2-18.

To fully explore these two recording modes, take a moment to perform a small test. First record a script using absolute mode:

1. **Open a new workbook.**
2. **On the Automate tab, select New Script and then select Create from Recording.**
3. **In the drop-down list in the Record Actions task pane, select Absolute Mode (refer to Figure 2-18).**
4. **On the worksheet, place your cursor in cell B2.**

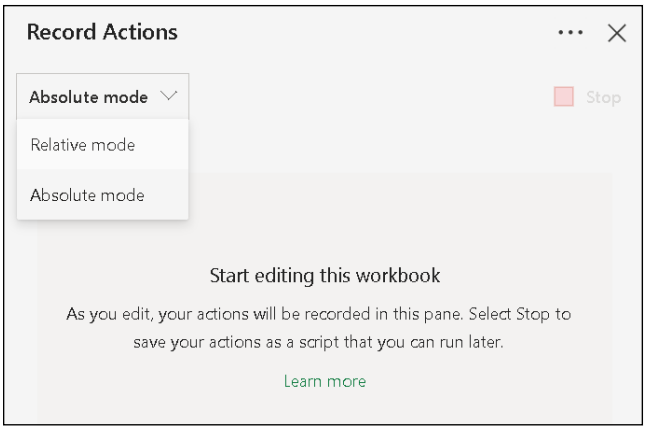


FIGURE 2-18: The Record Actions task pane lets you choose to record in relative or absolute mode.

5. **Click the up arrow key on your keyboard to go to cell B1.**
6. **Enter 1 in cell B1.**
7. **Click the right arrow key on your keyboard to go to cell B2.**
8. **Enter 2 in cell B2.**
9. **Stop recording by clicking the Stop button in the Record Actions task pane.**

Now record the same actions using relative mode. Follow the preceding steps, but in Step 3 select Relative Mode in the drop-down list. Take a gander at Figure 2-19 and compare how Excel references cells differently in absolute versus relative mode.

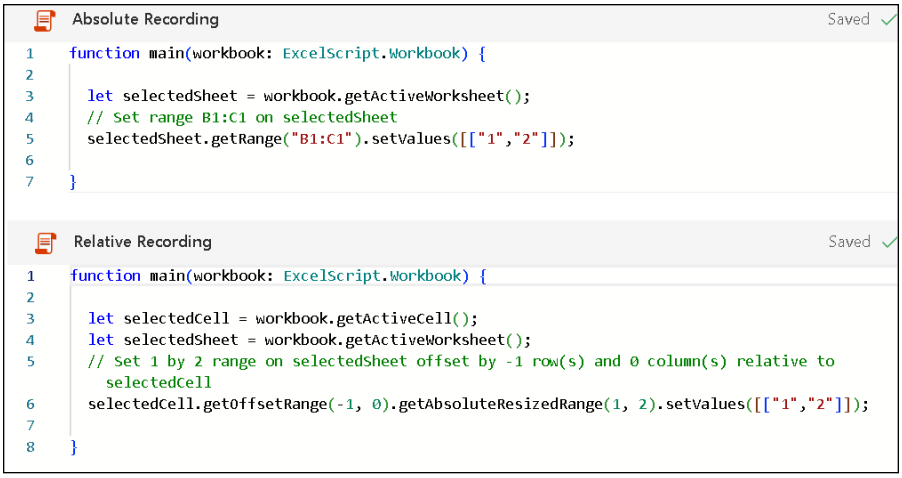


FIGURE 2-19: Compare the results of recording in absolute mode versus relative mode.

In the absolute recording script, the B1:C1 cell reference is hard-coded. No matter where your cursor is on a worksheet, this script will always edit only cell B1 and cell C1.

The relative recording script doesn't have a reference to a cell. Instead, the code uses methods such as `getOffsetRange` and `getAbsoluteResizedRange` because the script has been written using the relative position of the cell that was active when the recording started. When you run this script, your cursor's position in the worksheet is important. For instance, placing your cursor in cell D5 and running the relative recording script will edit cell D4 and cell E4. Why? Because this script does not specify an exact range. It tells Excel to go up one row and then go right one column relative to the cursor's position.

Table 2-1 is an at-a-glance guide for what recording mode is best given a project's requirement.

TABLE 2-1 Using Absolute versus Relative Recording

Recorded Script Must	Recording Mode
Work with exact cell ranges regardless of cursor starting position (for example, add a sum to a cell B2)	Absolute
Automate features based on specific ranges (for example, create a chart from data in cells A1:E15)	Absolute
Run based on a pattern (for example, move through the selected table row by row)	Relative
Process cells based on position (for example, write new values to the cell below the active cell)	Relative

Absolute recording mode is best when you need your code to predictably work with exact cell ranges regardless of the cursor's starting position or the layout of the workbook. Relative recording mode is best when your script needs to operate relative to the active cell, no matter where it is.

Relative recording mode makes your code more complex to read and manage but offers flexibility when you need to work with varying data layouts and starting points.

2 Basic Programming Concepts in Office Scripts

IN THIS PART . . .

Begin your journey into Office scripting by learning the fundamentals of the TypeScript language.

Explore the debugging and error-handling techniques that will help you anticipate and manage the errors in your scripts.

Go beyond simple scripts by introducing conditional logic and iterative actions to your automated processes.

IN THIS CHAPTER

- Introducing the object model
- Exploring methods and properties
- Simplifying with variables
- Clarifying with comments
- Getting flexible with functions
- Leveraging parameters

Chapter 3
Understanding Scripting Fundamentals

Excel thinks and acts in objects. In programming parlance, an *object* is a digital container that represents a specific element of an application. In the context of Excel, objects are things like workbooks, worksheets, and ranges.

You can't truly understand Excel scripts without a working knowledge of the Excel object model. So in this chapter, you explore the fundamental building blocks of the Excel object model and learn the core concepts vital for creating flexible scripts in Excel.

REMEMBER

The code snippets used in this chapter are available in the `Chapter 3 Code Snippets.xlsx` sample file. You can download the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Understanding the Object Model

In the real world, you can describe everything you see as an object. For example, your home is an object. The rooms in your home are separate objects. If a room has a closet, it is likewise an object. As you think about your home, its rooms, and its

closets, you may see a hierarchical relationship between them. Excel works in the same way.

In Office Scripts for Excel, the `workbook` object is the top object. Like your house, the `workbook` object is the container for all other objects.

Every script must contain the `main` function, which initiates the `workbook` object. All code is written in the function block (between the curly brackets) and applies to the currently active workbook:

```
function main(workbook: ExcelScript.Workbook){

}
```

Within the `workbook` object are collection objects. A *collection* is a group of objects of the same type. Some of the common collection objects in a workbook are:

- `worksheets` collection
- `charts` collection
- `tables` collection
- `pivotTables` collection

Each worksheet in your workbook lives in the `worksheets` collection. If you want to refer to a particular object in a collection, you can refer to it by its name. For example, the following code activates the worksheet called `Sheet1`:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1').activate();

}
```

In this code, Excel is calling specific `worksheet` objects in the `worksheets` collection in the current workbook.

Worksheets contain their own collection objects. Some of the common collection objects in a worksheet are:

- `range` collection
- `tables` collection
- `charts` collection

`pivotTables` collection

`comments` collection

To point to a specific object (such as a range), you need to traverse the object model. For example, the following code selects range A1:D5. Starting from the `workbook` object, you reference the `worksheet` object (`Sheet1`) and then the range object (`A1:D5`):

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1').getRange('A1:D5').select();

}
```

To access object members, you use the *dot operator (.)*. Each time you traverse the object hierarchy, you access the next level by using the dot operator.

Note that in the following code example, you don't have to traverse objects in a single line. You can chain the dot operators on different lines, improving readability for longer object chains:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1')
.getRange('A1:D5')
.select();

}
```

Ranges have their own `cells` collection. To continue our example, the following script selects a specific cell from the collection of cells in the `range` object (defined as `A1:D5`). In this case, you select the cell in row position 1 and column position 1 in the range:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1')
.getRange('A1:D5')
.getCell(1,1)
.select();

}
```

You work extensively with ranges and cells in subsequent chapters, so don't get hung up on the syntax here. For now, it's enough to understand how you reference objects by traversing the object model.

As you take in this information, think about all the features of Excel: pivot tables, charts, tables, slicers, comments, and more. These are all objects that have their own object hierarchy. For instance, a pivot table object has pivot field child objects, a chart object has a legend object, and a table object has header and footer objects. You can write scripts for almost any feature of Excel by simply navigating and referencing the objects in the Excel object model.

Understanding Methods

Methods are the actions you can perform on an object. It helps to think of methods as verbs. For instance, the following code uses the `select` method to select range A1:D5:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1')
.getRange('A1:D5')
.select();

}
```

In practice, methods enable your scripts to do things like activate a worksheet, select a range, add a worksheet, and set formatting for an object.

Every object has its own set of methods, although many methods are common across objects. For example, both the `chart` and `worksheet` objects have a `setName` method for renaming the object.

The following code uses the `setName` method to rename `Sheet1` to `MyFirstSheet`, and then uses the `add` method to add a worksheet called `MySecondSheet` to the workbook:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('Sheet1')
.setName('MyFirstSheet');
workbook.addWorksheet('MySecondSheet');

}
```

REMEMBER

Because Office Scripts is based on TypeScript, its core language features such as object names, method names, and variable names are case sensitive. For instance, `getWorksheet` is a valid method but `GetWorksheet` will result in an error. When working with the objects and methods in your Excel scripts, be sure to adhere to their proper case stylings.

TIP

You may have noticed the use of the semicolon (`;`) character in the code in this chapter. The primary role of the semicolon is to mark the end of a statement, so that anyone reading the script can easily see where one instruction ends and the next begins. Although the use of a semicolon at the end of each instruction is optional, it's good practice to include them for clarity.

Just as in the VBA environment, the Code Editor task pane provides IntelliSense to assist you in writing code. IntelliSense suggests possible code in response to what you're typing based on the framework of the Office Scripts API.

Using IntelliSense, you can quickly see the available methods for any object by simply typing the dot operator (see Figure 3-1). After entering the dot operator after `getWorksheet('Sheet1')`, IntelliSense lists all possible methods for that `worksheet` object. You can browse through the methods to find the one that best suits your needs.

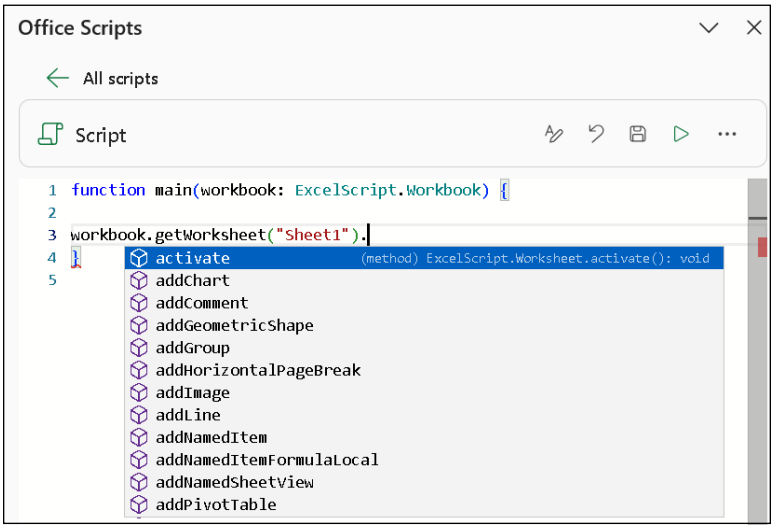


FIGURE 3-1: IntelliSense shows you all the methods for an object.

TIP

IntelliSense is truly your friend when writing Office Scripts. In addition to exposing all the possible methods available for an object, IntelliSense highlights syntax errors with wavy red underlines, displays required and optional arguments for methods, and sometimes offers options to automatically implement suggested changes.

As you look through an object's methods, note that many begin with `get` or `set`. Almost all methods that begin with `set` have a corresponding `get` method. For example, a `worksheet` object has a `setName` method that you use to rename a worksheet. A `getName` method returns the name of the worksheet.

There are generally more `get` methods than `set` methods because some objects have properties that can't be changed. The `getName` method of the `workbook` object is an example. You can find out the name of a workbook with this method, but the only way to change the name of a workbook is to save it with a different name. Office Scripts doesn't provide a `setName` method to save a workbook with a different name.

The other reason there are more `get` methods than `set` methods is due to the hierarchical nature of the objects in the ExcelScript package (and in Excel in general). Workbooks contain worksheets and worksheets contain ranges (that is, individual cells or groups of cells). Some `get` methods exist only to provide access to the object below the object that has the method. For example, the `workbook` object's `getWorksheets` method returns all the worksheets in that workbook. A `setWorksheets` method doesn't make sense.

TIP

Throughout the book, you'll see the term *properties,* which are the characteristics of an object. If you consider your house an object, it has properties such as a color, square footage, and age. You can change some properties, such as the color of the house, but not others, such as its age. Likewise, Excel objects have properties. An Excel worksheet has a `name` property, which can be changed, and a maximum row limit, which can't be changed.

Enhancing Code with Variables

Variables are memory containers that temporarily store information that you can use in multiple places in procedures. You can store data and complex objects in a variable using a user-friendly name, lessening the burden of writing code and making your scripts easier to read.

To understand variables, consider the following example script. Here, you're setting the fill color of cells B2 and B4 to yellow:

```
function main(workbook: ExcelScript.Workbook) {

 workbook.getWorksheet('Sheet1')
 .getRange('B2')
 .getFormat()
 .getFill()
```

```
 .setColor('#ffff00');

 workbook.getWorksheet('Sheet1')
 .getRange('B4')
 .getFormat()
 .getFill()
 .setColor('#ffff00');

}
```

Note how you need to reference the `worksheet` object (`Sheet1`) each time you point to a new range because you need to navigate the object model. Each time you need to reference a new object in `Sheet1`, it's cumbersome to type `workbook.getWorksheet('Sheet1')`. You can simplify this code by using variables.

Introducing a new variable to a script is referred to as *declaring* the variable. In the following script, you declare two variables, `MySheet` and `YellowColor`:

```
function main(workbook: ExcelScript.Workbook) {

 let MySheet = workbook.getWorksheet('Sheet1')
 const YellowColor = '#ffff00'

 MySheet.getRange('B2')
 .getFormat()
 .getFill()
 .setColor(YellowColor);

 MySheet.getRange('B4')
 .getFormat()
 .getFill()
 .setColor(YellowColor);

}
```

The `MySheet` variable stores the `worksheet` object representing `Sheet1`. You enter the long object navigation, `workbook.getWorksheet('Sheet1')`, just once and then simply use `MySheet` to represent that object each time you need to point to it.

The `YellowColor` variable stores the hexadecimal code for the color yellow. Again, you enter `#ffff00` once and then simply use that variable each time you want to apply that color.

Compare the two versions of this code in Figure 3-2. You can see how variables eliminate the need to type redundant syntax and reduce the possibility of typos and errors.

Before Variables

```
function main(workbook: ExcelScript.Workbook) {

    workbook.getWorksheet('Sheet1')
        .getRange('B2')
        .getFormat()
        .getFill()
        .setColor('#ffff00');

    workbook.getWorksheet('Sheet1')
        .getRange('B4')
        .getFormat()
        .getFill()
        .setColor('#ffff00');
}
```

After Variables

```
function main(workbook: ExcelScript.Workbook) {

    let MySheet = workbook.getWorksheet('Sheet1')
    const YellowColor = '#ffff00'

    MySheet.getRange('B2')
        .getFormat()
        .getFill()
        .setColor(YellowColor);

    MySheet.getRange('B4')
        .getFormat()
        .getFill()
        .setColor(YellowColor);

}
```

FIGURE 3-2: Examining the same code with and without variables.

TECHNICAL STUFF

A *hexadecimal* (*hex*) code is a six-character identifier that uses a system of numbers (0–9) and letters (A–F) to represent colors. Preceded by a hashtag (#), the code acts as a three-part instruction for red, green, and blue (RGB) intensity. In this system, 0 represents total darkness and F represents maximum brightness. For example, a hex code of #000000 is black (all color channels off). The code #FFFFFF is white (all color channels at maximum). A pure red color is hex code #FF0000 (only the red channel is fully engaged).

Using let versus const

In Office Scripts, you declare a variable by using the `let` keyword or the `const` keyword.

You use the `const` keyword for any variable that will remain constant throughout the script. In our previous example, the hex code #ffff00 represents the yellow you want. Because this won't change, you can declare the `YellowColor` variable as a constant.

You use the `let` keyword to declare a variable whose contents might change. Continuing with the previous example, the `MySheet` variable was declared using the `let` keyword. This means `MySheet` can point to another object if you want.

A FEW WORDS ABOUT NAMING VARIABLES

Because Office Scripts are written in TypeScript, they adhere to JavaScript variable-naming conventions. Let's take a moment to talk about some of the limits and restrictions when naming variables.

Variable names must begin with a letter or an underscore (_). They can't start with a number or any other symbol. Also, spaces and most special characters (!, @, #, $, %, ^, &, ?) are not allowed. In general, it's best to stay away from using any character that's not a letter or number.

Excel won't let you use names that are reserved TypeScript keywords, such as `if`, `for`, `while`, `function`, `let`, `const`, and `var`. Also, like other syntax in TypeScript, variable names are case sensitive. So a variable named `MyWorksheet`, for instance, is a different variable than one named `myworksheet`.

Finally, although variable names do not have a fixed maximum length, you'll want to shoot for descriptive yet concise names. Long variable names can reduce code readability and make it harder to understand and maintain your scripts.

For example, in Figure 3-3, `MySheet` is initially declared and assigned the `Sheet1` `worksheet` object. Any action applied to `MySheet` is applied to `Sheet1`. Then on line 7, `MySheet` is assigned a new object representing `Sheet2`. Any action applied to `MySheet` at that point is applied to `Sheet2`.

Reassigning Variable

```
1   function main(workbook: ExcelScript.Workbook) {
2
3       let MySheet = workbook.getWorksheet('Sheet1')
4       MySheet.activate();
5       MySheet.getRange('B2').select();
6
7       MySheet = workbook.getWorksheet('Sheet2')
8       MySheet.activate();
9       MySheet.getRange('B2').select();
10  }
```

FIGURE 3-3: You can reassign variables declared with the `let` keyword.

As you advance to more complex scenarios, such as working with dynamic rows, the `let` keyword will become more useful. Review the following script:

```
function main(workbook: ExcelScript.Workbook) {

  let MySheet = workbook.getWorksheet("Sheet1")
```

```
    let MyRow = 1
    let MyColumn = 1

    MySheet.getCell(MyRow, MyColumn).setValue('Microsoft');

    MyRow = 3
    MySheet.getCell(MyRow, MyColumn).setValue('Excel');

}
```

Here, you write `Microsoft` and `Excel` in two cells. Instead of referencing an exact range, such as B2 and B4, you use the `getCell` method to point to the target.

The `getCell` method requires a row position number and a column position number. You pass the required position numbers using two variables: `MyRow` and `MyColumn`. Note how the `MyRow` variable is initially assigned `1` and then later assigned `3` to dynamically change the target cell.

Figure 3-4 illustrates how the script writes to different cells even though you use the same syntax: `MySheet.getCell(MyRow, MyColumn).setValue`.

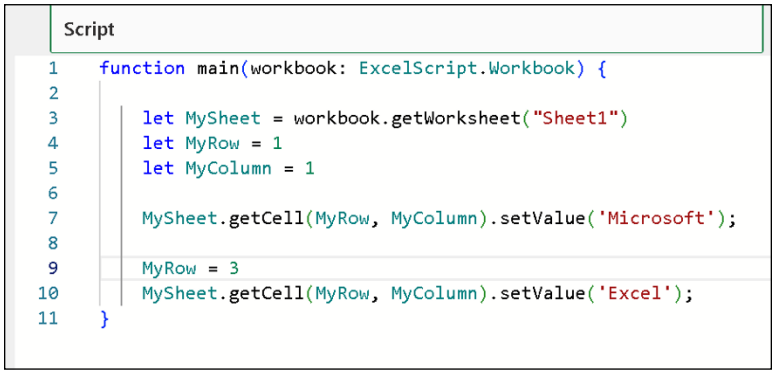


FIGURE 3-4: Using variables to dynamically point to different cells.

Although this example is simple, the capability to dynamically reference cells with variables opens the door to all kinds of possibilities. Later in the book, you'll see many examples of dynamic scripting using the `let` keyword.

REMEMBER

In TypeScript, indexes and ordinal position numbers are zero-based. This means the first index, row, or position number of any collection is always the 0th element. For example, according to TypeScript, cell A1 is in position 0,0 (row 0, column 0) and cell B1 is in position 0,1 (row 0, column 1). The first worksheet in a collection of worksheets is worksheet[0], and the second worksheet is worksheet[1].

Understanding data types

A *data type* defines the kind of value a variable can hold and the operations that can be performed on it. For example, a value can be stored in a variable as a number, text, or an array. (I describe arrays later in the chapter.) When you create a variable, Office Scripts automatically assigns it a data type, but for large or complex scripts, it's better for performance when you explicitly define data types for your variables.

You use data types to tell our script whether you're working with numbers, text, objects, or arrays of information. To explicitly declare a variable's data type, simply add a colon after the variable's name, followed by the data type, as follows:

```
function main(workbook: ExcelScript.Workbook) {

let Age: number = 30
workbook.getActiveCell().setValue(Age + 10);

}
```

By explicitly specifying the type for your variables, you ensure that your script accepts only appropriate values (in this case, numerical values) in the `Age` variable. You also improve clarity by letting the reader know exactly how your variable will be used.

TypeScript includes data types for strings (text values), numbers (numerical values), Boolean values (logical values), and arrays (lists). In this section, you take a closer look at how each type works.

String data types

The *string data type* is used to represent textual data. In TypeScript, the length of a string is technically limited only by your computer's memory. But an Excel cell can hold strings up to only 32,767 characters long. So, for all practical purposes, a string data type in Excel Office Scripts is limited to 32,767 characters. Strings can store file names, category values, range addresses (such as B3:D100), or any piece of useful text.

In the following example, a string variable named `MyRange` is assigned the value `A1:B5`. `MyRange` is subsequently used as the argument for the `getrange()` object:

```
function main(workbook: ExcelScript.Workbook) {

let MySheet = workbook.getWorksheet('Sheet1')
```

```
let MyRange: string = 'A1:B5'

MySheet.getRange(MyRange).select();

}
```

You can assign a string variable by enclosing the text value between single or double quotes. The following code uses double quotes to wrap the string. This code runs the same as the last example:

```
function main(workbook: ExcelScript.Workbook) {

let MySheet = workbook.getWorksheet("Sheet1")
let MyRange: string = "A1:B5"

MySheet.getRange(MyRange).select();

}
```

Number data types

VBA has four number data types: integer, long, single and double. Integer and long number types are used for whole numbers; long is the modern standard that handles large number values (up to 2.1 billion). Single and double number types are used for decimal numbers, with double providing higher precision for fractional values.

TypeScript, however, has only one number data type. The *number data type* in TypeScript is used for both integers (whole numbers) and floating-point numbers (numbers with decimals). Because TypeScript doesn't care about the size of the number or the quantity of decimal points, there is no need to worry about using the correct data type. Just use the number data type and you're golden.

In the following example, the number data type is used to store both the whole number for `Price` as well as the decimal number representing `Tax`. Because these numbers won't change, you use the `const` keyword for both variables. The `Price` and `Tax` variables calculate the total price with tax and output that value to the active cell:

```
function main(workbook: ExcelScript.Workbook) {

const Price: number = 100
const Tax: number = .25
```

```
workbook.getActiveCell().setValue(Price + (100 * Tax));

}
```

There's a limit to how large or small a number can be and still be stored as a number data type, but you'll probably never hit those limits. The technical limits for number storage in TypeScript are as follows:

- **Largest allowed positive number:** Approximately 1.79769313486232E+308 (one quattuornonagintillion)
- **Smallest allowed positive number:** Approximately 4.94065645841247E-324 (a positive decimal number starting with 323 zeros then before you even reach the first digit)
- **Largest allowed negative number:** Approximately –1.79769313486232E+308 (negative one quattuornonagintillion)
- **Smallest allowed negative number:** Approximately –4.94065645841247E-324 (a negative decimal number starting with 323 zeros before you even reach the first digit)

Although the Office Scripts number type has generous limits, Excel cells may not be able to display or store such large numbers. When dealing with very large numbers, do extra testing to ensure that your worksheets can handle your script outputs.

Boolean data types

A *Boolean data type* is a logical value that can be either true or false. This data type is most often used to check whether a condition is met or to output a response based on the outcome of a process.

For example, you might use a Boolean variable to send affirmation back to Excel indicating the completion of a step in your script. In the following example, you declare four variables. `Activecell` is a variable referencing the active cell in the workbook, `Price` is a constant number of `100`, `Tax` is a constant number of `.25`, and `isComplete` is a Boolean that will initially be set to `false`:

```
function main(workbook: ExcelScript.Workbook) {

 let ActiveCell = workbook.getActiveCell()
 const Price: number = 100
 const Tax: number = .25
 let isComplete: Boolean = false
```

```
  ActiveCell.setValue(Price + (100 * Tax));

  isComplete = true
  ActiveCell.getOffsetRange(1,0).setValue(isComplete);

}
```

After the variables are declared, the `Price` and `Tax` variables calculate the total price with tax and output a final value to the active cell. Next, the `isComplete` Boolean variable is set to `true`. Finally, the last line writes the result of the Boolean variable to the cell below the active cell.

The output of a Boolean variable is always `TRUE` or `FALSE`. In Figure 3-5, the output of the `isComplete` Boolean is `TRUE` in this case, as you see in column H.

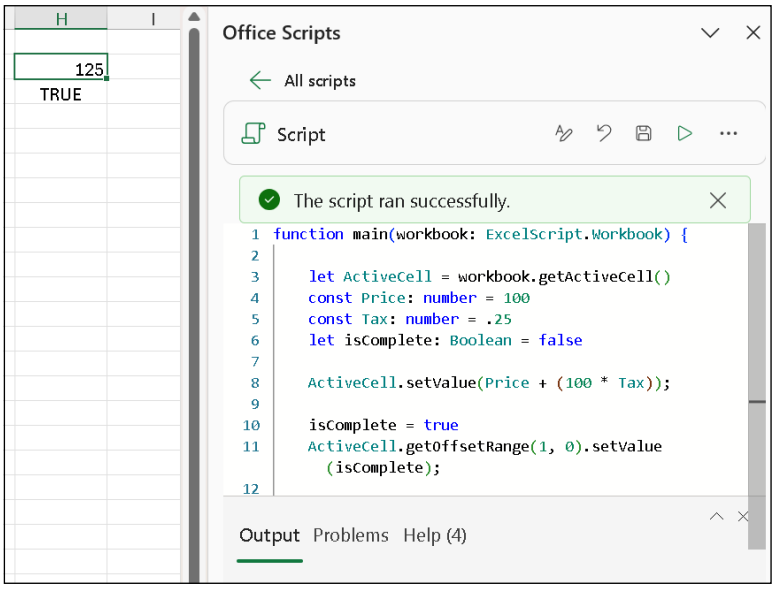


FIGURE 3-5: The output of a Boolean is either `TRUE` or `FALSE`.

Array data types

In programming, an *array* is a structured container for a list of values. Imagine a parking garage where each car is parked in its own numbered spot. Because each parking spot is in a fixed order (spot 5 is always between 4 and 6) you don't have to search the entire garage for a car; you can go directly to the numbered spot. In the same way, a data array contains values in a list where each item has a unique position number.

In terms of Office Scripts, an *array data type* is essentially a container for a data array. When you initialize a variable, you can define it as an array data type by using the open and close brackets (`[]`).

In the following example, you create an array called `MyHeaders` containing month names (`["Jan", "Feb", "Mar", "Apr", "May", "Jun"]`). To create the array, you surround the array elements with square brackets and separate each element with a comma. You then output the first value in the array to cell A1, the second value to cell B1, and the third to cell C1:

```
function main(workbook: ExcelScript.Workbook) {

let ActiveSheet = workbook.getActiveWorksheet()
let MyHeaders = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]

ActiveSheet.getRange("A1").setValue(MyHeaders[0]);
ActiveSheet.getRange("B1").setValue(MyHeaders[1]);
ActiveSheet.getRange("C1").setValue(MyHeaders[2]);

}
```

To read the array to a cell, you put the index of the element you want in square brackets (such as `MyHeaders[0]`). Because arrays in TypeScript are always zero-based, the first element is the 0th element. In this case, the word *Jan* will be written to cell A1 because `MyHeaders[0]` is the first element in the array, as shown in Figure 3-6.

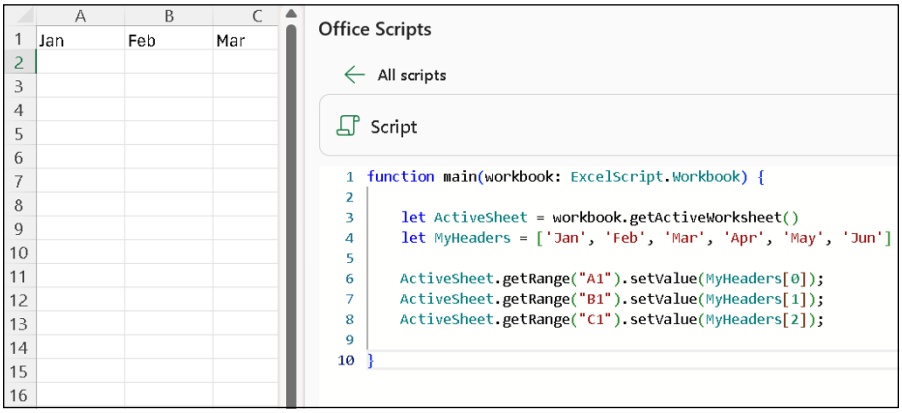


FIGURE 3-6: Arrays are zero-based, so Jan is the 0th element in this array.

The array in this example — `["Jan", "Feb", "Mar", "Apr", "May", "Jun"]` — is a one-dimensional array, meaning each element is accessed using a single

index. This approach is fine for simple scenarios, where you need to read and write one value at a time, but if you want to write data from an array to an array of cells, you need to use a two-dimensional array.

A *two-dimensional array* is a collection of elements arranged in rows and columns. Each element is accessed using two indices: one for the row and one for the column. To create a two-dimensional array, you wrap your array with another set of square brackets, creating an outer container:

```
[
         [1,2,3],
         ["a","b","c"]],
         ["AA","BB”,"CC"]
]
```

The first nested array `[1,2,3]` represents the first row in the outer container. The next nested array `["a","b","c"]` represents the second row in the outer container.

To read a two-dimensional array, you need two coordinates. The first coordinate is the row representing the nested array within the outer container that holds the data you want. The second coordinate is the position number of the value within the selected nested array. Yes, it's a bit confusing at first.

Let's say you want to enter the value `BB` in cell A1. Based on the two-dimensional array in the code, you need to point to the third row and second column to get to the value `BB`:

```
function main(workbook: ExcelScript.Workbook) {

 let ActiveSheet = workbook.getActiveWorksheet()
 let MyArray = [
          [1, 2, 3],
          ["a", "b", "c"],
          ["AA", "BB","CC"]
        ]

 ActiveSheet.getRange("A1").setValue(MyArray[2][1]);

}
```

But position numbers in TypeScript are zero-based, so the first row and column numbers are 0. This means you need to adjust your coordinates to row `[2]` (because the first row is 0) and column `[1]` (because the first column is 0). Running this final script outputs the value `BB` in cell A1, as shown in Figure 3-7.

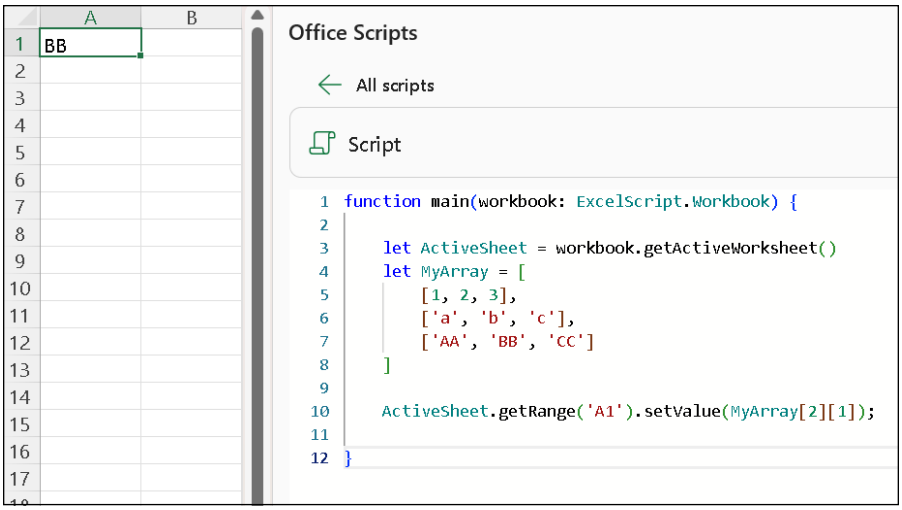


FIGURE 3-7: Referencing a two-dimensional array.

You can write any two-dimensional array directly to a range as long as it's the same size as the array. In Figure 3-8, you're writing the month names in the `MyHeaders` array directly to `A1:F1`:

```
function main(workbook: ExcelScript.Workbook) {

let ActiveSheet = workbook.getActiveWorksheet()
let MyHeaders = [ ["Jan", "Feb", "Mar", "Apr", "May", "Jun"] ]

ActiveSheet.getRange("A1:F1").setValue(MyHeaders);

}
```

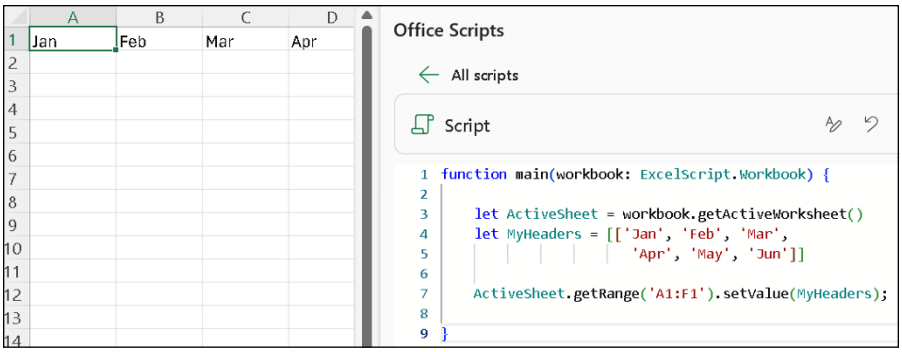
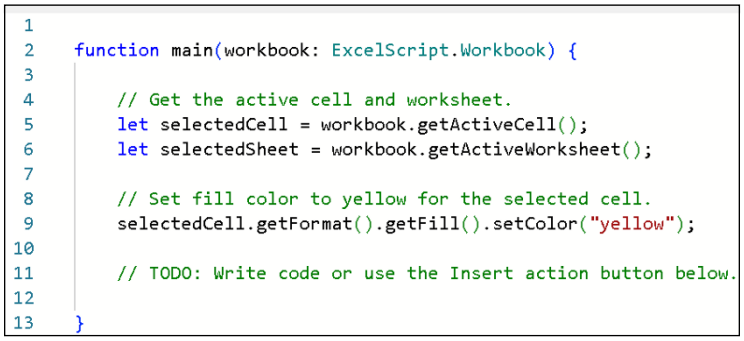


FIGURE 3-8: A two-dimensional array can be directly written to a range of the same size.

Adding Clarity with Comments

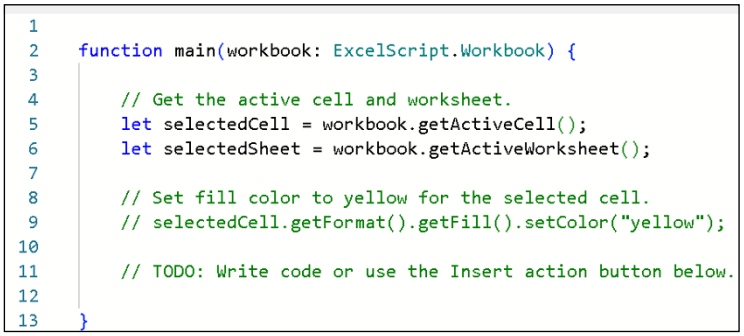
A *comment* is a line of TypeScript that's ignored when you run the script. These lines are colored green and start with two forward slashes (`//`), as shown in lines 4, 8, and 11 in Figure 3-9. Comments are typically used to describe what the code does (which isn't always obvious from reading the code itself).



```
1
2   function main(workbook: ExcelScript.Workbook) {
3
4       // Get the active cell and worksheet.
5       let selectedCell = workbook.getActiveCell();
6       let selectedSheet = workbook.getActiveWorksheet();
7
8       // Set fill color to yellow for the selected cell.
9       selectedCell.getFormat().getFill().setColor("yellow");
10
11      // TODO: Write code or use the Insert action button below.
12
13  }
```

FIGURE 3-9: Lines 4, 8 and 11 are comment lines.

You can use a comment also to exclude a statement (rather than deleting the statement and retyping it later). Simply enter two forward slashes (`//`) before the statement to convert that line to a comment; this action is called *commenting out* a line of code. In Figure 3-10, I commented out line 9. This code will run fine, but it won't do anything useful because the only action has been commented out.



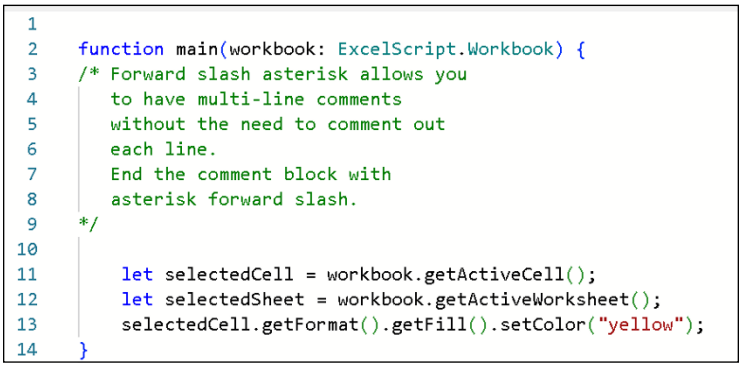
```
1
2   function main(workbook: ExcelScript.Workbook) {
3
4       // Get the active cell and worksheet.
5       let selectedCell = workbook.getActiveCell();
6       let selectedSheet = workbook.getActiveWorksheet();
7
8       // Set fill color to yellow for the selected cell.
9       // selectedCell.getFormat().getFill().setColor("yellow");
10
11      // TODO: Write code or use the Insert action button below.
12
13  }
```

FIGURE 3-10: Line 9 has been commented out.

When you're writing code, you'll often find it useful to comment out code while testing parts of your script. To reactivate commented-out statements, simply remove the forward slashes.

If you want to comment out several lines, it can be tedious to type forward slashes on each line. TypeScript has an alternative syntax for commenting blocks of code.

Start the first line with `/*` and end the block with `*/`. Everything between these symbol pairs will be treated as a comment and not executed. Figure 3-11 illustrates how this technique can be useful when adding multiline comments to your code.



```
function main(workbook: ExcelScript.Workbook) {
/* Forward slash asterisk allows you
   to have multi-line comments
   without the need to comment out
   each line.
   End the comment block with
   asterisk forward slash.
*/

    let selectedCell = workbook.getActiveCell();
    let selectedSheet = workbook.getActiveWorksheet();
    selectedCell.getFormat().getFill().setColor("yellow");
}
```

FIGURE 3-11: Use forward slash and asterisk pairs to create multiline comment blocks.

As you write more of your own scripts, you'll develop your own style of commenting code. For now, here are a few ways you'll see comments used in Office Scripts:

- To describe business logic not apparent in the code itself
- To add clarity by describing the purpose of complex statements
- To turn off old code so you can see a history of edits
- To add author credits and version info to the script
- To describe the variables, especially if the variable names aren't meaningful

Adding Flexibility with Functions

By now, you know that every script contains the `main` function, which initiates the `workbook` object. All code is written in the `main` function block (between the curly brackets) and applies to the active workbook:

```
function main(workbook: ExcelScript.Workbook) {

//All code goes here

}
```

In this section, you discover how to add custom functions to create flexible procedures that you can call on-demand to perform specific tasks.

Defining a function

A *function* is a set of instructions that perform a specific task. For example, Excel's `SUM` function is a procedure that adds some numbers and outputs an answer. You can call the `SUM` function as many times as needed.

In the same way, you can create a function that you can call repeatedly, performing the same action as many times as needed. To explore this concept further, look at the script in Figure 3-12. Here, I am setting the fill color of cells B2 and B4 to yellow.

```
1   function main(workbook: ExcelScript.Workbook) {
2
3       workbook.getWorksheet('Sheet1')
4           .getRange('B2')
5           .getFormat()
6           .getFill()
7           .setColor('#ffff00');
8
9       workbook.getWorksheet('Sheet1')
10          .getRange('B4')
11          .getFormat()
12          .getFill()
13          .setColor('#ffff00');
14  }
```

FIGURE 3-12: Code to color B2 and then color cell B4.

Note how the following code on lines 5, 6, and 7 are repeated on lines 11, 12 and 13:

```
.getFormat()
.getFill()
.setColor('#ffff00');
```

If I wanted to color more cells, I would need to repeat these steps for each new cell. This is where a function can help.

To add a function to your script, you enter a new function block in the `main` function. (As a reminder, a function block is defined by a function name and then code wrapped in curly brackets `{}`.)

The following code shows the `main` function block and, within that, a new function block called `ColorYellow`:

```
function main(workbook: ExcelScript.Workbook) {

//main code goes here

function ColorYellow
  {
  //function code goes here
  }

}
```

From here, you enter the code for the function inside the curly brackets (function block). Looking at the following script, you can see that the `ColorYellow` function will take the active cell and set the fill formatting to `#ffff00` (hex code for yellow):

```
function main(workbook: ExcelScript.Workbook) {

//main code goes here

function ColorYellow()
  {
  workbook.getActiveCell()
    .getFormat()
    .getFill()
    .setColor('#ffff00');
  }
}
```

Finally, the next script shows how you invoke your function by passing its name (`ColorYellow` in this case) followed by parentheses. You select each cell you want colored and then call the function:

```
function main(workbook: ExcelScript.Workbook) {

let MySheet = workbook.getWorksheet('Sheet1')
  MySheet.getRange('B2').select();
  ColorYellow();
  MySheet.getRange('B4').select();
  ColorYellow();
```

```
function ColorYellow()
  {
  workbook.getActiveCell()
    .getFormat()
    .getFill()
    .setColor('#ffff00');
  }
}
```

As you can see in Figure 3-13, because the steps needed to color each selected cell are typed only once (in the `ColorYellow` function), it's no trouble to apply formatting to a new cell. You simply call the function as many times as necessary.

```
1   function main(workbook: ExcelScript.Workbook) {
2
3   let MySheet = workbook.getWorksheet('Sheet1')
4   MySheet.getRange('B2').select(); ColorYellow();
5   MySheet.getRange('B4').select(); ColorYellow();
6   MySheet.getRange('B5').select(); ColorYellow();
7
8   function ColorYellow()
9       {
10      workbook.getActiveCell()
11          .getFormat()
12          .getFill()
13          .setColor('#ffff00');
14      }
15
16  }
```

FIGURE 3-13: Introducing a `ColorYellow` function simplifies the code.

REMEMBER

I can't overstate how important it is pay attention to the curly brackets when working with functions. The curly brackets are the boundaries for a function block. All relevant code for your function must be entered within its own block. Note also that any variables declared inside a function block can be used in only that function.

TIP

As mentioned, the semicolon marks the end of a statement. For example, line 4 in Figure 3-13 has the `select` statement — `MySheet.getRange('B2').select();` — on the same line as the function invocation statement, which is `ColorYellow();`. As long as you use the semicolon character to separate statements, you can put multiple statements on the same line.

TIP

Also note in Figure 3-13 that you call the `ColorYellow` function in line 4, before the function is defined in line 8. From a TypeScript perspective, where you place your function doesn't matter. As long as your function is somewhere in the `main` block, the script will run.

Understanding function return values

Some functions, like the last example, don't return a value. They simply perform an action on demand, such as formatting a cell. When a function does output a value, you use the `return` keyword to output information from the function to the main script.

Figure 3-14 uses the `getNextMonth` function to write the date representing one month from today. Lines 6–11 perform that calculation. Then on line 12, you use the `return` keyword to pass the `DateOutput` variable to the `main` function.

```
1   function main(workbook: ExcelScript.Workbook) {
2
3   let MyCell = workbook.getActiveCell();
4   MyCell.setValue(getNextMonth().toLocaleDateString());
5
6   function getNextMonth(): Date {
7       const today = new Date();
8       const year = today.getFullYear();
9       const month = today.getMonth() + 1; //0 based months so
                                              need to add 1
10      const day = today.getDate();
11      const DateOutput = new Date(year, month, day);
12      return DateOutput
13  }
14  }
```

FIGURE 3-14: Using a function that returns a value.

It's important to declare return types for any function that returns a value. In the example in Figure 3-14, line 6 shows you that the return type for the `getNextMonth` function is `Date`.

Similar to declaring a data type for variables, declaring a return type for your function ensures that the function's output matches the expected data type, which avoids errors and improves readability. If your function is expected to return a text, the return type should be set to `string`. If your function is expected to return a numerical value, the data type should be set to `number`.

Using function parameters

You might need to pass some information to your function for it to accomplish its task. A function that calculates total price with tax, for instance, would need to know an item's price and sales tax to output the result.

In the following code, you have a *parameterized* `CalculateTax` function. That is, inside the parentheses (the *function signature*), the function is expecting two

parameters: `price` and `tax`. Parameters are like variables in that you declare them with a data type and they store data for use in the function. In this scenario, both the `price` and `tax` parameters expect a number:

```
function main(workbook: ExcelScript.Workbook) {

    function CalculateTax(price: number, tax: number): number
    {
        const outputmath = price + (100 * tax)
        return outputmath;
    }
}
```

Figure 3-15 illustrates how to call the function with parameters. In line 5, you simply include your parameters in the function signature; `100` will be passed as the `price` parameter and `.25` will be passed as the `tax` parameter.

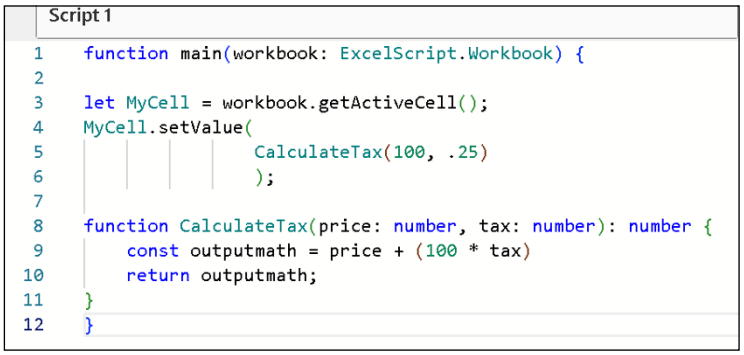


FIGURE 3-15: Passing `100` as the `price` parameter and `.25` as the `tax` parameter.

Defining default parameter values

Sometimes you want to define a default value for a parameter. For example, you can define a default tax rate of `.25` in your `CalculateTax` function. You set a default for a parameter after declaring it by typing an equal sign (`=`) and the default value, as shown in the following code snippet:

```
function CalculateTax(price: number, tax: number = .25):
  number {
```

To see your default parameter in action, look at Figure 3-16. Note how the statements in lines 4 and 5 pass both the `price` and `tax` parameters to the `CalculateTax` function, while the statement in line 6 passes only `price`. A quick glance at cell B4 confirms that, in absence of a `tax` parameter, the function used `.25` to calculate the total price with tax.

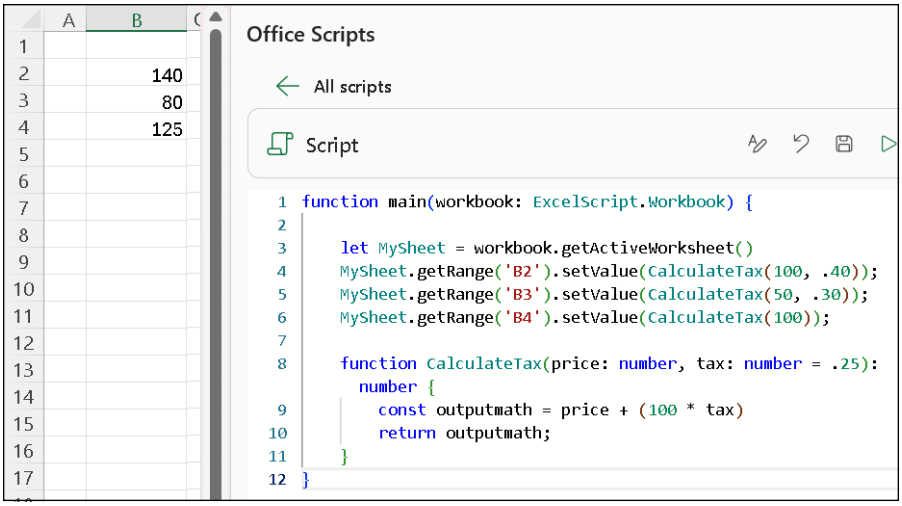


FIGURE 3-16: Defining a default value of .25 for the `tax` parameter.

REMEMBER

Attempting to use a parameterized function without the expected parameters will result in the function returning no data unless a default parameter value is defined.

Enhancing your scripts with input parameters

Now that you've discovered how to leverage parameters in your own functions, you might be wondering if you can use parameters in the workbook-level `main` function. The answer is yes! You can define what are known as input parameters at the workbook level.

Input parameters are defined just as you would custom functions, except you place parameters directly in the `main` function signature. The `ExcelScript.Workbook` parameter is always required and must be the first parameter for any script in Excel. After that, you can define additional parameters as needed. Each parameter must have a valid name and data type.

For example, in the following script, you once again calculate total price with tax. But this time, you enter parameters at the workbook level:

```
function main(workbook: ExcelScript.Workbook, price: number,
  tax: number)
{

 const outputmath = price + (100 * tax)
 let MySheet = workbook.getActiveWorksheet()
 MySheet.getRange('B2').setValue(outputmath);

}
```

Running this script triggers an input dialog box that will need to be completed for the script to successfully perform all the steps, as shown in Figure 3-17.

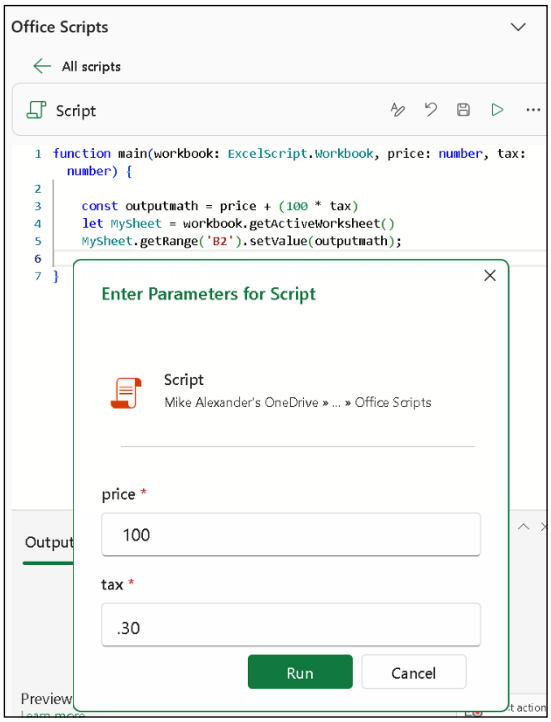


FIGURE 3-17: Running a script with an input parameter will display an input dialog box.

Using input parameters has several advantages. First, they increase script reusability by allowing one script to handle multiple use cases. This reduces the number of scripts you need to create and maintain.

Second, input parameters extend the utility of your scripts to Power Automate flows because flows can dynamically pass parameters (such as sheet names, ranges, dates, and other data) to your scripts. For instance, Power Automate can use the same script to process data on different worksheets by simply passing different sheet names. You explore integrating Office Scripts with Power Automate flows in Chapter 10.

Defining optional parameters

You can define as many input parameters as you need in a script, as long as you don't exceed the data limits covered in Chapter 1.

Note that you don't need to put all parameters on the same line. If they're within the function signature (between the parentheses), you can list them on separate lines. For instance, the following code takes five parameters: `sheetName`, `rangeAddress`, `lastname`, `color`, and `firstname`:

```
function main(workbook: ExcelScript.Workbook,
sheetName: string,
rangeAddress: string,
lastname: string,
color: string,
firstname?: string)

{
  let sheet = workbook.getWorksheet(sheetName);
  let range = sheet.getRange(rangeAddress);
  let fullname = lastname + ', ' + firstname
  range.setValue(fullname)
  range.getFormat().getFill().setColor(color);

}
```

The question mark (`?`) after the `firstname` parameter is an optional modifier. Any parameter with `?` after it is considered optional.

REMEMBER

Required parameters can't follow an optional parameter. In other words, you must define all required parameters before defining any optional parameters.

Creating a drop-down selector for a parameter

Sometimes you can help your users by providing a drop-down selector with predefined parameter choices. The following example contains a `title` parameter with four choices (`Mr.`, `Mrs.`, `Dr.`, and `Professor`) separated by pipe characters (`|`):

```
function main(workbook: ExcelScript.Workbook,
title: 'Mr.' | 'Mrs.' | 'Dr.' | 'Professor',
lastname: string,
firstname: string)

{
  let MyCell = workbook.getActiveCell()
  let fullname = lastname + ', ' + firstname
  MyCell.setValue(title + ' ' + fullname);
}
```

Running this script triggers an input dialog box that contain a title drop-down selector, as shown in Figure 3-18.

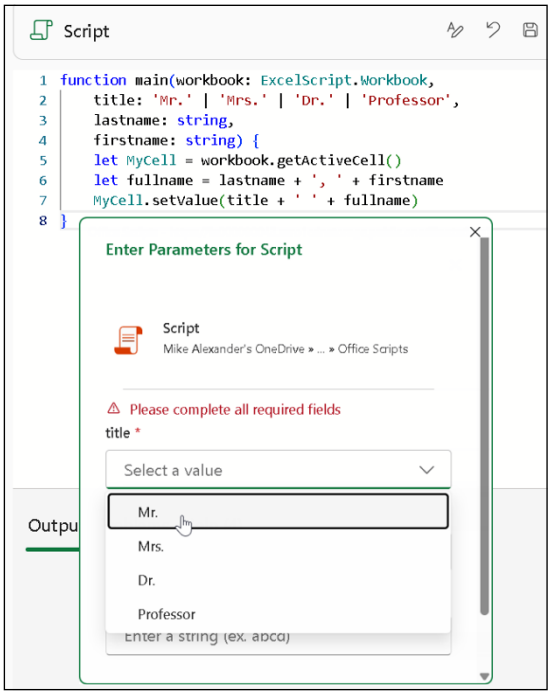


FIGURE 3-18: Using a drop-down selector to assist in parameter entry.

TIP

Although there is no limit for how many options you can define in a drop-down parameter, too many options can be unwieldy given the rudimentary interface Excel provides for entering parameters. Microsoft recommends keeping the number of options to around 20 for usability.

REMEMBER

Like all other areas of TypeScript, parameter drop-down options are case sensitive. For example, entering `Professor` and `professor` as parameter drop-down options will result in both versions of that word appearing in the drop-down selector.

IN THIS CHAPTER

- Using the console log
- Catching errors in runtime
- Exploring error-handling patterns
- Dealing with common error types

Chapter 4
Handling Errors with Style

Errors in programming are called bugs because, like real bugs, they are annoying, are often unexpected, and can ruin your day. Unfortunately, programming bugs are a fact of life. As perfect as you are, you will inevitably make silly syntax mistakes, take a misstep in your code logic, and generally introduce various errors into your scripts. But as John Walkenbach used to say, "To err is human. To anticipate errors is divine."

Before taking on the more advanced coding concepts in this book, let's pause here to explore some of the debugging and error-handling tools that will help you anticipate and manage the errors in your scripts.

REMEMBER

The code snippets in this chapter are available in the `Chapter 4 Code Snippets.xlsx` sample file. You can download the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Exploring Office Scripts Debugging Tools

Those of you coming from a VBA programming background may be used to a robust set of debugging tools that enable you to catch errors, step through code (running each line one at a time while you watch), and display error messages through the user interface. Unfortunately, you need to reset your expectations when it comes to tracking and handling errors in Office Scripts. As of this writing, Office Scripts doesn't offer a way to step through your code, nor can you relay error messages to users using a nicely formatted dialog box.

Debugging and error handling in Office Scripts are limited to two TypeScript language elements: the `console.log` method and the `try . . . catch` statement. The `console.log` method is your chief tool for debugging and communicating errors to you and your users via a console screen. The `try . . . catch` statement is the primary way to handle runtime errors in your code.

Using the console.log method

The `console.log` method enables you to output a message to the *console*, which is displayed in the Output tab at the bottom of the Office Scripts pane, as shown in Figure 4-1.

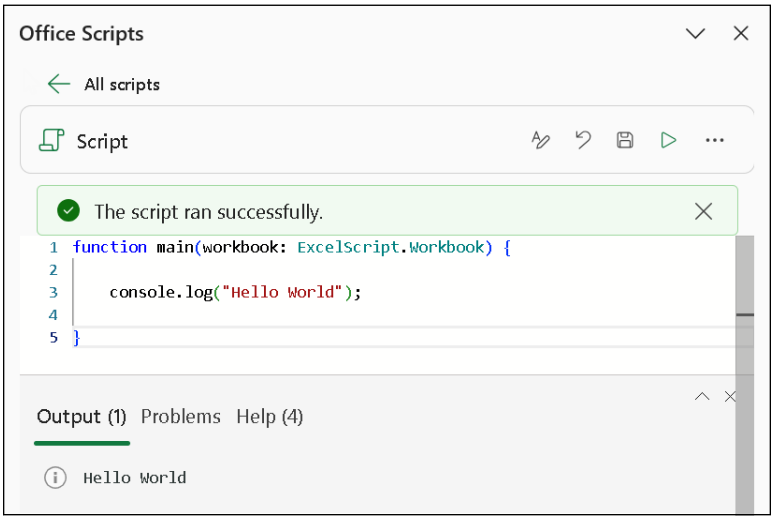


FIGURE 4-1: Use the `console.log` method to display a message at the bottom of the Office Scripts pane.

You generate messages in the console by typing `console.log` and then, in parentheses, the information you want displayed. The example in Figure 4-1 is a simple script that outputs *Hello World* to the console.

Like the `debug.print` statement in VBA, the `console.log` method lets you display all manner of information: the values of variables, the results of mathematical operations, the outputs of expressions, or anything that comes out of your code.

In the example in Figure 4-2, the name and address of the active sheet and range are output to the console. `Sheet1` is the output of the first `console.log` statement (line 7) and `Sheet1!B2` is the output of the second `console.log` statement (line 8).

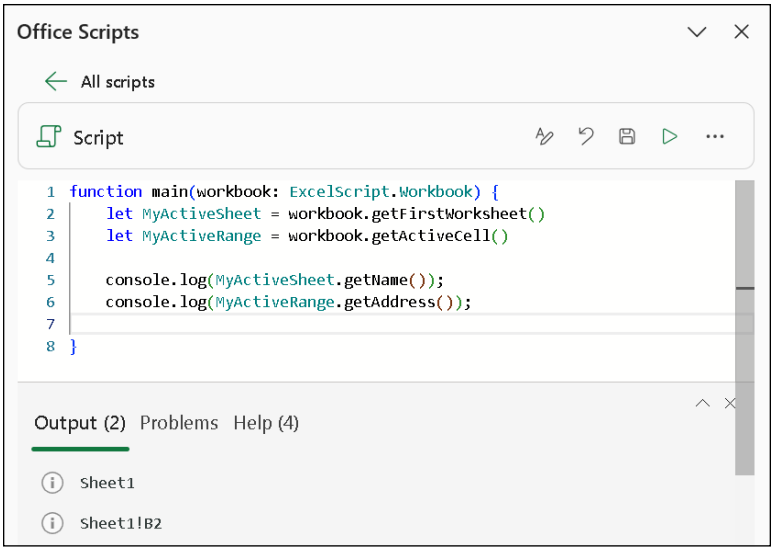


FIGURE 4-2: The `console.log` method can display any kind of value from your code.

The practical application of the console in debugging becomes evident when you encounter code where the resulting value isn't obvious. For instance, in the following code, imagine you want to determine the cell address of the `MyCell` variable. The full address is not known because it depends on the sheet name created when the `workbook.addWorkheet` statement is processed:

```
function main(workbook: ExcelScript.Workbook) {

let MyWorksheet = workbook.addWorksheet()
let Mycell = MyWorksheet.getCell(4,5)

console.log( Mycell.getAddress() );

}
```

By writing a quick `console.log` statement, Office Scripts can tell you the cell address, as shown in Figure 4-3.

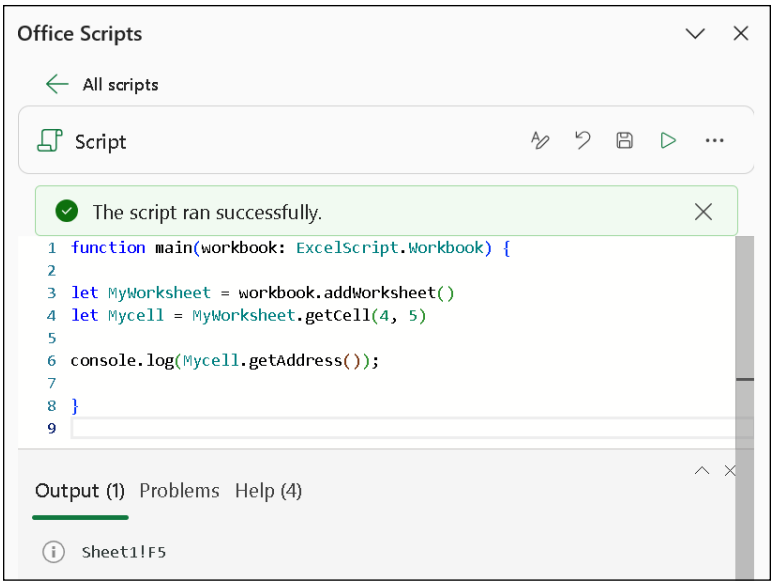


FIGURE 4-3: The console aids in debugging by exposing information that isn't obvious when looking at code.

You'll often be examining code others have written and will encounter variables that aren't clearly named. No problem. The console can show you the hidden meta data (the behind-the-scenes information) for a variable. You can enter any variable name in the `console.log` method to determine the variable type, as shown in Figure 4-4.

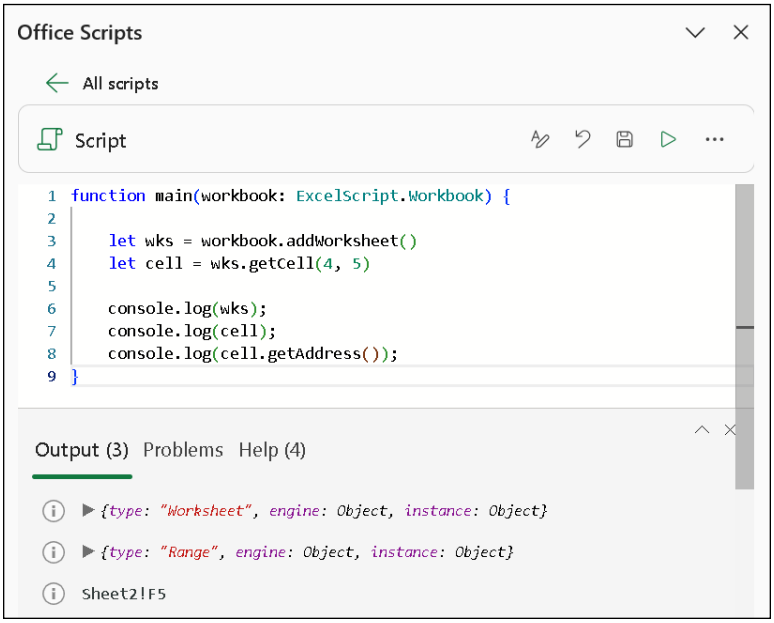


FIGURE 4-4: Enter a variable name in the `console.log` method to see meta data about that variable.

As useful as the `console.log` method can be on its own, coupling it with the `try . . . catch` statement enables you to add runtime error handling to your scripts.

Understanding try . . . catch statements

When you run a script in Excel, certain operations may fail. For example, the script might try to access a worksheet or table that doesn't exist or perform an action on an invalid cell range. If an error occurs and is not handled, your script will stop running and display a runtime error in Excel.

To make your scripts more reliable, you can employ `try . . . catch` statements to catch errors and gracefully exit the offending portion of code without abruptly terminating the script.

The following code snippet displays the basic structure of a `try . . . catch` statement:

```
try {
  // Code that might throw an error
}
catch (error) {
  // Code to handle the error
}
```

The `try` block contains code that you want to execute but that might throw an error. The `catch` block contains code that will be executed only if an error occurs in the `try` block. The `error` object displays the error message detailing what went wrong.

REMEMBER

The term *block* refers to the pair of curly brackets ({}) that follows a function or statement. All code must be contained in a block.

When a script throws an error in the `try` block (for example, your code fails due to a missing worksheet), the script will jump to the `catch` block. If there's no error, the `catch` block will be skipped.

For example, the script in Figure 4-5 fails abruptly because it's attempting to write data to the worksheet named `Data` but Excel can't find that sheet.

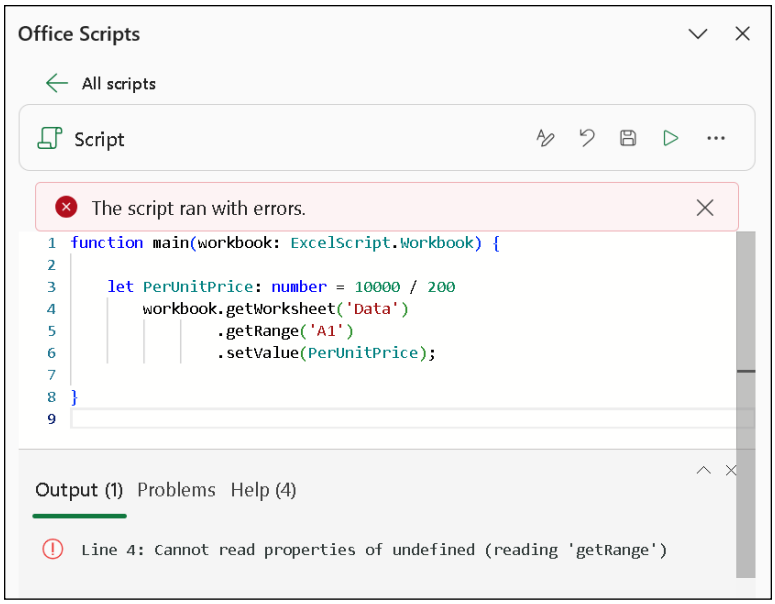


FIGURE 4-5: This code fails because a worksheet named Data doesn't exist.

You can handle this error a bit more gracefully by leveraging a `try . . . catch` statement, as shown in the following script:

```
function main(workbook: ExcelScript.Workbook)
 {

 let PerUnitPrice: number = 10000/200
  try {
    // Try writing to the Data worksheet
    workbook.getWorksheet('Data')
    .getRange('A1').setValue(PerUnitPrice);
  }
  catch (error) {
    // Handle the error gracefully
    console.log("An error occurred: " + error.message)
    console.log('Check to make sure the Data sheet exists')
  }
}
```

Note how you use the `console.log` method in the `catch` block to communicate what went wrong. Figure 4-6 shows how Office Scripts communicates the error in the console.

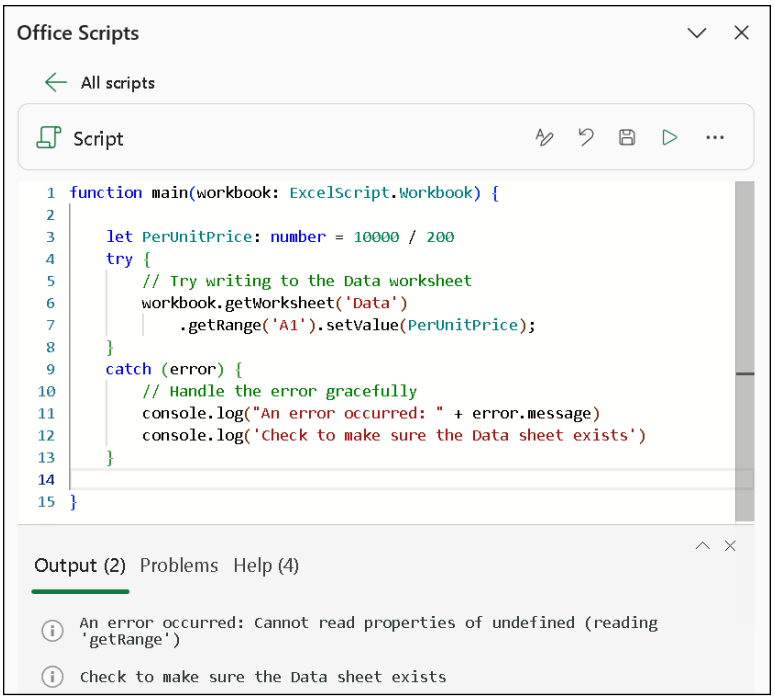


FIGURE 4-6: Use the `try . . . catch` statement gracefully handles and communicates errors.

Leveraging the finally block

You can extend the `try . . . catch` statement to include a `finally` block, which runs after the `try` block, whether or not an error occurs. The following script illustrates where the `finally` block fits into a `try . . . catch` statement:

```
try {
  // Code that might throw an error
}
catch (error) {
  // Code to handle the error
}
finally {
  // Code that runs, whether an error occurred or not
}
```

The `finally` block is entered after the `catch` block but will always run whether or not the catch block is triggered, as shown in Figure 4-7.

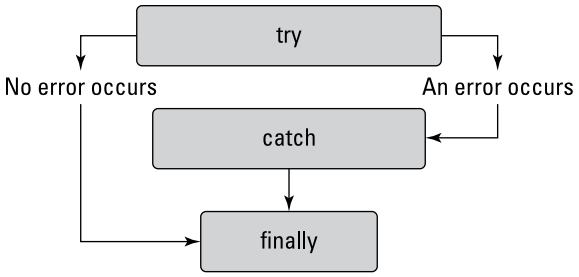


FIGURE 4-7: The process flow for a `try . . catch . . . finally` statement.

You'll often see the `finally` block used to clean up or reset the Excel environment after a script is run. For instance, the script in Figure 4-8 sets the value of cell B2 to `Processing Data` before the `try` statement executes. Note how cell B2 still displays `Processing Data`, even though an error was caught and communicated to the console.

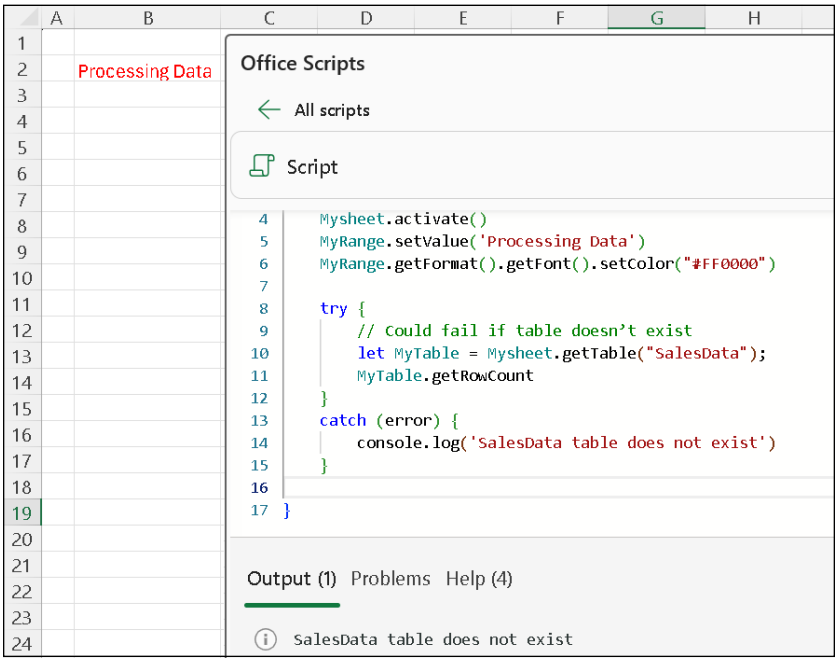


FIGURE 4-8: Cell B2 still shows `Processing Data` even after an error is thrown.

In this scenario, a `finally` block can be utilized to clear the contents of B2, as shown in Figure 4-9. Again, the code in the `finally` block runs whether or not an error is triggered.

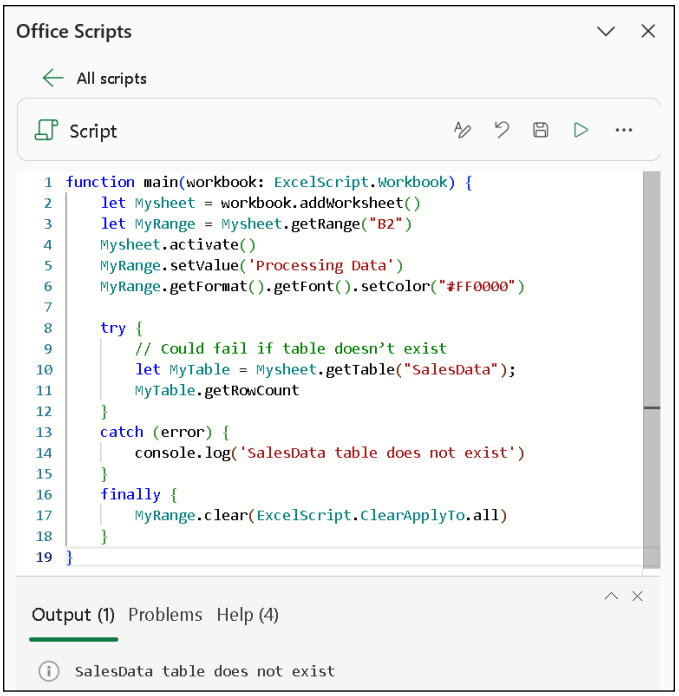


FIGURE 4-9: The `finally` block always runs its specified code after the `try` block.

Best practices for using try . . . catch statements

Before moving on, take a moment to review a few of the best practices for utilizing `try . . . catch` statements.

- **Use the error object.** When an error occurs, the error object contains the details of the error. It's best practice to include `console.log(error.message)` in the `catch` block to let the user know what went wrong. (Refer to Figure 4-6 for an example.)
- **Include descriptive messages.** The `catch` block can contain as many `console.log` statements as you need. Don't be shy about including extra console messages to communicate meaningful information, such as *"Missing table: SalesData."* Stay away from generic messages like *"An error occurred."*
- **Don't overdo it.** Overusing `try . . . catch` statements can make your code difficult to read and follow. It's best to use `try . . . catch` statements only for operations that are likely to fail.
- **Use `finally` blocks for cleanup.** Not every `try . . . catch` statement requires a `finally` block. Use a `finally` block only when you need to clean up the Excel environment or reset values after a portion of a script runs. Don't clutter your code with unnecessary blocks.

Examining Error-Handling Patterns

You'll find that not every scenario will require the same kind of error handling. In fact, if you examine various code examples from across the internet, you'll see all kinds of error-handling techniques (or patterns). In this section, you explore some common error-handling patterns and discover how you can leverage other parts of the TypeScript language to manage exceptions and errors in your scripts.

The try . . . catch . . . finally pattern

The `try . . . catch . . . finally` pattern is a favorite among many developers for its structured format and capability to respond to errors without halting the script. The following script illustrates a `try . . . catch . . . finally` pattern:

```
function main(workbook: ExcelScript.Workbook) {
let Mysheet = workbook.addWorksheet()
let MyRange = Mysheet.getRange('B2')
MyRange.setValue('Processing Data')

try {
  let MyTable = Mysheet.getTable('SalesData');
  MyTable.getRowCount
}
catch (error) {
  console.log('SalesData table does not exist')
}
finally {
  MyRange.clear(ExcelScript.ClearApplyTo.all)
}
}
```

First you assign cell B2 to the `MyRange` variable and set its value to `'Processing Data'`. Next, the script executes a `try . . . catch` statement, attempting to get the row count of the `SalesData` table. The script ends with `finally`, which clears the contents of `MyRange`.

The validation-first pattern

The *validation-first pattern* is not so much an error-handling technique as it is an error-avoidance method. With this pattern, you check a condition before executing the target script. The most common condition is checking whether an object exists before acting on the object.

For example, the following script will fail if the `Sales` worksheet does not exist:

```
function main(workbook: ExcelScript.Workbook) {
let MySheet = workbook.getWorksheet("Sales");
let MyRange = MySheet.getRange("A1");
MyRange.setValue("Validated");
}
```

Instead of handling the error after it occurs, you can avoid the error altogether with an `if{}` block. Take a gander at the following script:

```
function main(workbook: ExcelScript.Workbook) {

  let MySheet = workbook.getWorksheet("Sales");
  let MyRange = MySheet.getRange("A1");
  if (!MySheet) {

    console.log("Worksheet 'Sales' not found.");
    return;
  }

  MyRange.setValue("Validated");
}
```

You check if an object exists by first assigning that object (the `Sales` worksheet) to a variable named `MySheet`. If the `Sales` worksheet doesn't exist, the `MySheet` variable will be empty. If the `MySheet` variable is empty, it means an object is not assigned to it.

This portion of the script uses an `if{}` block with an exclamation operator (`!`):

```
  if (!MySheet) {
    console.log("Worksheet 'Sales' not found.");
    return;
  }
```

The exclamation operator (`!`) in TypeScript is a logical comparison called `NOT`. So `if(!MySheet)` means "if there is not a `MySheet` object." You can read the preceding code snippet as follows: If there is not a `MySheet` object, write to the console and then `return` (exit the script).

In the context of this pattern, you can think of the `if{}` block as analogous to the `catch{}` block in a `try . . . catch` statement. If the specified condition is true, the code in the `if{}` block will run; otherwise, it will be ignored.

TIP

You take a deeper dive into operators and conditional logic statements such as the `if` statement in Chapter 5. There, you get more examples of this type of pseudo error handling with conditional statements.

The guard clause pattern

The *guard clause pattern* prevents portions of code from running if a condition is not met, which reduces the chance for errors and avoids unnecessary code processing.

You typically see this kind of error prevention in the context of data. For instance, the following script uses a guard clause to check whether the `SalesData` table contains data rows (as well as a header row). If the row count is less than 1, the table is empty and there is no need to continue with the rest of the code:

```
function main(workbook: ExcelScript.Workbook) {

  const table = workbook.getTable("SalesData");

  if (table.getRowCount()<1) {
    console.log("SalesData table empty.");
    return;
  }

  table.getRange().getCell(1, 1).setValue("Processed");
}
```

Note how the `if{}` block uses the `return` clause to exit the script if the specified condition is true. If the specified condition is not true, the `if{}` block will be ignored.

A guard clause can be used also to validate parameters passed to a script to, for example, ensure that a required string is not empty or a number is within a valid range. This pattern is easy not only to read but also to adjust if requirements change or you need additional conditions.

The fallback pattern

The *fallback pattern* (sometimes called a *recovery pattern*) allows your script to continue running by attempting an alternative action instead of failing immediately. For example, if your script depends on a specific worksheet or table, you can use a fallback pattern to create the missing object before continuing to run the code.

The script shown here attempts to assign a worksheet named `Input` to a variable named `MySheet`. But what if there's no worksheet named `Input`? Instead of simply failing and ending the script, a `throw` statement triggers an error in the `try` block. This forces the script to jump to the `catch` block, where you simply add the needed worksheet:

```
function main(workbook: ExcelScript.Workbook) {

let MySheet = workbook.getWorksheet('Input');
try {
   if (!MySheet) throw new Error();
  }
  catch (error) {
  workbook.addWorksheet('Input');
  console.log('Error caught. Missing Input sheet created.');
  }

MySheet = workbook.getWorksheet('Input');
MySheet.getRange('A1').setValue('Fallback done');

}
```

The `throw new Error()` statement manually triggers an error and forces your code to jump to the `catch` block during script execution. The capability to trigger your own error based on a condition offers more flexibility when planning your code logic.

The simple example highlighted in this section provides a small window into the broader concept of self-healing, or self-correcting, code. With fallback patterns, your code can create missing objects and apply default values for missing data, making your scripts more resilient.

Contending with Common Error Types

Errors in Office Scripts fall into many categories, from simple syntax errors to more complex object reference errors. Although you'll encounter countless bugs and errors in your quest to build your Office Scripts skillset, many will be similar to the common error types outlined in this section.

Syntax errors

By far the most common errors are basic *syntax errors*, which include mistyped keywords, missing curly brackets, missing parentheses, or incorrect TypeScript annotations.

The good news is that syntax errors are caught early primarily because they're triggered at compile time (while you're in edit mode). Syntax errors generally prevent your script from running. For instance, Excel won't even attempt to run this script because a parenthesis is missing after `Sheet1`:

```
function main(workbook: ExcelScript.Workbook) {

  let MySheet = workbook.getWorksheet("Sheet1"
  console.log(MySheet.getName())

}
```

For its part, Office Scripts helps highlight syntax errors by displaying red squiggly lines where it recognizes syntax violations, as shown in Figure 4-10. In addition to the visual clues in Office Scripts, the Problems tab at the bottom of the Office Scripts task pane lists errors that need to be addressed before your script can run.

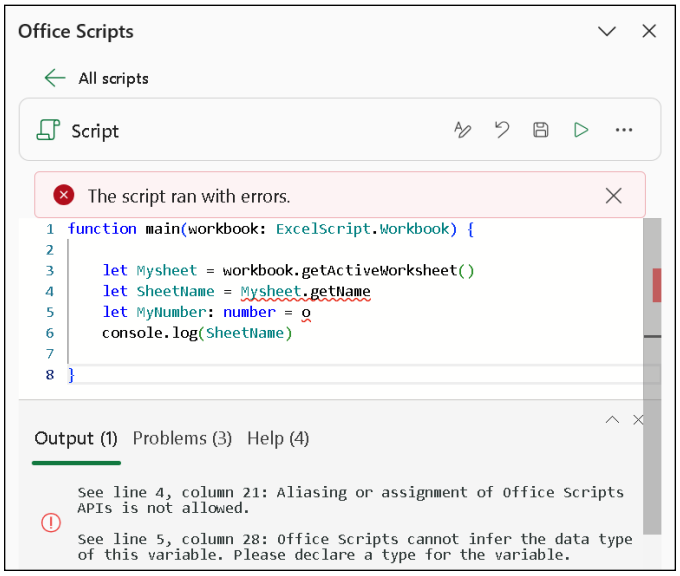


FIGURE 4-10: The Problems tab in the Office Scripts task pane lists errors that need to be fixed.

Each line on the Problems tab starts with the coordinates of the error. For instance, the first line in Figure 4-10 reads `See line 4, column 21: Aliasing or assignment of Office Scripts APIs is not allowed.`, which means that the first error is on line 4, at the 21st character.

In general, you can avoid most syntax errors by adhering to the following:

- **Close your curly brackets.** Each opening curly bracket and parenthesis must have a matching closed curly bracket and parenthesis.
- **End methods with parentheses.** Each TypeScript method must end with a parenthesis. For instance, `getName()`, `setName()`, and `getActiveWorksheet()` are all correct syntax. `getName`, `setName`, `getActiveWorksheet` are incorrect syntax.
- **Pay attention to the capitalization of variables and functions.** TypeScript is case sensitive. If you declare a variable called `MySheet`, attempting to reference that variable with `mysheet` would result in a syntax error.
- **Pay attention to the capitalization of TypeScript methods.** Enter method names with the appropriate case styling. For instance, `getActiveWorksheet()` is correct but `getactiveworksheet()` will cause an error.
- **Wrap object names in quotes.** Be sure to wrap all object names in single or double quotes. For example, if you want to refer to range A1:B1 on `Sheet1`, you would use either `workbook.getWorksheet("Sheet1").getRange("A1:B1")` or `workbook.getWorksheet('Sheet1').getRange('A1:B1')`. Referencing these without quotes, such as `workbook.getWorksheet(Sheet1).getRange(A1:B1)`, is a syntax violation.

Reference errors

A *reference error* is when you attempt to use a variable or function name that hasn't been declared. For example, the following script will fail because `myVar` has not been declared with the `let` or `const` keyword:

```
function main(workbook: ExcelScript.Workbook) {

  console.log(myVar)
}
```

A reference error can be triggered also if a variable or function is out of scope for the block. For instance, the following code will fail because `MyPrice` was declared in a separate function block. Remember that all variables declared in a function block (the space between the curly brackets, `{` and `}`) are available only in that block.

`MyPrice` can't be used outside the block in which it was declared:

```
function main(workbook: ExcelScript.Workbook) {

          let MyMath = MyPrice * (100 + .25)

function SomeMath() {
  let MyPrice: 100
}

}
```

Reference errors can be fixed by ensuring that all variable have a declaration statement and that those declarations are made in the same block in which you use them.

Object not found errors

When your code attempts to reference an object that doesn't exist, you will get an *object not found error.* An example is if you attempt to activate `Sheet1` when the workbook doesn't have a `Sheet1`.

You can guard against this type of error by using a validation-first error-handling pattern. (Refer to "The validation-first pattern" section, earlier in the chapter.) In the following example script, you check that `MySheet` exists before you attempt to write to a range in that sheet:

```
function main(workbook: ExcelScript.Workbook) {

  let MySheet = workbook.getWorksheet("Sales");
  let MyRange = MySheet.getRange("A1");

  if (!MySheet) {
    console.log("Worksheet 'Sales' not found.");
    return;
  }
  MyRange.setValue("Input Data")

}
```

Invalid range and out-of-bounds errors

You might accidentally reference a range that doesn't exist — an *invalid range error* — or a range that is out of bounds for the object being referenced — an *out-of-bounds error.*

In the following script, the first error is triggered because you're referencing cell A0 but there is no such cell. The second error occurs because you're referencing a cell using bad cell coordinates. There is no such thing as a cell in position (-1,-4):

```
function main(workbook: ExcelScript.Workbook) {

    let MySheet = workbook.getActiveWorksheet()

    /* This line will throw an invalid address error.
    MySheet.getRange("A0").setValue(1); /


    /* This line will throw an out of range error.
    MySheet.getCell(-1, -4).select() // out of range

}
}
```

The fix for range errors is to simply review the code and ensure that all ranges are valid.

Invalid arguments

Many Office Scripts methods require one or more arguments. For example, the `getRange()` method requires a string argument representing a range address, such as `getRange("A1")`.

Office Scripts checks for missing arguments at compile time. It won't let you get away with using the `getRange()` method without a range address. Your script simply won't run.

However, it's possible to pass an invalid argument to a method. In this case, your script will trigger a runtime error. Consider the example shown in Figure 4-11.

Here, two statements attempt to add a comment to the active worksheet by using the `addComment()` method. The `addComment()` method requires two arguments: the address of a single cell and a string representing the comment text.

The statement on line 7 runs fine because you provide the correct single argument for the `addComment()` method. However, the statement on line 10 triggers an invalid argument error because you pass a range of cells to the `addComment()` method.

FIGURE 4-11: Line 10 triggers an invalid argument error.

You can avoid mistakes like this by leveraging the built-in IntelliSense of the Office Scripts Editor. Before using a method, type the method name and the open parenthesis and then pause. Copy Editor will display a tool tip with the appropriate arguments for that method, as shown in Figure 4-12.

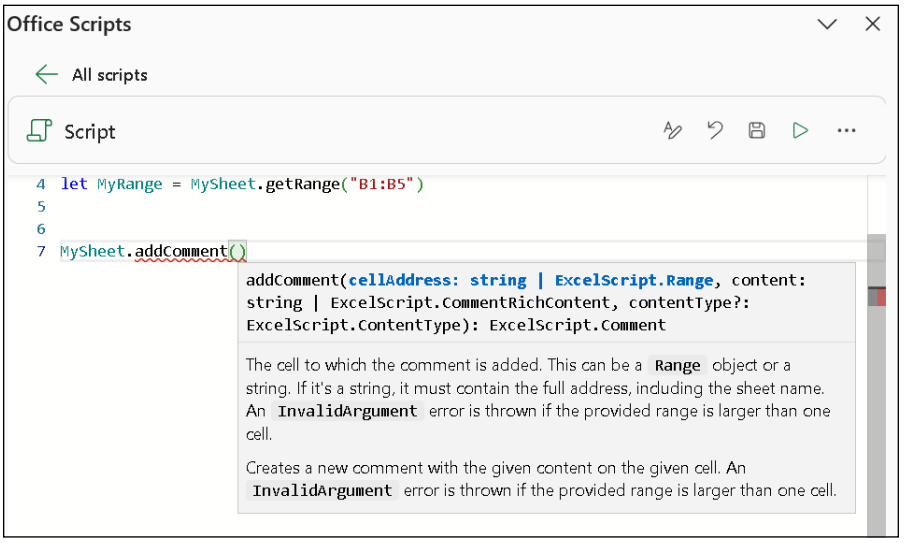


FIGURE 4-12: Leverage Copy Editor's built-in IntelliSense to see the required arguments for a given method.

IN THIS CHAPTER

- Introducing basic `if . . . then . . . else` conditions
- Performing multiple condition checks
- Understanding logical operators
- Leveraging loops for iterative code

Chapter 5

Controlling Program Flow and Conditions

If someone asked you to visualize a program running, you'd most likely imagine something like a script starting with the first line of code and running in sequence (line by line) from start to finish. After all, isn't that how most recorded macros work?

Well, as it turns out, programming can get complicated. Your scripts may need to execute different actions depending on the specific logic you need. Or your scripts may need to iterate over the same set of actions repeatedly until some condition is met. In this chapter, you take a deep dive into the techniques that enable you to build scripts robust enough to handle conditional logic, decision points, and iterative actions.

REMEMBER

The code snippets in this chapter are available in the `Chapter 5 Code Snippets.xlsx` sample file. You can download the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Getting Logical with Conditions

A *condition* is a value or an expression that is evaluated to determine which portion of code to execute based on whether that value or expression is true or false. If you've created an `IF` formula in Excel, you've worked with conditions. In an Excel `IF` formula, you test conditions and then take the appropriate actions based on the results of those tests.

That same need for conditional logic exists when it comes to scripting. You'll find plenty of scenarios where you need to build a script that takes a different path based on a condition or a logical test. Fortunately, Office Scripts offers several TypeScript language elements that enable you to do so:

- **`if {}`:** The basic `if` statement, which executes a script if a given condition is true
- **`if {} else {}`:** An expanded `if` statement that provides an alternative logical path if the initial given condition turns out to be false
- **`if {} else if {}`:** A multi-condition `if` statement that enables multiple logical checks, each leading to a separate execution path (like nested `IF` formulas in Excel)
- **`switch {}`:** A statement that simplifies complex nested `if` logic, resulting in cleaner scripts that execute different blocks of code based on the value of an expression

REMEMBER

The pair of curly brackets (`{ }`) that follows a function or statement creates a *block*. In the context of `if` and `switch` statements, the area inside the block (inside the curly brackets) is where you enter the code for the actions you want executed based on the conditional logic.

Using the basic if statement

The `if` statement is very much like Excel's `IF` function. You provide a condition, and the `if` statement determines whether the condition is true or false. If the condition is true, any script inside the curly brackets will be executed. The basic structure looks like the following:

```
if (condition to evaluate) {
code that will execute if the condition is true
}
```

The code within the `if` statement is executed only if the condition is true; the code is ignored if the condition is false. Figure 5-1 demonstrates this concept with a simple example.

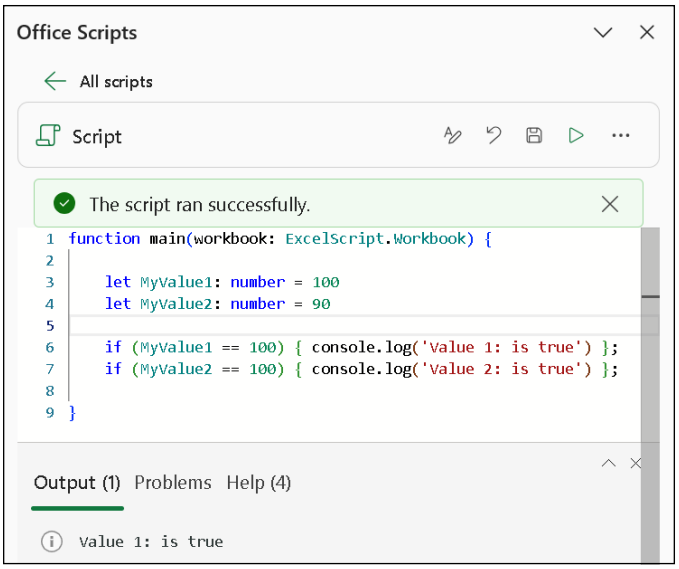


FIGURE 5-1: The `if` statement executes the code in its block only if the condition is true.

Two variables (`MyValue1` and `MyValue2`) are each assigned a number. Then on lines 6 and 7, statements check if the number assigned to each variable equals `100`. If true, the console will write a message for the given variable. Note how the console writes a line for only `MyValue1` because it's the only value that meets the condition.

REMEMBER

The semicolon (`;`) marks the end of a statement, making it easy to see where one instruction ends and the next begins. Although the use of a semicolon at the end of each instruction is optional, it's considered good practice to include it for clarity, especially when your code uses multiple conditional logic statements.

A word about equality operators

Before you go further, there's an important concept you need to understand. In standard Excel, the = operator (equal symbol) represents equality. TypeScript, however, has three equality operators:

- **Assignment operator (=):** Assigns a value to a variable. It does not perform any sort of equality comparisons. Refer to Figure 5-1, where the only place you see the single = is when you assign a value to a variable.
- **Loose equality operator (==):** Performs an equality comparison on two values (as seen in lines 6 and 7 of Figure 5-1), disregarding any data type differences between them.
- **Strict equality operator (===):** Performs an equality comparison on two values but takes into account any data type differences between them.

In the following code, `MyValue1` and `MyValue2` are different data types, so the strict equality operator (`===`) returns `false`. The loose equality operator (`==`) ignores the data types and returns `true`.

```
function main(workbook: ExcelScript.Workbook) {
let MyValue1 = 100
let MyValue2 = '100'

console.log(MyValue1 === MyValue2); //returns false

console.log(MyValue1 == MyValue2); //returns true

}
```

REMEMBER

In most cases, you can get away with using `==` (loose equality operator). However, it's considered a best practice to use `===` (strict equality operator) to avoid mismatches due to data type issues.

In addition to the equality operators (`==` and `===`), TypeScript allows other comparison operators commonly used in Excel formulas: less than (`<`), greater than (`>`), less than or equal to (`<=`), and greater than or equal to (`>=`). You use these operators in the same way you would the equality operators:

```
function main(workbook: ExcelScript.Workbook) {
let MyValue1: number = 100

if (MyValue1 < 100) {console.log('false'};

if (MyValue1 > 100) {console.log('false')};

if (MyValue1 <= 100) {console.log('true')};

if (MyValue1 >= 100) {console.log('true')};

}
```

Planning for false conditions with if . . . else statements

As mentioned, the code in a basic `if` statement block is executed only if the condition is true; the code is ignored if the condition is false. In the event you want

some action to be executed when a condition is false, you can couple the `if` statement with the `else` statement. The structure of an `if . . . else` statement follows:

```
if (condition to evaluate) {
code that will execute if the condition is true
}
else {
code that will execute if the condition is false
}
```

Figure 5-2 implements a simple `if . . . else` statement. In this script, you check whether `MyValue` is less than 50. Since the value is not less than 50, the console log message in the `else` block is executed.

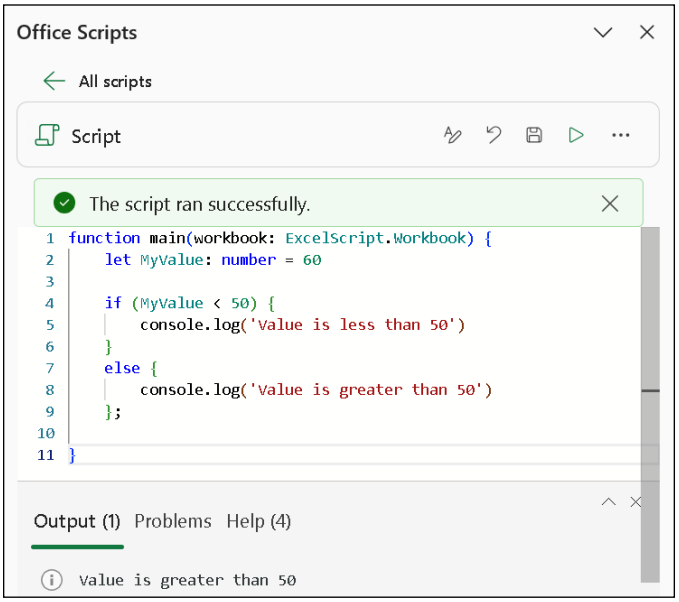


FIGURE 5-2: Use the `if . . . else` statement when you need an action to run when a condition is false.

Extending conditional checks with if . . . else if statements

You can extend your conditional analysis by adding multiple condition checks with the `else if` statement. The following snippet illustrates the structure of a

multi-condition check using `else if`. Note how each `else if` statement has its own block dictating what code to execute if the condition is true:

```
if (condition 1) {
code if condition 1 is true
}
else if (condition 2) {
code if condition 2 is true
}
else if (condition 3) {
code if condition 3 is true
}
else {
code if all conditions are false
}
```

Figure 5-3 shows the `else if` statement in action. The `BananaColor` variable is compared to three conditions, returning the corresponding console message based on which condition evaluates `true`. In this case, a Brown BananaColor corresponds with the second `else if`, causing the corresponding text (`Overripe`) to be written to the Office Scripts console.

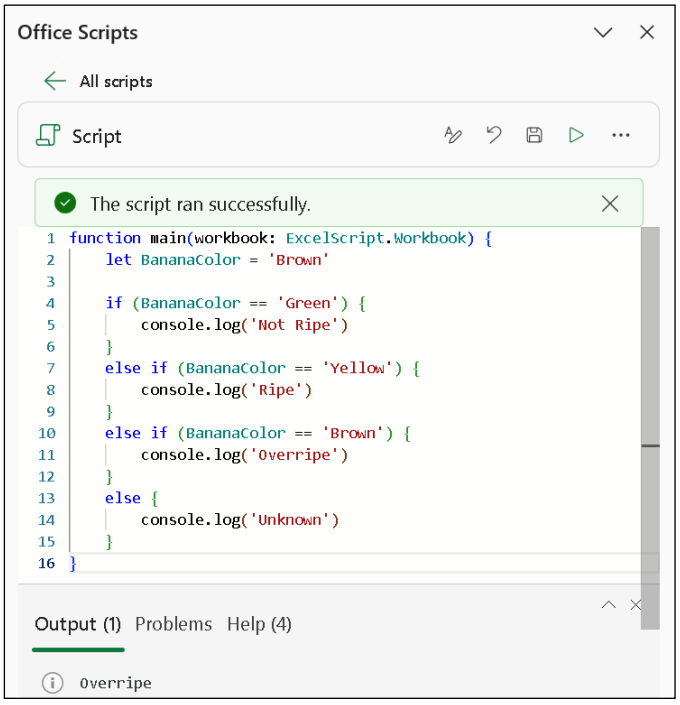


FIGURE 5-3: Utilizing the `else if` statement enables multiple condition checks.

Simplifying if statements with switch statements

Although there is no strict limit on how many `else if` conditions you can have as part of an `if` statement, too many can affect the readability of your script. As you can see in Figure 5-3, each statement adds another block you'll need to keep track of. When you need to check multiple conditions in your `if` statements, consider using the `switch` statement instead.

The `switch` statement evaluates an expression and then runs the code associated with the matching case. The following code snippet shows the basic structure of `switch`:

```
switch (InputValue) {
        case Input_Match_A: code to run
        break;
        case Input_Match_B: code to run
        break;
        case Input_Match_C: code to run
        break;
        default:
        Code to run when no match is found
        break;
        };
```

You first feed the `switch` statement an input value, which can be a string, a number, or any expression that produces a value. Next you define each possible `case` along with the code you want executed if the input value matches. The `break` clause after each `case` exits the `switch` statement when a match is found. Finally, it's good practice to include a `default` case as a catch-all in the event the input value does not match any defined cases.

Figure 5-4 illustrates the difference between using a `switch` statement versus `else if` statements. Although both scripts perform the same conditional analysis, the script utilizing `switch` is easier to read and edit.

Switch Statement

```
function main(workbook: ExcelScript.Workbook) {
    let BananaColor = 'Yellow'

    switch (BananaColor) {
        case 'Green': console.log('Not Ripe')
            break;
        case 'Yellow': console.log('Ripe')
            break;
        case 'Brown': console.log('Overripe')
            break;
        default:
            console.log('Unknown')
            break;
    };
}
```

Else If Statement

```
function main(workbook: ExcelScript.Workbook) {
    let BananaColor = 'Brown'

    if (BananaColor == 'Green') {
        console.log('Not Ripe')
    }
    else if (BananaColor == 'Yellow') {
        console.log('Ripe')
    }
    else if (BananaColor == 'Brown') {
        console.log('Overripe')
    }
    else {
        console.log('Unknown')
    }
}
```

FIGURE 5-4: Consider using `switch` statements to simplify complex nested `if` statements.

Using Logical Operators in Conditional Analyses

Some conditional analyses can't be satisfied by simply performing a value comparison such as `MyValue >10` or `MyValue = 'abc'`. You need a more nuanced analysis, such as `MyValue1 is not 100 AND MyValue1 > MyValue2`. This is where logical operators come into play.

If you've ever created `IF(AND())` and `IF(OR())` formulas in Excel, you've used logical operators. Just as in Excel formulas, Office Scripts logical operators evaluate values and return `true` or `false`. Table 5-1 lists the TypeScript logical operators along with their function.

You'll see these operators used throughout this book. For now, let's look at a few sample scripts to get a taste of some of the ways you can put logical operators into action.

TABLE 5-1 Logical Operators

Logical Operator	What It Does
&&	**The logical AND operator** combines two or more values and allows for conditional analysis as a set. Example: `A && B = "Yellow"` returns `true` only if the value of both A and B is `Yellow`.
||	**The logical OR operator** evaluates two or more values separately and allows for scenarios where an action is needed if any of the values meet a condition. Example: `A || B == "Yellow"` returns `true` if either A or B is `Yellow`.
!	**The logical NOT operator** evaluates a value and returns `true` if that value negates the given condition. This operator is often used to invert a condition. Example: `A !== B` returns `true` if A is not of the same value as B.
??	**The logical nullish coalescing operator** offers a concise way to handle null or undefined values. This operator is useful for providing default values when a variable or an expression might be null or undefined. Example: The expression `MyCellValue ?? "n/a"` returns `n/a` if `MyCellValue` is null or undefined.

The AND operator (&&)

Figure 5-5 demonstrates the use of the AND operator (`&&`) to evaluate two conditions as a set to decide what to do next. You first get the `status` and `amount` values from cells A2 and B2. In line 6, you use an `if` statement to check if the `status` value is equal to (`===`) `'Pending'` and (`&&`) the `amount` value is greater than (`>`) `1000`. If that condition is met, `Approve` is written in cell C2.

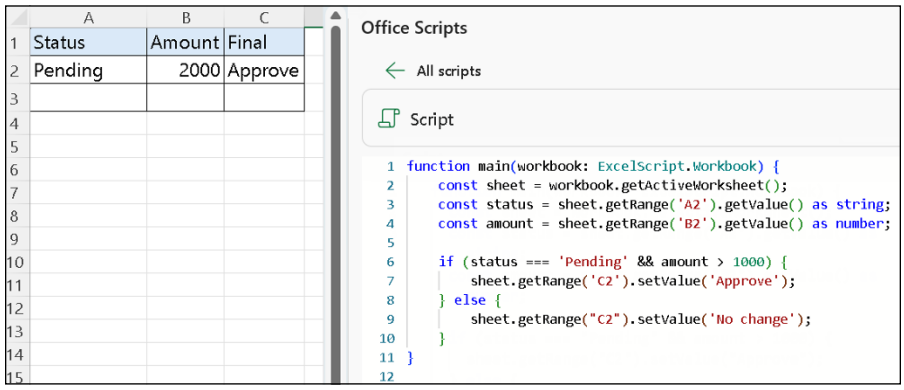


FIGURE 5-5: Use the AND operator (&&) to evaluate two or more conditions as a set.

The OR operator (||)

For an illustration of the OR operator (`||`), turn your attention to Figure 5-6, where you get the `user` and `manager` values from cells A2 and B2. Then in line 7, you initiate an `if` statement that checks for the `!user || !manager` condition.

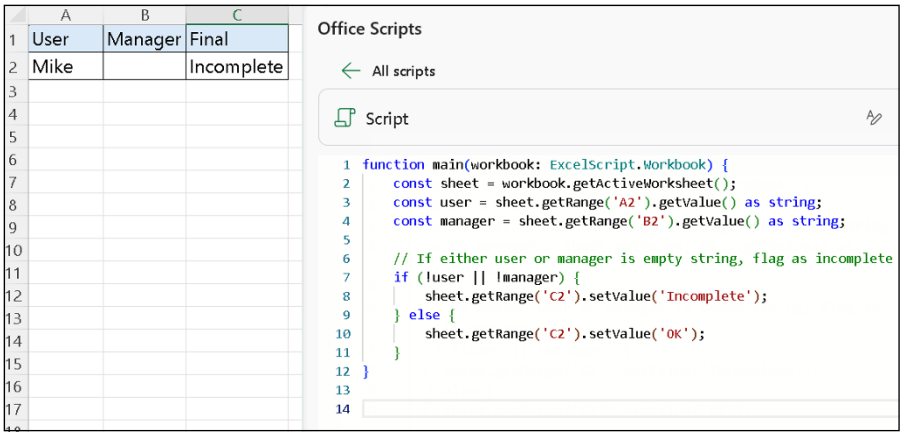


FIGURE 5-6: Use the OR operator (||) to evaluate two conditions separately, taking action if either condition is true.

When you use the NOT operator (`!`) in front of a variable, as in `!manager`, you initiate a condition check that will return `true` if that variable is empty or undefined. The condition in line 7 specifies that if `user` or `manager` is empty, mark cell C2 with the word `Incomplete`.

The NOT operator (!)

You'll find the NOT operator (`!`) useful when checking the state of a property for a given object. For example, suppose you want to check if cell A1 is unlocked before you write to that cell, as shown in Figure 5-7. In line 3, you initiate the `islocked` variable, which stores the result of the `getlocked` method for cell A1. The `getlocked` method outputs `true` if the cell is locked or `false` otherwise.

Then on line 8, you use the NOT operator (`!`) to check the `islocked` variable. Because the NOT operator inverts values, the script `if(!islocked)` translates to "if `islocked` is not true." This type of logical check on an object's properties is one of the more useful conditional analysis patterns. You'll encounter this technique in all kinds of scripts.

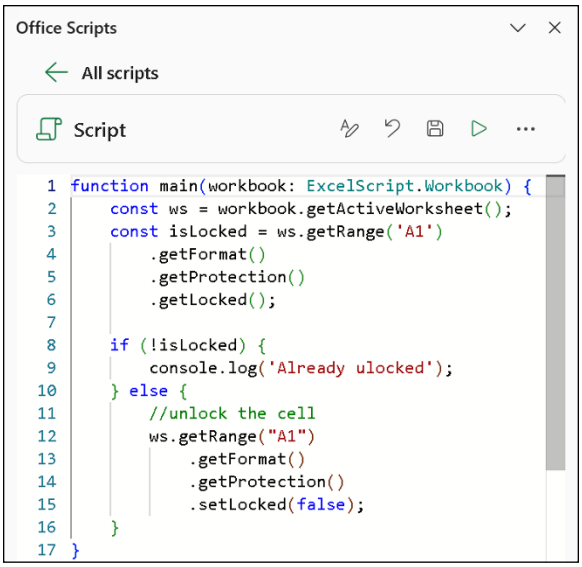


FIGURE 5-7: A useful pattern is to use the NOT operator (`!`) to check the state of a particular property for an object.

The nullish coalescing operator (??)

Despite having the most difficult name of all the operators, the nullish coalescing operator (`??`) is one of the simplest ones to employ. In short, this operator checks a given value to see if it's empty or undefined and then fills it with a default value. For instance, `Units ?? '0'` roughly translates to "if Units is empty, make the value 0."

In practice, the nullish coalescing operator (`??`) is most often used to guard against errors related to null or undefined values in your script. Figure 5-8 illustrates one scenario where you can employ this operator to avoid dividing errors due to a zero or an empty denominator.

In lines 2 and 3, you initiate two variables, `Revenue` and `Units`. Note that `Revenue` is assigned the value `5000` but `Units` remains undefined (so it remains empty). In line 8, you want to divide the revenue by the number of units. To avoid an error, you use a logical check roughly translating to "if Units = 0 OR if Units is empty":

```
If( (Units == 0) || (Units ?? '0') )
```

With this conditional check, the `if` statement can gracefully exit out of the script with a console message instead of producing an error due to an empty `Units` variable.

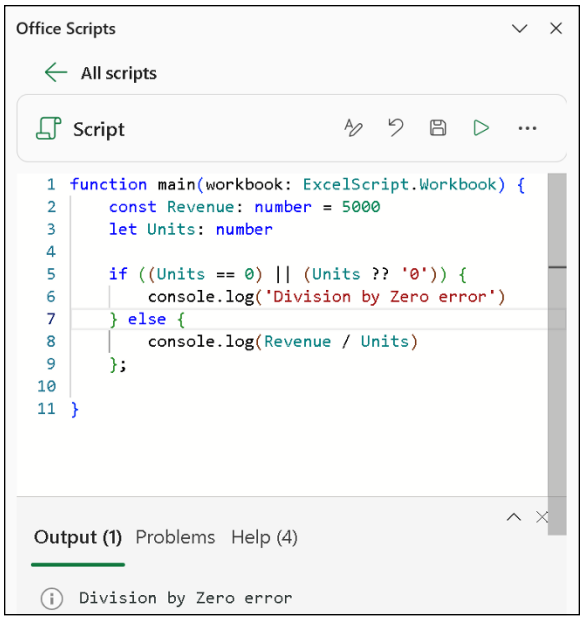


FIGURE 5-8: Use the nullish coalescing operator (??) in `if` statements to check for empty values before performing an operation.

PARENTHESES ARE YOUR FRIENDS

Logical operators have a defined order of precedence, meaning certain operators evaluate before others. The NOT operator (`!`) evaluates before the AND operator (&&), and AND (&&) evaluates before OR (||). When combining operators, the default order of precedence may not give you the expected result.

For example, the following conditional check is not only difficult to read but returns ambiguous and unintended results:

```
if (isAdmin || isManager && isActive)
//returns all Admins or Managers and all Active employees
```

Wrapping logical condition sets in parentheses explicitly defines the intended logical flow and ensures that each expression evaluates as expected. For instance, the following snippet is much clearer with the parentheses in place:

```
if ((isAdmin || isManager) && (isActive))
//returns only Admins or Managers who are Active
```

The ternary operator

For simple conditions, TypeScript offers a kind of shorthand alternative to the `if` statement called the *ternary operator*. The ternary operator uses the `?:` characters to represent an abridged `if . . . else` structure. You enter a condition to the left of the question mark, the expression if true to the right of the question mark, and the expression if false to the right of the colon:

```
Condition ? expression if true : expression if false
```

Some developers prefer the compact nature of ternary operators over `if {}` blocks. In Figure 5-9, note that there are no `if {}` blocks and the full conditional check is on one line.

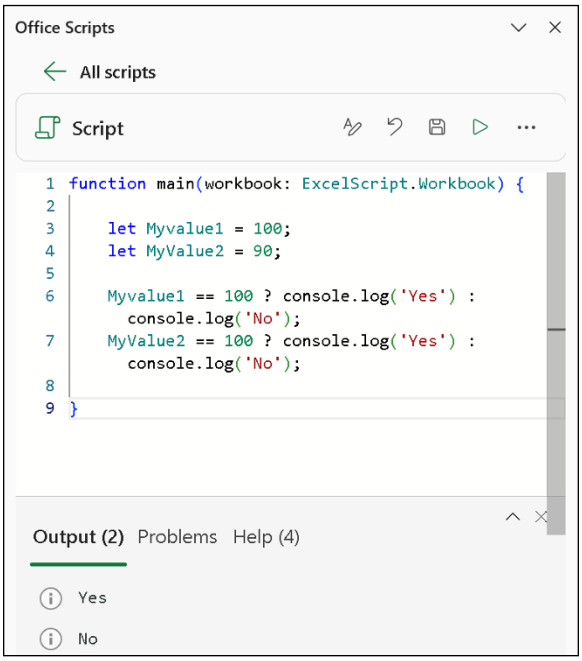


FIGURE 5-9: You can use ternary operators instead of `if` statements for simple true-or-false checks.

Ternary operators simplify your code but do not offer any performance gain. When you have simple true-or-false or yes-or-no conditional checks, ternary operators let you enter single-line, easy-to-read conditional checks without introducing blocks to manage.

Implementing Loops

A *loop* is a programming concept in which your script executes a set of actions more than once if needed. For instance, you may need to iterate over each worksheet in your workbook or insert an empty row where the value of a cell in column A is different than the value of the cell below it. You'll encounter many circumstances where you need to repeat similar commands to perform an operation.

Loops can be generally categorized as bounded loops and unbounded loops. *Bounded loops* iterate through a block of code a predetermined number of times. These loops are used when the number of repetitions is known, such as iterating through a fixed number of cells or a number of worksheets in a workbook. The two primary bounded loop constructs in Office Scripts are the `for` loop and `for . . . of` loop.

Unbounded loops are used when the number of iterations is unknown. These loops typically iterate over a set of commands until a condition is met, such as color each cell in a range until an empty cell is encountered. The unbounded loop construct in Office Scripts is the `while` loop.

Understanding for loops

The most common types of loops you will encounter are `for` loops. You start a `for` loop by defining a counter variable, specifying the number of iterations to run the loop, and setting an increment operator. The basic structure of a `for` loop is as follows:

```
for (let counter = 1; counter <= 10; counter++)
{
Code goes here
}
```

The `for` statement has three elements, separated by semicolons. The first segment declares a counter variable and sets its initial value. In the example snippet, the variable is named `counter` and starts with an initial value of 1.

The second segment defines the continuation condition. The loop will repeat until the continuation condition is `false`. The example snippet sets the continuation condition to `<=10`. As long as the counter variable is less than 11, the loop will continue. When the counter variable reaches 11, the loop stops.

The third segment defines the increment for the `counter` variable. In this sample snippet, `counter` increases by using the increment operator (`++`), which increases

the `counter` value by 1. The `counter` value starts at 1 and then, on each loop, is set to 2, 3, 4, and so on.

Following is a simple loop:

```
function main(workbook: ExcelScript.Workbook) {

   for (let i = 1; i <= 10; i++)
   {
    let current = i
    console.log('The counter is at ' + current);
  }
}
```

This loop defines `i` as the counter, and then increments `i` by 1 each time the loop iterates. Inside the `for` block, you specify the code you want the loop to run on each iteration. In this example, you output the value of `i` to the console, as shown in Figure 5-10.

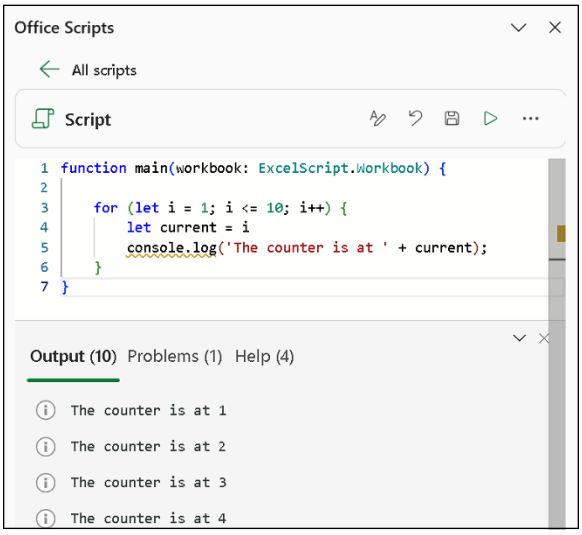


FIGURE 5-10: Using a basic `for` loop.

TIP

The squiggly line under console.log (in line 6) is a warning that `console.log()` statements in a loop can cause performance issues. While it's helpful to use `console.log()` statements in a loop for training and testing, be sure to remove them after you have finished testing.

Loop counters can also count backwards by using the decrement operator (--). The following sample script decrements `i` by 1 on each loop, counting backwards from 10 to 1. The code in the `for` block writes the current iteration count to the console, as shown in Figure 5-11:

```
function main(workbook: ExcelScript.Workbook) {

  for (let i = 10; i >= 1; i--) {
    let current = i
    console.log('The counter is at ' + current);
  }
}
```

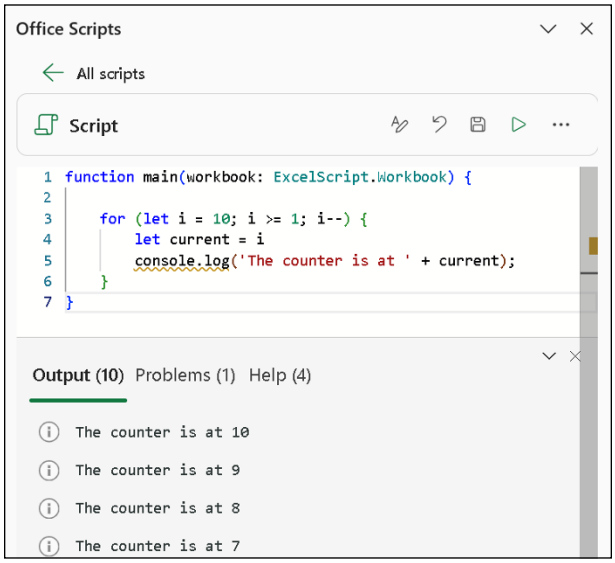


FIGURE 5-11: Use to decrement operator (--) to count backwards while looping.

To increment by a value greater than one, you can use the addition assignment operator (+=). In the following example, the loop initializes at 5 and increments by 5 on each iteration until it reaches 20. The `i += 5` statement is shorthand for `i = i + 5`. The exit condition is `i <= 20`. Once `i` reaches 25, the script recognizes that 25 is not less than or equal to 20, and stops immediately:

```
function main(workbook: ExcelScript.Workbook) {

    // Increment by 5
    for (let i = 5; i <= 20; i += 5) {
        let current = i;
```

```
        console.log('The counter is at ' + current);

    }
```

To decrement by a value greater than 1, use the subtraction assignment operator (`-=`). In the next example, you initialize the loop at 20 and subtract 5 on each iteration. The `i -= 5` statement is shorthand for `i = i - 5`. Note that the exit condition changes to `i >= 5` to ensure that the loop continues only until `i` reaches 5:

```
function main(workbook: ExcelScript.Workbook) {

    // Decrement by 5
    for (let i = 20; i >= 5; i -= 5) {
        console.log('The counter is at ' + i);
    }

}
```

Understanding prefixing versus postfixing

In most scenarios, the increment operator (`++`) and decrement operator (`--`) are used after a `counter` variable (for example, `counter++` or `counter--`). This procedure is called *postfixing the operator.* When you postfix the operator to your counter variable, the variable returns the original value before performing the increment.

TypeScript allows you to prefix operators (`++counter` or `--counter`). When you *prefix the operator* to your `counter` variable, TypeScript performs the increment or decrement before returning the value.

The distinction is admittedly a bit difficult to wrap your mind around, so let's run through an example.

In this first scenario, you postfix the increment operator (`x++`) so that the `x` variable is incremented after it's assigned to the `y` variable:

```
function main(workbook: ExcelScript.Workbook) {

  //x is assigned a value of 5
  let x = 5;

  //x is assigned to y and then incremented
  let y = x++;
}
```

The value of `x` will be 6 because the original value, 5, is incremented by 1. The value of `y` will be 5 because `y` gets assigned the original value in `x` *before* the increment.

Now let's prefix the increment operator (`++x`). Here, `x` is incremented before it's assigned to `y`:

```
function main(workbook: ExcelScript.Workbook) {

  //x is assigned a value of 5
  let x = 5;

  //x is incremented before being assigned to y
  let y = ++x;
}
```

Again, the value of `x` will be 6 because the original value, 5, is incremented by 1. But this time, the value of `y` will also be 6 because `x` is incremented before being assigned to `y`.

Most scripts you'll write will use the postfixed increment and decrement operators.

Leveraging for. . .of loops

`For . . . of` loops enable you iterate through objects that contain a list of items. Unlike in standard `for` loops, you don't define a specific number of times to iterate through the loop. Instead, `for . . . of` loops run once for each item in the list of items.

These kinds of loops are useful when working with a collection of Excel objects. Consider the following code:

```
function main(workbook: ExcelScript.Workbook)
{
  for (let ws of workbook.getWorksheets())
   {
   let SheetID = ws.getTabId()
   let SheetName = ws.getName()

   ws.getUsedRange().getFormat().autofitColumns();
   ws.setTabColor("Green");

   console.log(`Sheet: ${SheetID} Name: ${SheetName}`);
   }
}
```

The `for . . . of` statement iterates through the collection of worksheets returned by the `workbook.getWorksheets()` method. Unlike the `for . . . loop`, you don't need to define a counter. The loop automatically handles the logic: If there are five worksheets, it runs five times. With each iteration, the current worksheet is assigned to the `ws` variable, allowing the code block to perform the scripted actions — retrieving the ID, autofitting columns, and coloring the tab — before moving to the next sheet. Once every sheet has been processed, the loop terminates.

Figure 5-12 illustrates how the code outputs the console log for each sheet as the script loops through the collection of worksheets.

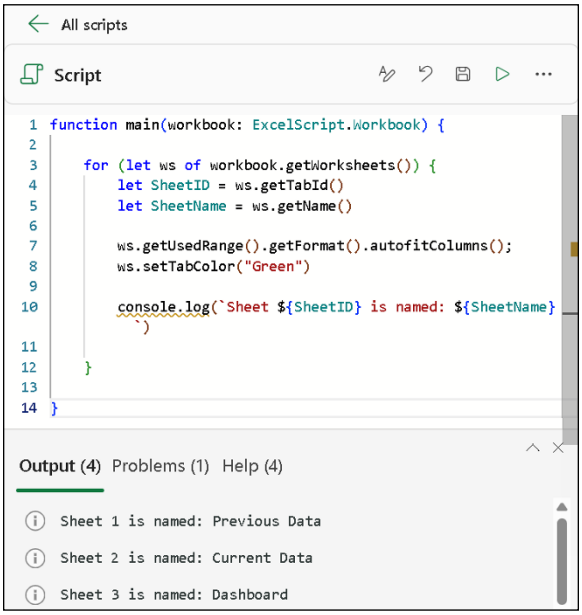


FIGURE 5-12: Use the `for . . . of` construct to loop through a collection of items and objects.

The `for . . . of` loop is one of the more useful looping constructs when working with all kinds of Excel objects (such as ranges, charts, and tables). You will be using `for . . . of` loops heavily in Part 2.

Line 10 in Figure 5-12 introduces a new syntactic feature of TypeScript called the template literal. A *template literal* enables the concatenation of text and variables without the messy syntax. For example, the following syntax requires the use of quotes and plus signs to join text and a variable:

```
Console.log('The cell value is ' + MyVariable + '.')
```

Instead, use a template literal syntax to concatenate text and variables:

```
Console.log(`The cell value is ${MyVariable}.`)
```

This approach is cleaner, easier to read, and less error prone.

A template literal is defined by two backticks (` `). These are not single quotes! The backtick is typically located on the same key as the tilde (~), below the Escape (ESC) key and above the Tab key. Between the backticks, enter text and any variables inside their own miniblock formed by `${ }`.

Template literals support multiline outputs. The following console.log statement outputs the template literal string on three lines: one for `Sheet`, one for `Name`, and one for `UsedRange`:

```
    console.log(`Sheet: ${ID}
                Name: ${sheetname}
                UsedRange: ${myRange}
                `);
```

You'll encounter template literals again in the next chapter, when you start working with range objects.

Understanding while loops

The `while` loop allows you to run a block of code repeatedly as long as a specific condition remains true. A `while` loop is *unbounded,* meaning there is no predefined number of times its loop will run. The loop keeps going as long as the specified condition is met.

The structure of a basic `while` loop is shown in the following snippet:

```
function main(workbook: ExcelScript.Workbook) {

while (Condition that needs to be met)
  {
  Code to execute if the condition is met
  }
}
```

You start the `while` statement with the condition that needs to be met. In the code block, you enter the code to execute if the condition is met. On every iteration, the script will continue to check the specified condition and will stop the moment the condition is no longer met.

The `while` loop in Figure 5-13 inspects column A. It checks if the current cell is empty (`=== ""`). If that condition is `true`, it writes `TBD` into the cell and moves to the next row. The moment it reaches a cell that already contains data, the condition becomes `false` and the loop terminates:

```
function main(workbook: ExcelScript.Workbook) {

  let ws = workbook.getActiveWorksheet();
  let row = 0;

  while (ws.getRange(`A${row + 1}`).getValue() === "")
   {
    ws.getRange(`A${row + 1}`).setValue("TBD");
    row++;
   }
}
```

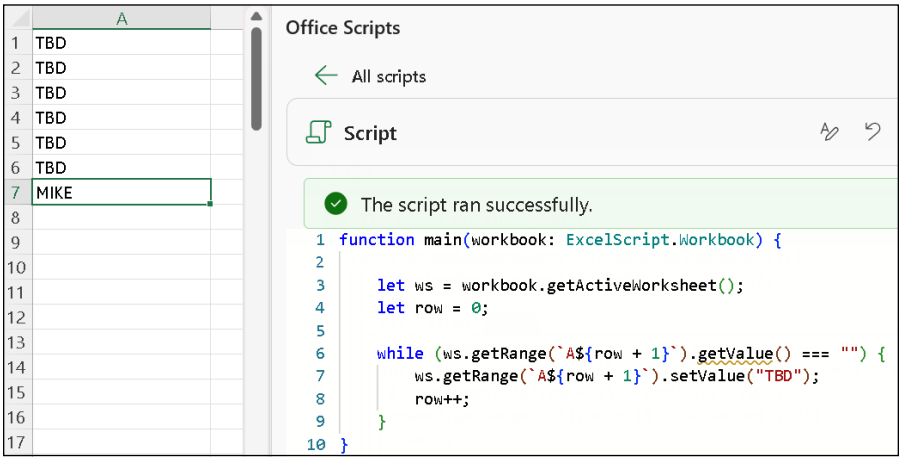


FIGURE 5-13: This `while` loop writes TBD in every cell in column A until it reaches a non-empty cell.

First, you declare a `row` variable and use it in the condition for the `while` statement (line 5). Note the use of the template literal to build a dynamic cell reference (`` `A${row + 1}` ``). This template literal translates to A1 when the loop starts, translates to A2 on the next loop, and so on for each loop.

After the `while` loop begins, each loop checks the condition, executes the code in line 7, and then increments the `row` variable for the next iteration of the loop. The `while` loop runs until it encounters the first non-empty cell, which makes the condition false (see row 7 in the column to the left of the code in Figure 5-13).

A `while` loop is useful when you don't know how many times the code needs to loop. However, because the loop is open-ended, there is the risk of a loop that never stops.

In Figure 5-13, the script seems reasonable because the loop stopped when it encountered a non-blank cell in row 7. But what if there are no empty cells in column A? The script would run over a million times as it iterated through every cell in column A!

In Figure 5-14, you add an `if` statement telling the script to stop the loop once your `row` variable hits 5000 rows. Note the `break` clause in line 11, which immediately exits the loop, helping to protect against runaway loops.

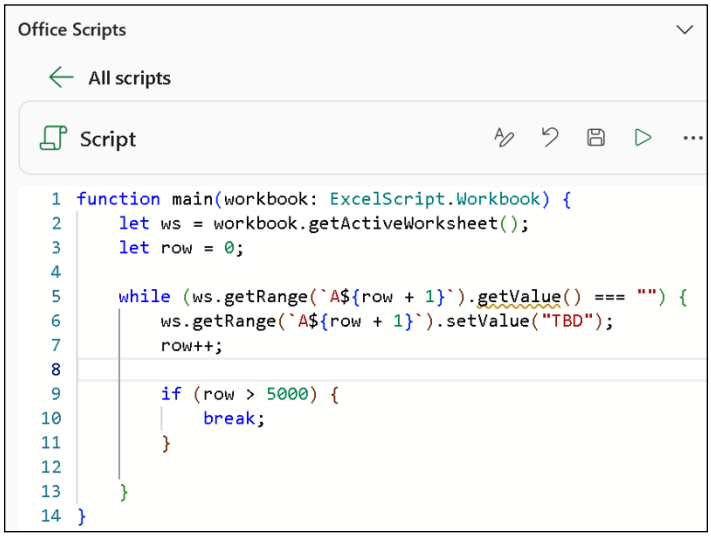


FIGURE 5-14: This `while` loop writes TBD in every cell in column A until it reaches a non-empty cell.

TIP

If your code is in a perpetual loop, you can halt execution and exit the run by clicking the Stop button in the Office Scripts task pane.

3 Scripting Common Excel Objects

IN THIS PART . . .

Automate the creation and management of worksheet objects.

Understand the ins and outs of navigating and manipulating Excel ranges and tables.

Take a deep dive into pivot table automation.

Dynamically create charts, apply conditional formatting, and build visual elements in your reporting.

IN THIS CHAPTER

- Adding and deleting worksheets
- Hiding and unhiding worksheets
- Moving worksheets
- Duplicating worksheets
- Automating worksheet protection
- Extending the worksheets collection

Chapter 6
Starting with Worksheets

Congratulations! You've made it past the core programming fundamentals of Office Scripts. This chapter starts Part 3, where you finally get to work with Excel objects in depth. First up, the `worksheet` object!

I use a "learning by doing" approach, focusing on real-world scenarios you'll come across when working with worksheets. Each section in this chapter focuses on a common scenario and provides a script that automates actions you would normally do by hand. As you work through each script, you build your understanding of the methods and techniques that will enable you to dynamically navigate and manipulate your worksheets.

REMEMBER

The code snippets in this chapter are available in the `Chapter 6 Code` folder, which you can download with the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Working with Worksheets

TypeScript enables you to automate most worksheet tasks — such as creating worksheets, changing the order of worksheets in a workbook, or protecting a worksheet — to save time and gain efficiency. In this section, I cover some of the more useful automations related to worksheets.

Adding a new worksheet

You start with one of the simplest worksheet-related automations you can apply with a script — adding and naming a new worksheet.

In the following script, you use the `addWorksheet` method to simultaneously add and name a new worksheet named `January`:

```
function main(workbook: ExcelScript.Workbook) {
    let newSheet = workbook.addWorksheet('January');
    newSheet.activate();
    }
```

However, there's an issue with this script. If it runs more than once, Excel will throw an error because the sheet name `January` will already exist. When automating the creation of a worksheet, it's best practice to first check to see if the target sheet name already exists in the workbook, as follows:

```
function main(workbook: ExcelScript.Workbook) {

// Step 1: Declare newSheet variable
let newSheet = workbook.getWorksheet('January');

// Step 2: Check if a sheet named January exists
if (newSheet) { newSheet.activate();
                return;
}

//Step 3: Create sheet if needed then activate it
newSheet = workbook.addWorksheet('January');
newSheet.activate();

}
```

In Step 1, you declare a variable called `newSheet` and attempt to fill it with a sheet object called `January`. If there is no sheet named `January`, the `newSheet` variable will be empty.

In Step 2, you check the `newSheet` variable to see if it's empty. The `if` statement, `if (newSheet)`, returns a value of `FALSE` if the `newSheet` variable is empty (meaning there is no sheet named `January`). If a sheet named `January` already exists, the `if` statement returns a value of `TRUE` and the `return` clause exits the function.

Step 3 runs only if there is not already a sheet named `January`. In this step, you add the new sheet and make it the active sheet.

Adding a new worksheet for each month

You can streamline the script showcased in the last section by leveraging what is called a *short-circuit evaluation.* The term *short circuit* is an analogy to electrical circuits. If an electrical current can complete the circuit early, it doesn't travel the longer path.

The following script creates a short-circuit evaluation by using the `||` (OR) operator to assign a value to the `mySheet` variable:

```
function main(workbook: ExcelScript.Workbook) {

let newSheet = workbook.getWorksheet('January') ||
               workbook.addWorksheet('January')

newSheet.activate()

}
```

If `getWorksheet('January')` finds the sheet, it returns the worksheet object and the `addWorksheet` method is never called. If `getWorksheet('January')` doesn't find the sheet, it returns undefined, so it continues and calls `addWorksheet('January')`.

You can take this core concept to create a function that enables you to create any number of worksheets at once. For example, the script in Figure 6-1 creates a new worksheet for each month. The script starts by creating a `months` array containing the list of the month names you want as worksheets.

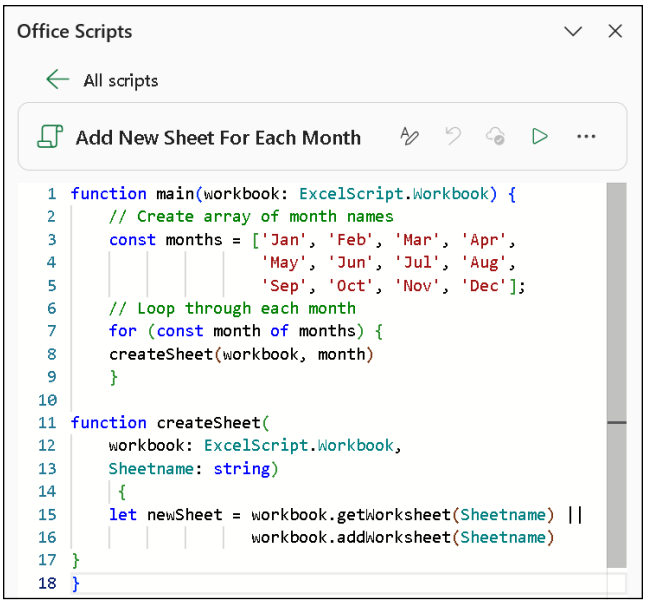


FIGURE 6-1: Script to create a new worksheet for each month in an array.

In line 7 in Figure 6-1, you start a `for` loop that loops through the array of month names, filling a `month` variable with each name one at a time. Then you trigger (call) the `createSheet` function in line 8, passing the `month` name. The `createSheet` function creates a sheet if it doesn't already exist.

Deleting all but a specified sheet

Sometimes you want to take action on all but a specified sheet. In this case, you want to loop through all worksheets and take action only if the sheet is not the specified sheet.

For example, in the following script you delete all sheets except `Sheet1`:

```
function main(workbook: ExcelScript.Workbook){

//Step 1: Check if Sheet1 exists
    let mySheet = workbook.getWorksheet('Sheet1');
    if (!mySheet) {
        return;
    }
```

```
//Step 2: Start looping through all sheets
    let allSheets = workbook.getWorksheets();
    for (const sheet of allSheets) {

//Step 3: Assign each sheet to a variable
        let sheetName = sheet.getName();

//Step 4: Check sheet name and delete if not Sheet1
        if (sheetName !== 'Sheet1') {
            sheet.delete();
            }
    }
}
```

In Step 1, you declare a variable called `mySheet` and assign `Sheet1` to it. Then you check to make sure the variable is not empty with the statement `if (!mySheet)`, which essentially says that if the `mySheet` variable is empty, exit the function using the `return` clause. This step ensures that the function doesn't run if `Sheet1` doesn't exist.

In Step 2, the script assigns the `worksheets` collection to a variable called `allSheets`. You then start looping through each sheet in the `allSheets` collection.

In Step 3, you capture in a variable called `sheetName` the name of the sheet currently being looped.

Step 4 uses an `if` statement to delete the sheet only if the `sheetName` variable is not `Sheet1`. In other words, delete the sheet currently being looped if it has a different name than `Sheet1`.

The takeaway from this example is that you can loop through all the sheets in a workbook, check each sheet for a condition, and then perform some action on that sheet. This kind of automation lends itself nicely to a custom function that can be called from the main procedure. In Figure 6-2, you place the code you just reviewed into the `deleteAllButSheet1` function (see line 5).

REMEMBER

You can trigger (call) a function by entering its name along with any required parameters. In line 2, you call the `deleteAllButSheet1` function (see Figure 6-2). Feel free to revisit the section on adding flexibility with functions in Chapter 3 for a quick refresher on leveraging reusable functions in your scripts.

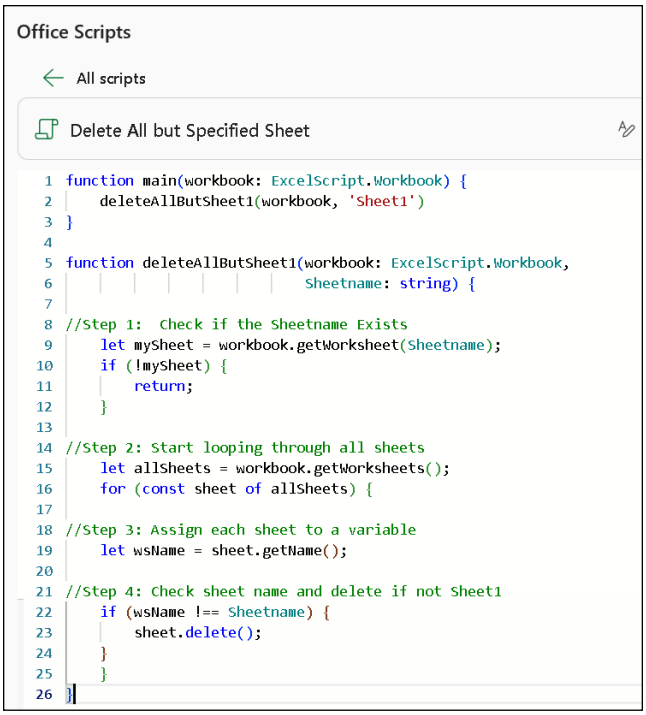


FIGURE 6-2: You can create reusable custom function that can be called from the main procedure.

Unhiding all worksheets in a workbook

Another common scenario is the need to unhide all worksheets in a workbook. The core steps are the same as those in the preceding section: Loop through every worksheet, check a condition (in this case, whether the sheet is hidden), and then perform the appropriate action.

Follow along with this script to understand how you can unhide all worksheets in a workbook:

```
function main(workbook: ExcelScript.Workbook) {
//Step 1: Get all worksheets in the workbook
    let allSheets = workbook.getWorksheets();

//Step 2: Loop through all sheets and get visibility
    for (const sheet of allSheets) {
        let visibility = sheet.getVisibility();
```

```
//Step 3: Check if the sheet is hidden
    if (visibility === ExcelScript.SheetVisibility.hidden
        ||
        visibility === ExcelScript.SheetVisibility.veryHidden)
    {
        // Unhide the sheet
        sheet.setVisibility(
            ExcelScript.SheetVisibility.visible);
    }
}
//Step 4: Activate the first sheet
    allSheets[0].activate();
}
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you start looping through each sheet in the `allSheets` collection, capturing the visibility state for the sheet currently being looped. The visibility state is stored in the `visibility` variable.

Step 3 says that if the `visibility` variable is equal to `hidden` or `veryHidden`, set the visibility for the sheet to `visible`.

Step 4 finishes the script by activating the first sheet in the `allSheets` collection. Remember that TypeScript is a base-zero language, meaning all arrays start at 0. This means the first worksheet in a workbook is in position 0, the second worksheet is in position 1, and so on. So, `allSheets[0].activate` activates the first sheet in the collection.

Just as in the last example, you can use this script as a custom function, which you can call from the main procedure, as shown in Figure 6-3.

You may be wondering what the difference is between hidden sheets and very hidden sheets. Sheets that have a `hidden` state are not visible but can be unhidden by using the standard Unhide dialog box. Sheets in a `veryHidden` state can be made visible only by using a script or VBA procedure.

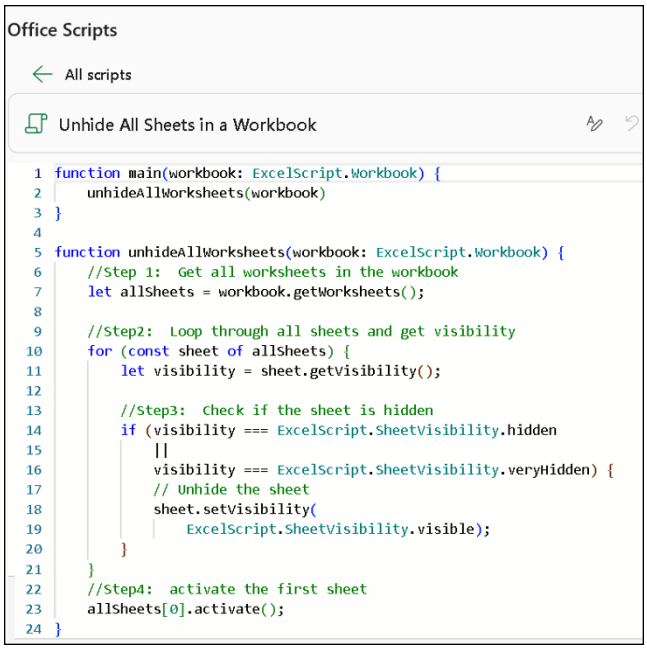


FIGURE 6-3: You can make this code into a reusable custom function named unhideAll Workhseets.

Hiding all but a specified worksheet

Using the techniques covered in the chapter so far, you can create a custom function that hides every worksheet except the one you specify. The idea is simple: Identify a target worksheet you want to keep unhidden, passing its name to a function that hides all other sheets.

The `hideAllbutTargetSheet` function shown next takes two parameters, a `workbook` parameter and a `tgtsheet` parameter:

```
function hideAllbutTargetSheet(
    workbook: ExcelScript.Workbook,
    tgtSheet: ExcelScript.Worksheet)
{

//Step 1: Get all worksheets in the workbook
    let allSheets = workbook.getWorksheets();

//Step 2: Loop through all sheets
    for (const sheet of allSheets)
    {
```

```
    //Step 3: Check if the sheet name is same as target
        if (sheet.getName() !== tgtSheet.getName())
            sheet.setVisibility(
            ExcelScript.SheetVisibility.hidden
            );
        }
    }
}
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you start looping through each sheet in the `allSheets` collection.

Step 3 compares the sheet name for the sheet that is currently being looped to the sheet name of the target sheet. If they are not the same, you set the visibility of the current sheet to `hidden`.

Figure 6-4 illustrates how to use the `hideAllbutTargetSheet` function. In line 3, you define a `tgtSheet` variable (`Sheet1` in this case). Then in line 4, the code uses `if (!tgtSheet) {return;}` to tell Excel to exit the function if the target sheet (`Sheet1` in this case) doesn't exist. Line 5 calls the `hideAllbutTargetSheet` function, passing the required `workbook` and `tgtSheet` parameters.

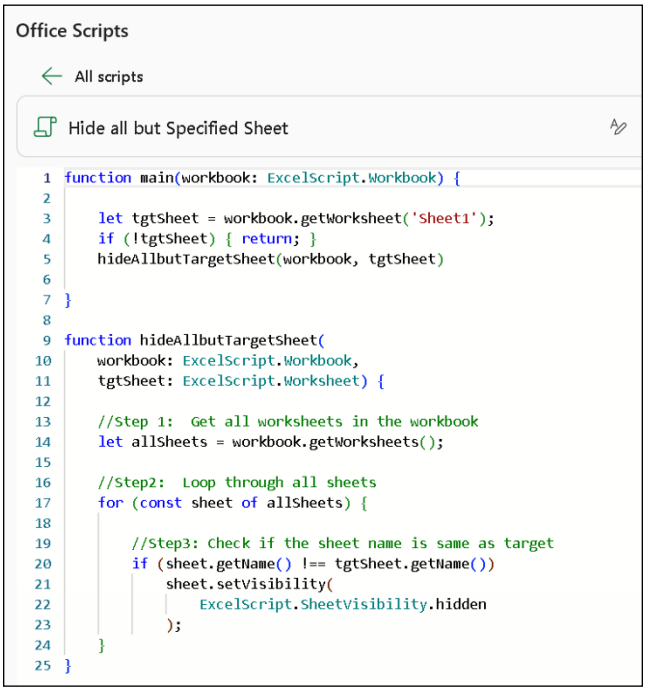


FIGURE 6-4: Using the `hideAllbut TargetSheet` function.

Because the `hideAllbutTargetSheet` function accepts a `tgtsheet` parameter, it becomes a versatile function that you can use in any number of automation projects. Simply pass a new target sheet as a parameter and the function will hide all but the specified sheet.

Moving Worksheets Around

We've all had to manually rearrange our workbook so that some sheets came before or after other sheets. In this section, you discover some of the scripting techniques for programmatically sorting and arranging your worksheets.

Moving a sheet to the first position

Every worksheet has a number indicating its position in relation to the other worksheets in the workbook. Because TypeScript is a base-zero programming language, the first worksheet in a workbook is always in position 0. The second worksheet is in position 1, the third worksheet is in position 2, and so on.

When you want to rearrange sheets, you use the `setPosition` method to specify a new position number. For example, the following snippet sets `Sheet5` to position 0, making it the first worksheet in the workbook:

```
function main(workbook: ExcelScript.Workbook) {

   let firstPos = 0
   workbook.getWorksheet('Sheet5').setPosition(firstPos)

}
```

Moving a sheet to the last position

To set a worksheet to the last position of the workbook, you first need to know the number of worksheets in the workbook. You use the `length` property of a collection to get the total number of objects in the collection.

The following snippet demonstrates the `length` property in action. Running this script in a workbook containing five sheets will write the number 5 in the console log:

```
function main(workbook: ExcelScript.Workbook) {

    let numberofsheets = workbook.getWorksheets().length
    console.log(numberofsheets)

}
```

Keep in mind that the number of worksheets is not the same as the position number. If a workbook has five sheets, the fifth sheet is in position 4 (due to the base-zero nature of TypeScript). So you must subtract 1 from the `length` property of the `worksheets` collection.

The following snippet makes `Sheet1` the last worksheet in the workbook. Note how you simply subtract 1 from `getWorksheets().length` to get the number that represents the last position:

```
function main(workbook: ExcelScript.Workbook) {

let lastPos = workbook.getWorksheets().length-1
workbook.getWorksheet('Sheet1').setPosition(lastPos)

}
```

Moving a specific sheet before another sheet

To move a specific sheet in relation to another sheet, you simply need to capture each of their position numbers and then define new position numbers depending on your needs.

The following code illustrates a custom function called `moveWorksheetBefore`, which takes the `sheetToMove` and `targetSheet` sheets as parameters:

```
    function moveWorksheetBefore(
        workbook: ExcelScript.Workbook,
        sheetToMove: string,
        targetSheet: string) {

// Step 1: Confirm sheetToMove
        let sheetToMoveObj = workbook.getWorksheet(sheetToMove);
        if (!sheetToMoveObj) { return; }
```

```
// Step 2: Confirm targetSheet
        let targetSheetObj = workbook.getWorksheet(targetSheet);
        if (!targetSheetObj) { return; }

// Step 3: Get position numbers
        let targetPos = targetSheetObj.getPosition();
        let sheetToMovePos = sheetToMoveObj.getPosition();

// Step 4: Set new position for sheetToMoveObj
        if (sheetToMovePos > targetPos) {
            sheetToMoveObj.setPosition(targetPos);
        }
}
```

In Step 1, the script is confirming that the sheet assigned to the `sheetToMove` variable exists (`Sheet3` in this case). If not, the `return` statement ends the code run.

Step 2 does the same for `targetSheet` (`Sheet2` in this case). If the target sheet does not exist, the `return` statement ends the code run.

In Step 3, you capture the position numbers for both sheets.

Step 4 compares the two position numbers. If the position of `sheetToMoveObj` is greater than the position of the target sheet, you set the position number to the target sheet's position. This action forces the target sheet to the next position number, placing it after the sheet being moved.

Figure 6-5 demonstrates how to use the `moveWorksheetBefore` function. In line 3, `Sheet3` is being passed as the `sheetToMove` parameter, and `Sheet2` is being passed as the `targetSheet` parameter.

Duplicating worksheets

Sometimes your script must duplicate (copy) a worksheet while processing an action. When you duplicate a worksheet in Office Scripts, Excel positions and renames the new worksheet automatically.

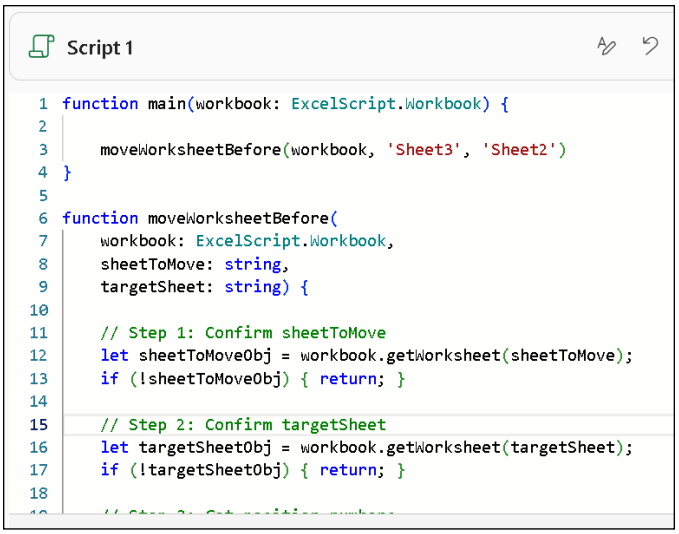


FIGURE 6-5: Using the `moveWorksheetBefore` function.

If you were to run the following basic script, Excel would create a copy of `Sheet1`, name the new worksheet `Sheet1 (2)`, and automatically position the newly created worksheet as the first sheet in the workbook:

```
function main(workbook: ExcelScript.Workbook) {

    let sourceSheet = workbook.getWorksheet('Sheet1');
    sourceSheet.copy();

}
```

You can enhance this action by specifying a name for the newly created worksheet and by leveraging the positioning options available through the `copy` method. Excel provides several positioning options when adding or copying worksheets:

- **`WorksheetPositionType.beginning`** places the sheet at the start (leftmost position).
- **`WorksheetPositionType.end`** places the sheet at the end (rightmost position).
- **`WorksheetPositionType.before`** places the sheet before a specified worksheet.
- **`WorksheetPositionType.after`** places the sheet after a specified worksheet.

The following script performs the same copy action just shown, but this time you specify a name and position for the new worksheet. You use the `WorksheetPositionType.end` option to ensure that the newly created sheet is positioned as the last worksheet:

```
function main(workbook: ExcelScript.Workbook) {

// Get the worksheet to duplicate
  let sourceSheet = workbook.getWorksheet('Sheet1');

// Copy it to the end of the workbook and rename it
  sourceSheet
  .copy(ExcelScript.WorksheetPositionType.end)
  .setName('New Copied Sheet');

}
```

The `WorksheetPositionType.before` and `WorksheetPositionType.after` options offer a bit more flexibility by allowing you to position the copied worksheet before a specific sheet or after a specific sheet, respectively. For instance, the following script copies `sourceSheet` (`Sheet1`) and places the copy before `targetSheet` (`Sheet5`):

```
function main(workbook: ExcelScript.Workbook) {
let sourceSheet = workbook.getWorksheet('Sheet1');
let targetSheet = workbook.getWorksheet('Sheet5');

// Copy sourceSheet and place before targetSheet
 sourceSheet
 .copy(ExcelScript.WorksheetPositionType.before,
        targetSheet)
 .setName('New Sheet');
}
```

Understanding Worksheet Protection

Each worksheet has a `getProtection()` method that returns the `WorksheetProtection` object, which defines the protection state of a given sheet (whether or not it's protected). You can check whether a sheet is protected by testing for the existence of the `WorksheetProtection` object.

Checking if a worksheet is protected

The following code snippet initiates a variable named `protection` and then attempts to fill it with the `WorksheetProtection` object for `Sheet1`. You can then use an `if` statement to test if the `protection` object exists:

```
function main(workbook: ExcelScript.Workbook) {

const mySheet = workbook.getWorksheet('Sheet1')
const protection = mySheet.getProtection();

    if (!protection) {
        console.log ('Sheet is not protected')
    }
}
```

Protecting a worksheet

Once you determine that a sheet is not protected, you can use the `protect` method and its options to protect the sheet. Protection options are defined by `true` or `false`. For example, if you set the `allowFormatCells` option to `true`, users can format cells although the worksheet is protected.

The following script protects `Sheet1` with the password `RED123`:

```
function main(workbook: ExcelScript.Workbook) {

const mySheet = workbook.getWorksheet('Sheet1')
const protection = mySheet.getProtection();
const password = 'RED123'

    if (!protection.getProtected()) {
          protection.protect(
                {
                    allowFormatCells: false,
                    allowFormatColumns: false,
                    allowFormatRows: false,
                    allowInsertColumns: false,
                    allowInsertRows: false,
                    allowInsertHyperlinks: false,
                    allowDeleteColumns: false,
                    allowDeleteRows: false,
```

```
                    allowSort: false,
                    allowAutoFilter: false,
                    allowPivotTables: false
                }
                , password);
    }
}
```

Note the `password` argument applied after the protection options. If a password has been supplied, users will need the password to unprotect the sheet. The `password` argument is optional. If you omit it, the sheet is still protected, but you won't need to enter a password to unprotect it.

Unprotecting a worksheet

You use the `unprotect` method to remove worksheet protection. In the following script, note that you use `try . . . catch` error handling to avoid any errors due to a bad password or an invalid sheet:

```
function main(workbook: ExcelScript.Workbook) {

    const mySheet = workbook.getWorksheet('Sheet2')
    const protection = mySheet.getProtection();
    const password = 'RED1d23'

    try {protection.unprotect(password);}
    catch (error)
        {console.log('unprotect failed')}
}
```

REMEMBER

If you need a refresher on the `try . . . catch` error-handling method, revisit Chapter 4.

Protecting all worksheets in a workbook

Before you distribute your workbook, you may want to apply sheet protection to all its worksheets. However, as you can see in Figure 6-6, Excel disables the Protect Sheet command if you try to protect multiple sheets at one time. You are forced to protect one sheet at a time.

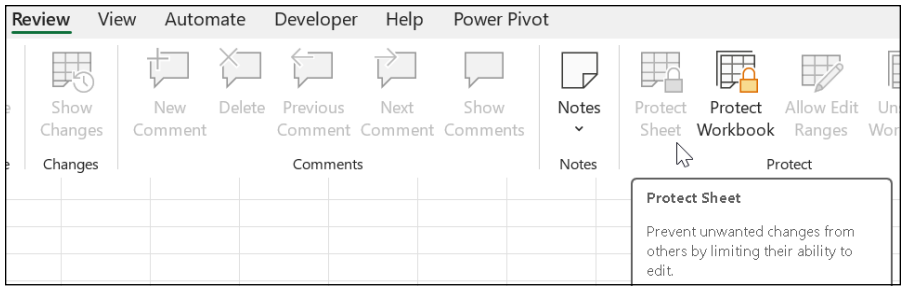


FIGURE 6-6: The Protect Sheet command is disabled if you try to protect more than one sheet at a time.

You can use the following custom function to save you from protecting each sheet manually. This function accepts a `password` parameter defining the password needed to remove the protection. The `password` parameter is optional. If you omit it, the sheet will still be protected but you won't need to enter a password to unprotect it.

```
function protectAllWorksheets(
    workbook: ExcelScript.Workbook,
    password = null as string)
{

// Step 1: Get all worksheets in the workbook
    let allSheets = workbook.getWorksheets();

// Step 2: Start looping through worksheets
    for (const sheet of allSheets)
    {

// Step 3: Get protection object for each sheet
    let protection = sheet.getProtection();

// Step 4: Protect if not already protected
        if (!protection.getProtected()) {
            protection.protect(
                {
                allowFormatCells: false,
                allowFormatColumns: false,
                allowFormatRows: false,
                allowInsertColumns: false,
                allowInsertRows: false,
                allowInsertHyperlinks: false,
                allowDeleteColumns: false,
                allowDeleteRows: false,
```

```
                allowSort: false,
                allowAutoFilter: false,
                allowPivotTables: false
                }
            , password);
        }
    }
}
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you start looping through each sheet in the `allSheets` collection.

Step 3 gets the protection object for the sheet currently being looped.

In Step 4, the code uses an `if` statement to check that the sheet is not currently protected. If the worksheet is not already protected, it will be protected using the protection options listed. Note the `password` parameter applied after the protection options. If a password has been supplied, users will need the password to unprotect each sheet.

In Figure 6-7, the `protectAllWorksheets` custom function is used to protect all sheets with the password `RED123`.

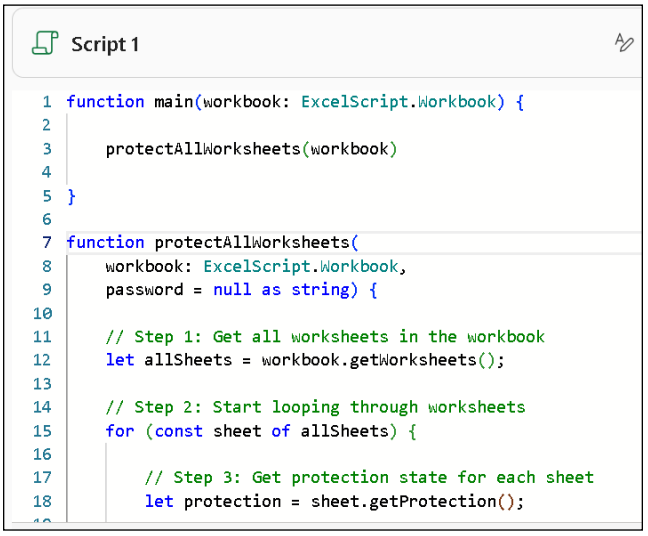


FIGURE 6-7: Calling the `protectAllWorksheets` to protect all sheets with the password `RED123`.

Unprotecting all worksheets in a workbook

You can use the following custom function to dynamically unprotect all worksheets at once. This function accepts a `password` parameter defining the password needed to remove protection:

```
function unprotectAllWorksheets(
    workbook: ExcelScript.Workbook,
    password = null as string)
    {

// Step 1: Get all worksheets in the workbook
    let allSheets = workbook.getWorksheets();

// Step 2: Start looping through worksheets
    for (const sheet of allSheets)
    {

// Step 3: Get protection object for each sheet
    let protection = sheet.getProtection();

// Step 4: Unprotect the sheet
    try {protection.unprotect(password);}
    catch (error) {}
    }
}
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you start looping through each sheet in the `allSheets` collection.

Step 3 gets the protection object for the sheet currently being looped.

In Step 4, you unprotect the sheet using the provided password. Attempting to unprotect a sheet with the wrong password will result in an error. Therefore, you perform the `unprotect` action inside a `try . . . catch` error handler. This error handling allows the code to gracefully bypass any errors due to a bad password.

In Figure 6-8, you call the `unprotectAllWorksheets` custom function with the password `red123`.

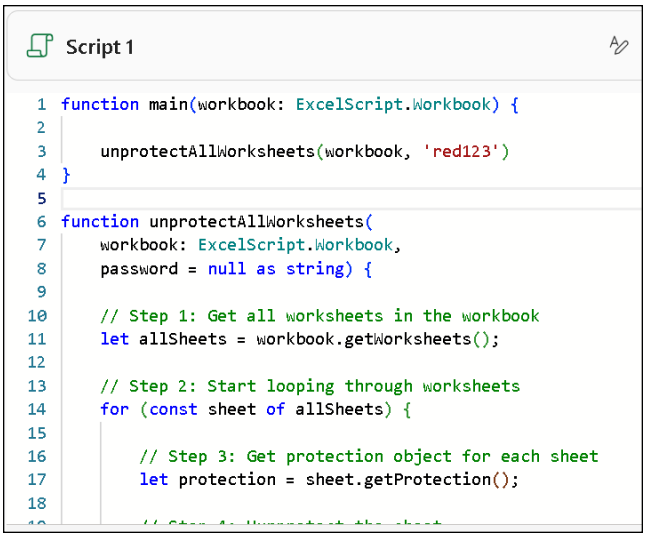


FIGURE 6-8: Calling the `unprotectAllWorksheets` function from the main procedure.

Extending the Worksheets Collection

At this point, you've seen several examples of looping through the `worksheets` collection to take actions on each sheet in your workbook. You can extend the utility of the `worksheets` collection by leveraging a TypeScript method called the `map()` method.

The `map()` method allows you to alter the elements in a given collection, outputting the transformed values into a new array that can be used to perform additional processing. In the context of worksheets, the `map()` method lets you extract properties of each worksheet into a separate array that you can then use in some clever ways.

A practical use case for this concept is sorting worksheet names:

```
function main(workbook: ExcelScript.Workbook)
    {

// Step 1: Capture the worksheets collection
    let allSheets = workbook.getWorksheets();

// Step 2: Create a new array of sheet names
    let sheetNames = allSheets.map(mySheet =>
        mySheet.getName());
```

```
//Step 3: Sort the array of sheet names alphabetically
    sheetNames.sort((a, b) => {
        return a.localeCompare(b)
     });

//Step 4: Output the sorted list of names
    sheetNames.forEach((name) => {
        console.log (name)
     });
    }
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you use the `map()` method to iterate through all the worksheets in the `allSheets` collection, extracting the name property of each sheet. The resulting output of the `map()` method is a new array of names stored in the `sheetNames` variable.

Step 3 sorts the values in the `sheetNames` array by using TypeScript's `localeCompare` method, which iterates through the values in `sheetNames`, comparing each value with the next one. It then assigns a sorting index value based on whether each value comes before or after the next value alphabetically. The `sheetNames` array is sorted by this index.

In Step 4, you write each name in the `sheetNames` array to the console log.

Figure 6-9 shows the output of this script. Note the original order of the sheet names at the bottom of Figure 6-9 versus the sorted order of the sheet names written to the console log.

Sorting worksheets alphabetically

You can alter Step 4 of the script shown in Figure 6-9 to use the sorted `sheetNames` array to reorder the worksheets in the workbook instead of writing to the console:

```
function main(workbook: ExcelScript.Workbook) {

// Step 1: Capture the worksheets collection
    let allSheets = workbook.getWorksheets();
```

```
// Step 2: Create a new array of sheet names
    let sheetNames = allSheets.map(mySheet =>
  mySheet.getName());

//Step 3: Sort the array of sheet names alphabetically
    sheetNames.sort((a, b) => {
        return a.localeCompare(b)
    });

//Step 4: Reorder the worksheets
    sheetNames.forEach((name, index) => {
        let sheet = workbook.getWorksheet(name);
        sheet.setPosition(index);
    });
}
```

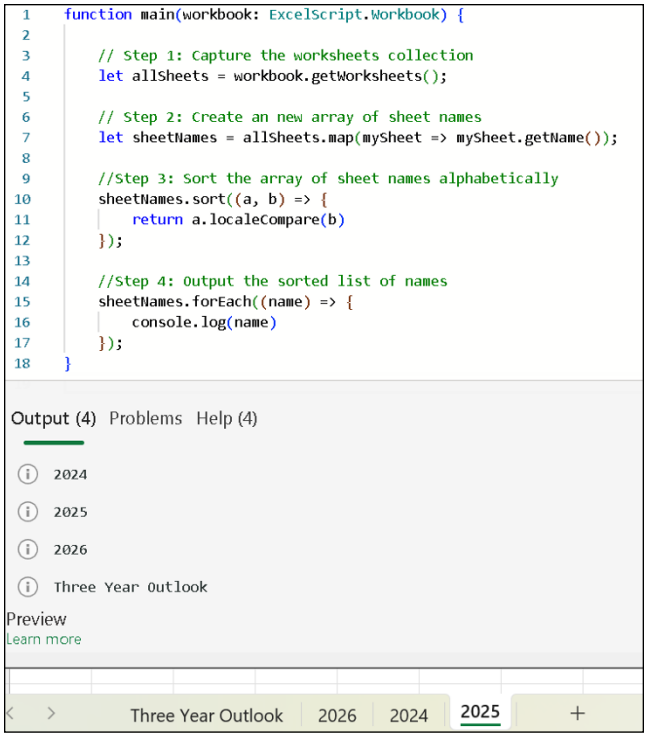
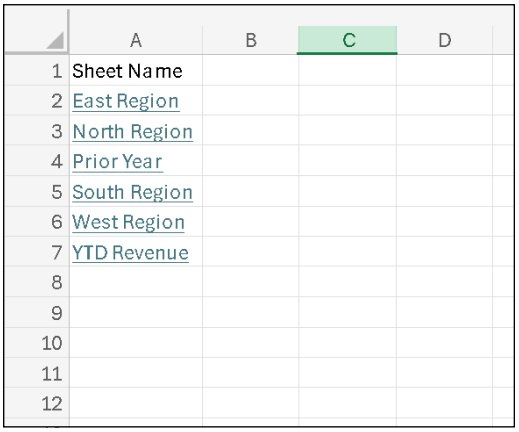


FIGURE 6-9: Using the `map( )` method to enable sorting of sheet names.

Step 4 now uses the index of each element in the `sheetNames` array to set the position number for each respective sheet. As a result, the worksheets are sorted alphabetically.

Creating a table of contents for your workbook

Outside of sorting worksheets, creating a table of contents for the worksheets in a workbook is the most requested Office Scripts. The reason is probably not lost on you. We often work with files that have more worksheet tabs than can easily be seen or navigated. A table of contents like the one in Figure 6-10 helps by providing an alphabetized list of worksheets.



	A	B	C	D
1	Sheet Name			
2	East Region			
3	North Region			
4	Prior Year			
5	South Region			
6	West Region			
7	YTD Revenue			
8				
9				
10				
11				
12				

FIGURE 6-10: A table of contents can help you more easily navigate your workbook.

The following script not only creates a list of worksheet names in the workbook but also adds hyperlinks so that you can easily jump to a sheet with a simple click:

```
function main(workbook: ExcelScript.Workbook) {

// Step 1: Capture the worksheets collection
    let allSheets = workbook.getWorksheets();

// Step 2: Create an array of sheet names
    let sheetNames = allSheets.map(mySheet =>
  mySheet.getName());

//Step 3: Sort the array of sheet names alphabetically
    sheetNames.sort((a, b) => {
        return a.localeCompare(b)
    });
```

```
//Step 4: Create the TOC sheet if it does not exist
    let TOCSheet = workbook.getWorksheet('TOC');
    if (!TOCSheet) {
        TOCSheet = workbook.addWorksheet('TOC');
        TOCSheet.setPosition(0);
    }

//Step 5: Reset TOC sheet
    TOCSheet.getUsedRange()?.clear();
    TOCSheet.getRange('A1').setValue('Sheet Name');


//Step 6: Write sheet names with hyperlinks
    sheetNames.forEach((name, index) => {
    if (name !== 'TOC') {
        let cell = TOCSheet.getRange(`A${index + 2}`);
        cell.setValue(name);
        cell.setHyperlink({
            textToDisplay: name,
            documentReference: `'${name}'!A1`
        });

//Step 7: Do final formatting
        cell.getFormat()
        .getFont()
          .setUnderline(ExcelScript.RangeUnderlineStyle.single);
        TOCSheet.getRange("A:A").getFormat()
        .autofitColumns();
        TOCSheet.activate()
        }
    });
}
```

In Step 1, the script assigns the `worksheets` collection to the `allSheets` variable.

In Step 2, you use the `map()` method to iterate through all the worksheets in the `allSheets` collection, extracting the name property of each sheet. The resulting output of the `map()` method is a new array of names stored in the `sheetNames` variable.

Step 3 sorts the values in the `sheetNames` array by using TypeScript's `localeCompare` method, which iterates through all the values in `sheetNames`, comparing each value with the next one. It then assigns a sorting index value based on whether each value comes before or after the next value alphabetically. The `sheetNames` array is sorted by this index.

In Step 4, you create a worksheet named `TOC` to hold the table of contents. Note the use of `if (!TOCSheet)`. This `if` statement ensures that the new TOC worksheet is added only if it doesn't already exist.

Step 5 clears any data from the TOC sheet and adds a header called `Sheet Names` to cell A1.

In Step 6 you write the sheet names to the new `TOC` worksheet, using a `for . . . each` loop to iterate through the items in the `sheetNames` array. Each item in the array has a `name` and an `index` property. As the `for . . . each` loop iterates through each item, `index` determines which cell in the `TOC` sheet to enter and the `name` property contains the sheet name to enter.

Step 7 performs some final formatting.

REMEMBER

Note in Step 6 that you use `` `'${name}'!A1` ``. This syntactic feature, called a template literal, lets you build a string by concatenating text and variables without the messy syntax you would normally need. In this case, `` `'${name}'!A1` `` takes the name of the sheet stored in the `name` property and joins it with `!A1`, creating a string representing a cell address (for example, `Sheet1!A1`). For more information on how template literals work, see Chapter 5.

IN THIS CHAPTER

- Navigating cell ranges
- Leveraging range metadata
- Using methods that return ranges
- Copying data between ranges
- Clearing and deleting ranges
- Working with tables

Chapter 7
Working with Ranges and Tables

One of the most important things you do in Excel is navigate the worksheet. When you work with Excel manually, you're constantly navigating to ranges, finding the last row, moving to the last column, hiding and unhiding ranges, and so on. These actions all come instinctively as part of doing work in Excel.

When you begin to automate your work, you'll find that navigating your spreadsheet remains an important part of the automation process. In many cases, you'll need to dynamically navigate and manipulate Excel ranges, just as you would manually. This chapter provides some of the most useful scripting techniques for navigating and working with ranges and tables.

REMEMBER

The code snippets used in this chapter are available in the `Chapter 7 Code` folder, which you can download with the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Navigating Ranges

When working with data in any capacity, you'll undoubtedly need to navigate the cells in your worksheet. One of the core actions you'll consistently do with Office Scripts is to reference specific ranges to apply your automation on them.

You use the `getRange` method of the `worksheet` object to initialize a `range` object for the range of cells you're referencing. When using the `getRange` method, you need to specify the `worksheet` object first so Excel knows what worksheet you're working with. The following script selects range A3 of Sheet1:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')

mySheet.getRange('A3').select()

}
```

The following script demonstrates that you can use any standard Excel reference:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')

//Selects a single cell
mySheet.getRange('A3').select()

//Selects a range of cells
mySheet.getRange('E2:F5').select()

//Selects entire column C
mySheet.getRange('C:C').select()

//Selects entire row 7
mySheet.getRange('7:7').select()

}
```

Note in the last code example that the `select` method is used to select each range. In practice, you don't have to explicitly select a range to work with it. For example, in the next script, you initialize a `range` object for cell B2, and then fill it with `Merry Christmas` and color it yellow — all without selecting the range:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')
let myRange = mySheet.getRange('B2')

myRange.setValue('Merry Christmas');
myRange.getFormat().getFill().setColor('#ffff00');
}
```

After a `range` object is initialized, you can use IntelliSense to see all the methods and properties available, as shown in Figure 7-1. To activate IntelliSense, simply enter the dot operator after the range object (such as `myRange.`). You can then scroll through the methods to find the one that best suits your needs.

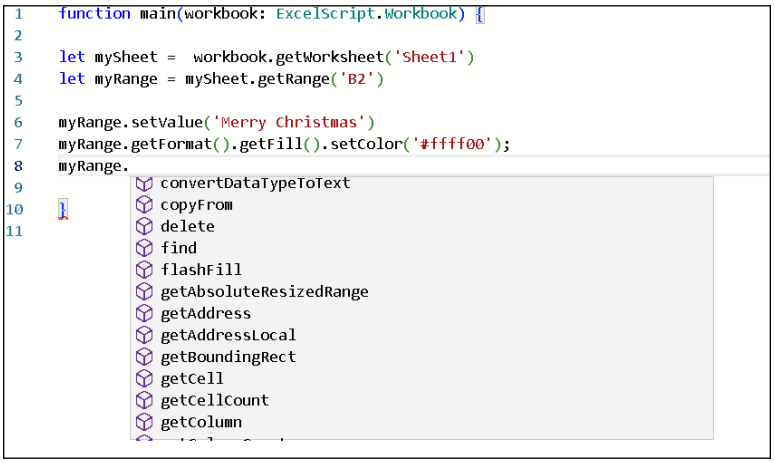


FIGURE 7-1: Use IntelliSense to explore the methods and properties of your `range` objects.

TIP

You don't need to memorize all the methods and properties of the `range` object. You can record a script while you take the actions needed on the worksheet and then examine the code Excel has written. After you've seen the correct syntax, you can apply it as needed. Even seasoned coders use Action Recorder to get coding help. See Chapter 2 for details on Action Recorder.

Using methods that return metadata

Some methods of the `range` object return metadata that exposes valuable information that you can use to extend the functionality of your scripts. Table 7-1 lists some commonly used methods that return metadata.

TABLE 7-1 Methods that Return Range Metadata

Method	Example	Resulting Output
`getAddress()`	`getRange('A1:F5'). getAddress()`	**Sheet1!A1:F5:** The cell address for the range.
`getCellCount()`	`getRange('A1:B10'). getCellCount()`	**20:** The total number of cells in the range A1:B10.
`getRowCount()`	`getRange('A1:F5'). getRowCount()`	**5:** The total number of rows in the range A1:F5.
`getRowIndex()`	`getRange('F5'). getRowIndex()`	**4:** The 0-based index of row 5. Indexes in TypeScript always start at 0, so row 1 has an index of 0, row 2 has an index of 1, and so on. Row 5 has an index of 4.
`getColumnIndex()`	`getRange('F5'). getColumnIndex()`	**5:** The 0-based index of column F. Column F has an index of 5.

Figure 7-2 is an example of leveraging the `getRowIndex()` method to enable the building of dynamic cell references. The full script in Figure 7-2 follows:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')
let myRange = mySheet.getRange('A1')
let row = myRange.getRowIndex()+1

while (row<5)
 {let myNewRange = mySheet.getRange('A${row}');
 console.log(myNewRange.getAddress());
 row++;
}
}
```

In this example script, you start by referencing `Sheet1` and then you initialize a `range` object pointing to A1. You then initialize a variable named `row`, which will hold the row index for cell A1. The `row` variable is filled with `myRange.getRowIndex()+1`. (You add 1 because indexes in TypeScript start at 0.) At this point, the `row` variable equals `1`.

Next, you start a `while` loop (see line 7 in Figure 7-2), incrementing the number in the `row` variable and building a new range with each loop. The address of the range after each loop is written to the console. The loop repeats until the `row` variable equals `5`.

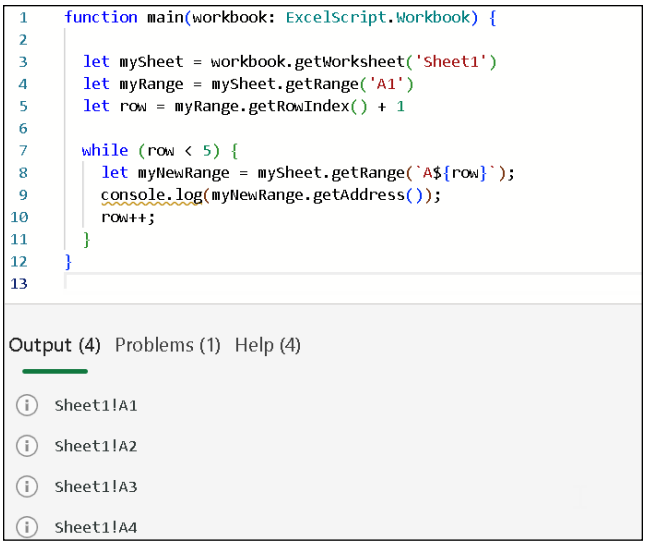


FIGURE 7-2: Using methods like `getRowIndex()` enable more dynamic coding than simply referencing ranges.

This small example demonstrates how using the row index of A1 as a starting point enables flexibility by allowing you to increase the row index to dynamically build new cell references and call out their cell addresses.

Leveraging range shortcut methods

The Excel scripting package offers range methods that return other range objects. These methods are convenient shortcuts that let you reference specific ranges even when you don't know their precise coordinates. Table 7-2 highlights some of the more commonly used shortcut methods.

TABLE 7-2 Methods that Return Other Ranges

Method	Example	Resulting Output
`getActiveCell()`	`workbook.getActiveCell()`	A range object that references the currently active cell
`getSelectedRange()`	`workbook.getSelectedRange()`	A range object that references the selected cell or cells
`getUsedRange()`	`worksheet.getUsedRange()`	A range object that references a contiguous range that encapsulates any cell that contains data or formatting

(continued)

TABLE 7-2 *(continued)*

Method	Example	Resulting Output
`getLastCell()`	`getRange('A1:F25'). getLastCell()`	A range object that references cell F25, the last cell of the last column in the contiguous range A1:F25
`getLastColumn()`	`getRange('A1:F25'). getLastColumn()`	A range object that references cells F1:F25, the range of the last column in the contiguous range A1:F25
`getLastRow()`	`getRange('A1:F25'). getLastRow()`	A range object that references cells A25:F25, the range of the last row in the contiguous range A1:F25
`getColumn()`	`getRange('A1:F25'). getColumn(0)`	A range object that references cells A1:A25, the range of the column in position 0 (the first column) in the contiguous range A1:F25
`getRow()`	`getRange('A1:F25'). getRow(0)`	A range object that references cells A1:F1, the range of the row in position 0 (the first row) in the contiguous range A1:F25
`getOffsetRange()`	`getRange('A1:F1'). getOffsetRange(1,0)`	A range object that references cells A2:F2, the range of the row offset from the original range by a specified number of rows and columns (in this case, one row down from the contiguous range A1:F1)

Most of the methods in Table 7-2 are intuitive, but the `getOffsetRange()` method deserves a bit more explanation.

Understanding the getOffsetRange method

The `getOffsetRange()` method returns a new range offset from the original range by a specified number of rows and columns. The basic syntax requires two arguments: `rowOffset` and `columnOffset`:

```
getOffsetRange(rowOffset, columnOffset)
```

The `rowOffset` argument specifies the number of rows to offset the range. You can use positive numbers to offset the range down rows or negative numbers to offset the range up rows.

The `columnOffset` argument specifies the number of columns to offset the range. You can use positive numbers to offset columns to the right or negative numbers to offset columns to the left.

Figure 7-3 demonstrates a few simple `getOffsetRange()` statements.

```
function main(workbook: ExcelScript.Workbook) {

    let mySheet = workbook.getWorksheet('Sheet1');
    let StartingRange = mySheet.getRange('C3')

    StartingRange.getOffsetRange(0, -2).select() //selects cell A3
    StartingRange.getOffsetRange(-2, 0).select() //selects cell C1
    StartingRange.getOffsetRange(0, 2).select()  //selects cell E5
    StartingRange.getOffsetRange(2, 0).select()  //selects cell C5

}
```

FIGURE 7-3: The `getOffsetRange()` method is useful for navigating around a worksheet programmatically.

REPLICATING CTRL+SHIFT+ARROW IN OFFICE SCRIPTS

When working with Excel manually, you can select a starting point (a cell or range) and then press Ctrl+Shift+arrow key. Excel then extends the selection in the specified arrow direction until it hits a blank cell or the edge of the worksheet. In this way, Excel lets you select a range of data without knowing beforehand the size of the range.

This same functionality is available in Office Scripts via the `getExtendedRange` method. The `getExtendedRange` method replicates what happens when you manually use the Ctrl+Shift+arrow key shortcut in Excel. It selects a contiguous block of cells, starting from the current range and extending to the edge of the data in a specified direction.

The `getExtendedRange` method takes two arguments: `KeyboardDirection` and `activeCell`. For the `KeyboardDirection` argument, you select one of the following enumerations:

```
ExcelScript.KeyboardDirection.down

ExcelScript.KeyboardDirection.up
```

(continued)

(continued)

```
ExcelScript.KeyboardDirection.left

ExcelScript.KeyboardDirection.right
```

For the `activeCell` argument, you provide a range representing the active range (the starting point) from which you want to extend. As an example, imagine your worksheet contains data in a single column from cell A1 to A50. The following script returns the range A1:A50:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getActiveWorksheet();
// Start on cell A1
let startCell = mySheet.getRange('A1');

// Extend the range down
let extendedRange = startCell.getExtendedRange(
ExcelScript.KeyboardDirection.down, startCell);

console.log(extendedRange.getAddress());

}
```

As you continue to build scripts, you'll find that the `getExtendedRange` method is a practical way to dynamically select a column or row of data without knowing its length.

Dynamically adding a total column

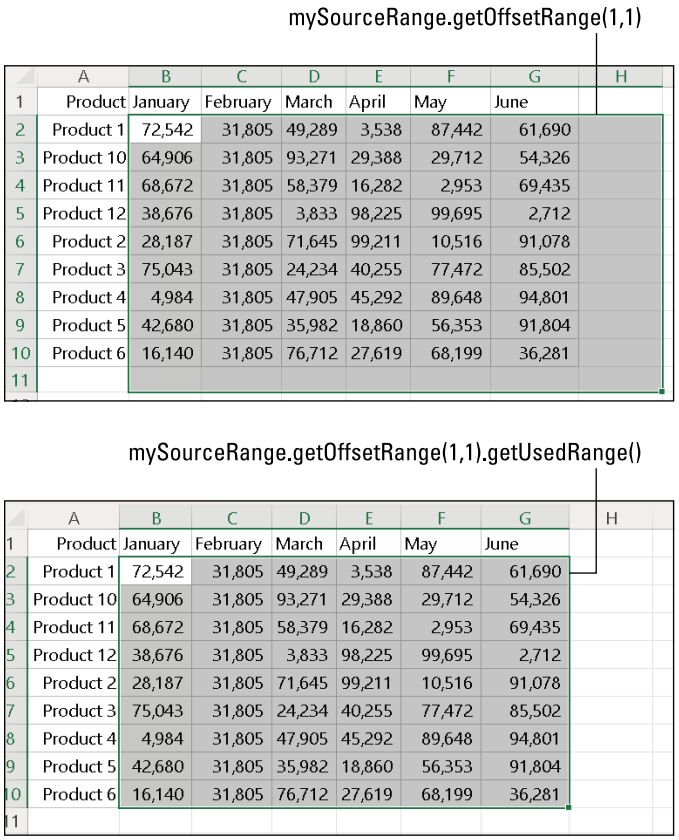
Let's take a moment to see some of these range shortcut methods in action. A common requirement is to dynamically calculate totals in a data table. The following script formats a data table and continues to work properly even if the number of rows and columns changes:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Define the source range and header row
 let mySheet = workbook.getWorksheet('DataSheet');
 let mySourceRange = mySheet.getUsedRange();
 let myHeaderRow = mySourceRange.getRow(0);

//Step 2: Get the range containing data
 let myDataRange = mySourceRange
```

```
          .getOffsetRange(1,1)
          .getUsedRange();

//Step 3: Define a column to enter totals
 let myTotalColumn = myDataRange
           .getLastColumn()
           .getOffsetRange(0,1);

//Step 4: Format the header row
 myHeaderRow.getFormat().getFill().setColor('black');
 myHeaderRow.getFormat().getFont().setColor('white');
 myHeaderRow.getFormat().getFont().setBold(true);

//Step 5: Enter a formula in the first row of the total column
 myTotalColumn.getRow(0).setFormula(
  '=SUM(' +
       myDataRange
       .getRow(0)
       .getAddress()
      +
    ')'
  );
 myTotalColumn.getRow(0).setNumberFormat('#,##0');

//Step 6: Autofill a formula to all rows in the total column
 myTotalColumn
 .getRow(0)
 .autoFill(myTotalColumn,
  ExcelScript.AutoFillType.fillDefault);

}
```

In Step 1, you define a variable named `mySourceRange`, which contains the used range (the range encompassing all cells containing data) in the DataSheet tab of the workbook. The `myHeaderRow` variable is also defined, pointing to the first row of `mySourceRange`.

Step 2 initiates a variable named `myDataRange`, which holds the range for just the data portion of the data table. Take a look at Figure 7-4 to understand what the code in Step 2 is doing. First, you offset `mySourceRange` by one column and one row. Then you get the used range of that offset range to land on the cells you need. This technique of chaining range methods to pinpoint exact ranges is incredibly useful for working with different parts of a dataset.



	A	B	C	D	E	F	G	H
1	Product	January	February	March	April	May	June	
2	Product 1	72,542	31,805	49,289	3,538	87,442	61,690	
3	Product 10	64,906	31,805	93,271	29,388	29,712	54,326	
4	Product 11	68,672	31,805	58,379	16,282	2,953	69,435	
5	Product 12	38,676	31,805	3,833	98,225	99,695	2,712	
6	Product 2	28,187	31,805	71,645	99,211	10,516	91,078	
7	Product 3	75,043	31,805	24,234	40,255	77,472	85,502	
8	Product 4	4,984	31,805	47,905	45,292	89,648	94,801	
9	Product 5	42,680	31,805	35,982	18,860	56,353	91,804	
10	Product 6	16,140	31,805	76,712	27,619	68,199	36,281	
11								

FIGURE 7-4: Chaining range methods is a useful technique for referencing specific parts of a dataset.

In Step 3, you create a range named `myTotalColumn` to house totals for each product. First, you use `myDataRange.getLastColumn()` to get the last column of `myDataRange` (G2:G10 in this case). Then you use `.getOffsetRange(0,1)` to offset that by one column to the right (H2:H10).

At this point, all the necessary ranges are in their own variables. You can start manipulating the worksheet with Step 4, where you format `myHeaderRow`.

Step 5 enters a new formula on the first row of `myTotalColumn` (this happens to be cell H2). The formula entered is the `SUM` of the first row in the `myDataRange` range (B2:G2 in this case). Step 5 finally formats the cell as a number.

Step 6 ends the script by autofilling the formula in the first row of the `myTotalColumn` range down to all the cells in the range.

Running the script transforms the data to look like Figure 7-5.

	A	B	C	D	E	F	G	H
1	Product	January	February	March	April	May	June	
2	Product 1	72,542	31,805	49,289	3,538	87,442	61,690	306,305
3	Product 10	64,906	31,805	93,271	29,388	29,712	54,326	303,407
4	Product 11	68,672	31,805	58,379	16,282	2,953	69,435	247,526
5	Product 12	38,676	31,805	3,833	98,225	99,695	2,712	274,947
6	Product 2	28,187	31,805	71,645	99,211	10,516	91,078	332,442
7	Product 3	75,043	31,805	24,234	40,255	77,472	85,502	334,310
8	Product 4	4,984	31,805	47,905	45,292	89,648	94,801	314,435
9	Product 5	42,680	31,805	35,982	18,860	56,353	91,804	277,485
10	Product 6	16,140	31,805	76,712	27,619	68,199	36,281	256,755

FIGURE 7-5: The transformed data after running the script.

If the number of columns or rows in the Datasheet tab changes, the script will run fine because you used shortcut range methods instead of hard-coding cell references.

TIP

Get in the habit of confirming that your range variables reference the correct cells by using the console log in Code Editor. When testing and debugging your code, enter each range name followed by `.getAddress()` in a `console.log` statement, such as `console.log(myDataRange.getAddress())`.

Getting flexible with the getCell method

The `getCell` method returns a single-cell `range` object based on a row index and a column index. It requires two arguments: a row position index and a column position index.

```
getCell(row, column)
```

TypeScript is a base-zero programming language, meaning the first item in any array is always considered to be in position 0. The second item is in position 1, the third item in position 2, and so on. The statement `getCell(0,0)` returns cell A1 — the first row and first column. The statement `getCell(0,2)` returns the first row and third column (cell C1).

Figure 7-6 demonstrates how to reference a range using the `getCell` method. Here, you reference cell C1 and write the address to the console log.

While the `getRange` method feels more intuitive with its A1 notation style (`getRange('A1:F5')` for instance), the `getCell` method offers advantages when your script requires looping through rows or columns or changing focus to different cells during processing. In these scenarios, referencing cells using numeric positions makes it much easier to dynamically switch the focus to various cells without the need to initialize a new variable for each cell your script needs to touch.

```
1  function main(workbook: ExcelScript.Workbook) {
2
3    let mySheet = workbook.getWorksheet('Sheet1')
4    let myRange = mySheet.getCell(0,2)
5
6    console.log(myRange.getAddress());
7
8  }
9
```

Output (1) Problems Help (4)

Sheet1!C1

FIGURE 7-6: Referencing a range using the `getCell` method.

To get a full appreciation of the power of the `getCell` method, let's examine another scenario, where you calculate a running total. Figure 7-7 illustrates a running total in column C based on the values in column B. You want a script that can automatically create the running totals in column C even if you add more months of data in column B.

	A	B	C	D
1		ProductA	Running Total	
2	January	72,542	72,542	
3	February	64,906	137,448	
4	March	68,672	206,120	
5	April	38,676	244,796	
6	May	28,187	272,983	
7	June	75,043	348,026	
8	July	4,984	353,011	
9	August	42,680	395,691	
10	September	16,140	411,831	
11				
12				

FIGURE 7-7: You need a script that will automatically create running totals in column C.

The following script uses a combination of range navigation techniques to add totals in column C:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Define the source range and starting cell
let mySheet = workbook.getWorksheet('DataSheet2');
let myStartCell = mySheet.getRange('B2');
```

```
//Step 2: Get the working range and count of rows
let myWorkingRange = myStartCell.getExtendedRange(
          ExcelScript.KeyboardDirection.down,
          myStartCell)
let rowCount = myWorkingRange.getRowCount()

/*Step 3: Initialize a runningTotal variable
 Initialize the i counter and start the loop*/
  let runningTotal = 0
  for (let i = 0; i < rowCount; i++) {

/*Step 4: Define valueCell as the current cell
 Define totalCell as the cell next to valueCell*/
  let valueCell = myWorkingRange.getCell(i, 0);
  let totalCell = myWorkingRange.getCell(i, 1);

//Step 5: Add the value of valueCell to runningtotal
  runningTotal += valueCell.getValue() as number;

//Step 6: Set the value of totalCell to runningTotal
  totalCell.setValue(runningTotal);
 }

}
```

Step 1 defines the worksheet you're working with and initializes the `myStartCell` variable, which points to cell B2.

In Step 2, you use the `getExtendedRange` method to dynamically get the range of cells containing data. You store that range in the `myWorkingRange` variable and capture the row count (in the `rowCount` variable) for use later in the script.

Step 3 initiates a new `runningTotal` variable, which will hold the running total value. The starting value for `runningTotal` is `0`. Step 3 also starts a `for` loop with the counter variable (`i`). The intent here is to iterate from `0` to the count of rows in `myWorkingRange` (represented by `rowCount`).

Step 4 uses the `i` counter as the row argument in the `getcell()` method, effectively getting the range row associated with each loop. That range is stored in the `valueCell` variable. Step 4 also initializes the `targetCell` variable pointing to the cell one column over (where you want the running total written). On each loop, the `i` counter increments by `1`, effectively iterating through all the cells in `myWorkingRange`.

Step 5 adds the value of the `valueCell` range to `runningTotal`. Note the use of the += compound assignment operator. The += operator performs addition and writes the result back to the existing variable:

```
runningTotal+= valueCell.getValue()
```

Think of this line as shorthand for the following longer statement:

```
runningTotal = runningTotal + valueCell.getValue()
```

Step 6 writes the value in `runningTotal` to the `targetCell` range.

Creating custom range functions

Although Excel offers a wide array of methods to help you navigate your worksheets, some navigation needs aren't supported by the Excel object hierarchy. The most cited example is that Excel doesn't offer a shortcut method to get the first empty row in a worksheet. The good news is that you can fill gaps in functionality by creating custom range functions.

A common task in Excel is finding the first empty row. When appending rows, you don't want to overwrite existing data, so you need to determine the first row that doesn't contain data.

When creating a script, it helps to first think about how you would accomplish the given task manually. For instance, one way to navigate to the first empty row is to take the following steps:

1. Go to the very bottom of the worksheet by selecting cell A1048576.
2. Press Ctrl+Shift+up arrow to jump to the last used cell in column A.
3. Go down one cell below the last used cell and select the first empty cell.

After deciding how to approach the task, create a starter script for testing. The following script serves as your starter script, translating the manual steps into codified navigation steps:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Initialize variables
let mySheet = workbook.getWorksheet('DataSheet')
let myRange = mySheet.getRange('A1')
let lastWsCell = 1048575
```

```
let firstColIndex = myRange.getColumnIndex()

//Step 2: Get the row index for the last used row
let lastUsedRow = mySheet
.getCell(lastWsCell, firstColIndex)
.getExtendedRange(ExcelScript.KeyboardDirection.up)
.getRowIndex()

//Step 3: Offset 1 row down and get the address
console.log (
 mySheet
.getCell(lastUsedRow + 1, firstColIndex)
.getAddress()
)

}
```

Step 1 initializes the starting variables. The main goal in this step is to establish the two indexes you'll use in the `.getcell` method in Step 2. The first of these is `lastWsCell`, which contains the number `1048575` (representing the last zero-based positional row in the worksheet). The second is `firstColIndex`, which contains the index number for the first column in `myRange`.

Step 2 navigates to `A1048576` with the `getCell(lastWsCell, firstColIndex)` statement, and then uses the `getExtendedRange()` method to replicate pressing Ctrl+Shift+up arrow. The `lastUsedRow` variable stores the row index of the resulting cell.

Finally, Step 3 uses `getCell()` to point to the first empty cell. Note how the `row` argument is `lastUsedRow + 1`. You add `1` to increment the row index to the next row down (the empty cell). The final address is output to `console.log`.

Once you're satisfied with your starter script, you can convert it to a custom function by changing the function name:

```
function getFirstEmptyRow(
workbook: ExcelScript.Workbook,
mySheet: ExcelScript.Worksheet,
myRange: ExcelScript.Range)
{
 let lastWorksheetCell = 1048575
 let firstColumnIndex = myRange.getColumnIndex()
 let lastUsedRow = mySheet
```

```
  .getCell(lastWorksheetCell, firstColumnIndex)
  .getExtendedRange(ExcelScript.KeyboardDirection.up)
  .getRowIndex()
return mySheet.getCell(
lastUsedRow + 1, firstColumnIndex)
}
```

You changed the function name to `getFirstEmptyRow`. You also moved the `mySheet` and `myRange` variables to the function signature (inside the parentheses after the function name). Any variables inside the function signature become input parameters for the function.

Finally, you replace the `console.log` statement with the `return` statement:

```
return mySheet.getCell(lastUsedRow + 1, firstColumnIndex)
```

Figure 7-8 demonstrates how to use the newly created custom range function.

```
1   function main(workbook: ExcelScript.Workbook) {
2     let mySheet = workbook.getWorksheet('DataSheet')
3     let myRange = mySheet.getRange('A1')
4     let FirstEmptyRow = getFirstEmptyRow(workbook, mySheet, myRange)
5
6     console.log('First empty row is ' + FirstEmptyRow.getAddress());
7   }
8
9   function getFirstEmptyRow(workbook: ExcelScript.Workbook,
10                            mySheet: ExcelScript.Worksheet,
11                            myRange: ExcelScript.Range) {
12    let lastWorksheetCell = 1048575
13    let firstColumnIndex = myRange.getColumnIndex()
14    let lastUsedRow = mySheet
15      .getCell(lastWorksheetCell, firstColumnIndex)
16      .getExtendedRange(ExcelScript.KeyboardDirection.up)
17      .getRowIndex()
18    return mySheet.getCell(lastUsedRow + 1, firstColumnIndex)
19  }
20

Output (1)  Problems  Help (4)

ⓘ  First empty row is DataSheet!A11
```

FIGURE 7-8: Implementing your newly created custom range function.

Manipulating Data in Ranges

When working with information in Excel, you often have to transform the data in some way: cleaning, standardizing, or shaping the data so that it's appropriate for your work. Perhaps you need to clean out extra spaces, pad numbers with zeros, or filter data for certain criteria.

This section shows you some of the more useful scripts you can use to dynamically transform the data in your workbooks. If you like, you can combine these scripts, running each piece of code in a sequence that automates the scrubbing and shaping of your data.

Copying data from one range to another

Given that copying data from one range to another is a common Excel task, it's not surprising that Office Scripts offers methods to copy ranges. Strangely, though, there's no such thing as pasting in Office Scripts.

Instead, Office Scripts uses the `copyFrom` method to tell a target range to pull data from a source range. For example, the following script copies data from the source range (A1:G10) to the target range (A15):

```
function main(workbook: ExcelScript.Workbook) {

 let mySheet = workbook.getWorksheet('Sheet1')
 let targetRange = mySheet.getRange('A15')
 let sourceRange = mySheet.getRange('A1:G10')

 targetRange.copyFrom(sourceRange);

}
```

TIP

The target range is always treated as the top-left cell of where you want to paste the data. Even if you define the target range as more than just one cell (B5:C200, for instance) the `copyFrom` method considers only B5 as the target range. Think of the target range as the cell to start pasting rather than the range to paste into. Put another way, the target range does not determine the paste area; the source range dimensions determine the paste area.

Simulating paste-special functionality

The `copyFrom` method accepts an optional `copytype` parameter allowing you to simulate the paste-special functionality of Excel. You can use any one of the following members of the `ExcelScript.RangeCopyType` enumerations to specify what gets copied from the source:

- **`ExcelScript.RangeCopyType.all`:** Copies everything (values, formatting and formulas). This type is the default, which is used if you don't provide a `copytype` parameter.
- **`ExcelScript.RangeCopyType.values`:** Copies only the raw text or numbers. This type mimics the paste-special values functionality in Excel.
- **`ExcelScript.RangeCopyType.formulas`:** Copies the formulas without the formatting. This type mimics the paste-special formulas functionality in Excel.
- **`ExcelScript.RangeCopyType.formats`:** Copies only the formatting (colors, borders, fonts, and so on), not the data.

The following sample script copies only values, meaning it will transfer only raw text and numbers from the `sourceRange` (not formulas or formatting):

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')

let sourceRange = mySheet.getRange('A1:G10')
let targetRange = mySheet.getRange('A15')

// Copy only values (no formulas or formatting)
targetRange.copyFrom(sourceRange,
  ExcelScript.RangeCopyType.values);

}
```

Copying data between worksheets

The `copyFrom` method also supports copying data between worksheets. The mechanics are essentially the same as copying data from one range to another in the same worksheet. The only difference is that you need to specify the `worksheet` objects for both the source and destination ranges.

The following script copies A1:G10 from the source worksheet to A1 on the destination worksheet:

```
function main(workbook: ExcelScript.Workbook){

//Define source and destination sheets
let sourceSheet = workbook.getWorksheet('Sheet1')
let destSheet = workbook.getWorksheet('Sheet2')

//Check if destination sheet exists
if (!destSheet) {
console.log ('Destination sheet missing')
return
}

//Define source and target ranges
let sourceRange = sourceSheet.getRange('A1:G10');
let targetRange = destSheet.getRange('A1');

//Copy sourceRange to targetRange
targetRange.copyFrom(sourceRange);

}
```

You initialize two variables to hold the worksheet objects: `sourceSheet` and `destSheet`. Then you check to see if the `destSheet` variable is empty by using the `if(!destSheet)` statement (if the `destSheet` variable is empty, exit the function).

The rest of the script follows the basic pattern for copying data from one range to another.

Converting all formulas in a range to values

In some situations, you may want to temporarily use formulas in a certain capacity but don't necessarily want to keep or distribute the formulas with your workbook. In these situations, you may want to convert all the formulas in a given range to values. There are two ways to accomplish this task.

First, you can use the `copyFrom` method you just explored in the preceding section. Simply use the same range as both the source and target range while using the `copyType` argument to limit what is copied. Note in the following sample script

that you can choose to keep formatting by including a statement that copies formats:

```
function main(workbook: ExcelScript.Workbook) {

 let mySheet = workbook.getWorksheet('Sheet1')
 let sourceRange = mySheet.getRange('A1:G10')

 //Copy values
        sourceRange.copyFrom(sourceRange,
        ExcelScript.RangeCopyType.values);

 //Copy only formats
        sourceRange.copyFrom(sourceRange,
        ExcelScript.RangeCopyType.formats);

}
```

The second way to remove formulas from a range is to store the values in that range into an array. The `range` object offers a `getvalues` method that you can use to get values from the given range and store them in an array variable. When you store values into an array, any associated formulas fall away, leaving only the raw values.

The following script demonstrates this method:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('Sheet1')
let sourceRange = mySheet.getRange('A1:G10')

//Store the values into an array
let sourceValues = sourceRange.getValues()

//Write the values back to the source range
sourceRange.setValues(sourceValues)
}
```

Note how you initialize a variable named `sourceValues` and fill it with `sourceRange.getValues()`. Next, you use the `setValues` method to replace the values in the source range with those in the `sourceValues` array. Because the `sourceValues` array contains only the values and not the formulas, this technique effectively simulates the paste-special values functionality in Excel.

WARNING

Any actions taken via Office Scripts do not get logged in Excel's undo stack, so there is no undo function when you overwrite data or formulas with your script. Be careful.

Clearing cell contents

Sometimes you'll need to programmatically delete data from cells. For instance, you may want to clear cell contents to prepare for a fresh data import. Or maybe you need to reset a data entry template by clearing previous values. Office Scripts has you covered with the `clear()` method.

The `clear()` method enables you to clear different kinds of content from your range. Its optional `applyTo` parameter allows you to specify what gets cleared using one the following members of the `ClearApplyTo` enumerations:

- **`ExcelScript.ClearApplyTo.all`:** Clears all contents including values, formulas, formats, and hyperlinks. This type is the default, which will be used if you don't provide an `applyto` parameter.
- **`ExcelScript.ClearApplyTo.contents`:** Clears everything except formatting. This option lets you reset cells while maintaining all formatting, which is perfect for templates.
- **`ExcelScript.ClearApplyTo.formats`:** Clears cell formatting. This option can clear only what's in the range. It will not affect data validation rules, comments, worksheet row heights, or column widths.
- **`ExcelScript.ClearApplyTo.hyperlinks`:** Clears only hyperlinks. Using this option on a range will leave the URL or hyperlink text, but the cell will no longer contain a working hyperlink.

In the following example script, you don't need to provide an `applyTo` parameter when you want to clear all contents:

```
function main(workbook: ExcelScript.Workbook) {
let mySheet = workbook.getWorksheet('Sheet1')

//Clear all contents
  mySheet.getRange('A1:C1').clear();

//Clear everything but formatting
  mySheet.getRange('A2:C2').clear(
  ExcelScript.ClearApplyTo.contents);
```

```
//Clear only formatting
  mySheet.getRange('A3:C3').clear(
  ExcelScript.ClearApplyTo.formats);

//Clear only hyperlinks
  mySheet.getRange('A4:C4').clear(
  ExcelScript.ClearApplyTo.hyperlinks);

}
```

WARNING

Once you run your script, the `clear()` method will do its thing without giving you a warning or a chance to cancel the action. You can't undo actions taken through Office Scripts. Always test your scripts on a sample workbook to avoid accidental data loss.

Deleting rows and columns

While the `clear()` method empties range contents, the `delete()` method removes a given range from the worksheet and shifts the neighboring cells to fill the gap. When deleting a range, you always specify a shift direction:

- **`ExcelScript.DeleteShiftDirection.up`:** Cells below the deleted range move up.
- **`ExcelScript.DeleteShiftDirection.left`:** Cells to the right of the deleted range move left.

Deleting ranges using the A1 notation style is fairly intuitive. Simply use the `getRange` method to reference the row, column, or range, as in the following example script:

```
function main(workbook: ExcelScript.Workbook) {

let mysheet = workbook.getActiveWorksheet();

//Delete row 4
mysheet.getRange('4:4').delete(
ExcelScript.DeleteShiftDirection.up)

//Delete first 10 rows
mysheet.getRange('1:10').delete(
ExcelScript.DeleteShiftDirection.up)
```

```
//Delete column C
mysheet.getRange('C:C').delete(
ExcelScript.DeleteShiftDirection.left)

//Delete columns B through E
mysheet.getRange('B:E').delete(
ExcelScript.DeleteShiftDirection.left)

//Delete A1:G10 and then shift cells up
mysheet.getRange('A1:G10').delete(
ExcelScript.DeleteShiftDirection.up)

}
```

When using a single-cell range reference such as with the `getCell()` method, you need to couple it with `getEntireRow()` or `getEntireColumn()` to delete rows or columns:

```
function main(workbook: ExcelScript.Workbook) {
let mysheet = workbook.getActiveWorksheet();

//Delete row 1
 mysheet.getCell(0,0).getEntireRow().delete(
 ExcelScript.DeleteShiftDirection.up)

//Delete column C
mysheet.getCell(0, 2).getEntireColumn().delete(
ExcelScript.DeleteShiftDirection.left)

}
```

When you delete a row, all rows below shift up. This action is expected and is not an issue when deleting a single row or a single block of rows. However, if you're looping through a range and deleting rows based on a condition, you need to consider that each deletion shifts rows up. If you loop through a range forward (from the first row to the last row), you'll skip rows after each deletion.

Imagine your code loops through a range, deleting each row that is empty. It checks row 1, then checks row 2, and then finds that row 3 is empty. It deletes row 3. But after the deletion, what *was* row 4 has shifted up, and is now row 3. Your code continues to check row 4. But wait! It skipped the row that used to be row 4 and is now row 3, but the loop has already processed row 3 and has moved on.

To avoid this issue, you need to loop through your range backward. Check each row starting from the last row to the first. This way, when rows are deleted, the loop moves up the range, so it's not affected by rows shifting below it.

To understand how this works, look at the following script. Here, you want to loop through range A1:A15, check each value in the range, and delete the row if the value is empty:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Get the target range
let mySheet = workbook.getWorksheet('DataSheet3')
let myRange = mySheet.getRange('A1:A15')

//Step 2: Get the row count and starting row of myRange
let rowCount = myRange.getRowCount()
let myRangeFirstRow = myRange.getRowIndex()

//Step 3: Start looping backward from the last row
for (let i = rowCount; i >= 1; i--) {

//Step 4: Get the value of column A for the current row
let cellValue = myRange.getCell(i, 0).getValue()

//Step 5: If cellValue is empty get row number
  if (cellValue === "" || cellValue === null){
  let deleteRowNumber = myRangeFirstRow + i

//Step 6: Delete the row
mySheet
.getCell(deleteRowNumber, deleteRowNumber)
.getEntireRow()
.delete(ExcelScript.DeleteShiftDirection.up);
 }
 }
}
```

In Step 1, you initialize the `myRange` variable and fill it with range A1:A15.

Step 2 captures the count of rows in the `rowCount` variable and stores the row index of the first row in `myRangeFirstRow`.

Step 3 starts a `for` loop with the following: `for (let i = rowCount; i >= 1; i--)`. The statement `let i = rowCount` initializes a counter variable named `i`

and sets its starting value as the total row count for the range. The next portion, `i >= 1;`, tells the loop to keep going as long as the value of `i` is greater than or equal to `1`. Finally, `i--` is a decrement operator that reduces `i` by `1` on every loop. To summarize, Step 3 says start the loop with `i` equaling the total number of rows in your range. On every loop, reduce the value of `i` by `1` and keep looping while `i` is greater than or equal to `1`.

Step 4 gets and stores the value of the cell currently being looped in `cellValue`. Then in Step 5, an `if` statement checks `cellValue`. If the value is empty or is null, you initialize the `deleteRowNumber` variable and set it to the row number where the empty cell was found.

Finally, Step 6 deletes the row where the empty cell was found.

TIP

For a refresher on how loops work, see Chapter 5.

Working with Excel Tables

An Excel table is different than a plain range of cells. It provides structured data objects with features such as calculated columns and row totals. From an Office Scripts perspective, it is a distinct object separate from a range object.

TECHNICAL STUFF

You can manually create a table in Excel by selecting any cell in your data range and pressing Ctrl+T. The Create Table dialog will open, and you can confirm the data range and indicate whether your table has headers. Every table is assigned a generic default name, such as Table1 or Table2. You can rename a table by clicking anywhere in the table and selecting the Table Design tab on the ribbon. In the Properties group on the far left of this tab, simply enter a new name in the Table Name input box.

To start working with tables, use the `getTable()` method as shown in the following example:

```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('DataSheet4')
let myTable = mySheet.getTable('Table1')

//Always make sure the table exists
if (!myTable) {
console.log ('Table does not exist')
return
```

```
}

//Get the table name
console.log(myTable.getRange())

//Get the row count
console.log(myTable.getRowCount())

//Get the column count
console.log(myTable.getColumns().length)

}
```

When working with tables, always check that the table exists by using `if (!myTable)`. Once you have the table loaded into an object variable, you can use its methods to get the row count, column count, and data range.

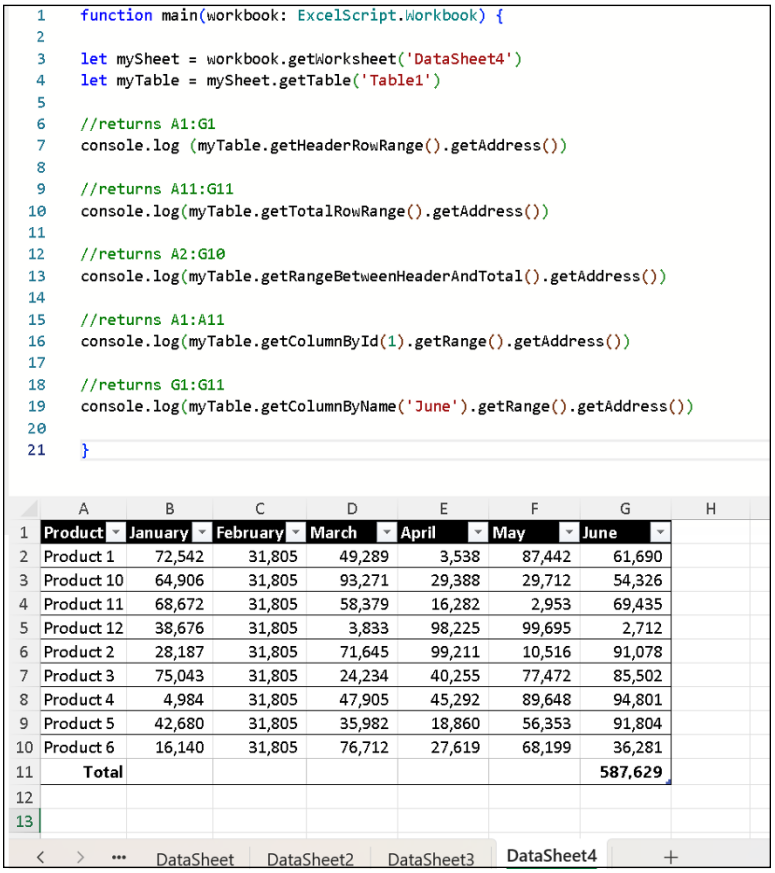
Referencing parts of a table

Tables contain a hierarchy of objects. You can reference the parts of a table using its object hierarchy. Table 7-3 highlights some of the most useful methods for getting range objects from Excel tables.

TABLE 7-3 Useful Methods for getting Table range objects

Object	Resulting Output
`myTable.getHeaderRowRange()`	The range of the table header row.
`myTable.getTotalRowRange()`	The range of the table total row.
`myTable.getRangeBetweenHeaderAndTotal()`	The range of the data area between the header and total rows.
`myTable.getColumnById(1).getRange()`	The range of the first column in the table. Note the number parameter (`1`). The number is a column number, not an index. In tables, the first column is 1, not 0 (like column A in the worksheet).
`myTable.getColumnByName('June').getRange()`	The range of the column named June.
`myTable.getRange()`	The range object for the entire table.

Figure 7-9 demonstrates how each of these methods references each range of the Excel table shown.



```
function main(workbook: ExcelScript.Workbook) {

let mySheet = workbook.getWorksheet('DataSheet4')
let myTable = mySheet.getTable('Table1')

//returns A1:G1
console.log (myTable.getHeaderRowRange().getAddress())

//returns A11:G11
console.log(myTable.getTotalRowRange().getAddress())

//returns A2:G10
console.log(myTable.getRangeBetweenHeaderAndTotal().getAddress())

//returns A1:A11
console.log(myTable.getColumnById(1).getRange().getAddress())

//returns G1:G11
console.log(myTable.getColumnByName('June').getRange().getAddress())

}
```

Product	January	February	March	April	May	June
Product 1	72,542	31,805	49,289	3,538	87,442	61,690
Product 10	64,906	31,805	93,271	29,388	29,712	54,326
Product 11	68,672	31,805	58,379	16,282	2,953	69,435
Product 12	38,676	31,805	3,833	98,225	99,695	2,712
Product 2	28,187	31,805	71,645	99,211	10,516	91,078
Product 3	75,043	31,805	24,234	40,255	77,472	85,502
Product 4	4,984	31,805	47,905	45,292	89,648	94,801
Product 5	42,680	31,805	35,982	18,860	56,353	91,804
Product 6	16,140	31,805	76,712	27,619	68,199	36,281
Total						587,629

DataSheet | DataSheet2 | DataSheet3 | DataSheet4

FIGURE 7-9: Referencing the component parts of a table.

Creating a table from a range

A common data scenario is the need to convert an existing range to an Excel table. You can create a table from a range by using the `addTable()` method, which takes two parameters: `address` and `hasHeaders`. The `address` parameter defines the range to be converted to a table, and the `hasHeaders` `true` or `false` parameter specifies whether or not the source range has headers. The following script creates a table called `MyNewTable` from the range A1:G10:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Get source range
let mySheet = workbook.getWorksheet('Sheet1')
```

```
let myRange = mySheet.getRange('A1:G10').getAddress()

//Step 2: Ensure new table name is not being used
let myTableName = 'MyNewTable'
let myTable = workbook.getTable(myTableName)
  if (myTable) {
  console.log ('table name already exists')
  return
  }

//Step 3: Create table with headers
myTable = mySheet.addTable(myRange, true)

//Step 4: Change table name
myTable.setName(myTableName)
console.log("Table created successfully!");
}
```

In Step 1, you initialize the `myRange` variable to hold the address of the range you want made into a table (Sheet1!A1:G10) in this case.

Step 2 tests to see if the current workbook already has a table with the same name as the proposed name from your new table. Here, you use `getTable(myTableName)` to attempt to load a table object into the `myTable` variable. The `if` statement returns `true` if the `myTable` variable is not empty (meaning a table with the same name exists), which triggers a `console.log` message and exits the function.

If the `myTable` variable is empty, Step 3 creates the Excel table using `addTable(myRange, true)`. The `myRange` variable defines the address containing your data, and `true` indicates that the first row in your source data contains headers.

Step 4 rounds out the script by naming the newly created table using the `myTableName` variable.

Applying table filters

Excel tables offer built-in filtering capabilities, allowing you to present only data that meets specified criteria. Note that, from an Office Scripts perspective, you don't filter a table; you filter a column.

Each column in a table has its own `Filter` object. To apply a filter to a table, you get the column you want to target, access its `Filter` object, and then use the `applyValuesFilter` method to define the filter criteria.

In the sample script shown next, you filter Table1 on the Product field to show only rows related to `'Product 12'`:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Get the table
let mySheet = workbook.getWorksheet('DataSheet4')
let myTable = mySheet.getTable('Table1')

//Step 2: Get the Product column's Filter object
let productFilter =
    myTable.getColumnByName('Product').getFilter();

//Step 3: Apply the filter to show only 'Product 12'
productFilter.applyValuesFilter(['Product 12']);

}
```

In Step 2, you define a `productFilter` variable to store the `Filter` object for the `Product` column. Then in Step 3, you use the `applyValuesFilter` method to specify your criteria.

To filter by multiple values, simply provide an array of values separated by commas. For instance:

```
productFilter.applyValuesFilter(['Product 12', 'Product 5'])
```

Filtering numeric values

To filter numeric values, use the `applyCustomFilter` method to define the number range you're targeting. For example, the following code filters the `February` column for values greater than 30,000:

```
function main(workbook: ExcelScript.Workbook) {

//Step1: Get the table
let mySheet = workbook.getWorksheet('DataSheet4')
let myTable = mySheet.getTable('Table1')

//Step 2: Get the February filter object
let febFilter =
myTable.getColumnByName('February').getFilter();
```

```
//Step 3: Filter for values greater than 30,000
febFilter.applyCustomFilter('>30,000');

}
```

The following statements are also valid:

```
//less than 30,000
febFilter.applyCustomFilter('<30,000');

//less than or equal to 30,000
febFilter.applyCustomFilter('<=30,000');

//greater than or equal to 30,000
febFilter.applyCustomFilter('>=30,000');
```

When you need to check more than one criterion on a single numeric column, you can use the `ExcelScript.FilterOperator` enumeration to specify a conjunction in the criteria. For instance, the following snippet uses the `and` enumerator to filter the `February` column for values less than 30,000 and greater than 50,000:

```
//less than 30,000 AND greater than 50,000
febFilter.applyCustomFilter('<30,000', '>50,000',
ExcelScript.FilterOperator.and);
```

Here, you use `or` to filter the `February` column for values less than 30,000 or greater than 50,000:

```
//less than 30,000 OR greater than 50,000
febFilter.applyCustomFilter('<30,000', '>50,000',
ExcelScript.FilterOperator.or);
```

TIP

The `applyCustomFilter` method supports a maximum of two criteria joined by the `and` operator or the `or` operator. If you provide more than two, the custom filter will ignore all but the first criteria.

Performing advance text filtering

Sometimes you may need to filter values that contain a certain word or start with a specific set of characters. In these cases, you can use the asterisk (*) wildcard. Think of the asterisk as a placeholder representing any number of characters. Using it allows Excel to filter for patterns rather than exact matches. The asterisk wildcard works as follows:

car*: Search for any string that starts with *car.*

***car:** Search for any string that ends with *car.*

***car*:** Search for any string that contains *car.*

The following code snippets illustrate how wildcards can help with advanced text filtering:

```
//Filter values that contain the word oil
productFilter.applyCustomFilter('=*oil*');

//Filter values that start with the word synthetic
productFilter.applyCustomFilter('=synthetic*');

//Filter values that do NOT contain the word oil
productFilter.applyCustomFilter('<>*oil*');

//Filter values that start with the work synthetic
//OR values that contain the word oil
productFilter.applyCustomFilter('synthetic*', '*oil*',
  ExcelScript.FilterOperator.or);
```

Clearing filters

To clear filters for a single column, simply reference the column's `Filter` object and use the `clear` method. The next sample script illustrates how to clear the filter for the `Product` column:

```
function main(workbook: ExcelScript.Workbook) {

workbook.getWorksheet('DataSheet4')
        .getTable('Table1')
        .getColumnByName('Product')
        .getFilter()
        .clear();

}
```

To clear filters for all columns in a table, use the `getcolumns` method to load all the columns into an array variable, and then simply loop through the array, calling the `clear` method for each column:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Get the table
let table = workbook.getTable('Table1')

//Step 2: Get all columns in the table
let columns = table.getColumns();

//Step 3: Loop through each column and clear its filter
columns.forEach(column => {
    column.getFilter().clear();
  });

}
```

IN THIS CHAPTER

- Creating a pivot table object
- Adding pivot table data items
- Customizing pivot table data fields
- Refreshing pivot tables in a workbook
- Changing the layout of pivot table reports
- Creating and filtering slicers

Chapter 8

Pivoting with Pivot Tables

Anyone who uses Excel knows that pivot tables are the gold standard for data communication. Without pivot tables, a report feels like a fireplace without a fire. Pivot tables democratize data like no other feature, allowing even novice end users to explore information as effectively as a pro.

Given the importance of pivot tables, it's only natural that Office Scripts offers a robust object model for automating them. In this chapter, you learn about the code behind pivot table automation. I also cover the most common scripting scenarios when automating pivot table solutions.

REMEMBER

The code snippets used in this chapter are available in the `Chapter 8 Code` folder, which you can download along with the sample files at `www.dummies.com/go/microsoft365excelofficescriptsfd`. The sample files for this book include a workbook called `SamplePivotData.xlsx`, which contains the data used in the examples for this chapter. Simply open the workbook, start a new script there, and paste the code snippets to test them.

Understanding Pivot Table Hierarchy

You're accustomed to thinking about a pivot table as a kind of container object that holds data. That container has different areas for filters, columns, rows, or values. When you create a pivot table, you simply drag data elements to these areas by using the PivotTable Fields task pane, as shown in Figure 8-1.

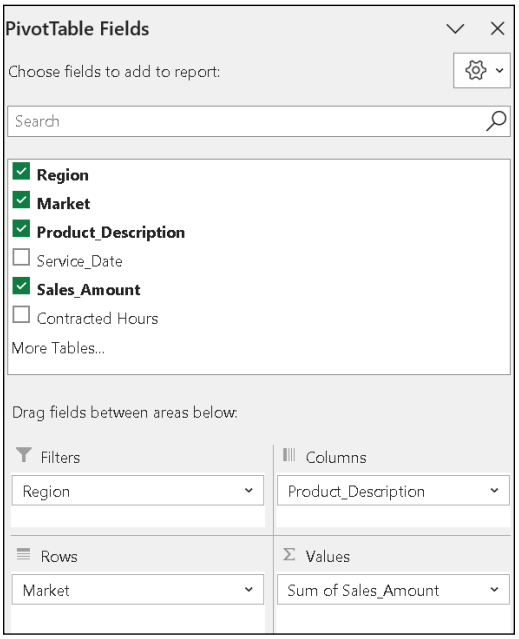


FIGURE 8-1: Think of pivot tables in terms of fields that can be placed in the Filters, Columns, Rows, or Values area.

But scripts don't work this way. They consider each area of a pivot table as a distinct container referred to as a *hierarchy.* As shown in Figure 8-2, there are five distinct hierarchies:

- **`PivotHierarchy`:** The central container object for all data fields in the pivot table. When you create a pivot table, `PivotHierarchy` is the first container to be filled with data items. You can think of `PivotHierarchy` as the main list of data items you see at the top of the PivotTable Fields dialog box in Figure 8-1. You add data items to the other hierarchies from this central `PivotHierarchy` object.
- **`RowHierarchy`:** The container corresponding to the Rows area in Figure 8-1. In Office Scripts, you use the `addRowHierarchy` method to mimic dragging a data field to the Rows area.

- **`ColumnHierarchy`:** The container corresponding to the Columns area in Figure 8-1. You use the `addColumnHierarchy` method to mimic dragging a data field to the Columns area.
- **`FilterHierarchy`:** The container corresponding to the Filters area in Figure 8-1. You use the `addFilterHierarchy` method to simulate dragging a data field to the Filter area.
- **`DataHierarchy`:** The container corresponding to the Values area in Figure 8-1. In Office Scripts, you use the `addDataHierarchy` method to add data fields to the Values area.

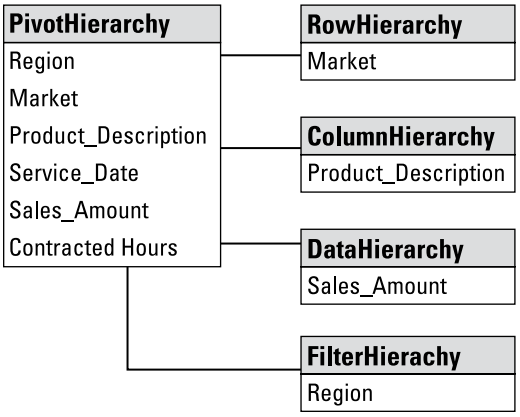


FIGURE 8-2: From an Office Scripts perspective, a pivot table has five distinct hierarchies.

Starting a pivot table

In Office Scripts, the first step in creating a pivot table is to create a `PivotHierarchy` object. You do this by using the `addPivotTable` method, which requires three arguments — a name for the pivot table, the data source, and the destination for the pivot table:

```
addPivotTable(Name, Source, Destination)
```

The following code demonstrates the basic pattern for creating a pivot table:

```
function main(workbook: ExcelScript.Workbook) {

  const mySheet = workbook.getWorksheet('Data')
  const pivotName = 'MyPivotTable'
  const sourceData = mySheet.getTable('MyDataTable')
  const pivotSheet = workbook.addWorksheet('PivotSheet')
```

```
  pivotSheet.addPivotTable(
    pivotName,
    sourceData,
    pivotSheet.getRange('A1')
  )
}
```

As you can see, you first define a `pivotName` variable, supplying the name you want to use for the pivot table. You then define the `sourceData` variable, which contains the data for the pivot table. In this case, the source is defined as a table called `MyDataTable`. Finally, you define a `pivotSheet` variable specifying the worksheet you want to use as the destination for the pivot table. You can use an existing worksheet, but in this example, you use the `addWorksheet` method to create a worksheet named `PivotSheet`.

TIP

The source data for a pivot table doesn't have to be a named table object. You can specify any valid range as the source, such as `getUsedRange()` or `getRange('A1:G5000')`. However, it's considered best practice to use a named table object as the source because the pivot table will automatically include any new rows or columns. (If you use a standard range, the pivot table will not automatically include new rows or columns.) If you're sure that the source range for the pivot table will remain constant and will not grow, you can settle for using a standard range.

You can improve the sample script by guarding against errors. For example, you can first check if the source (`MyDataTable`) exists. You can also ensure that there is no sheet already named `PivotSheet` before attempting to create one. Compare the following enhanced script with its error-handling techniques to the script shown previously:

```
function main(workbook: ExcelScript.Workbook) {

// Define the name for the pivot table
  const pivotName = 'MyPivotTable'

//Define MyDataTable as the source
  const mySheet = workbook.getWorksheet('Data');
  const sourceData = mySheet.getTable('MyDataTable');
     if (!sourceData) {
     console.log('Table does not exist');
     return; //exit function if table does not exist
     };
```

```
//Create PivotSheet
  try { workbook.addWorksheet('PivotSheet') }
  catch {
  console.log('Sheet already exists');
  return; //exit function if PivotSheet already exists
  };

//Create the pivot table
  let pivotSheet = workbook.getWorksheet('PivotSheet')
  pivotSheet.activate()
  pivotSheet.addPivotTable(
  pivotName,
  sourceData,
  pivotSheet.getRange('A1')
  );
}
```

TIP

Feel free to revisit Chapter 4 for a refresher on the error-handling techniques illustrated in the code.

Running this script creates a pivot table object. As you can see in Figure 8-3, the pivot table is placed in the new `PivotSheet` worksheet and named `MyPivotTable`.

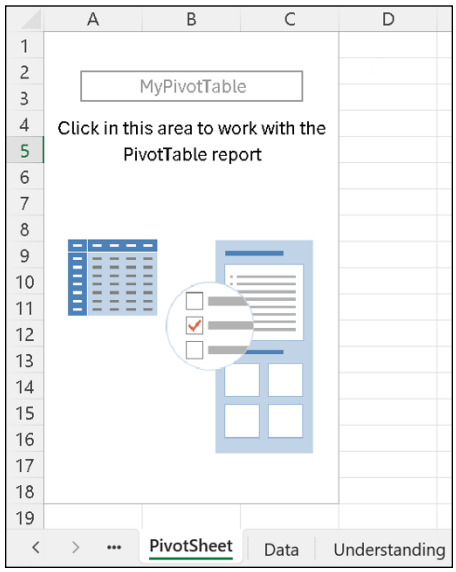


FIGURE 8-3: Running the starter script will create a pivot table object.

Adding data items to the pivot table

At this point, the pivot table in Figure 8-3 is empty. If you were building this pivot table manually, you would use the PivotTable Fields task pane to start dragging data items to rows, columns, and so on. But as mentioned, scripts don't work that way.

Instead of dragging data items to each area, you use the `add` method for each of the four hierarchies representing the four areas of the pivot table: `RowHierarchy`, `ColumnHierarchy`, `FilterHierarchy`, and `DataHierarchy`. You point to the data items in the central `PivotHierarchy` and add them to one of the four areas (refer to Figure 8-2).

The following code illustrates the basic pattern. You first reference the pivot table you're working with. Then for each area, you use the `add` method to tell Excel what field to include in that area. For example, to add the `Market` field to the Rows area, you would use the `addRowHierarchy` method:

```
pivotTable.addRowHierarchy(pivotTable.getHierarchy('Market'))
```

Note how each area has the same syntax pattern. You specify the hierarchy and then get the field by using `getHierarchy()` method:

```
function main(workbook: ExcelScript.Workbook) {

//Specify your pivot sheet
 const sheet = workbook.getWorksheet('PivotSheet');

//Specify the pivot table
 const pivotTable = sheet.getPivotTable('MyPivotTable');

//Add Row fields
 pivotTable.addRowHierarchy(
 pivotTable.getHierarchy('Market'))

//Add Column fields
 pivotTable.addColumnHierarchy(
 pivotTable.getHierarchy('Product_Description'))

//Add Data fields
 pivotTable.addDataHierarchy(
 pivotTable.getHierarchy('Sales_Amount'))
```

```
 pivotTable.addDataHierarchy(
 pivotTable.getHierarchy('Contracted Hours'))

//Add Filter fields
 pivotTable.addFilterHierarchy(
 pivotTable.getHierarchy('Region'))

}
```

Figure 8-4 displays the final pivot table after running the code.

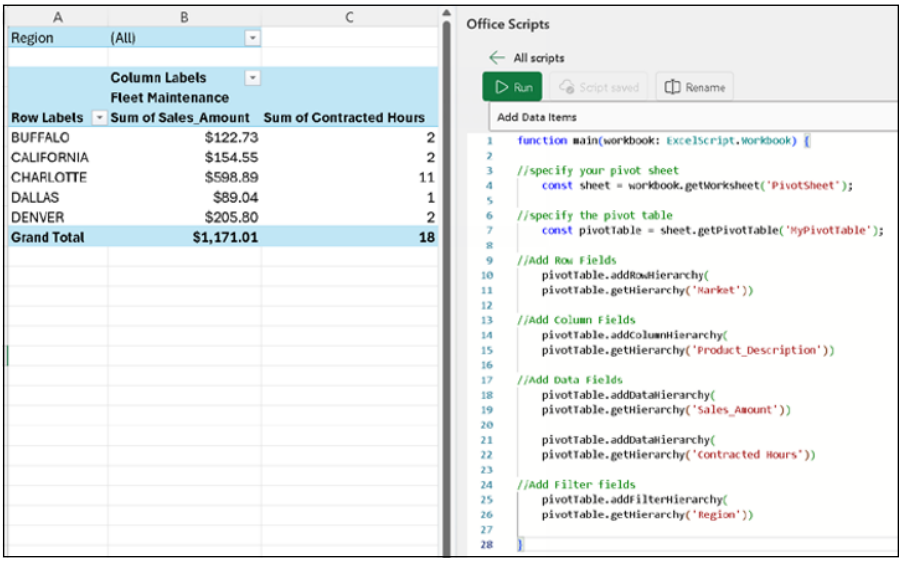


FIGURE 8-4: Fill the pivot table by using the add method for each of four hierarchies representing the four areas of the pivot table.

Customizing data field items

As you can see in Figure 8-4, the code produces the desired pivot table structure, but there's room for improvement. Specifically, you can customize the items in the Values area of the pivot table to better suit your needs.

`DataHierarchy` corresponds to the Values area of the pivot table; each field in the Values area is a data item in the `DataHierarchy` collection, Each data item in `DataHierarchy` has properties such as `NumberFormat`, `Name`, and `SummarizeBy` (which sets the aggregation type for the data item).

The code snippet in Figure 8-5 illustrates how to create a customized data item in the Values area.

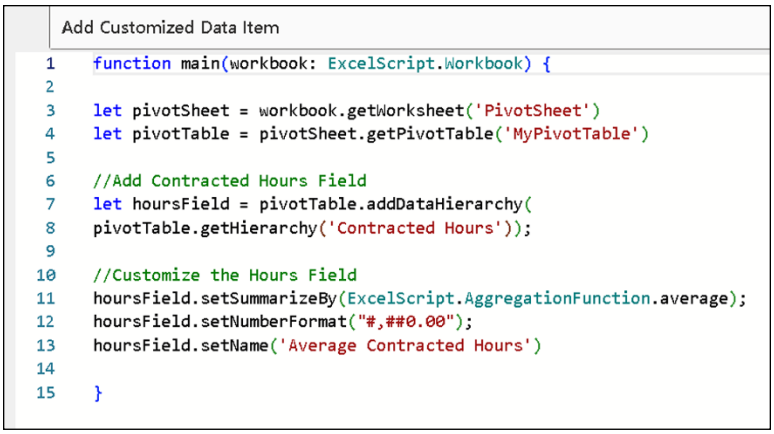


FIGURE 8-5: Use the `setSummarizeBy`, `setNumberFormat`, and `setName` methods to customize fields in `DataHierarchy`.

In line 4, you define the pivot table you're working with. In line 7, you create a variable called `hoursField` to reference the Contracted Hours data field. This variable mainly helps you avoid having to type `pivotTable.getRowHierarchy('Contracted Hours')` every time you need to reference that data item.

Then starting on line 11, you use the `setSummarizeBy`, `setNumberFormat`, and `setName` methods to customize fields in `DataHierarchy`. Let's cover what each does in detail.

The setSummarizeBy method

The first customization you employ in Figure 8-5 (see line 11) is the `setSummarizeBy` method, specifying the aggregation type for the data item. When creating a pivot table, Excel, by default, summarizes your data by either counting or summing the items. If all the cells in a column contain numeric data, Excel will sum them. If the field you're adding contains a blank or text, Excel will count it.

However, in many instances, a pivot field that should be summed legitimately contains blanks. In these cases, you're forced to manually go in after Excel and change the calculation type from `count` back to `sum`.

You can avoid this problem by explicitly setting the `summarizeBy` property. For example:

```
setSummarizeBy(ExcelScript.AggregationFunction.sum)
```

In this case, you want the average contracted hours, so you use:

```
setSummarizeBy(ExcelScript.AggregationFunction.average)
```

The following list shows the aggregation options in the `ExcelScript.AggregationFunction` enumeration:

```
ExcelScript.AggregationFunction.sum
ExcelScript.AggregationFunction.count
ExcelScript.AggregationFunction.average
ExcelScript.AggregationFunction.max
ExcelScript.AggregationFunction.min
ExcelScript.AggregationFunction.product
ExcelScript.AggregationFunction.countNumbers
ExcelScript.AggregationFunction.standardDeviation
ExcelScript.AggregationFunction.standardDeviationP
ExcelScript.AggregationFunction.variance
ExcelScript.AggregationFunction.varianceP
```

The setNumberFormat method

The next customization in Figure 8-5 (line 12) is setting the number formatting. For this, you use the `setNumberFormat` method to define the format. In this scenario, you are using `setNumberFormat("#,##0.00")`, which is standard formatting for a number with two decimal places.

Some other examples follow:

- **`setNumberFormat("0.00%")`:** Percentage with two decimal places
- **`setNumberFormat("$#,##0.00")`:** Currency
- **`setNumberFormat("#,##0.00;(#,##0.00)")`:** Negative numbers in parentheses: (1,234.56)

The setName method

The final customization in Figure 8-5 (line 13) sets the name for the data item:

```
hoursField.setName('Average Contracted Hours')
```

When you create a pivot table, Excel tries to help by prefacing each data field header with `Sum of`, `Count of`, or whichever operation you use. Often, this is not

what you want. Having a field called Sum of Sales Amount is not as clean and snappy as, say, Total Revenue. The `setName` method lets you display clear and helpful titles in your reports.

TIP

Unfortunately, pivot tables won't allow you to name a data field with the same name as the source data field. For example, if a column in the source data is called `Sales Amount`, you can't name the corresponding value in the pivot table `Sales Amount`. The workaround for this is to add a space to the end of the field name. For example: `setName('Sales Amount')`.

Avoiding errors when adding data items

When programmatically adding data items to a pivot table, you'll encounter errors if your script attempts to add a data item to a hierarchy that already contains that item. To avoid such errors, you can utilize the `try-catch` method when attempting to add a data item. For instance, the following sample script adds the `Market` field to the Columns area of the pivot table. Note how the `addColumnHierarchy` method is in a `try` statement. This effectively tells the script to attempt to add the fields, but if it fails, just ignore the error and skip that step:

```
function main(workbook: ExcelScript.Workbook) {

let pivotSheet = workbook.getWorksheet('PivotSheet')
let pivotTable = pivotSheet.getPivotTable('MyPivotTable')

//Add Column fields
  try {
  pivotTable.addColumnHierarchy(
  pivotTable.getHierarchy('Market'))
  }
  catch {console.log('Field is already there')}

}
```

Another technique is to clear the entire hierarchy before adding fields to it. The next script first clears the entire `DataHierarchy`, effectively removing all fields from the Values area before adding them back in. This resets the pivot Values area, allows you to read data items with your own custom formatting:

```
function main(workbook: ExcelScript.Workbook) {

let pivotSheet = workbook.getWorksheet('PivotSheet')
let pivotTable = pivotSheet.getPivotTable('MyPivotTable')
```

```
//Clear all Data fields
 const dataHierarchies=pivotTable.getDataHierarchies();
       dataHierarchies.forEach(hierarchy =>
       {pivotTable.removeDataHierarchy(hierarchy)});

//Add Sales Field
let salesField = pivotTable.addDataHierarchy(
pivotTable.getHierarchy('Sales_Amount'));
salesField.setNumberFormat("$#,##0.00");
salesField.setName('Sales Amount');

//Add Contracted Hours Field
let hoursField = pivotTable.addDataHierarchy(
pivotTable.getHierarchy('Contracted Hours'));
```

```
hoursField.setNumberFormat("#,##0.00");
hoursField.setName('Average Hours');

}
```

You first fill the `dataHierarchies` variable with all the data items in the `DataHierarchy` collection. Then you use a `forEach` statement to loop through each item in the collection. For each item, you use the `removeDataHierarchy` method to delete it from the collection.

Putting it all together

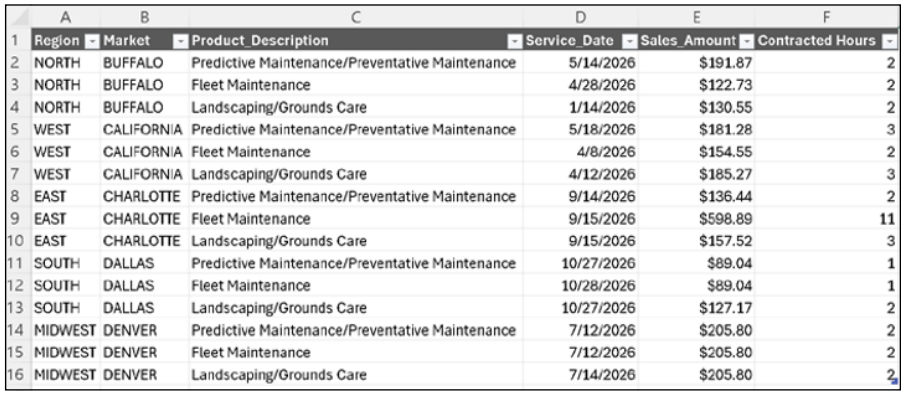
So far in this chapter, you've explored the different concepts you need to know to automate the creation of a pivot table: from initiating a pivot table object, to adding data items, to customizing and error-proofing your pivot table script.

Next, you review the full script with all its component parts so you can see how they fit together. The following script starts with the table shown in Figure 8-6 and creates the final pivot table shown in Figure 8-7:

```
function main(workbook: ExcelScript.Workbook) {

//Define the source table
  const mySheet = workbook.getWorksheet('Data');
  const sourceData = mySheet.getTable('MyDataTable');
  if (!sourceData) {
      console.log('Table does not exist')
```

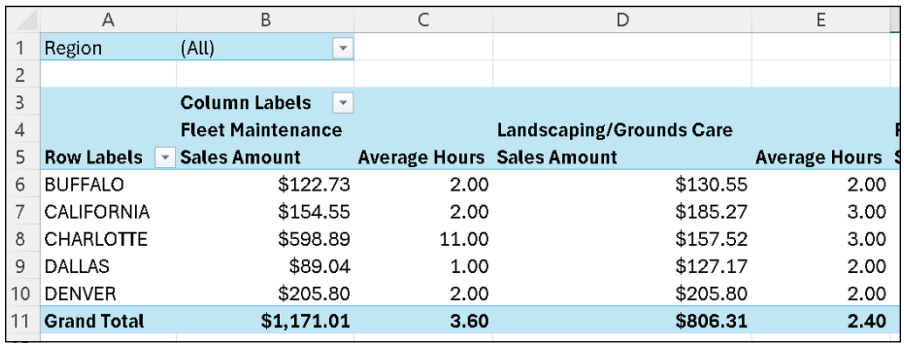
```
      return
      };

//Create PivotSheet
  try { workbook.addWorksheet('PivotSheet') }
  catch { console.log('Sheet already exists')
        return };

//Create the pivot table
  let pivotSheet = workbook.getWorksheet('PivotSheet');
  pivotSheet.activate()
  pivotSheet.addPivotTable(
    'MyPivotTable',
    sourceData,
    pivotSheet.getRange('A1')
  );

//Specify the pivot table
  const pivotTable = pivotSheet.getPivotTable('MyPivotTable');

//Add Filter fields
  try { pivotTable.addFilterHierarchy(
        pivotTable.getHierarchy('Region'))
      }
  catch { console.log('Field is already there') };

//Add Row fields
  try { pivotTable.addRowHierarchy(
        pivotTable.getHierarchy('Market'))
      }
  catch { console.log('Field is already there') };

//Add Column fields
  try { pivotTable.addColumnHierarchy(
        pivotTable.getHierarchy('Product_Description'))
      }
  catch { console.log('Field is already there') };

//Clear all Data fields
  const dataHierarchies = pivotTable.getDataHierarchies();
        dataHierarchies.forEach(hierarchy => {
        pivotTable.removeDataHierarchy(hierarchy)
        });
```

```
//Add Sales field
  let salesField = pivotTable.addDataHierarchy(
      pivotTable.getHierarchy('Sales_Amount'));
      salesField.setSummarizeBy(
      ExcelScript.AggregationFunction.sum);
      salesField.setNumberFormat("$#,##0.00");
      salesField.setName('Sales Amount');

//Add Contracted Hours field
  let hoursField = pivotTable.addDataHierarchy(
      pivotTable.getHierarchy('Contracted Hours'));
      hoursField.setSummarizeBy(
      ExcelScript.AggregationFunction.average);
      hoursField.setNumberFormat("#,##0.00");
      hoursField.setName('Average Hours');
}
```



	A	B	C	D	E	F
1	Region	Market	Product_Description	Service_Date	Sales_Amount	Contracted Hours
2	NORTH	BUFFALO	Predictive Maintenance/Preventative Maintenance	5/14/2026	$191.87	2
3	NORTH	BUFFALO	Fleet Maintenance	4/28/2026	$122.73	2
4	NORTH	BUFFALO	Landscaping/Grounds Care	1/14/2026	$130.55	2
5	WEST	CALIFORNIA	Predictive Maintenance/Preventative Maintenance	5/18/2026	$181.28	3
6	WEST	CALIFORNIA	Fleet Maintenance	4/8/2026	$154.55	2
7	WEST	CALIFORNIA	Landscaping/Grounds Care	4/12/2026	$185.27	3
8	EAST	CHARLOTTE	Predictive Maintenance/Preventative Maintenance	9/14/2026	$136.44	2
9	EAST	CHARLOTTE	Fleet Maintenance	9/15/2026	$598.89	11
10	EAST	CHARLOTTE	Landscaping/Grounds Care	9/15/2026	$157.52	3
11	SOUTH	DALLAS	Predictive Maintenance/Preventative Maintenance	10/27/2026	$89.04	1
12	SOUTH	DALLAS	Fleet Maintenance	10/28/2026	$89.04	1
13	SOUTH	DALLAS	Landscaping/Grounds Care	10/27/2026	$127.17	2
14	MIDWEST	DENVER	Predictive Maintenance/Preventative Maintenance	7/12/2026	$205.80	2
15	MIDWEST	DENVER	Fleet Maintenance	7/12/2026	$205.80	2
16	MIDWEST	DENVER	Landscaping/Grounds Care	7/14/2026	$205.80	2

FIGURE 8-6: The starting table named `MyDataTable`.



	A	B	C	D	E
1	Region	(All)			
2					
3		Column Labels			
4		Fleet Maintenance		Landscaping/Grounds Care	
5	Row Labels	Sales Amount	Average Hours	Sales Amount	Average Hours
6	BUFFALO	$122.73	2.00	$130.55	2.00
7	CALIFORNIA	$154.55	2.00	$185.27	3.00
8	CHARLOTTE	$598.89	11.00	$157.52	3.00
9	DALLAS	$89.04	1.00	$127.17	2.00
10	DENVER	$205.80	2.00	$205.80	2.00
11	Grand Total	$1,171.01	3.60	$806.31	2.40

FIGURE 8-7: The final pivot table after running the script.

You can use this as a template of sorts to create your own pivot table scripts. Simply replace the variables with ones appropriate for your situation.

Refreshing All Pivot Tables in a Workbook

It's common to manage multiple pivot tables in a single workbook. Rather than manually refreshing each one as your underlying data changes, you can use a script to update all pivot tables instantly and on demand.

The following script gets a list of all the worksheets in the workbook and all the pivot tables on each worksheet, and then refreshes each pivot table:

```
function main(workbook: ExcelScript.Workbook) {

// Step 1: Get worksheets collection
  const worksheets = workbook.getWorksheets();

// Step 2: Start looping through the worksheets
    worksheets.forEach(sheet => {

// Step 3: On each sheet, get pivottables collection
      const pivotTables = sheet.getPivotTables();

// Step 4: Loop through all pivot tables and refresh
      pivotTables.forEach(pivotTable => {
      pivotTable.refresh();
    });
  });
}
```

This script makes heavy use of the `forEach` statement (covered in Chapter 5). In Step 1, you initiate a `worksheets` collection variable that contains all the worksheets in the workbook. Then Step 2 starts looping through each sheet in the `worksheets` collection.

In Step 3, you initiate a `pivotTables` collection variable that contains all the pivot tables in the worksheet. Then Step 4 loops through each pivot table, refreshing each one using the `pivotTable.refresh` method.

It's handy to know how to traverse the object hierarchy, looping through all worksheets in a workbook and all pivot tables in a worksheet. In this case, you're simply refreshing each pivot table. But this script can serve as a template for

performing more complex actions, such as adding subtotals to all pivot tables or changing the layout of all pivot tables.

Changing Pivot Report Layouts

Pivot tables can be configured in three layouts: compact form, outline form and tabular form, as shown in Figure 8-8.

Outline Form

Region	Market	Sales Amount	Average Hours
EAST			
	CHARLOTTE	$892.85	5.33
MIDWEST			
	DENVER	$617.40	2.00
NORTH			
	BUFFALO	$445.15	2.00
SOUTH			
	DALLAS	$305.25	1.33
WEST			
	CALIFORNIA	$521.10	2.67
Grand Total		$2,781.75	2.67

Compact Form

Row Labels	Sales Amount	Average Hours
EAST		
CHARLOTTE	$892.85	5.33
MIDWEST		
DENVER	$617.40	2.00
NORTH		
BUFFALO	$445.15	2.00
SOUTH		
DALLAS	$305.25	1.33
WEST		
CALIFORNIA	$521.10	2.67
Grand Total	$2,781.75	2.67

Tabular Form

Region	Market	Sales Amount	Average Hours
EAST	CHARLOTTE	$892.85	5.33
MIDWEST	DENVER	$617.40	2.00
NORTH	BUFFALO	$445.15	2.00
SOUTH	DALLAS	$305.25	1.33
WEST	CALIFORNIA	$521.10	2.67
Grand Total		$2,781.75	2.67

FIGURE 8-8: Pivot tables can be configured in three different layouts.

Whether created manually or via automation, pivot tables will by default start in compact form with subtotals and grand totals. However, this layout isn't suitable for all data and reporting situations. It's common to manually adjust the layout after creating a pivot table.

The pivot table object model includes a few methods to help automate layout styles as well as the visibility settings for subtotals and grand totals.

Setting the report layout

The `setLayoutType` method's `PivotLayoutType` property enables you to specify the report layout. The `PivotLayoutType` property offers three options, corresponding to the layouts in Figure 8-8:

```
setLayoutType(ExcelScript.PivotLayoutType.compact)

setLayoutType(ExcelScript.PivotLayoutType.outline)

setLayoutType(ExcelScript.PivotLayoutType.tabular)
```

The following code snippet demonstrates how to change the report layout to tabular form:

```
function main(workbook: ExcelScript.Workbook) {

  //Define the pivot table
  const mySheet = workbook.getWorksheet('PivotSheet');
  const pivotTable = mySheet.getPivotTable('Pivot1');

 //Change report layout
  pivotTable.getLayout()
  .setLayoutType(ExcelScript.PivotLayoutType.tabular)

}
```

Note how you first invoke `pivotTable.getlayout()` to access the layout object for the pivot table. Once you have the layout object in focus, you use the `setLayoutType` method to apply a tabular layout.

Adjusting grand totals and subtotals

Another common formatting task is hiding grand totals and subtotals when the data doesn't require them. To do this in code, you use the following methods:

- **`setShowColumnGrandTotals`:** This method unhides or hides grand totals for columns based on a `true` or `false` setting.
- **`setShowRowGrandTotals`:** This method unhides or hides grand totals for rows based on a `true` or `false` setting.
- **`setSubtotalLocation`:** This method defines the visibility state of all subtotals in the pivot table based on the selected member of the `ExcelScript.SubtotalLocationType` enumeration — `SubtotalLocationType.atTop`,

`SubtotalLocationType.atBottom` or `SubtotalLocationType.off`. The first two options make subtotals visible either above the data or below the data. That last option turns off subtotals.

The following example script changes the report layout for `MyPivot` to tabular form, turns off grand totals, and removes subtotals:

```
function main(workbook: ExcelScript.Workbook) {

//Define the pivot table
const mySheet = workbook.getWorksheet('PivotSheet')
const pivotTable = mySheet.getPivotTable('MyPivot')

//Change the report layout to tabular
pivotTable.getLayout()
.setLayoutType(ExcelScript.PivotLayoutType.tabular);

//Turn off grand totals
pivotTable.getLayout().setShowColumnGrandTotals(false);
pivotTable.getLayout().setShowRowGrandTotals(false);

//Turn off subtotals
pivotTable.getLayout()
.setSubtotalLocation(
ExcelScript.SubtotalLocationType.off);

}
```

Working with Slicers

A slicer is an interactive control that makes it easy to filter data in a pivot table. Figure 8-9 shows a slicer for the `Market` field (on the left) that is filtering the pivot table on the right. When dynamically creating a pivot table, it's often useful that you also have your code create associated slicers for your pivot table.

Creating a slicer

Office Scripts enables the automated creation of slicers with the `addSlicer` method, which requires two arguments, `slicerSource` and `sourceField`:

```
sheet.addSlicer(slicerSource, sourceField)
```

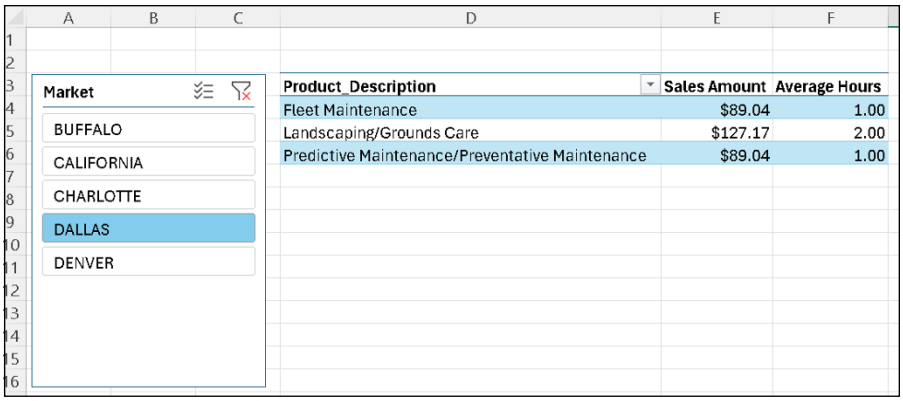


FIGURE 8-9: A slicer gives users an easy way to filter data in a pivot table.

For the `slicerSource`, you provide the pivot table you're using. For the `sourceField`, you specify what pivot field the slicer is for.

The sample script in this section demonstrates the creation of a slicer for the `Market` field. The `addSlicer` method uses two arguments. The first is `pivoTable` (a variable representing the pivot table called `'MyPivot'`). The second is a pointer to the fields of the `Market` data item in the pivot table hierarchy:

```
function main(workbook: ExcelScript.Workbook) {

  //Define source pivot table
  const mySheet = workbook.getWorksheet('PivotSheet');
  const pivotTable = mySheet.getPivotTable('MyPivot');

  //Add a slicer for the Market pivot field
  let marketSlicer = mySheet.addSlicer(
  pivotTable,
  pivotTable.getHierarchy('Market').getFields()[0])
  marketSlicer.setName('MarketSlicer');

  //Set position (in points from top left of worksheet)
  marketSlicer.setTop(50);
  marketSlicer.setLeft(10);

  //Set size
  marketSlicer.setHeight(200);
  marketSlicer.setWidth(150);

}
```

Note how you need to specify `getFields()[0]`. This tells the slicer to read all the fields starting from the first field. Running this script outputs a slicer for the `Market` pivot field.

After the slicer is created, you use some positioning methods to reposition the slicer where you need it.

```
marketSlicer.setTop(50);
marketSlicer.setLeft(10);
```

Excel uses a 2D coordinate system to determine the position of objects. The origin (0,0) is the top-left corner of the usable worksheet area (just below the column headers and to the right of the row numbers). We position an object by setting the top position separately from the left position:

- setTop: Sets how far from the top of the page to place the slicer
- setLeft: Sets how far from the left of the page to place the slicer

Excel doesn't use pixels or inches directly in the code; it uses *points.* One point represents 1/72 of an inch. If you want a slicer to be exactly 1 inch from the top, you would use the statement `setTop(72)`.

In addition to position, you can define an object's height and width using the `setHeight` and `setWidth` methods:

- `setHeight`: Sets the height of the slicer
- `setWidth`: Sets the width of the slicer

Filtering a slicer

You may need your script to dynamically filter a slicer for a specific value. For instance, imagine you need your script to create a slicer and have the default selection be the value `DALLAS`, as shown in Figure 8-10.

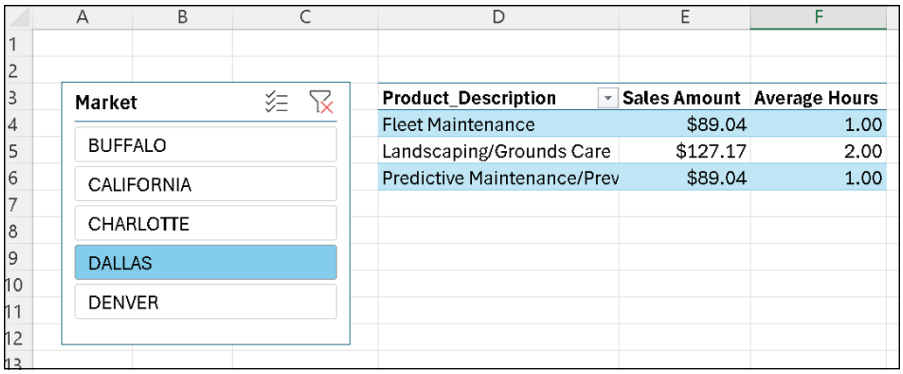


FIGURE 8-10: You want a script that dynamically creates a `Market` slicer with DALLAS selected by default.

The following script dynamically creates a slicer for the `Market` pivot field, filters the slicer to the `DALLAS` selection, and then positions the slicer next to the pivot table:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Define the source pivot table
  const mySheet = workbook.getWorksheet('PivotSheet')
  const pivotTable = mySheet.getPivotTable('MyPivot')

//Step 2: Create a slicer for Market
  let marketSlicer = mySheet.addSlicer(
    pivotTable,
    pivotTable.getHierarchy('Market').getFields()[0])
    marketSlicer.setName('MarketSlicer')

//Step 3:Get slicer items for the Market slicer
  let slicer = mySheet.getSlicer('MarketSlicer')
  let slicerItems = slicer.getSlicerItems()

//Step 4:Make sure all filters are clear
  slicer.clearFilters()

//Step 5:Start looping through each slicer item
  slicerItems.forEach(item => {

//Step 6:Set item to IsSelected if the name is DALLAS
    if (item.getName() === 'DALLAS') {
      item.setIsSelected(true);
    } else { item.setIsSelected(false) }
  });
```

```
//Step 7: Set the slicer position
  marketSlicer.setTop(20)
  marketSlicer.setLeft(10)
  marketSlicer.setHeight(200)
  marketSlicer.setWidth(150)
}
```

In Step 1 you define the source pivot table with the `pivotTable` variable. In Step 2, you use this variable to create a slicer for the `Market` field and set the name for the slicer to `MarketSlicer`. At this point you have a slicer!

In Step 3, you use the `getSlicerItems` method to store all available data items in a new variable called `slicerItems`. You will eventually use this variable in Step 5 to loop through each item. But before that, you clear any existing slicer filters in Step 4. Step 4 is not strictly necessary, but it's good practice to clear existing filters before programmatically manipulating slicer selections.

In Step 5, you start looping through each slicer item. Step 6 evaluates each item to see if the item name is equal to `DALLAS`. If the condition is met, you set the `IsSelected` property to `true`. The `Else` statement sets the `IsSelected` property to `false` for any item not meeting the condition.

Finally, Step 7 positions the slicer where you want it placed.

REMEMBER

At this point, you should have a better understanding of pivot table scripts. As you wrap up this chapter, remember that you don't need to start from scratch. Action Recorder remains the best way to generate starter code; simply record your steps, and then optimize and customize the resulting script to fit your needs.

IN THIS CHAPTER

- Understanding custom number formatting
- Working with conditional formatting
- Automating charts
- Converting visualizations into images
- Creating custom visuals

Chapter 9
Scripting Visual Elements

Visualization is the presentation of abstract concepts or data in visual terms through some sort of graphic. A trendline chart, for example, is a visualization of the abstract concept *magnitude of change over time.* In the business world, visualizations help us communicate and process the meaning of data faster than simple tables of numbers.

Excel offers business analysts a wide array of features that they can use to add visualizations to dashboards and reports: charts, number formatting, and conditional formatting to name a few. In this chapter, you explore some of the techniques for automating common visualizations in Excel reports.

REMEMBER

The code snippets used in this chapter are available in the `Chapter 9 Code` folder which can be downloaded with the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Understanding Custom Number Formatting

Custom number formatting tells Excel how a number should look in various scenarios. Number formatting syntax consists of individual number formats separated by semicolons. For example:

```
+#,##0_);(#,##0)
```

By default, any formatting to the left of the semicolon is applied to positive numbers and any formatting to the right of the semicolon is applied to negative numbers. So, with the preceding syntax, negative numbers will be formatted with parentheses, and positive numbers will be formatted with a plus sign. For example:

```
(1,500)
 +1,500
```

The script to format a number is straightforward. Simply reference the range and use the `setNumberFormat` method to define the format you'd like to use. The general pattern is:

```
function main(workbook: ExcelScript.Workbook) {

    let sheet = workbook.getWorksheet('MySheet');
    let range = sheet.getRange('D17:D22')

    range.setNumberFormat('+#,##0_);(#,##0)')

}
```

You can edit the syntax for the number format so that the numbers are formatted differently. For example, if you change the syntax to:

```
[blue]+#,##0;[red]-#,##0
```

positive numbers will be colored blue and will start with the + symbol, and negative numbers will be red and start with the – symbol. Figure 9-1 illustrates an example of customizing two ranges with two number formats.

```
function main(workbook: ExcelScript.Workbook) {

    let sheet = workbook.getWorksheet('Number Formatting');
    let range: ExcelScript.Range

    range = sheet.getRange('D17:D22')
    range.setNumberFormat('#,##0_);(#,##0)')

    range = sheet.getRange('F17:F22')
    range.setNumberFormat('[blue]+#,##0;[red]-#,##0')

}
```

	A	B	C	D	E	F
16		Before Formatting				
17		-1890		(1,890)		-1,890
18		-525		(525)		-525
19		1982		1,982		+1,982
20		2744		2,744		+2,744
21		526		526		+526
22		7419		7,419		+7,419

FIGURE 9-1: Use the `setNumberFormat` method to apply various types of number formatting to defined ranges.

A WORD ABOUT CUSTOM COLOR FORMATTING

When using colors in custom number formatting, you can call out by name only the eight Visual Basic colors. These colors make up the first eight colors of the default Excel 56-color palette: `[black]`, `[blue]`, `[cyan]`, `[green]`, `[magenta]`, `[red]`, `[white]`, and `[yellow]`. Although it's easy to specify a custom color by name, not all Visual Basic colors are pleasing. For example, Visual Basic Green is notoriously difficult to look at due to its bright neon green color. You can see for yourself by testing out this custom number syntax:

```
[Green]+0%;[Red]-0%
```

The good news is that custom number formatting will also accept any of the 56 colors in the standard color palette. Each color is represented by a number. To call up a color by number, you use `[ColorN]`, where *N* represents a number from 1 to 56.

For example, you can use `[Color10]` to present a much more acceptable green:

```
[Color10]+0%;[Red]-0%
```

So how do you know what color number to use? Well, the sample files for this chapter contain a text file called `GetVisualBasice56ColorPalette.txt`, which has a sample script to output all 56 Visual Basic colors to a new sheet called 56ColorPalette. Simply run the script, review the colors, and select the color number that works for you.

Hiding and suppressing zeroes

In addition to formatting positive and negative numbers, you can also format zero values. You do this by adding another semicolon to your custom number syntax. By default, any format syntax placed after the second semicolon is applied to any number that evaluates to zero.

For example, the following syntax applies a format that shows `n/a` for any cells that contain zeroes:

```
#,##0_);(#,##0);"n/a"
```

You can also suppress zeroes entirely. If you add the second semicolon but don't follow it with any syntax, cells containing zeroes will appear blank:

```
#,##0_);(#,##0);
```

Custom number formatting affects only the cosmetic look of the cell, not the data, as shown in Figure 9-2. The selected cell is formatted so that zeroes appear as `n/a`, but its unformatted cell contents appear in the formula bar.

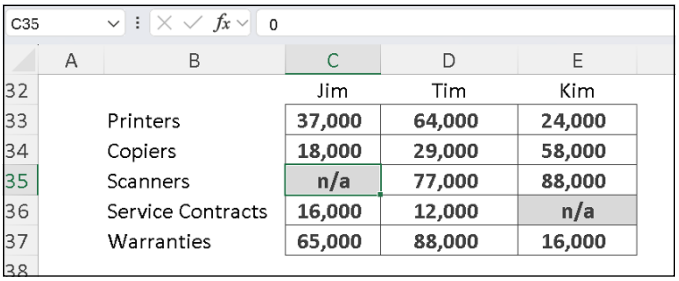


FIGURE 9-2: Cell C35 displays `n/a` but the formula bar reveals the true value beneath the custom formatted display.

Formatting numbers in thousands and millions

It's a common requirement to format very large numbers in thousands and millions to avoid inundating your audience with too many numbers. Using the following syntax will format numbers to appear in thousands:

```
#,##0,
```

If needed, you can indicate that the number is in thousands by adding `"k"` to the number syntax:

```
#,##0,"k"
```

This syntax would display your numbers as:

```
118k
318k
```

You can use this syntax to format both positive and negative numbers to appear in thousands:

```
'$#,##0,"k";–$#,##0,"k"'
```

After applying this syntax, negative numbers also appear in thousands:

```
 $118k
–$318k
```

Need to show numbers in millions? Easy. Simply add two commas to the number format syntax in the Type input box:

```
#,##0.00,, "m"
```

Note the use of the decimal places (`.00`). When converting numbers to millions, it's often useful to show additional precision points, as in 24.65 m.

TIP

Custom number formatting has advantages over other techniques to format numbers to thousands. For instance, many beginning analysts convert numbers to thousands by dividing them by 1,000 in a formula. But that changes the integrity of the number dramatically. When you perform a mathematical operation in a cell, you're changing the value represented in that cell. This method forces you to keep track of and maintain the formulas you introduced simply to achieve a cosmetic effect. Using custom number formatting avoids this issue by changing only how the number looks, keeping the actual number intact.

Formatting dates and times

Custom number formatting isn't just for numbers. You can also format dates and times. Several variations are available for formatting dates and times. Table 9-1 lists some common date and time format codes you can use with the `setNumberFormat` method.

TABLE 9-1 Common Date and Time Format Codes

Format Code	12/25/2026 7:42:53 PM Is Displayed As
`setNumberFormat ('M')`	12
`setNumberFormat ('mmm')`	Dec
`setNumberFormat ('mmmm')`	December
`setNumberFormat ('mmmmm')`	D
`setNumberFormat ('dd')`	25
`setNumberFormat ('ddd')`	Fri
`setNumberFormat ('dddd')`	Friday
`setNumberFormat ('yy')`	26
`setNumberFormat ('yyyy')`	2026
`setNumberFormat ('mmm-yy')`	Dec-26
`setNumberFormat ('dd/mm/yyyy')`	25/12/2026
`setNumberFormat ('dddd mmm yyyy')`	Friday Dec 2026
`setNumberFormat ('mm-dd-yyyy h:mm AM/PM')`	12-25-2026 7:42 PM
`setNumberFormat ('h:mm:ss AM/PM')`	7:42:53 PM

Enhancing Data with Conditional Formatting

Conditional formatting is the term given to Excel's capability to dynamically change the formatting of a value, cell, or range of cells based on a set of conditions you define. When a report has conditional formatting, you can make split-second determinations as to which values are good and which are bad, based simply on formatting.

The syntax for automating conditional formatting is predicated on the type of condition you want Excel to evaluate. You start by specifying the appropriate member of the `ExcelScript.ConditionalFormatType` enumeration. Each condition has its own interface that requires programmatic inputs, as shown in Table 9-2.

Looking at Table 9-2, you probably feel overwhelmed, and that's okay! Conditional formatting has a complicated object model with a different syntax pattern for almost every kind of use case. Even the experts don't script conditional formatting logic off the top of their head. Instead, they lean heavily on Action Recorder.

TABLE 9-2 Conditional Formatting Types

Conditional Formatting Type	Purpose and Utility	Interface Options
`cellValue`	Formats cells based on a comparison to a specified value or formula	Initiates the `ConditionalCellValueOperator` interface, letting you define one of the following comparison operators: `EqualTo`, `NotEqualTo`, `GreaterThan`, `LessThan`, `GreaterThanOrEqualTo`, `LessThanOrEqualTo`, `Between` and `NotBetween`
`presetCriteria`	Applies formatting based on common, predefined rules, such as blank cells, duplicate values, or commonly used time comparisons	Initiates the `ConditionalFormatPresetCriterion` interface, which lets you format cells based on a common evaluation criterion, such as `blankCells`, `nonBlankCells`, `duplicateValues`, `uniqueValues`, and `containsText`
`topBottom`	Formats the highest or lowest values or percentages based on defined criteria	Initiates the `ConditionalTopBottomCriterionType`, interface, which lets you format cells based on a common evaluation criterion, such as: `topItems`, `bottomItems`, `topPercent`, and `bottomPercent`
`containsText`	Formats cells based on the content of their text	Initiates the `ConditionalTextOperator` interface, which lets you specify text comparisons such as `beginsWith`, `endWith`, and `contains`
`dataBar`	Adds a horizontal bar in the cell background, visually representing the cell's value relative to others in the range	Initiates options for formatting axis, color and direction of the horizontal bars
`colorScale`	Formats cells with a color gradient across the range, with different colors representing the relative value of each cell	Initiates options for setting `ConditionalFormatColorCriterionType`, which defines the lowest, middle, and highest value in the selected range, allowing you to control how the color scale is applied
iconSet	Adds icons to cells representing each cell's value in relation to other cells in the range	Adds icons to cells based on specified criteria and value ranges

Capturing conditional formatting code with Action Recorder

Some features in Excel are just easier to script by using Action Recorder. Conditional formatting is one of the those features.

For example, suppose you want to add conditional formatting that highlights the bottom 50 percent of values in a range. You want Action Recorder to give us some starter code, and then you'll modify that code to suit your needs:

1. **If necessary, click the Automate tab in Action Recorder.**
2. **Start recording by clicking Record Actions.**
3. **Highlight a sample range of data.**
4. **Click the Home tab.**
5. **Select Conditional Formatting ➪ Top/Bottom Rules ➪ Bottom 10%, as shown in Figure 9-3.**

   The Bottom 10% dialog box appears.
6. **Change the percentage value to 50, as shown in Figure 9-4, and then click OK.**

   The Code Editor task pane appears.
7. **To stop recording, click the Stop button.**

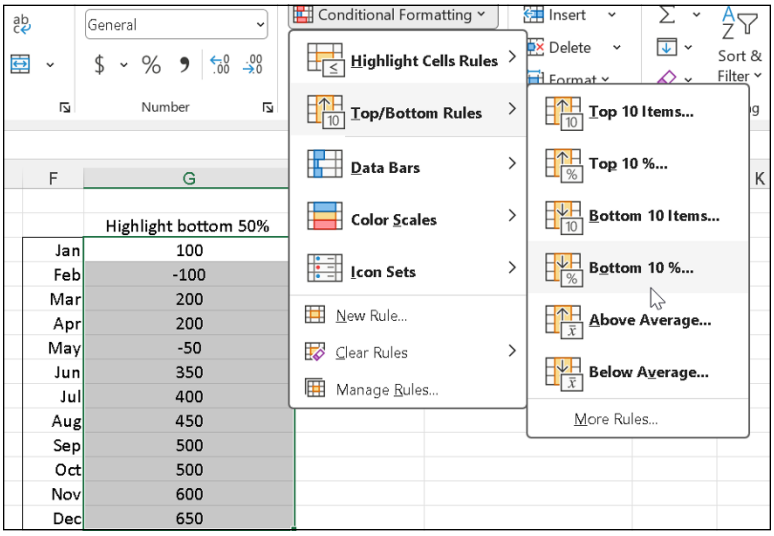


FIGURE 9-3: While recording, you highlight a range and choose the Bottom 10% conditional formatting option.

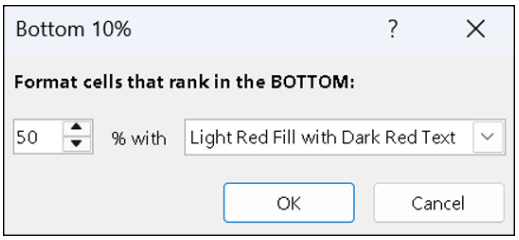


FIGURE 9-4: Change the percentage value to 50.

As you can see from Figure 9-5, Excel writes all the code you need to apply this conditional formatting. At this point, you can review the recorded code and refine the syntax to enhance readability and ensure long-term usability.

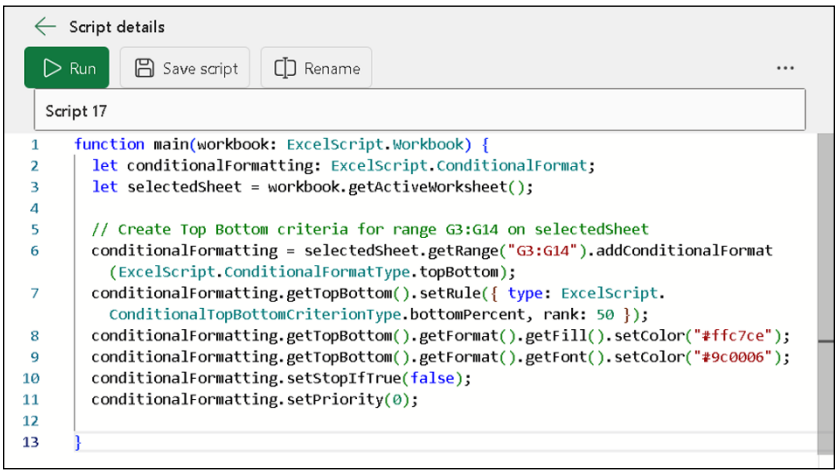


FIGURE 9-5: The resulting recorded code.

It's always a good idea to clean up recorded code. Even small changes make a difference. For example, you can shorten bulky variable names to reduce text clutter. You can move variable declarations to the top of the function to improve readability. And finally, you can create your own variables to enhance your ability to reuse the code.

Figure 9-6 shows how a few simple changes can improve recorded code and make it your own. Note how I shortened the `conditionalformatting` variable to `cf` to reduce clutter. I also brought the range declaration to the top and changed the variable for the worksheet to capture a specified sheet instead of simply capturing the active sheet. This change lets me supply any worksheet and range I want, making this code reusable.

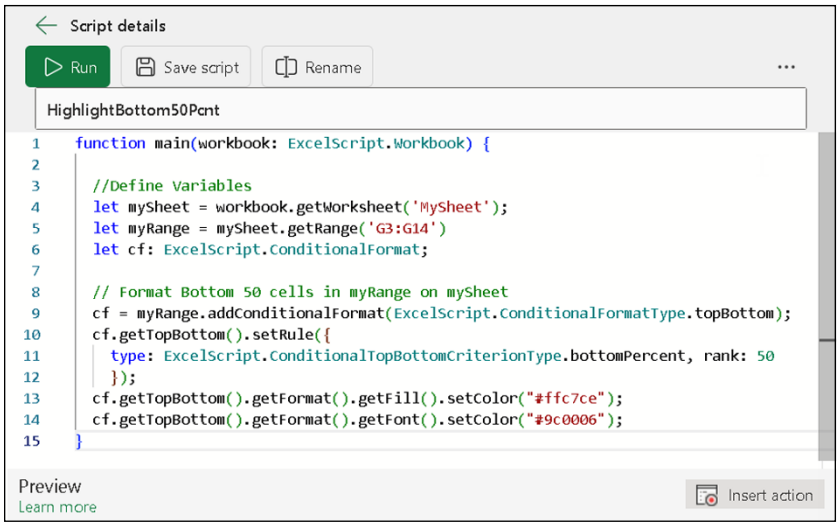


FIGURE 9-6: Refining recorded code improves readability and improves reusability.

Although this example demonstrates a specific conditional formatting rule, the steps here apply to any formatting rule you want to automate. Once you decide on a conditional format to implement, let Action Recorder capture the steps, and then clean up and optimize the code for reusability.

Conditional formatting that fails to record properly

When recording an advanced feature such as conditional formatting, you'll inevitably encounter actions that simply can't be translated into the required TypeScript code. When you record an action that can't be recorded, Excel inserts the following comment into the code:

```
// This action currently can't be recorded.
```

Data bars, color scales, and icon sets are particularly tricky for Action Recorder. Action Recorder often fails to correctly capture the complex parameters for data bars and icon sets, leaving you with a script that doesn't apply the logic you intended. As for color scale formatting, Action Recorder can't record steps to create a color scale condition (as of this writing).

Just because an action can't be recorded, however, doesn't mean that it can't be automated. The underlying Office Scripts object model is much more powerful than Action Recorder alone, and plenty of resources are available to help you write the code manually.

In that spirit, this section offers simplified code for those trickier conditional formatting features: data bars, color scales, and icon sets. Use these scripts as templates to bypass the recorder's limitations and get your automation up and running quickly.

REMEMBER

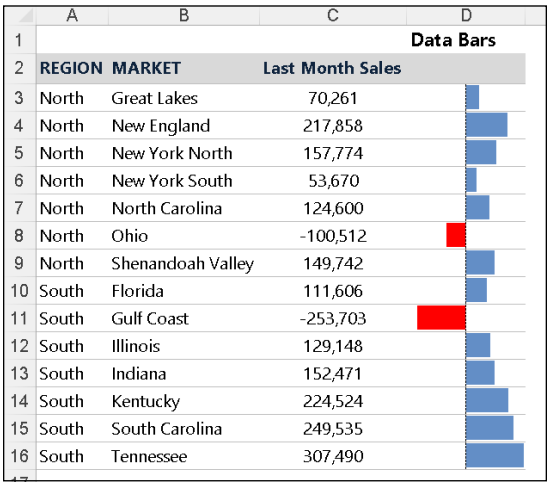
You can download the `SampleVisualization.xlsx` workbook to test out the code snippets in this section. Download the sample files for this chapter at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Examining a starter script for data bars

You can use the following script as a template for data bars. Use the `sheet` and `range` variables to define the data you want formatted. The optional settings at the bottom of the script are true-or-false settings that determine if a gradient fill is used and if values are shown with the bar:

```
function main(workbook: ExcelScript.Workbook){

//Define the sheet and range
  let sheet = workbook.getWorksheet('MySheet');
  let range = sheet.getRange('D3:D16');

//Clear any existing conditional formatting
  range.clearAllConditionalFormats();

//Define the data bar object
  let cf = range.addConditionalFormat(
  ExcelScript.ConditionalFormatType.dataBar);
  let dataBar = cf.getDataBar();

//Essential settings
dataBar.setAxisColor("#000000");
dataBar.getPositiveFormat().setFillColor("#638ec6");
dataBar.getNegativeFormat().setFillColor("#ff0000");
dataBar.setAxisFormat(ExcelScript.ConditionalDataBarAxisFormat.
   cellMidPoint)

//Optional settings
dataBar.getPositiveFormat().setGradientFill(false)
dataBar.setShowDataBarOnly(true)

}
```

When I run this script, it creates data bars in the specified range, as shown in Figure 9-7. The data bars are colored according to the hex codes specified. In this example, #638ec6 is used to color positive values blue, and #ff0000 is used to color negative values red (see the sample file for the full color experience).



	A	B	C	D
1				Data Bars
2	REGION	MARKET	Last Month Sales	
3	North	Great Lakes	70,261	
4	North	New England	217,858	
5	North	New York North	157,774	
6	North	New York South	53,670	
7	North	North Carolina	124,600	
8	North	Ohio	-100,512	
9	North	Shenandoah Valley	149,742	
10	South	Florida	111,606	
11	South	Gulf Coast	-253,703	
12	South	Illinois	129,148	
13	South	Indiana	152,471	
14	South	Kentucky	224,524	
15	South	South Carolina	249,535	
16	South	Tennessee	307,490	

FIGURE 9-7: Data bar formatting applied after running the simplified code.

Examining a starter script for color scales

You can use the following script as a template for color scales. Use the `sheet` and `range` variables to define the data you want formatted:

```
function main(workbook: ExcelScript.Workbook){

//Define the sheet and range
  let sheet = workbook.getWorksheet('MySheet');
  let range = sheet.getRange('H3:H16');

//Clear any existing conditional formatting
  range.clearAllConditionalFormats();

//Define the color scale object
  let cf = range.addConditionalFormat(
  ExcelScript.ConditionalFormatType.colorScale);
   let colorScale = cf.getColorScale();

//Essential settings for a 3-color scale (low, mid, high)
  colorScale.setCriteria({
  minimum: { color: "#F8696B", type:
  ExcelScript.ConditionalFormatColorCriterionType
```

```
    .lowestValue },

    midpoint: { color: "#FFEB84", type:
    ExcelScript.ConditionalFormatColorCriterionType
    .percentile, formula: "=50" },

    maximum: { color: "#63BE7B", type:
    ExcelScript.ConditionalFormatColorCriterionType
    .highestValue }
    });

}
```

The midpoint setting is set to use `percentile` and is configured to `50`, which ensures that color banding is applied with the 50th percentile as the midpoint. This behavior can be altered by choosing any of the alternative members of the `ExcelScript.ConditionalFormatColorCriterionType` enumeration. The following are commonly used midpoints:

```
ExcelScript.ConditionalFormatColorCriterionType.number

ExcelScript.ConditionalFormatColorCriterionType.percent

ExcelScript.ConditionalFormatColorCriterionType.formula
```

Figure 9-8 demonstrates the resulting conditional formatting for the specified range. Again, the color hex codes (#F8696B, #FFEB84 and #63BE7B) define the color scale used. It's difficult to see here in black-and-white print, but you can refer to the sample file for this chapter to see the result in full color.

	E	F	G	H
1				Color Scale
2		REGION	MARKET	Last Month Sales
3		North	Great Lakes	70,261
4		North	New England	217,858
5		North	New York North	157,774
6		North	New York South	53,670
7		North	North Carolina	124,600
8		North	Ohio	100,512
9		North	Shenandoah Valley	149,742
10		South	Florida	111,606
11		South	Gulf Coast	253,703
12		South	Illinois	129,148
13		South	Indiana	152,471
14		South	Kentucky	224,524
15		South	South Carolina	249,535
16		South	Tennessee	307,490

FIGURE 9-8: Color scale formatting applied after running the simplified code.

Examining a starter script for icon sets

The following script can be used as a template for a three-color icon set. Use the `sheet` and `range` variables to define the data you want formatted. Below the `Essential settings` comment are three blocks of code, each governing the criteria for the respective icon. The idea here is to adjust the rules and criteria for each icon to fit your scenario:

```
function main(workbook: ExcelScript.Workbook) {

//Define the sheet and range
  let sheet = workbook.getWorksheet('MySheet');
  let range = sheet.getRange('N3:N16');

//Clear any existing conditional formatting
  range.clearAllConditionalFormats();

//Define the color scale object
  let cf = range.addConditionalFormat(
  ExcelScript.ConditionalFormatType.iconSet);
  let iconSet = cf.getIconSet();

// Essential settings
// Using three-triangle icon set
iconSet.setStyle(ExcelScript.IconSet.threeTriangles);
iconSet.setCriteria([

   //Red down triangle if other conditions not met
  {formula: '0', operator:
  ExcelScript.ConditionalIconCriterionOperator
  .greaterThanOrEqual,
  type: ExcelScript.ConditionalFormatIconRuleType.number
  },

    //Yellow flat triangle if value is -03 to 2.9
  {formula: '-.03', operator:
  ExcelScript.ConditionalIconCriterionOperator
  .greaterThanOrEqual,
  type: ExcelScript.ConditionalFormatIconRuleType.number
  },

    //Green up triangle if value is .03 or higher
  {formula: '.03', operator:
  ExcelScript.ConditionalIconCriterionOperator
  .greaterThanOrEqual,
```

```
    type: ExcelScript.ConditionalFormatIconRuleType.number
    }
  ]);
  }
```

Figure 9-9 demonstrates the resulting conditional formatting for the specified range.

	J	K	L	M	N
1					Icon Set
2	REGION	MARKET	Previous Month	Current Month	Variance
3	North	Great Lakes	70,261	72,505	3.2%
4	North	New England	217,858	283,324	30.0%
5	North	New York North	157,774	148,790	-5.7%
6	North	New York South	53,670	68,009	26.7%
7	North	Ohio	100,512	98,308	-2.2%
8	North	Shenandoah Valley	149,742	200,076	33.6%
9	South	South Carolina	249,535	229,473	-8.0%
10	South	Florida	111,606	136,104	22.0%
11	South	Gulf Coast	253,703	245,881	-3.1%
12	South	Illinois	129,148	131,538	1.9%
13	South	Indiana	152,471	151,699	-0.5%
14	South	Kentucky	224,524	225,461	0.4%
15	North	North Carolina	124,600	130,791	5.0%
16	South	Tennessee	307,490	268,010	-12.8%
17					

FIGURE 9-9: Icon set formatting applied after running the simplified code.

Automating Charts

Nothing says "Excel reporting" quite like the classic chart. It's the ultimate tool for highlighting trends, spotting differences, and communicating data faster. So it's no surprise that Office Scripts offers a robust object model to automate charts. In this section, you break down the chart object model and explore scripts for common real-world reporting tasks.

Understanding the chart object model

A *chart* is an object that lives in a `Charts` collection. Every worksheet has its own `Charts` collection. You tap into this collection using the `getCharts` method of the `worksheet` object. You can use the `length` property to count the number of charts in the collection.

The following code snippet outputs the count of all the charts in `Sheet1`:

```
function main(workbook: ExcelScript.Workbook) {

  const sheet = workbook.getWorksheet('Sheet1');
  const myCharts = sheet.getCharts();
  console.log (myCharts.length);

}
```

You reference a chart by using the `getChart` method. The following sample script demonstrates two ways to point to a specific chart:

```
function main(workbook: ExcelScript.Workbook) {

  const sheet = workbook.getWorksheet('Components');

  //get chart from the collection by index number
  const Chart1 = sheet.getCharts()[0]

  //get chart by name
  const Chart2 = sheet.getChart('Revenue Trend')

}
```

You can point to a chart by its index number in the collection or its name. In this example, you get the first chart in the worksheet (the first chart is always in position 0). Then you get a chart by name, specifically a chart called `Revenue Trend`.

Once you have a chart loaded into a variable, you can see its available methods by typing the dot operator after the variable name (see Figure 9-10). These methods enable you to script any chart action that you can do manually: resize the chart, add a title, add a legend, and so on.

To add a new chart to the worksheet, you use the `addChart` method, providing two arguments: a chart type and a range. For example, the following code snippet creates a line chart from the data in cells A1:A10:

```
Sheet.addChart(
ExcelScript.ChartType.line,
Sheet.getRange(sheet.getRange('A1:A10')
);
```

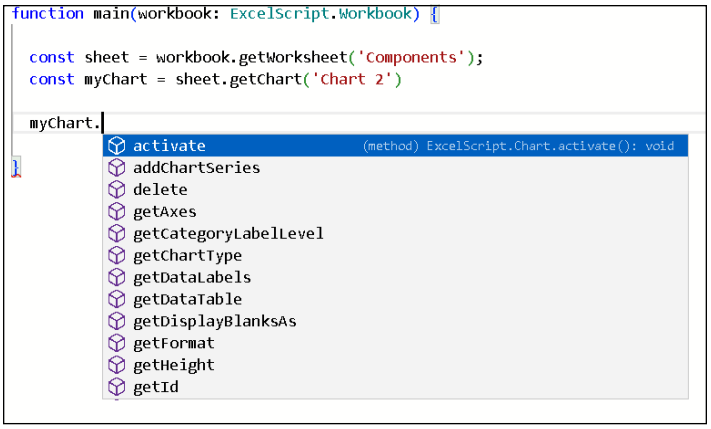


FIGURE 9-10: Load a chart into a new variable to see all the available methods.

You don't have to memorize the chart types. As Figure 9-11 illustrates, typing the dot operator activates IntelliSense, which displays the available options.

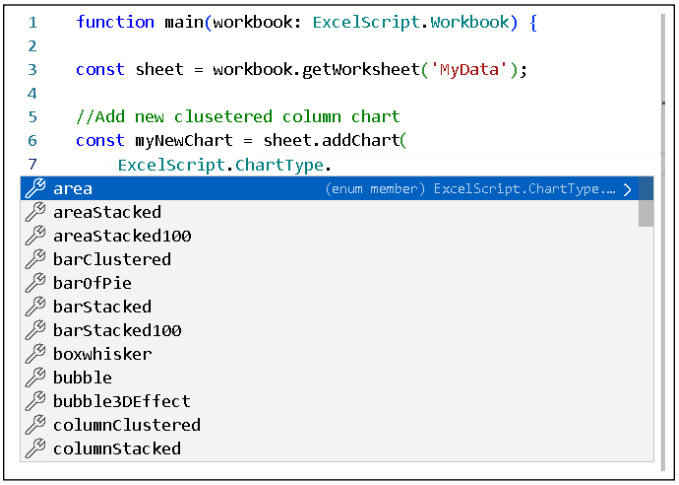


FIGURE 9-11: IntelliSense will show you all the available chart types.

The next sample script demonstrates the general pattern for adding a chart:

```
function main(workbook: ExcelScript.Workbook) {

//Define the data
const sheet = workbook.getWorksheet('MyData');
const myRange = sheet.getRange('A2:B13')
```

```
//Add the new line chart
const myNewChart = sheet.addChart(
      ExcelScript.ChartType.line, myRange)

//Name the new chart
      myNewChart.setName('Revenue Trend');
}
```

First you define the range that holds your data (`myRange` in this case). Next, you use that range in the `addChart` method. Finally, you give the new chart a friendly name. This last step is optional because Excel automatically assigns a name. However, explicitly naming your chart will make it easier to use the `getChart` method.

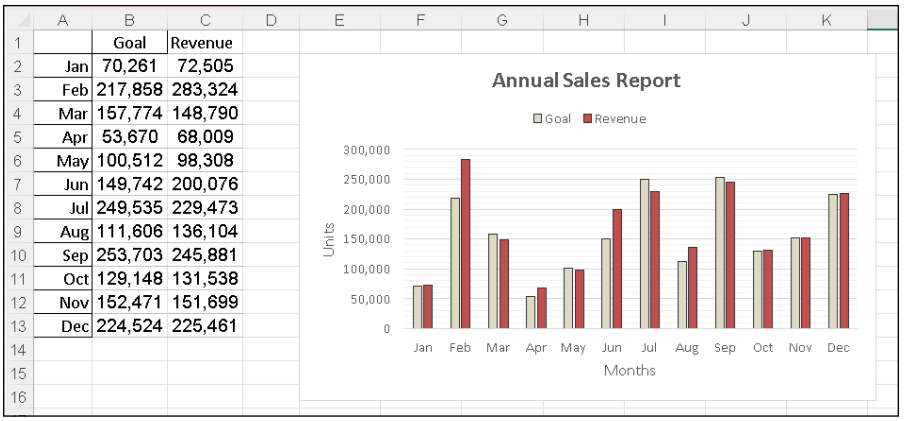
Working the chart elements

Excel applies default formatting and colors to charts when they're created, but you can customize some chart elements, such as titles, colors, legends, and gridlines. In this section, you work with some of the more common chart elements.

For this look into chart elements, consider the following script, which creates the chart shown in Figure 9-12:

```
function main(workbook: ExcelScript.Workbook) {

const sheet = workbook.getWorksheet('MyData');
const myRange = sheet.getRange('A1:C13')

//Add a new clustered column chart
const myChart = sheet.addChart(
      ExcelScript.ChartType.columnClustered, myRange)

//Name the new chart
  myChart.setName('Chart 1');

//Set the chart position
  myChart.setWidth(400)
  myChart.setHeight(200)
  myChart.setPosition('E2')
```

```
//Format the chart title
let myChartTitle = myChart.getTitle()
    myChartTitle.setVisible(true)
    myChartTitle.setText('Annual Sales Report');
    myChartTitle.getFormat().getFont().setBold (true);
    myChartTitle.getFormat().getFont().setSize(14);

//Set the legend (top, bottom, left, right, etc.)
let myChartLegend = myChart.getLegend()
    myChartLegend.setVisible(true)
    myChartLegend.setPosition(
    ExcelScript.ChartLegendPosition.top);

//Format the y axis
let myChartYaxis = myChart.getAxes().getValueAxis()
    myChartYaxis.getTitle().setText('Units')
    myChartYaxis.getMajorGridlines().setVisible(true)
    myChartYaxis.getMinorGridlines().setVisible(true)

//Format the x axis
let myChartXaxis = myChart.getAxes().getCategoryAxis()
    myChartXaxis.getTitle().setText('Months')
    myChartXaxis.getMajorGridlines().setVisible(false)
    myChartXaxis.getMinorGridlines().setVisible(false)

//Set the series format
let goalSeries = myChart.getSeries()[0]
    goalSeries.getFormat()
    .getFill().setSolidColor('#DDD9C4')
    goalSeries.getFormat().getLine().setColor('#333333')

let revSeries = myChart.getSeries()[1]
    revSeries.getFormat()
    .getFill().setSolidColor('#C0504D')
    revSeries.getFormat().getLine().setColor('#333333')
}
```



	Goal	Revenue
Jan	70,261	72,505
Feb	217,858	283,324
Mar	157,774	148,790
Apr	53,670	68,009
May	100,512	98,308
Jun	149,742	200,076
Jul	249,535	229,473
Aug	111,606	136,104
Sep	253,703	245,881
Oct	129,148	131,538
Nov	152,471	151,699
Dec	224,524	225,461

FIGURE 9-12: The final chart with formatting applied.

Setting the chart position

When Excel creates a chart, it assigns it a default height, width, and starting position. However, these defaults rarely align perfectly with your reporting layout. You can use the `setLeft`, `setTop`, `setHeight`, and `setWidth` methods to adjust the size and position of your chart:

```
myChart.setWidth(400)
myChart.setHeight(200)
myChart.setLeft(200)
myChart.setTop(50)
```

An alternative way to set the position of your chart is to use the `setPosition` method:

```
myChart.setPosition('E2')
```

With this method, you can anchor your chart to the top-left corner of a specific cell. If you resize the columns or rows later, the chart stays locked to that cell.

Formatting the chart title

The `getTitle` method of the `Chart` object shows you all the underlying methods needed to customize the chart title.

You can display or hide the title by using `setVisible(true)` or `setVisible(false)`, respectively.

The `setText` method defines the title text, while `getFormat` displays all the methods needed to format the title font, size, background, and color:

```
let myChartTitle = myChart.getTitle()
  myChartTitle.setVisible(true)
  myChartTitle.setText('Annual Sales Report');
  myChartTitle.getFormat().getFont().setBold (true);
  myChartTitle.getFormat().getFont().setSize(14);
```

Setting the legend

Use the `getLegend` method to configure the legend for your chart:

```
let myChartLegend = myChart.getLegend()
  myChartLegend.setVisible(true)
  myChartLegend.setPosition(
  ExcelScript.ChartLegendPosition.top);
```

You can display or hide the legend by using `setVisible(true)` or `setVisible(false)`, respectively.

The `setPosition` method initiates the `ExcelScript.ChartLegendPosition` enumerations whose members define where on the chart the legend should snap into place:

- `ExcelScript.ChartLegendPosition.top`: Aligns the legend above the plot area
- `ExcelScript.ChartLegendPosition.bottom`: Aligns the legend below the plot area (standard default)
- `ExcelScript.ChartLegendPosition.left`: Places the legend to the left of the plot area
- `ExcelScript.ChartLegendPosition.right`: Places the legend to the right of the plot area
- `ExcelScript.ChartLegendPosition.corner`: Positions the legend in the top-right corner of the chart frame

Formatting the x-axis and y-axis

In Office Scripts, axes are managed via the `getAxes()` collection. The `ValueAxis` object handles numerical data on the y-axis, while the `CategoryAxis` object handles labels on the x-axis.

To customize the x- and y-axes, you first call the `get` method for the respective axis, and then drill down into its various properties, such as title or gridlines:

```
let myChartYaxis = myChart.getAxes().getValueAxis()
    myChartYaxis.getTitle().setText('Units')
    myChartYaxis.getMajorGridlines().setVisible(true)
    myChartYaxis.getMinorGridlines().setVisible(true)

let myChartXaxis = myChart.getAxes().getCategoryAxis()
    myChartXaxis.getTitle().setText('Months')
    myChartXaxis.getMajorGridlines().setVisible(false)
    myChartXaxis.getMinorGridlines().setVisible(false)
```

Setting series formats

To get to any data series in Office Scripts, you call the `getSeries()` method to first get the array of all the data series in the chart. Then you specify which series you want by using its position number. The first series in a chart is always in position 0. So, to get the first series in the chart, you would use `myChart.getSeries()[0]`. To get the second series, use `myChart.getSeries()[1]`. And so on.

Once you get the series in focus, you can customize it using `getformat()` sub-methods. A few examples follow:

- `getFill().setSolidColor`: Sets the color for the body of columns, bars, and area charts
- `getLine().setColor`: Sets the color for the outline of columns, bars, or the line in a line chart
- `setWeigh`: Adjusts the thickness in points and lines

The following sample script illustrates how to format two data series in a chart:

```
let Series1 = myChart.getSeries()[0]
Series1.getFormat().getFill().setSolidColor('#DDD9C4')
Series1.getFormat().getLine().setColor('#333333')

let Series2 = myChart.getSeries()[1]
Series2.getFormat().getFill().setSolidColor('#C0504D')
Series2.getFormat().getLine().setColor('#333333')
```

Overcoming chart recording challenges

It's only natural to think you can lean on Action Recorder to help with chart formatting. After all, why not let Excel do the heavy lifting by recording your manual formatting steps?

Unfortunately, Action Recorder often falls short when it comes to chart formatting, frequently triggering the "this action can't be recorded" notification. Although Action Recorder does a good job capturing structural actions, such as creating a chart or changing the chart type, it tends to fail on actions that fine-tune or format granular chart options. This failure occurs because many formatting changes are driven by user interface interactions that don't map cleanly to the Office Scripts object model. In addition, some chart actions depend on which chart element is currently selected, which Action Recorder can't always determine.

Don't be discouraged when Action Recorder fails. Instead, use it as a starting point and bridge the gap by using these essential resources:

- **Microsoft Office Scripts documentation:** Official documentation doesn't always offer a copy-paste solution, but it's the best place to find the specific methods and enumerations required for your task. The documentation provides a technical blueprint for what the object model can do.
- **Community forums:** Platforms like Stack Overflow connect you with developers who have likely faced the same hurdles. You can often find code snippets that reveal the syntax needed to solve your problem.
- **AI coding assistants:** Tools like ChatGPT, Claude, and Gemini are invaluable for connecting the dots. Try pasting your recorded script directly into the chat with a prompt like: "I have a script that creates a chart. How do I modify it to set the value axis font size to 12 and make the third series red?" Most of the time, you'll get functional code that you can immediately build upon.

Converting Visualizations into Images

Often, your reporting solutions will be based on analyses spread across multiple worksheets. The goal is often the creation of a final presentation layer that pulls the best visuals into one view.

Without scripting, Excel users rely on linked pictures to create pictures of ranges to place on a single sheet. Office Scripts offers an equivalent feature with the `getImage` method. This method converts a given range into a Base64-encoded

string. The sister method, `addImage`, renders the string back into a physical image on a target worksheet.

To illustrate, consider the following sample script. Here, you load range B2:L14 into the `picRange` object. Then you create an image object with `picRange.getImage`. Finally, you output that image to the sheet by using the `addImage` method:

```
function main(workbook: ExcelScript.Workbook) {

//Define the range to capture
let sheet = workbook.getWorksheet("Waffle Charts")
let picRange = sheet.getRange('B2:L14')

//Load the range into the image object
let image = picRange.getImage();

//Add the image to the sheet
let output1 = sheet.addImage(image);

//Position the image
output1.setLeft(50);
output1.setTop(200);

}
```

As Figure 9-13 demonstrates, the image will be an exact picture of the cells that encompass the specified range, including any object that is in the range (such as a chart, a text box, or conditional formatting).

TIP

The images created with the `getImage` method are static. They don't dynamically update when the source range changes (unlike images created with Excel's linked picture functionality). The best you can do is explicitly delete and recreate the converted images when you know that the source range will change.

When building a consolidated dashboard, you often run into screen real estate issues. Column widths in one range might ruin the layout of a summary chart in another range. By converting ranges into images, you essentially decouple visuals from the grid. Images become like floating tiles on a canvas, making it easy to stack, resize, and align mismatched elements into a single presentation layer.

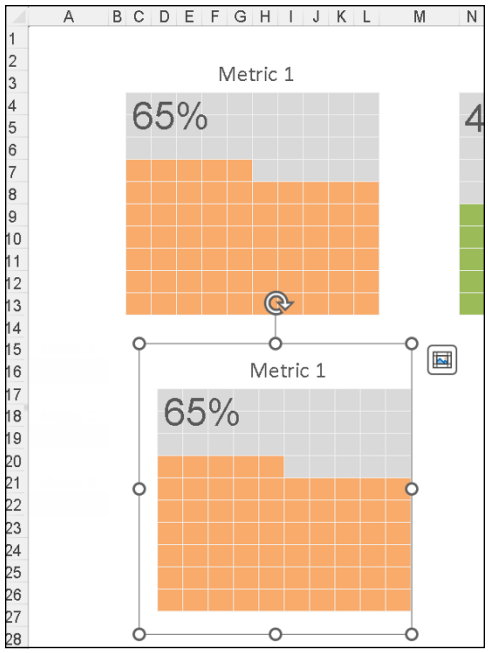
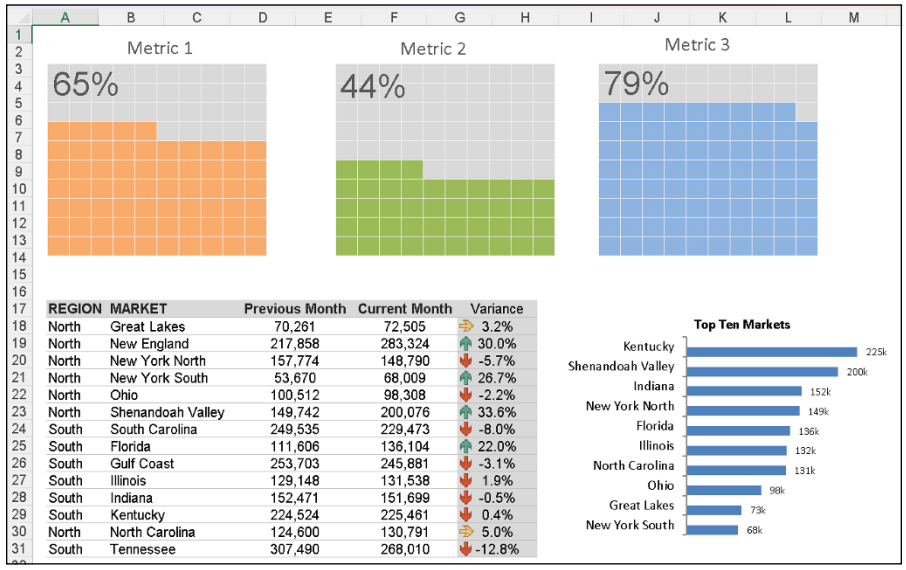


FIGURE 9-13: The `getImage` method creates an exact image of the contents of a given range.

The dashboard view shown in Figure 9-14 was created from four disparate ranges by using the following code:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Delete the Dashboard sheet if it exists
workbook.getWorksheet('Dashboard')?.delete();

//Step 2: Create an empty Dashboard sheet
let dashboardSheet = workbook.addWorksheet('Dashboard')

//Step 3: Get all the ranges we want included
let sheet = workbook.getWorksheet("Components")
let sheet2 = workbook.getWorksheet("Waffle Charts")
let picRange1 = sheet.getRange('A1:E15')
let picRange2 = sheet.getRange('E18:K32')
let picRange3 = sheet.getRange('A35:F52')
let picRange4 = sheet2.getRange('B2:AJ14')

//Step 4: Capture the images for each range
let image1 = picRange1.getImage();
let image2 = picRange2.getImage();
```

```
let image3 = picRange3.getImage();
let image4 = picRange4.getImage();

//Step 5: Output each image to the Dashboard sheet
let output1 = dashboardSheet.addImage(image1);
let output2 = dashboardSheet.addImage(image2);
let output3 = dashboardSheet.addImage(image3);
let output4 = dashboardSheet.addImage(image4);

//Step 6: Arrange the images
output1.setLeft(10); output1.setTop(200);
output2.setLeft(375); output2.setTop(200);
output3.setLeft(10); output3.setTop(400);
output4.setLeft(0); output4.setTop(0);

//Step 7: Go to the new Dashboard sheet
dashboardSheet.activate()
dashboardSheet.setShowGridlines(false)

}
```



REGION	MARKET	Previous Month	Current Month	Variance
North	Great Lakes	70,261	72,505	3.2%
North	New England	217,858	283,324	30.0%
North	New York North	157,774	148,790	-5.7%
North	New York South	53,670	68,009	26.7%
North	Ohio	100,512	98,308	-2.2%
North	Shenandoah Valley	149,742	200,076	33.6%
South	South Carolina	249,535	229,473	-8.0%
South	Florida	111,606	136,104	22.0%
South	Gulf Coast	253,703	245,881	-3.1%
South	Illinois	129,148	131,538	1.9%
South	Indiana	152,471	151,699	-0.5%
South	Kentucky	224,524	225,461	0.4%
North	North Carolina	124,600	130,791	5.0%
South	Tennessee	307,490	268,010	-12.8%

FIGURE 9-14: An automated dashboard presentation layer.

Steps 1 and 2 delete any existing `Dashboard` sheet, starting from scratch with a new empty worksheet.

Step 3 captures all the ranges you want in the final presentation layer. Then in Step 4, you create image objects for all the captured ranges.

Step 5 adds all the images to the `Dashboard` sheet, and Step 6 positions each image at its final place on the dashboard. Step 7 rounds out the script by hiding the gridlines on the final dashboard.

As mentioned, images won't be updated if the underlying data changes. You'll need to re-run the script to create the images to capture any updated data.

Creating Custom Visuals

With a little creativity, you can leverage the Office Scripts object model to create custom visuals that go beyond standard charts and graphs. In this section, you explore two examples of custom visuals that introduce innovative ways to display your analysis.

Creating a bullet graph

A *bullet graph* is a type of chart developed by visualization expert Stephen Few to serve as a replacement for dashboard gauges and meters. He developed bullet graphs so you could clearly display multiple layers of information without occupying a lot of space on a dashboard.

A bullet graph, as shown in Figure 9-15, contains a single performance measure (such as revenue value), a horizontal marker representing a comparative measure (such as target or plan), and qualitative ranges (such as Poor, Fair, and Good).

Figure 9-16 illustrates what that source data table needs to contain. Note that the starting table contains data for the three main parts of the bullet graph.

The first four values in the dataset (Poor, Fair, Good, and Very Good) make up the qualitative range. You don't need to have four values; you can have as many or as few as you need. The fifth value (Value) creates the performance bar. The sixth value (Target) makes the target marker.

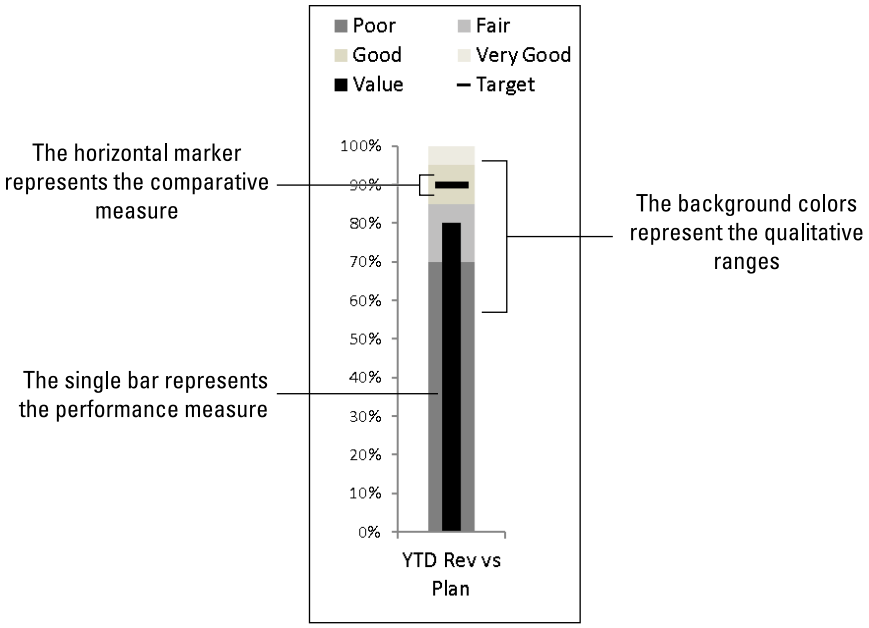


FIGURE 9-15: The parts of a bullet graph.

	A	B
1		YTD Rev vs Plan
2	Poor	70%
3	Fair	15%
4	Good	10%
5	Very Good	5%
6	Value	80%
7	Target	90%

FIGURE 9-16: The starting data needed for a bullet graph.

As you can see in Figure 9-17, you can include the data for multiple metrics to create a bullet graph that displays a column for each metric.

Since Excel doesn't have a native bullet graph option, you must engineer your own by repurposing a stacked column chart. The process involves a sequence of layering and formatting steps that create the overlapping data series visual.

The following code demonstrates how a script can help automate the creation of a bullet chart. This script is a function called `createBulletChart`. Looking at the function signature, you can see it expects a `workbook`, a `worksheet`, and a `range` object:

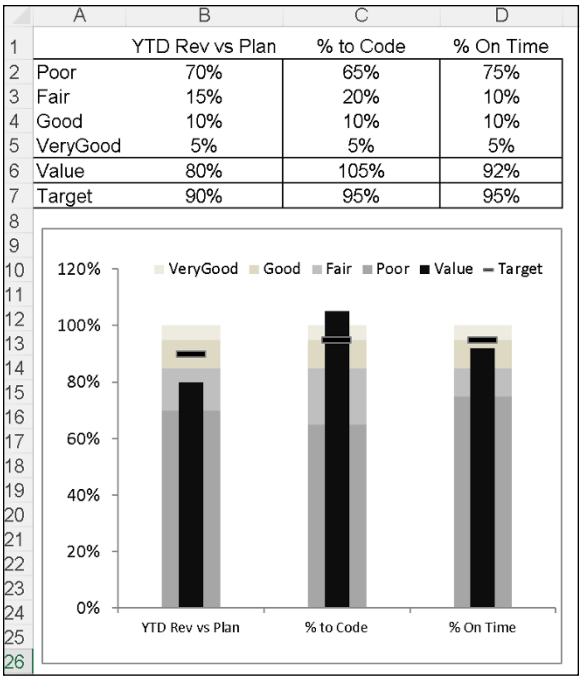


FIGURE 9-17: Your starting table can contain the data for multiple metrics.

```
function createBulletChart(
  workbook: ExcelScript.Workbook,
  worksheet: ExcelScript.Worksheet,
  range: ExcelScript.Range
  )
{

//Step 1: Create a stacked column chart
let chart = worksheet.addChart(
    ExcelScript.ChartType.columnStacked, range);
//Switch to rows
  chart.setPlotBy(ExcelScript.ChartPlotBy.rows);

//Step 2: Put the series[4] (Value) on secondary axis
const series = chart.getSeries();
series[4].setAxisGroup(
ExcelScript.ChartAxisGroup.secondary);


/*Step 3: Change the chart type for series[5] (Target)
Put on the secondary axis
Set style to no line and a dash marker*/
```

```
series[5].setChartType(
ExcelScript.ChartType.lineMarkers);
series[5].setAxisGroup(
ExcelScript.ChartAxisGroup.secondary);
series[5].getFormat().getLine().
setLineStyle(ExcelScript.ChartLineStyle.none);
series[5].setMarkerStyle(
ExcelScript.ChartMarkerStyle.dash);
series[5].setMarkerSize(15);
series[5].setMarkerBackgroundColor("#C0504D");
series[5].setMarkerForegroundColor("#C0504D");

/*Step 4: Set width of series[4] (Value)
to be smaller than the rest and
then set the fill color for each series */
series[4].setGapWidth(30);
series[0].getFormat().getFill().setSolidColor("#808080");
series[1].getFormat().getFill().setSolidColor("#D9D9D9");
series[2].getFormat().getFill().setSolidColor("#DDD9C4");
series[3].getFormat().getFill().setSolidColor("#EEECE1");
series[4].getFormat().getFill().setSolidColor("#404040");

//Step 5: Add a legend and final formatting
chart.getLegend().setPosition(
ExcelScript.ChartLegendPosition.right);
chart.getTitle().setText('Performance vs Target')
chart.getAxes()
.getValueAxis(ExcelScript.ChartAxisGroup.primary)
.setVisible(false);
chart.setLeft(38.25); chart.setTop(126.5);
chart.setWidth(250); chart.setHeight(216);

}
```

Step 1 uses the `range` object to create an initial stacked column chart on the specified `worksheet` object.

Step 2 places the `Value` series on the secondary axis, while Step 3 does the same for the `Target` series.

Step 3 changes the chart type for the `Target` series to the line with marker chart type. Step 3 finishes by displaying the `Target` series as a dash marker with no line.

Step 4 changes the gap width of the `Value` series so that it is narrower than the other bars. Then each series is given a separate color.

Step 5 finishes by adding a legend and positioning the chart.

Figure 9-18 demonstrates how to call this `createBulletChart` function. Simply provide the needed worksheet and range parameters.

By creating a reusable function, you expanded Excel's native capabilities. Now, any time you want a bullet graph, you just call the `createBulletChart` function with a new data range. This is like adding a new chart type to Excel!

Bullet Chart Example

```
function main(workbook: ExcelScript.Workbook)
{
  let mySheet = workbook.getWorksheet('Bullet Chart')
  let myData = mySheet.getRange('A1:D7')

  createBulletChart(workbook, mySheet, myData)

}


function createBulletChart(
  workbook: ExcelScript.Workbook,
  worksheet: ExcelScript.Worksheet,
  range: ExcelScript.Range
  ) {

```

FIGURE 9-18: This function can be reused to create a bullet graph for any data range you specify.

Creating a waffle chart with conditional formatting

A *waffle chart* is an interesting visualization that helps display progress toward a goal. As you can see in Figure 9-19, a waffle chart is basically a square divided into a 10×10 grid. Each grid box represents 1 percent toward a goal of 100 percent. The number of colored or shaded grid boxes is determined by the associated metric. This kind of chart is an effective option when you want to add an interesting visualization to your dashboard without distorting the data or taking up too much dashboard real estate.

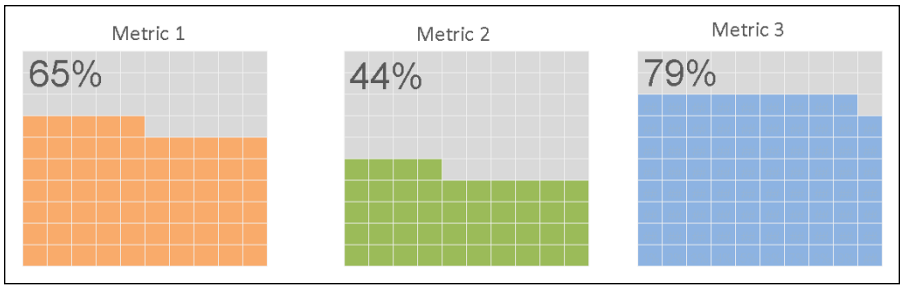


FIGURE 9-19: Three waffle charts side by side.

A waffle chart is built on a 10×10 grid of percentages that range from 1 percent to 100 percent. Figure 9-20 shows the initial setup.

	A	B	C	D	E	F	G	H	I	J	K
1											
2											
3	Cell to hold data value>>	**75%**									
4	10X10 grid >>	0.91	0.92	0.93	0.94	0.95	0.96	0.97	0.98	0.99	1
5		0.81	0.82	0.83	0.84	0.85	0.86	0.87	0.88	0.89	0.9
6		0.71	0.72	0.73	0.74	0.75	0.76	0.77	0.78	0.79	0.8
7		0.61	0.62	0.63	0.64	0.65	0.66	0.67	0.68	0.69	0.7
8		0.51	0.52	0.53	0.54	0.55	0.56	0.57	0.58	0.59	0.6
9		0.41	0.42	0.43	0.44	0.45	0.46	0.47	0.48	0.49	0.5
10		0.31	0.32	0.33	0.34	0.35	0.36	0.37	0.38	0.39	0.4
11		0.21	0.22	0.23	0.24	0.25	0.26	0.27	0.28	0.29	0.3
12		0.11	0.12	0.13	0.14	0.15	0.16	0.17	0.18	0.19	0.2
13		0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09	0.1

FIGURE 9-20: The initial setup needed for a waffle chart.

A conditional formatting rule drives the visualization. The rule colors each cell in the 10×10 grid if the cell value is less than or equal to the value shown in the input cell (B2 in this example). Figure 9-21 illustrates what the formatting rule looks like.

The following code automates the creation of a waffle chart. This script is a function called `createWaffleChart` that requires three parameters, `workbook`, `sheetName`, and `rawValue`:

```
function createWaffleChart(
  workbook: ExcelScript.Workbook,
  sheetName: string,
  rawValue: number)
{
```

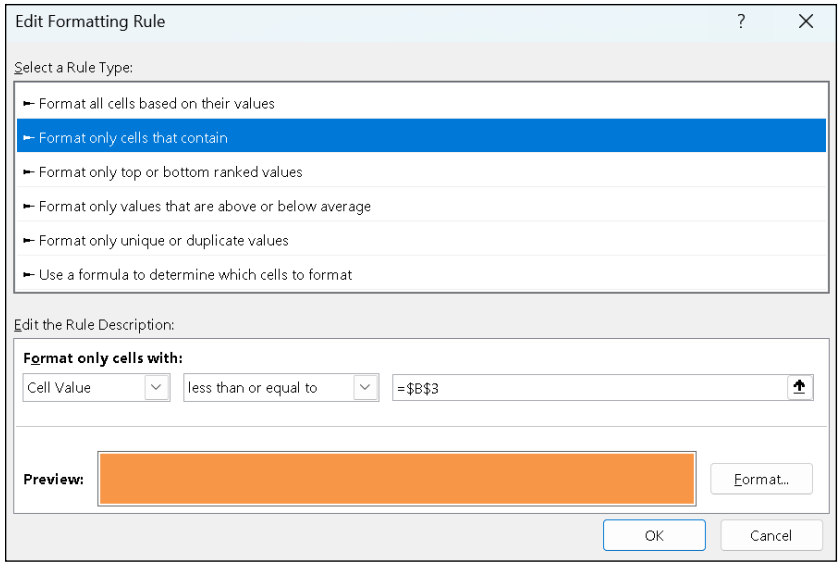


FIGURE 9-21: The conditional formatting rule that drives the waffle chart visual.

```
//Step1: Add a new worksheet for the sheetName given
workbook.getWorksheet(sheetName)?.delete();
let waffleSheet = workbook.addWorksheet(sheetName);
waffleSheet.activate()
waffleSheet.setShowGridlines(false)


//Step2: Fill and format input cell 'B3'
let rawValuecell = waffleSheet.getRange('B3');
rawValuecell.setValue(rawValue);
rawValuecell.setNumberFormatLocal("0%");
rawValuecell.getFormat().getFont().setColor("#595959");

let formatRange =
rawValuecell.getResizedRange(0,1).getFormat();
formatRange.setHorizontalAlignment(
ExcelScript.HorizontalAlignment.centerAcrossSelection);
formatRange.getFill().setColor("#D9D9D9");
formatRange.getFont().setSize(12);
formatRange.getFont().setBold(true);
formatRange.setColumnWidth(20);
```

```
//Step3: Create the 10X10 grid
const data = [
[.91, .92, .93, .94, .95, .96, .97, .98, .99, 1],
[.81, .82, .83, .84, .85, .86, .87, .88, .89, .90],
[.71, .72, .73, .74, .75, .76, .77, .78, .79, .80],
[.61, .62, .63, .64, .65, .66, .67, .68, .69, .70],
[.51, .52, .53, .54, .55, .56, .57, .58, .59, .60],
[.41, .42, .43, .44, .45, .46, .47, .48, .49, .50],
[.31, .32, .33, .34, .35, .36, .37, .38, .39, .40],
[.21, .22, .23, .24, .25, .26, .27, .28, .29, .30],
[.11, .12, .13, .14, .15, .16, .17, .18, .19, .20],
[.01, .02, .03, .04, .05, .06, .07, .08, .09, .10]
];
const endRow = data.length;
const endCol = data[0].length;
const waffleRange =
waffleSheet.getRangeByIndexes(3, 1, endRow, endCol);
waffleRange.setValues(data);


//Step4: Format the grid
let waffleFormat = waffleRange.getFormat()
waffleFormat.setColumnWidth(15);
waffleFormat.getFill().setColor("#D9D9D9");
waffleFormat.getFont().setColor("#D9D9D9");
waffleFormat.getRangeBorder(
ExcelScript.BorderIndex.insideVertical)
.setStyle(ExcelScript.BorderLineStyle.continuous);
waffleFormat.getRangeBorder(
ExcelScript.BorderIndex.insideVertical).setColor("#FFFFFF");
waffleFormat.getRangeBorder(
ExcelScript.BorderIndex.insideHorizontal)
.setStyle(ExcelScript.BorderLineStyle.continuous);
waffleFormat.getRangeBorder(
ExcelScript.BorderIndex.insideHorizontal).setColor("#FFFFFF");


//Step 5: Create the conditional formatting rule
let cf = waffleRange.addConditionalFormat(
ExcelScript.ConditionalFormatType.cellValue);
cf.getCellValue().setRule(
```

```
{
operator:   ExcelScript.ConditionalCellValueOperator
.lessThanOrEqual,formula1: '=$B$3'
});

//Step 6:  Set the color for squares
cf.getCellValue().getFormat()
.getFill().setColor('#f9ab6b');
cf.getCellValue().getFormat()
.getFont().setColor('#f9ab6b');

}
```

Step 1 creates a new worksheet and gives it the name provided by the `sheetName` parameter. Step 2 fills the input cell (B3) with the value provided by the `rawValue` parameter, and then formats it.

In Step 3 you create a data array with the 10×10 grid values. You then fill the `endRow` variable with the number of rows in the data array and fill the `endColumn` variable with the number of columns in the data array. Using those variables, you define the `waffleRange` as:

```
waffleSheet.getRangeByIndexes(3, 1, endRow, endCol)
```

This statement essentially says start on cell B3 (row 3 column 1) and expand the range by the same number of rows and columns in the data array. Once `waffleRange` is defined, you use the `setValues` method to create the grid.

Step 4 formats the grid with default fill and border colors. Step 5 creates the conditional formatting rule, and Step 6 defines the grid colors for those cells that meet the rule.

Figure 9-22 demonstrates how to call the `createWaffleChart` function. Simply provide the needed sheet name and raw value parameters.

Again, by creating a reusable function, you just added a new visualization option to your toolkit. Each time you want to employ this visualization, you simply call the `createWaffleChart` function.

FIGURE 9-22: This function can be reused to create a waffle chart on a specified sheet with the specified value.

Waffle Chart Example

```
function main(workbook: ExcelScript.Workbook) {

  //Give sheet name and a decimal number
  createWaffleChart(workbook, 'WaffleSheet', .75)

}
function createWaffleChart(
  workbook: ExcelScript.Workbook,
  sheetName: string,
  rawValue: number
) {

  //Step1: Add new worksheet for the sheetname given
  workbook.getWorksheet(sheetName)?.delete();
```

4 Going Beyond Excel with Power Automate

IN THIS PART . . .

Get an introduction to the world of Power Automate and create your first cloud workflow.

Explore techniques for distributing Excel data by email.

Create and manipulate SharePoint files by using Power Automate.

IN THIS CHAPTER

- Understanding Power Automate requirements
- Exploring the Power Automate interface
- Building a flow
- Adding parameters to flows
- Avoiding the limitations of Power Automate

Chapter 10
Introducing Power Automate

Organizations today operate in a broad ecosystem of cloud applications. Employees work across a variety of devices, from laptops to tablets and touchscreens. Sharing information is no longer about passing files around with email; it's about integrated reporting that runs on an organization's cloud platform.

Power Automate supports today's cloud-based reporting systems by providing a centralized platform for building intelligent workflows. In this chapter, I introduce Power Automate and show you how to leverage its capabilities to integrate Excel Office Scripts into automated reporting solutions.

REMEMBER

The code snippets used in this chapter are available in the `Chapter 10 Code` folder, which you can download at `www.dummies.com/go/microsoft365exceloffice scriptsfd`.

Going with the Flow

A *workflow*, in the context of data, is a structured sequence of steps that moves data from one stage of processing to the next. Think about the manual steps you take to create a report. You collect data; clean, validate, shape, and transform the data; and finally deliver an output. That process is a workflow.

Power Automate is a cloud-based automation service that enables you to create automated workflows (or *flows*). When paired with Office Scripts, Power Automate flows let you integrate your custom scripts into broader reporting processes. For example, a Power Automate flow can pull a file from SharePoint, send it through an Office Scripts for data transformation or analysis, and then deliver the processed results as a new file in an email to another user.

Exploring the benefits of flows

One of the most compelling benefits of automated cloud flows is the reduction in manual data processing. Power Automate flows can run on a recurring schedule or in response to specific events, enabling workflows to execute without human intervention.

Scheduled flows let you define precise intervals to run your automation (hourly, daily, weekly, and so on), while *event-driven flows* trigger when something happens in a connected cloud application. For example, a flow can run the moment a file is uploaded to SharePoint, a new email arrives, or a row is added to a database.

Flows execute the same logic in the same way every time, eliminating variability and reducing the risk of human error. By removing manual steps, Power Automate ensures that your data is processed precisely as intended.

Most importantly, Power Automate is a true no-code platform. Even mere mortals, like you and me, can build complex automations that once required professional developers. Tasks that previously demanded heavy programming or full applications can now be built with an intuitive, visual, drag-and-drop interface — no programming knowledge required!

Understanding Power Automate requirements

To start creating cloud flows with Power Automate, you must sign in with a Business account associated with a valid Microsoft 365 license. Power Automate capabilities are included in Microsoft 365 Business and Enterprise plans, but the available features differ by license tier.

Users with Business Basic, Business Standard, or Enterprise E-series plans gain access to standard connectors, which enable you to integrate your flow with the associated cloud application. For example, if you want to work with Excel online, you need access to the Excel Online Business connector. The standard connectors are:

- SharePoint
- Outlook or Office 365 Outlook
- Excel Online
- OneDrive for Business
- Planner
- Microsoft Teams
- Microsoft Forms

TIP

Microsoft does not offer a special license for schools and educational organizations. Instead, they typically receive discounted pricing for Business Standard licensing. If you're using a .edu email account, you'll likely have access to standard connectors.

Some organizations purchase Power Automate Premium licenses, which are supplemental licenses assigned to individuals who need to work with premium connectors. Dozens of premium connectors are available; the commonly used ones are:

- Microsoft Dataverse
- SQL Server
- Azure SQL
- Salesforce
- ServiceNow
- Dynamics 365

If you want to build flows using a premium connector, ask your Microsoft administrator to assign you a license.

TIP

Personal Microsoft accounts do not have access to Power Automate cloud automation. Microsoft does offer Power Automate Desktop to all Windows 10/11 users, including Personal accounts, but it's like a macro recorder for your local PC. It doesn't allow you to build cloud automation.

Exploring the Power Automate Interface

To navigate to the Power Automate portal, enter the following URL in your favorite browser: `https://make.powerautomate.com/`. The home screen, shown in Figure 10-1, opens with options on the left. Many of these options let you interact with features that are outside the scope of this book. The options I focus on here are Create, Templates, and My Flows.

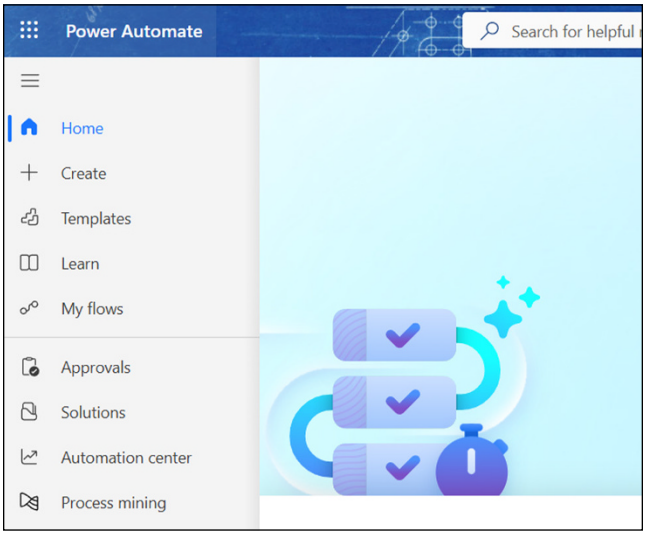


FIGURE 10-1: The Power Automate home screen.

The Create option

The Create option starts the process of creating a flow from scratch. When you select this option, the screen shown in Figure 10-2 will ask you to first choose how your flow will be triggered:

- **Automated cloud flow:** Triggered when a specific event occurs (such as when a file is uploaded to SharePoint)
- **Instant cloud flow:** Runs only when a user manually triggers it
- **Scheduled cloud flow:** Runs at a specified interval (hourly, daily, monthly and so on)

I dive deeper into creating a flow later in the chapter.

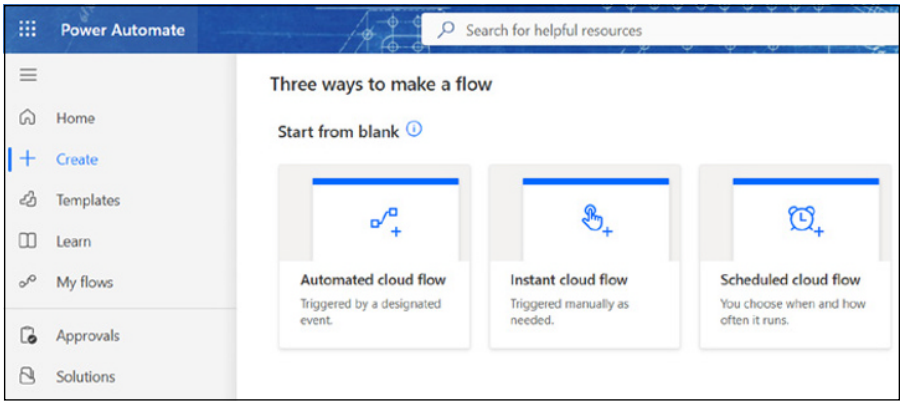


FIGURE 10-2: You must first choose a trigger when creating a flow.

The Templates option

You use the Templates option to browse a wide variety of prebuilt, ready-to-use automations that can help you create a flow quickly. These templates were created by both Microsoft and the broader Power Automate community to address common business scenarios.

To help you find an appropriate template, use the search bar. For example, in Figure 10-3, I typed **Excel Script** to show all available templates related to running an Excel script.

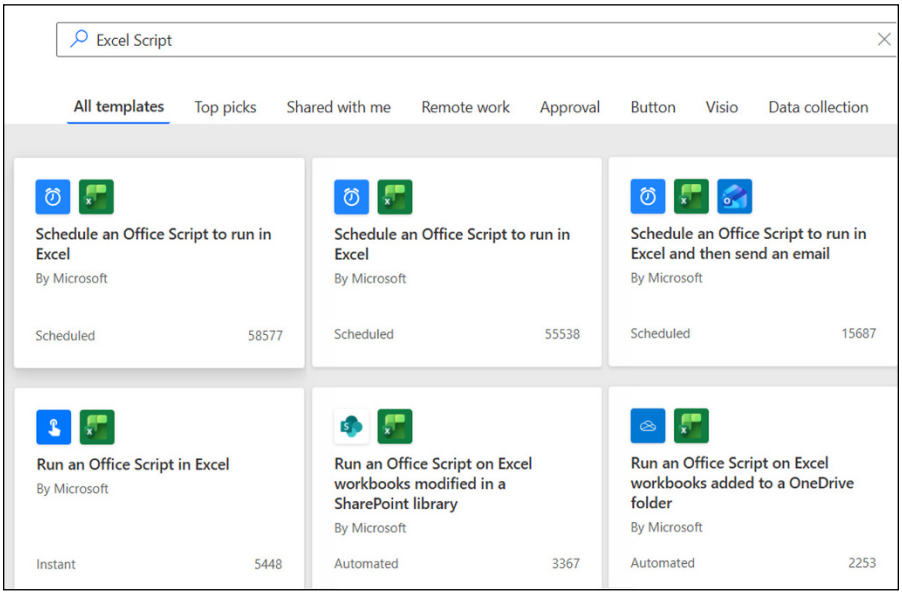


FIGURE 10-3: You can start a flow based on a preconfigured template.

When you select a template, Power Automate takes you to the *flow designer* (see Figure 10-4), where you provide the required parameters and add any additional actions needed to tailor the flow to your requirements.

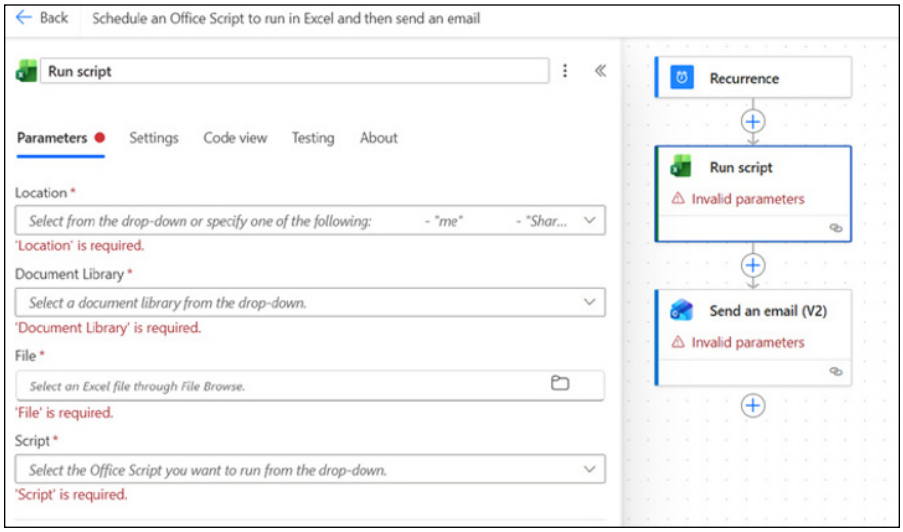


FIGURE 10-4: Use the flow designer to configure the prepopulated actions in the template and add new actions.

TIP

Actions are the individual steps that make up a Power Automate flow. They represent specific operations that the flow performs, such as sending an email, creating a file, or running an Office Scripts. Every flow consists of a trigger followed by one or more actions, which execute sequentially. You can chain actions to form complex, multistep processes. You delve further into the topic of actions shortly.

The My Flows option

The My Flows option takes you to the screen shown in Figure 10-5, where you can view and manage all the flows you've created. You can browse your flows and open a flow for deeper review. Clicking the three vertical dots next to a flow displays options to edit it, share it with others, export it, or view its run history (a report detailing every execution of the flow).

You can select the Shared with Me tab to view flows that other team members have shared with you. These are flows where you've been granted permission to view or edit, allowing for easier collaboration across your team.

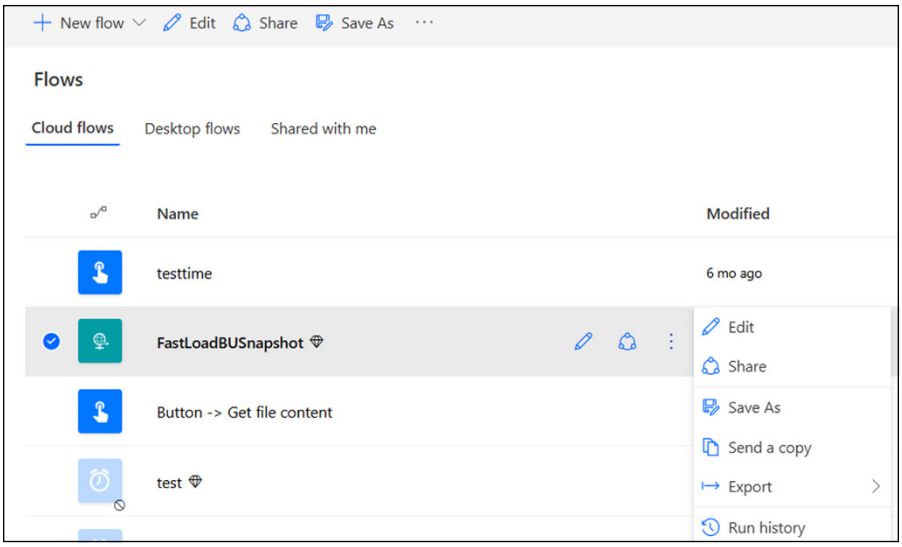


FIGURE 10-5: Use the My Flows menu option to browse and manage your flows.

TECHNICAL STUFF

The diamond icon next to some flows indicates that the flow is using Premium actions, which are available only to those with a Power Automate Premium license. (For example, connecting to a SQL server or running a PowerBI query are premium actions.)

Creating Your First Flow

In this section, you walk through the steps for creating a Power Automate flow that runs an Excel script, using the sample code provided in the `Chapter 10 Code` folder (at `www.dummies.com/go/microsoft365excelofficescriptsfd`). Note that you must be able to sign in to the Power Automate portal (see "Understanding Power Automate requirements," shown previously in the chapter) to complete these steps.

Understanding triggers

Triggers are important to understand because they define when a Power Automate flow runs. Every cloud flow has exactly one trigger, and the selection of that trigger defines when the flow begins executing its actions. Choosing the right trigger is one of the most important design decisions in building a Power Automate flow because it sets the context for all subsequent actions and determines how well the flow aligns with the solution it is meant to automate.

An *automated (event-based) trigger* responds to something happening in a connected service, such as when a new email arrives, an item is created or modified in SharePoint, or a file is created in a folder on OneDrive. This type of trigger enables a reactive flow that runs the moment a specified event occurs, without delay or manual intervention.

A *scheduled trigger* causes a flow to run automatically at a fixed time interval regardless of whether an event has occurred. A scheduled trigger is ideal for periodic reporting, regular data synchronization, nightly cleanup tasks, or whenever an action needs to happen with a predictable cadence rather than in response to a specific event.

An *instant (manual) trigger* allows a flow to run on demand. Instant triggers can be configured to accept input parameters at the time of execution, allowing the flow to collect information such as a date or a name, and then use that information in the workflow logic. This approach makes instant cloud flows versatile tools for ad hoc operations.

Gathering your thoughts and files

Before creating a flow, it's important to think through what you want to accomplish and confirm that you have the necessary resources. The type of flow you create depends on how you intend it to run:

- If your flow needs to run on a regular schedule, create a scheduled cloud flow.
- If you want to trigger a flow manually, choose an instant cloud flow.
- If you want your flow to trigger on a specific event, create an automated cloud flow.

Clarifying your desired outcome and how the flow should behave will guide you toward the correct flow type from the start.

You should also take inventory of any files your flow depends on. Make sure all required Office Scripts are saved in your `Documents/Office Scripts` folder in OneDrive or SharePoint. If the flow will reference specific files, such as `SalesReport.xlsx`, verify the location of the files and ensure that they can be opened without passwords or restricted permissions. This preparation helps avoid errors and interruptions when building your flow.

REMEMBER

A Power Automate flow requires existing file content to create or modify Excel workbooks. In other words, a flow must start with a valid Excel file structure. Even when you want to generate a new workbook from scratch, the flow still needs a starter workbook as a template. To support this, it's beneficial to create an empty Excel workbook and save it as `BlankWorkbook.xlsx` in your `Documents/OfficeScripts` folder on OneDrive or SharePoint. This file will be the starter template that your flow references when it requires file content.

Building the flow

For your first flow, you will create an on-demand process that runs a simple script named `CreateSampleData.osts`, as shown in Figure 10-6. This script creates a worksheet called `DataSheet`, generates some sample data, and then outputs the data to a table starting in cell A1.

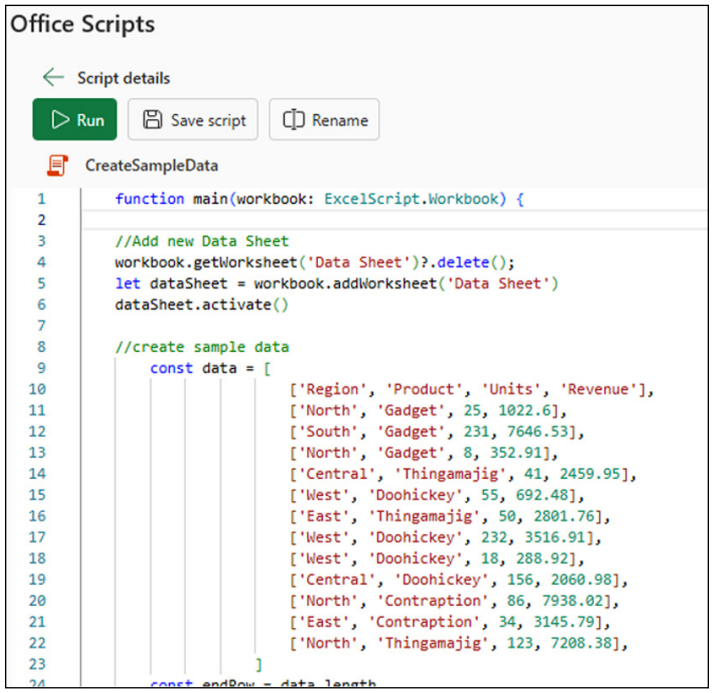


FIGURE 10-6: The flow will run the `CreateSampleData` script, which writes sample data to a new worksheet.

REMEMBER

To do these steps, you must have access to OneDrive for Business and you must be able to create and save Excel Online workbooks to the `Documents` directory. Also, the `CreateSampleData.osts` script and an empty workbook called `BlankWorkbook.xlsx` must be in your `Documents/Office Scripts` folder.

For this first flow, do the following:

1. **Sign into OneDrive.**

   Make sure the `CreateSampleData.osts` and `BlankWorkbook.xlsx` files are in your `Documents/Office Scripts` folder.

2. **Sign into `https://make.powerautomate.com/`.**

3. **Select the Create menu option, and then choose Instant Cloud Flow (refer to Figure 10-2).**

   The Build an Instant Cloud Flow dialog box appears.

4. **Under Flow Name, enter** My First Flow**, and then choose the Manually Trigger a Flow option (see Figure 10-7).**

5. **Click the Create button.**

   The flow designer appears, as shown in Figure 10-8. The first step in the flow is the trigger. In this case, Power Automate prepopulated the trigger based on your selection in Figure 10-7.

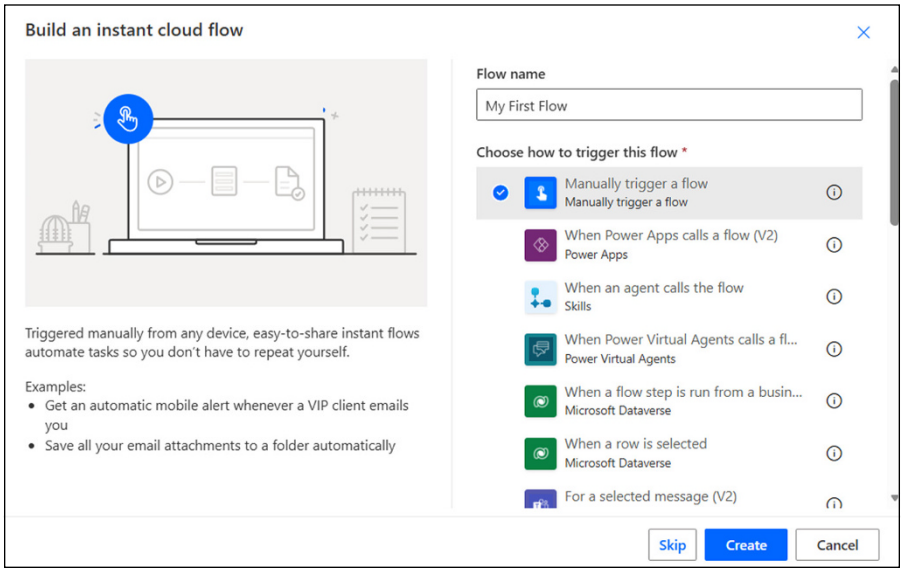


FIGURE 10-7: Name your flow and choose a trigger option.

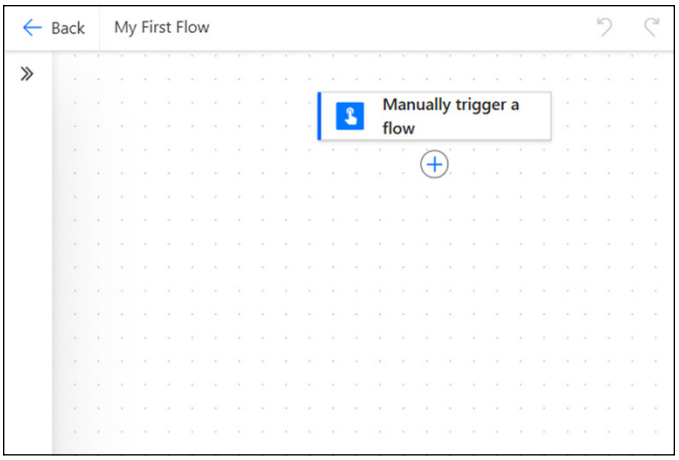


FIGURE 10-8: Use the flow designer to build your flow by adding actions.

Once your trigger is defined, you can start adding actions. For this flow, you start by adding an action to get the file contents:

1. **Click the plus symbol below the trigger.**

   The Add an Action task pane appears, displaying all available connectors and actions. You need to think about the action you want to take, find the appropriate connector, and then select the action below the connector.

2. **Enter** get file content **in the search bar, as shown in Figure 10-9.**

   You need to create an empty Excel file in your OneDrive account. To accomplish this, you need to give Power Automate a valid file structure by using the Get File Content action.

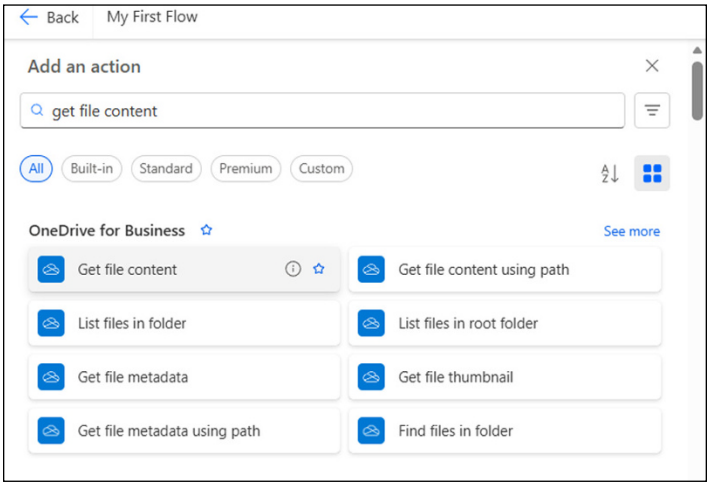


FIGURE 10-9: Use the search bar to find the Get File Content action under OneDrive for Business.

3. **Under the OneDrive for Business connector, select the Get File Content action.**

   A configuration pane for the Get File Content action appears, as shown in Figure 10-10. (Each action has a unique configuration pane with the parameters it needs to function properly.) The Get File Content action has one required parameter: the path to the file you want to use as the starter content.

4. **Under File, click the folder icon and browse to and select the `BlankExcelWorkbook.xlsx` file.**

5. **Click the Back button in the upper left to return to the flow designer, as shown in Figure 10-11.**

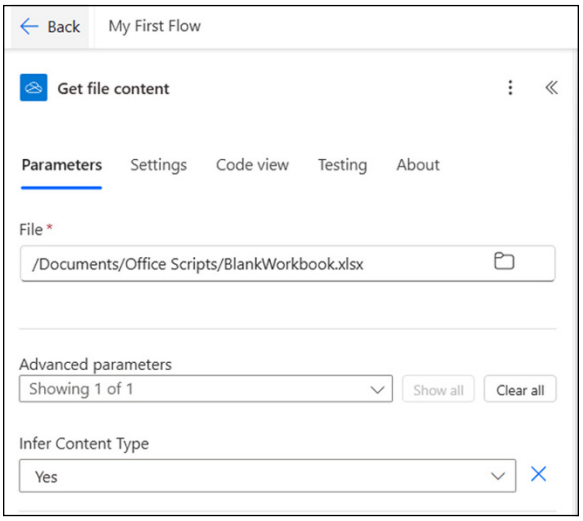


FIGURE 10-10: Click the folder icon to browse to the workbook that will be used as the file content.

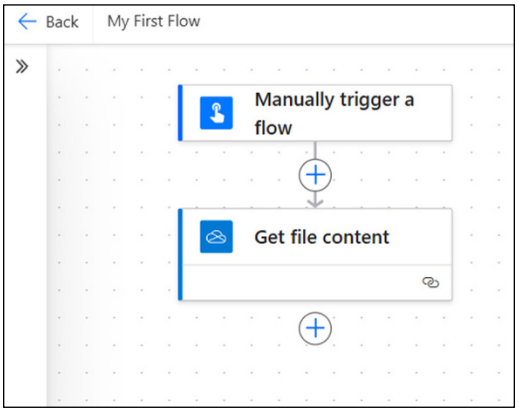


FIGURE 10-11: The flow designer now contains the trigger and one action.

Now, let's add an action to create a new Excel file:

1. **In the flow designer, click the plus symbol below the trigger to the Get File Content action.**

   The Add an Action task pane appears.

2. **In the search bar, type** create file**.**

   The screen shown in Figure 10-12 appears.

3. **Under OneDrive for Business, choose the Create File action.**

   The Create File action requires three parameters, as shown in Figure 10-13: the folder path of the directory that will store the new file, the file name, and the file contents.

4. **Under Folder Path, click the folder icon and navigate to and select the `/Documents` folder.**

5. **Under File Name, enter** MyNewWorkbook.xlsx**.**

6. **Under File Content, click the lightning bolt to open the menu of dynamic content, and choose File Content.**

7. **Return to the flow designer by clicking the designer canvas.**

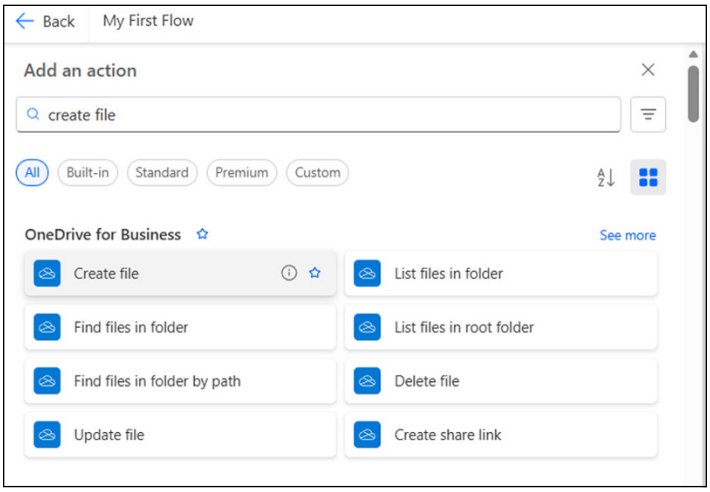


FIGURE 10-12: Find and select the Create File action under OneDrive for Business.

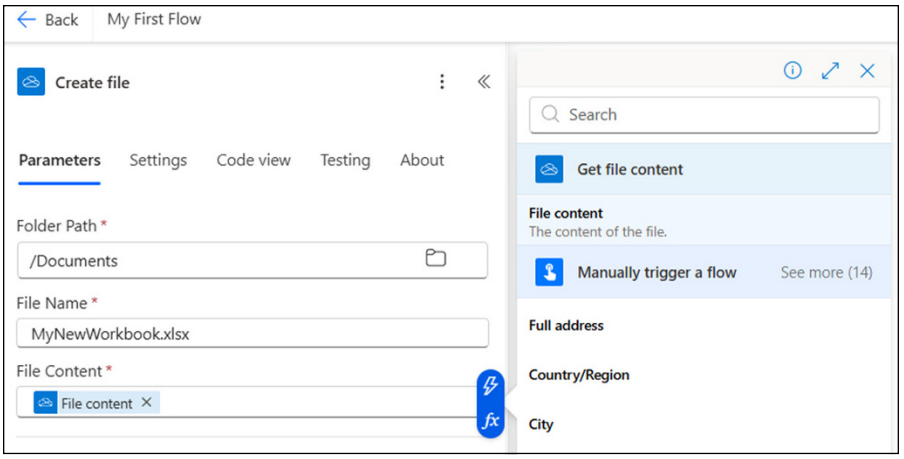


FIGURE 10-13: Enter the parameters for the Create File action.

For the File Content input, you need to supply the file content that holds the file structure you want to start with. Here is where the concept of dynamic content comes into play. Most actions generate *dynamic content* — that is, metadata that can be passed to subsequent actions. This metadata can include things like a file name, a file path, or a file ID. In this case, the Get File Content action creates a piece of metadata called File Content.

Now it's time to add the last action:

1. **In the flow designer, click the plus symbol below the Create File action.**

   The Add an Action task pane opens.

2. **In the search bar, type** Run Script **and then select the Run Script action.**

   The Run Script configuration pane opens, as shown in Figure 10-14, asking for four parameters: the location of the Excel file you want to run the script on, the document library of the file, the file name, and the script you want to run.

3. **Under Location, click the down arrow and select OneDrive for Business.**

   The Location and Document Library inputs dictate where the new Excel file (created with the Create File action) will be saved.

4. **Under Document, click the down arrow and select OneDrive.**

5. **Under File, click the lightning bolt icon to open the menu of dynamic content, and select Path (under the Create File action on the right in Figure 10-14).**

   For the File input, you supply the filename of the workbook you want your script to be run against.

6. **Under Script, click the down arrow and find and select the target script, which is `CreateSampleData`.**

7. **Click the Back button in the upper left to return to the flow designer.**

   As shown in Figure 10-15, the flow now has a trigger and three actions.

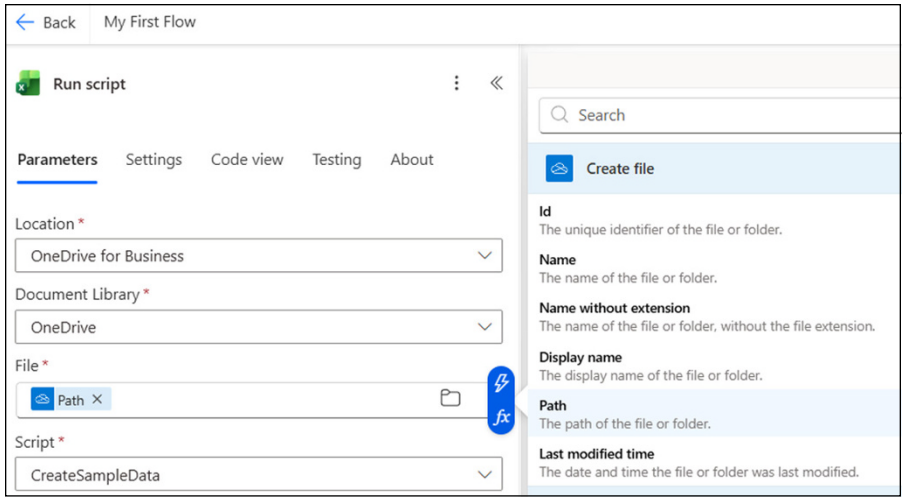


FIGURE 10-14: Enter the parameters for the Run Script action.

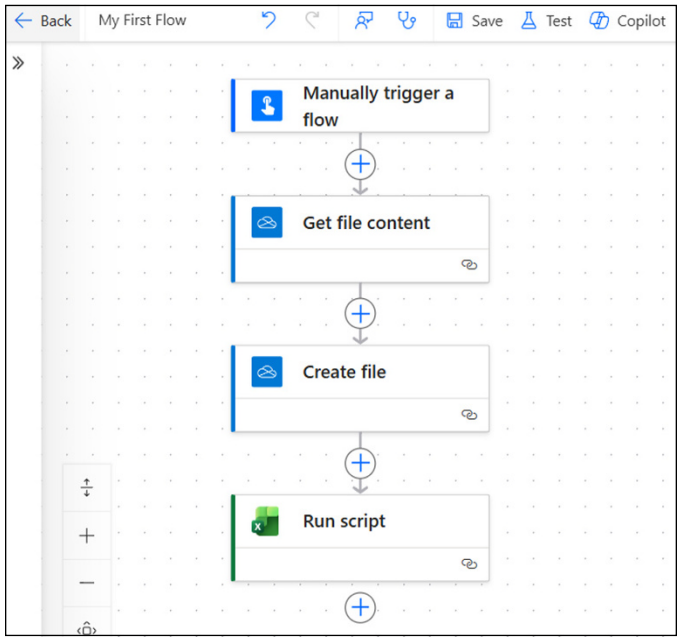


FIGURE 10-15: The final flow is ready to test.

You're ready to save and test the flow to make sure that each action is configured correctly, the connections are working, and the steps don't produce errors:

1. **Click the Save icon.**
2. **Click Test.**

   The Test Flow task pane appears, as shown in Figure 10-16.

3. **Select Manually, and then click the Test button.**

   During testing, each action in the flow runs in real time. As each step completes successfully, a green checkmark appears next to the action, indicating that it ran without errors. If any action fails, Power Automate will highlight it and display an error message so you can troubleshoot and make adjustments.

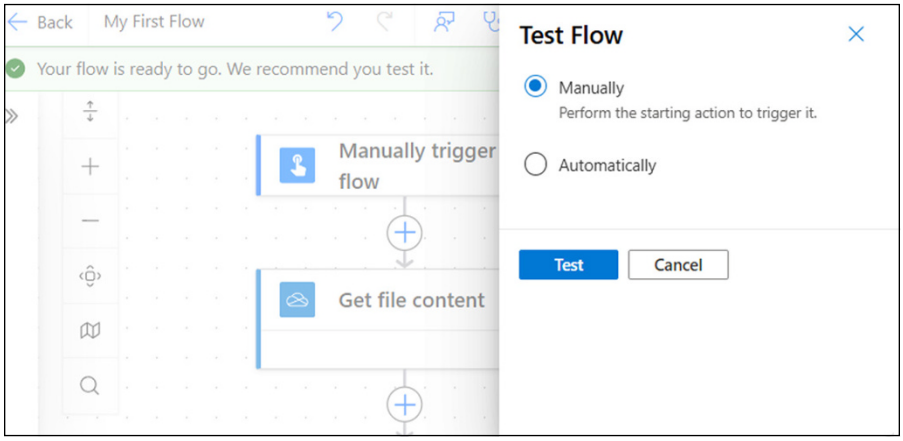


FIGURE 10-16: Manually trigger a test.

You know a flow has completed successfully when every action in the flow displays a green checkmark, as shown in Figure 10-17.

After running your flow, it creates the `MyNewWorkbook.xlsx` file (see Figure 10-18).

You aren't limited to using a single script in a flow. For example, in the sample flow, you could add another action to execute the `AddDataBars` script after the first script finishes, as shown in Figure 10-19.

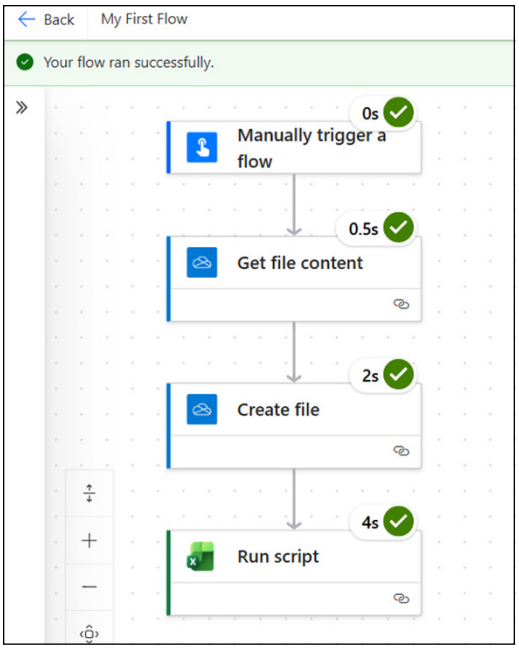
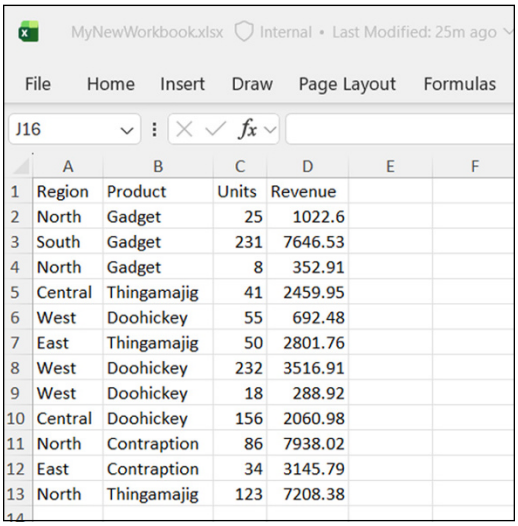


FIGURE 10-17: A successful test displays green checkmarks next to each action.



	A	B	C	D	E	F
1	Region	Product	Units	Revenue		
2	North	Gadget	25	1022.6		
3	South	Gadget	231	7646.53		
4	North	Gadget	8	352.91		
5	Central	Thingamajig	41	2459.95		
6	West	Doohickey	55	692.48		
7	East	Thingamajig	50	2801.76		
8	West	Doohickey	232	3516.91		
9	West	Doohickey	18	288.92		
10	Central	Doohickey	156	2060.98		
11	North	Contraption	86	7938.02		
12	East	Contraption	34	3145.79		
13	North	Thingamajig	123	7208.38		

FIGURE 10-18: The resulting workbook from the flow.

You can also rename each action by editing the label at the top of its configuration pane (see the top of Figure 10-19). For example, I changed the name of the action in Figure 10-19 from Run Script to Run AddDataBars Script. Renaming actions helps keep your flow organized and makes it easier to understand the purpose of each step at a glance.

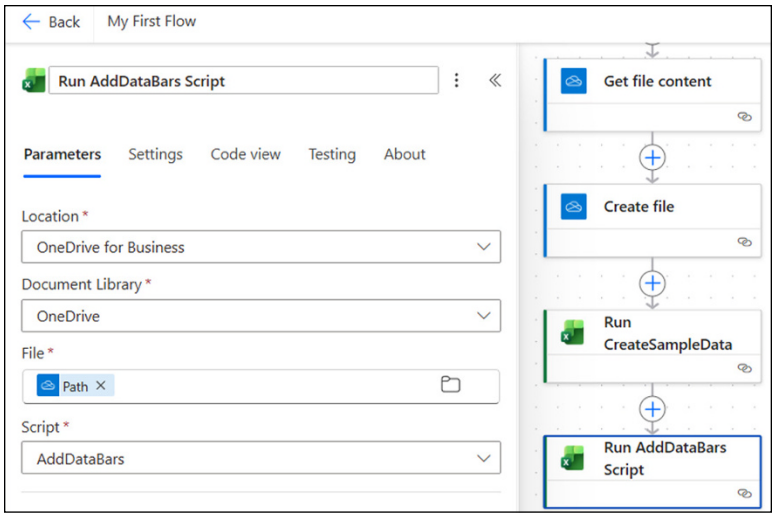


FIGURE 10-19: You run multiple scripts in a single flow.

TIP

You can rename the flow itself on the flow designer screen. Simply click the flow name, enter the new name, and then click the Save icon at the top of the page.

Passing Parameters to a Script from Power Automate

You can pass parameters from Power Automate to an Office Scripts by defining them in the script's function signature (the area between the first opening and closing parentheses). For example, the script shown in Figure 10-20 includes two parameters in its function signature (on lines 3 and 4): `createRegionWorksheets` and `sheetName`.

The `createRegionWorksheets` parameter is a Boolean that allows the user to specify whether a worksheet for each region should be created. The `sheetName` parameter is a string that lets the user define the name of the data sheet. When the script is run directly from Excel, the Enter Parameters dialog box appears and prompts the user to provide values for these required inputs, as shown in Figure 10-21.

Now here's the nifty part. When you use a script that includes parameters in a flow, Power Automate automatically detects those parameters by reading the script's function signature. Any parameter defined in the script appears in the Run Script action as configurable input fields.

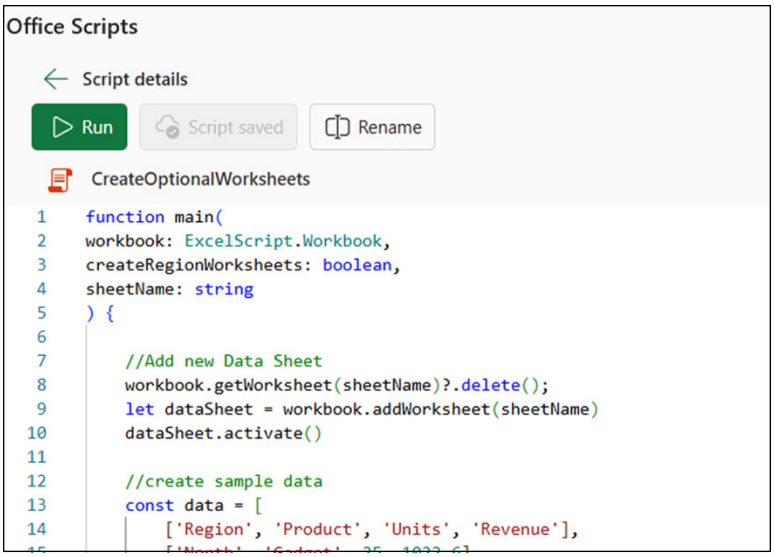


FIGURE 10-20: The `Create Optional Worksheets` script includes parameters in the function signature.

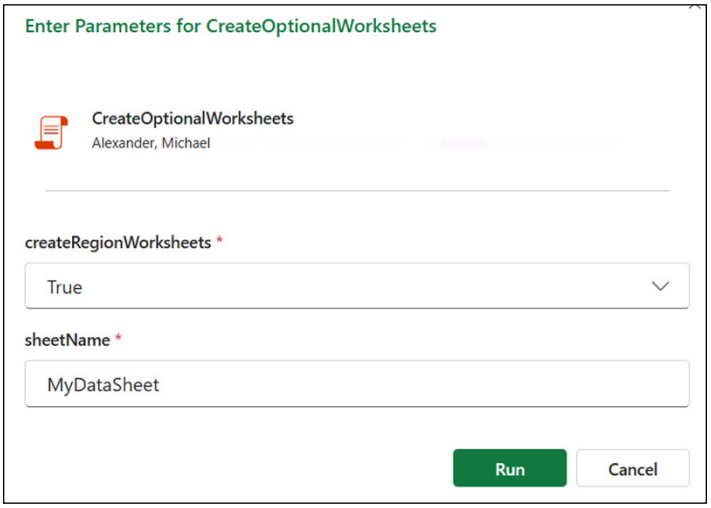


FIGURE 10-21: Running the `Create Optional Worksheets` script prompts the user to supply the required parameters.

In the configuration pane of the Run Script action, these parameters appear at the bottom of the card. Note the two script parameters at the bottom of the configuration pane shown in Figure 10-22. Each parameter corresponds directly to the arguments you defined in the Office Scripts.

In Figure 10-22, the parameter values are entered in the configuration. But ideally, they would be provided automatically. As you discover in the next chapter, parameters can be supplied automatically through steps in the flow. Instead of

typing values manually when configuring the flow, you can pull parameter values from actions such as outputs from Excel, SharePoint actions, email triggers, or logical expressions (such as using an `IF` statement to determine parameter values).

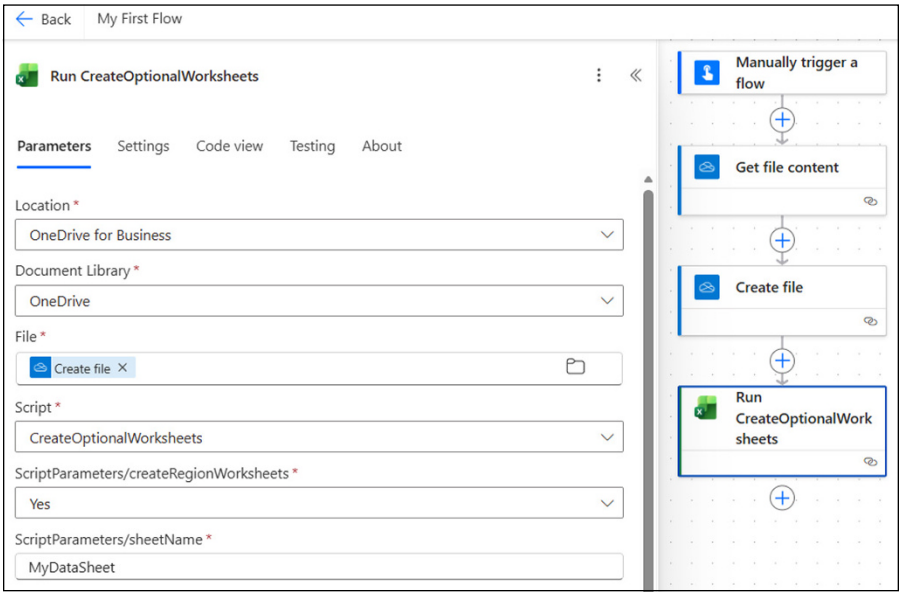


FIGURE 10-22: Power Automate detects and asks for any script parameters.

Understanding the Limitations of Power Automate

Running Office Scripts through Power Automate comes with several technical limitations important to understand. Power Automate and Office Scripts are two separate platforms, with their own platform and behavioral limitations.

Excel data and API limitations

Office Scripts running in a Power Automate flow will time out after 2 minutes. If the script doesn't fully execute and return a response within 120 seconds, the Run Script action will fail with a timeout error.

Office Scripts will throw an error when the data sent to or returned from the script exceeds the 5 MB request/response size limit. Any parameter values, JSON objects, arrays, and strings must remain below this size cap. Additionally, any Excel range that exceeds 5 million cells will fail even if the data size is smaller.

Scripts that perform numerous small operations may run into platform limits. Power Automate enforces a cap of 1,600 Run Script actions per user per day, and the Excel Online (Business) connector applies throttling for high-frequency operations (for example, approximately 100 calls per minute). These limits can affect flows that trigger scripts in rapid succession.

Power Automate platform limits

Power Automate can run only Office Scripts stored in OneDrive for Business or SharePoint, specifically in the user's `/Documents/Office Scripts` directory. A script stored on a local PC or a network drive can't be accessed or executed by Power Automate.

Power Automate executes Office Scripts in a *headless* Excel session, meaning it can't simulate a user interface. As a result, any methods that rely on an active user session will fail. Methods such as `Workbook.getActiveWorksheet()`, `Range.select()`, `Chart.activate()`, or `Workbook.getActiveCell()` work in Excel but will throw errors when the script runs inside a flow. These methods must be removed or replaced before using a script in the Run Script action.

Refresh operations, such as `PivotTable.refresh()`, are not supported when Office Scripts run through Power Automate. They simply do nothing and fail silently without raising errors. This is important to remember when working with data connections, queries, or pivot tables, because the scripts can't refresh them from a flow.

Ensure that only one script runs against an Excel workbook at a time. If multiple flows or scripts attempt to open or modify the same workbook simultaneously, the workbook may become locked, causing subsequent scripts to fail due to file-locking conflicts.

Best practices to avoid Office Scripts failures in flows

Now that you know the limitations of integrating Office Scripts and Power Automate, look at the following best practices to avoid potential failures in your flows:

- **Design scripts to complete within 2 minutes.** Power Automate enforces a strict 120-second timeout for the Run Script action. If a script takes longer, it fails. Avoid lengthy workbook operations like sorting large ranges, formatting in loops, and forcing workbook recalculations.

- **Keep data transfers under 5 MB.** Pass only essential data to and from scripts. Don't pass entire datasets when a summary will do. If working with large data tables, use filters or summaries before returning results.
- **Avoid ranges of more than 5 million cells.** Use range shortcut methods like `Range.getUsedCells()` when you can. Avoid using entire column and row ranges like A:A or 1:1.
- **Remove `Activate` and `Select` methods.** Avoid using interface-dependent methods such as `range.Select()` or `chart.Activate()` in your scripts.
- **Implement error handling in your scripts.** Adding try/catch blocks to your Office Scripts can protect your flows from unexpected interruptions. While errors inside scripts will still cause the Power Automate action to fail, structured error handling allows you to handle errors gracefully.
- **Use batch operations instead of repeated single-cell manipulations.** Scripts that access and write to cells one at a time can quickly hit internal Excel API call limits or cause performance bottlenecks. Consolidate repeated logic into a single operation when possible. And reduce the number of script calls by bundling related operations into fewer, more streamlined scripts.

Discovering the Classic Flow Designer

The flow designer you've been working with up until now is what is known as the *modern flow designer.* As you've seen, the modern designer makes heavy use of dedicated task panes to add and configure actions.

The *classic flow designer* is the original Power Automate authoring experience. You can switch to the classic flow designer by clicking the New Designer toggle in the upper-right corner of the page. Figure 10-23 illustrates the modern designer (on the left) versus the classic designer (on the right).

In the classic designer, there are no task panes. The configuration options for actions are embedded in each action card. Clicking each action expands the card, revealing the configurations for that action (see Figure 10-24). Many users prefer the classic designer because configurations can be seen at a glance, without the need for multiple clicks to drill into specific task panes. In fact, many online training resources utilize the classic designer because a single screenshot can easily show all relevant action settings.

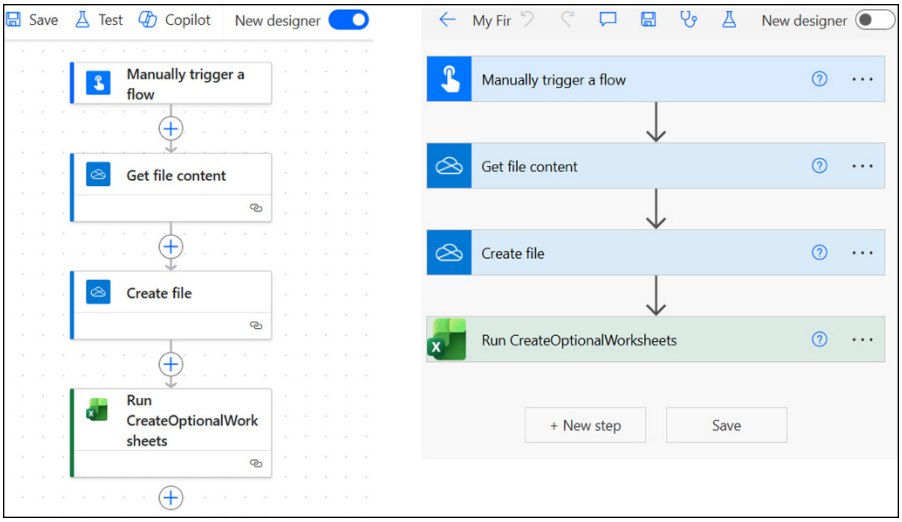


FIGURE 10-23: The modern designer (left) and the classic designer (right).

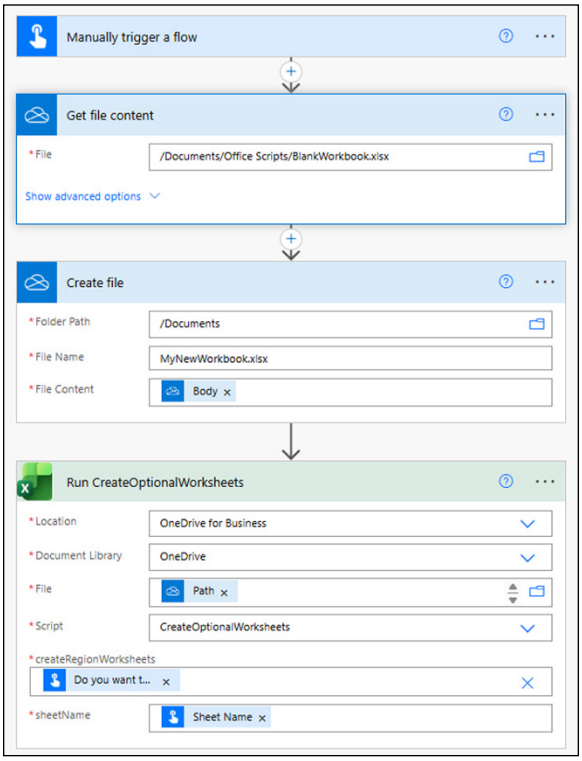


FIGURE 10-24: The classic designer allows or a more transparent configuration experience.

However, very large flows become difficult to navigate in the classic designer because actions must be repeatedly expanded and collapsed to manage configurations. Additionally, increased vertical stacking leads to excessive scrolling, making complex flows cumbersome to work with.

The modern designer has the benefit of making large flows easier to navigate by displaying configuration settings in task panes on the left side of the screen. This design makes large flows easier to navigate because actions remain compact and stable on the canvas. Configurations no longer cause actions to expand and shift, so the overall experience is smoother and less visually disruptive.

The drawback of the modern designer is that many configuration options are hidden behind multiple clicks. Some users complain that the modern designer is less transparent. Understanding an action's full configuration often requires opening several task panes rather than seeing everything at once.

Which designer should you use? Whichever one you feel suits the situation. Many users take a hybrid approach, using the classic designer to quickly troubleshoot actions and review configurations without clicking through multiple task panes, and then switching to the modern designer to more easily navigate and manage large, complex flows.

IN THIS CHAPTER

- Emailing attachments
- Embedding Excel data in emails
- Distributing Excel data via PDF

Chapter 11
Integrating Excel with Email

While Office Scripts are powerful on their own, their real value emerges when you integrate them with other cloud applications across your organization. One of the most natural integration points in the Microsoft ecosystem is between Excel and Outlook. When paired with Power Automate, the combination of Excel and Outlook enables an efficient model for distributing data without requiring ongoing manual effort.

In this chapter, you discover some of the techniques for distributing Excel reporting through emails.

REMEMBER

The code sample files used in this chapter are available in the `Chapter 11 Code` folder which can be downloaded with the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Integrating Excel and Outlook

In this section, you explore practical techniques for using Power Automate to distribute Excel data via Outlook. These techniques form the foundation for scalable, hands-off report distribution solutions.

REMEMBER

To follow along with the examples in this section, you need access to OneDrive for Business and the capability to save the sample files to the appropriate folders. Place the `SalesData.xlsx` file in the `/Documents` folder in OneDrive. Place the `E1_GetEmailList.osts` and `E2_EmailImages.osts` files in the `/Documents/Office Scripts` folder.

Emailing an attachment

One of the more common work scenarios is the need to email an Excel report as an attachment on a recurring basis. For example, imagine you need to email the `SalesData.xlsx` file every day. You can use the steps outlined here to create a scheduled cloud flow that emails the given workbook at the same time every day:

1. **Sign into OneDrive and make sure the `SalesData.xlsx` sample file is in the `Documents` folder.**
2. **Sign into `https://make.powerautomate.com/`.**
3. **Select the Create menu option and then choose Scheduled Cloud Flow.**

   The Build a Scheduled Cloud Flow dialog box appears, as shown in Figure 11-1.
4. **Name your flow and define when your flow will run. Then click Create to activate the flow designer.**

   In the example, the flow will run at 7 am every day starting on August 3rd.

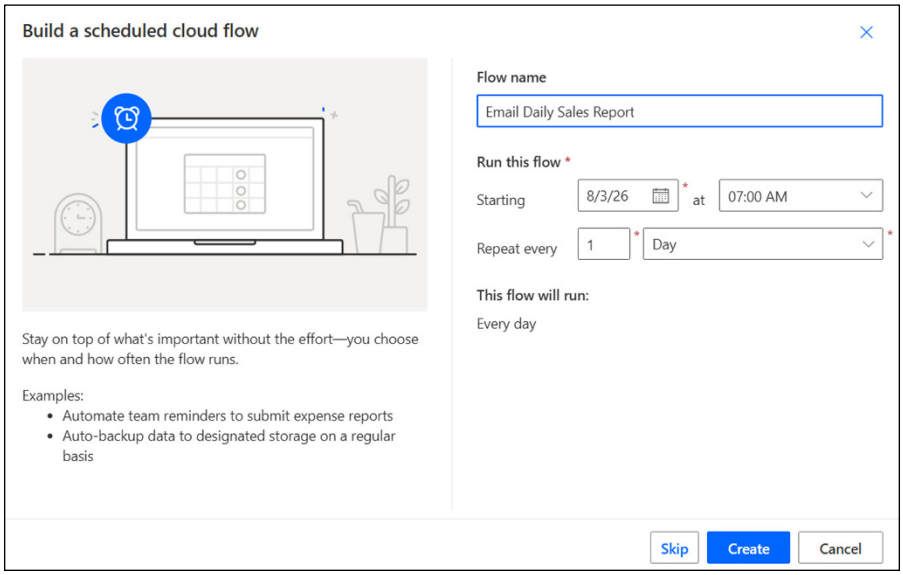


FIGURE 11-1: Define the schedule for your recurring flow.

5. **In the flow designer, click the plus symbol below the Recurrence trigger to add an action.**

   The Add an Action dialog box appears.

6. **In the search field, type** get file content**, and then select the Get File Content action under OneDrive for Business, as shown in Figure 11-2.**

7. **In the Get File Content task pane, click the folder icon to browse for and select the `SalesData.xlsx` file, as shown in Figure 11-3.**

8. **In the flow designer, click the plus symbol below the Get File Content action to add an action.**

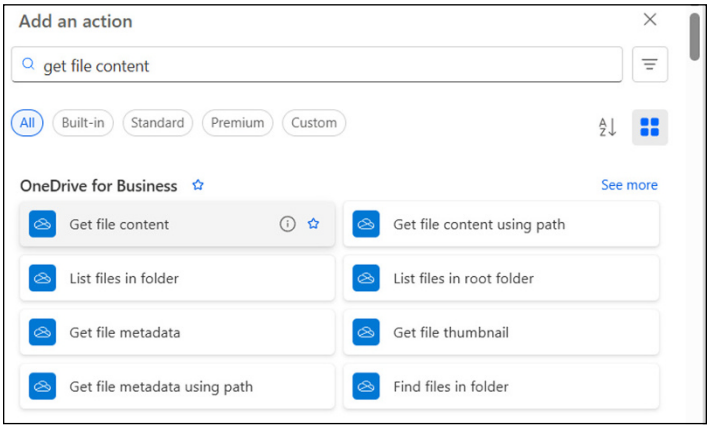


FIGURE 11-2: Choose the Get File Content action for OneDrive for Business.

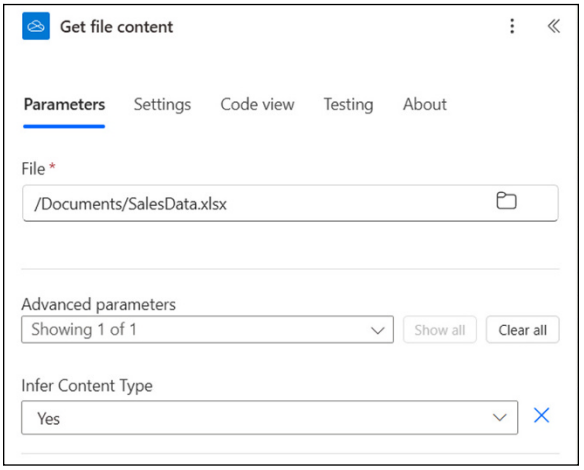


FIGURE 11-3: Select the file you want to email.

9. **In the Add an Action dialog box, enter** send an email **in the search bar, and then select the Send an Email action under the Office 365 Outlook connector (see Figure 11-4).**

   The Send an Email task pane appears. This is where you will input the information needed for the email.

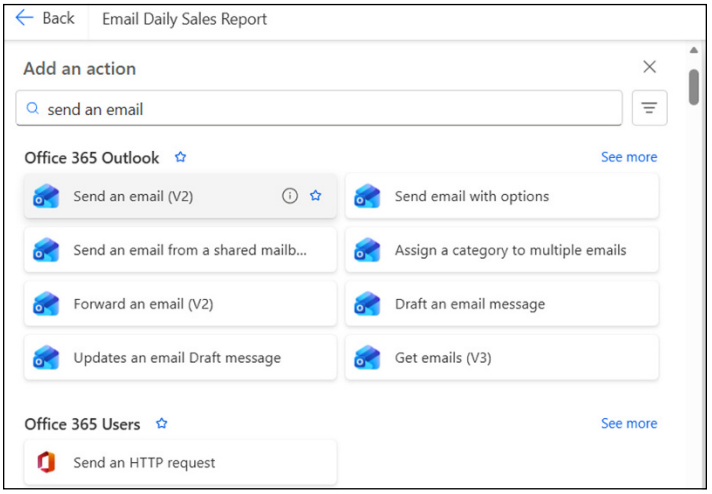


FIGURE 11-4: Choose the Send an Email action for Office 365 Outlook.

10. **In the To field, enter the target email addresses. Then fill in the Subject and Body fields.**

11. **Click the Show All button in the bottom right to display additional options, as shown in Figure 11-6.**

    You can specify additional email options, including adding an attachment.

12. **Under Attachments, click the Add New Item button.**

    The screen shown in Figure 11-7 appears.

13. **In the Name input box, enter the name for the attachment.**

14. **In the Content input box, click the lightning bolt icon and choose File Content.**

    This is the file you referenced in Get File Content task pane in Step 7. Figure 11-8 shows the final flow.

15. **Click the Save icon and then click the Test icon to make sure that the flow runs without issue.**

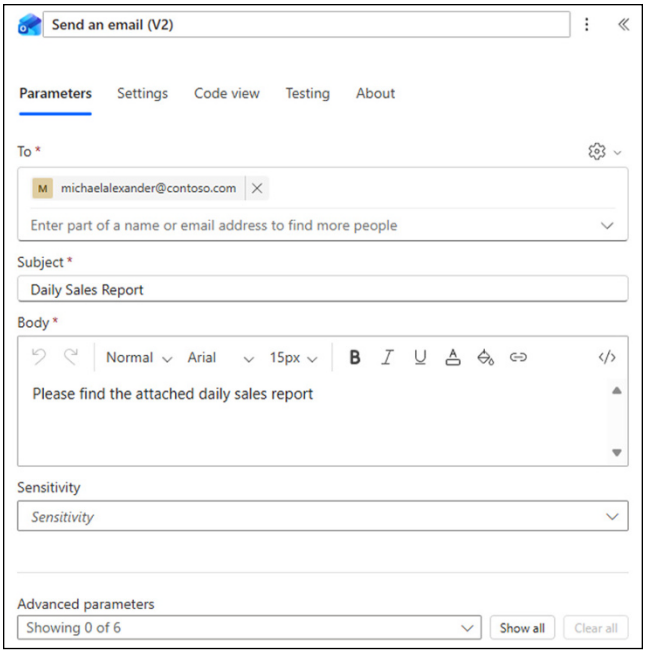


FIGURE 11-5: Enter the target email address, subject and body of the email.

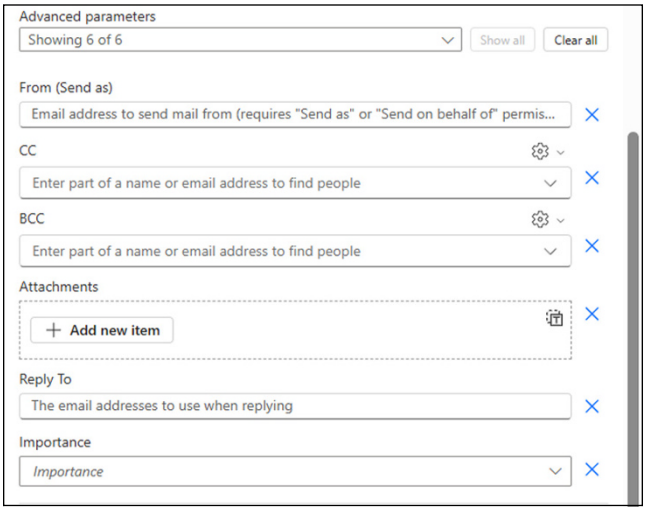


FIGURE 11-6: The Advanced parameters let you specify additional option for the email.

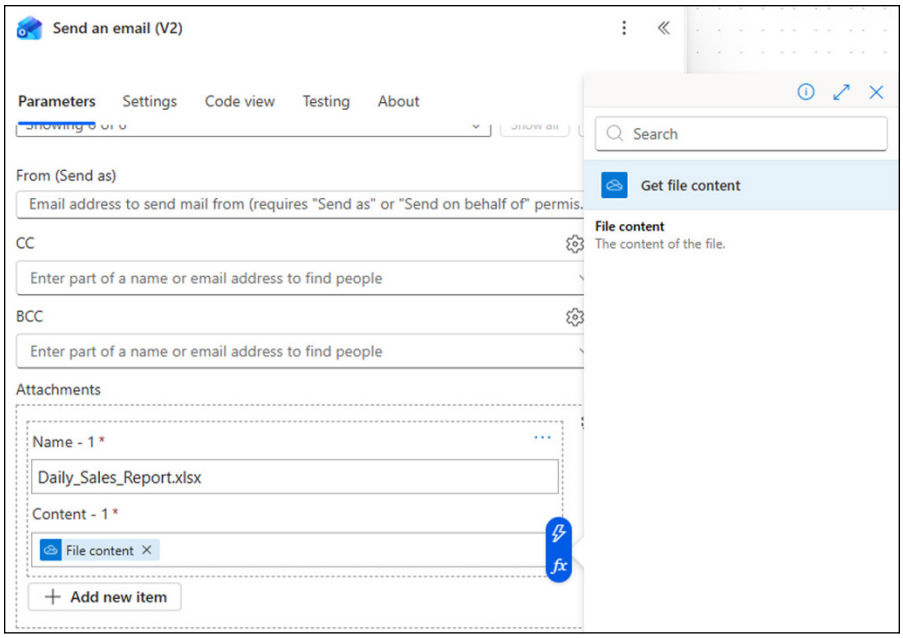


FIGURE 11-7: Point to the file referenced in the Get File Contents action.

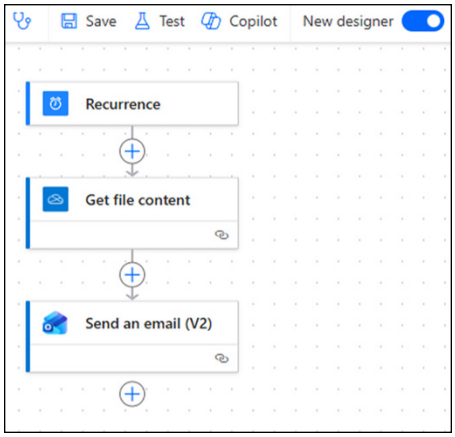


FIGURE 11-8: You can now save and test the flow.

Sending emails to a dynamic list of recipients

Note in Figure 11-5 that you use `michaelalexander@contoso.com` as the target email address. You can enter as many email addresses as you'd like, and each recipient will receive the same message.

In some scenarios, the list of recipients changes over time. As you can imagine, updating the flow every time the list of emails changes isn't practical. In these cases, you'll want to dynamically pass the email addresses to the flow.

To facilitate this, you can include a table of email addresses in a dedicated worksheet in the workbook you're emailing, such as the `Emails` sheet in the `SalesData.xlsx` file. A script can capture the email addresses and return an array that you can pass to Power Automate (see Figure 11-9).

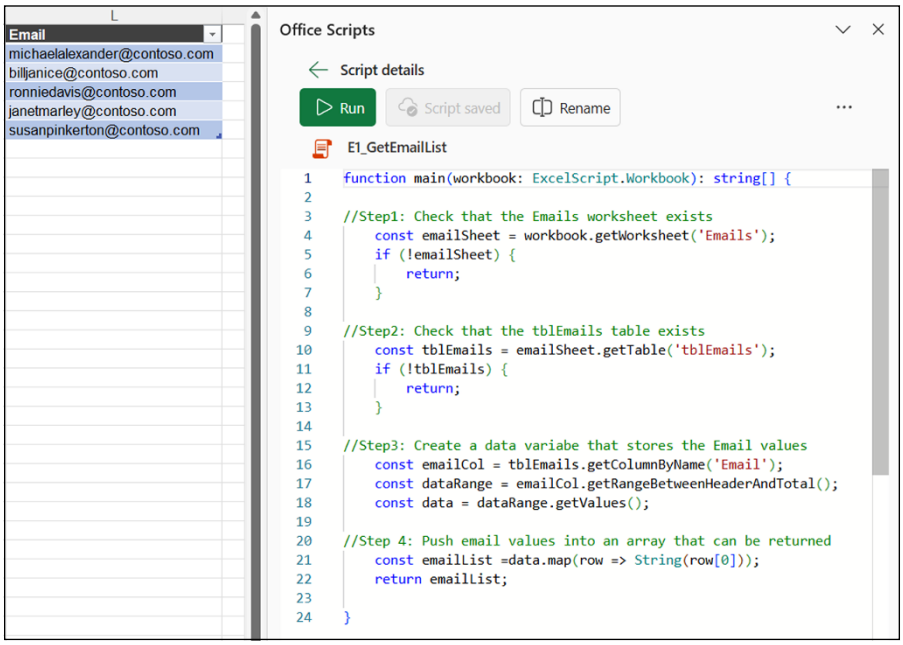


FIGURE 11-9: A table of email addresses can provide an array of recipients to Power Automate via a script.

Reviewing the script

Let's take a moment to review the `E1_GetEmailList` script shown in Figure 11-9:

```
function main(workbook: ExcelScript.Workbook): string[]{

//Step1: Check that the Emails worksheet exists
    const emailSheet = workbook.getWorksheet('Emails');
    if (!emailSheet) {
        return;
    }
```

```
//Step2: Check that the tblEmails table exists
    const tblEmails = emailSheet.getTable('tblEmails');
    if (!tblEmails) {
        return;
    }

//Step3: Create a data variable that stores the Email values
    const emailCol = tblEmails.getColumnByName('Email');
    const dataRange = emailCol.getRangeBetweenHeaderAndTotal();
    const data = dataRange.getValues();

//Step 4: Push email values into an array
    const emailList = data.map(row => String(row[0]));
    return emailList;
}
```

In Steps 1 and 2, you guard against runtime errors by verifying that both the `Emails` worksheet and the `tblEmails` table exist. If either is missing, the script exits early.

In Step 3, you retrieve the Email column from the table and store all its values (excluding the header and total rows) in the `data` variable. The values are returned as a two-dimensional array.

In Step 4, you use the map method to extract each email address from the array and convert it to a string, storing the results in a new one-dimensional array named `emailList`.

Finally, the script returns `emailList`. This return value allows the array of email addresses to be fed to Power Automate and used as the list of recipients.

Updating the flow

The following steps update the flow you built in the last section (see Figure 11-8) by adding a Run Script action that executes the `E1_GetEmailList` script. This addition enables you to implement a dynamic email list:

1. **In the flow designer, click the plus symbol below the Get File Contents action.**

   As you can see in Figure 11-10, you can add an action between steps.

2. **In the Add an Action task pane, select the Run Script action.**

3. **Click the Run Script action and configure it to run the `E1_GetEmailList` script on the `SalesData.xlsx` file (see Figure 11-11).**

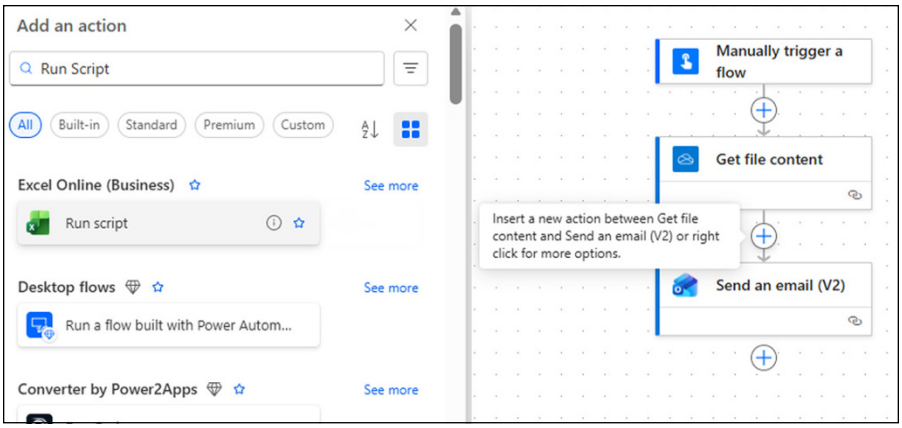


FIGURE 11-10: Add a new Run Script action after the Get File Contents step.

FIGURE 11-11: Configure the Run Script action to run the `E1_GetEmailList` script on the `SalesData.xlsx` workbook.

4. **In the flow designer, select the Send an Email action.**

   The Send an Email pane appears.

5. **Click the gear icon and then select the Use Dynamic Content option, as shown in Figure 11-12.**

6. **Remove any manually entered email addresses in the To input box, and then click the lightning bolt icon.**

7. **Under For Each, select the Current Item option, as shown in Figure 11-13.**

   By choosing Current Item, you tell the Send an Email action to send an email for each email address in the array coming from the Run Script action.

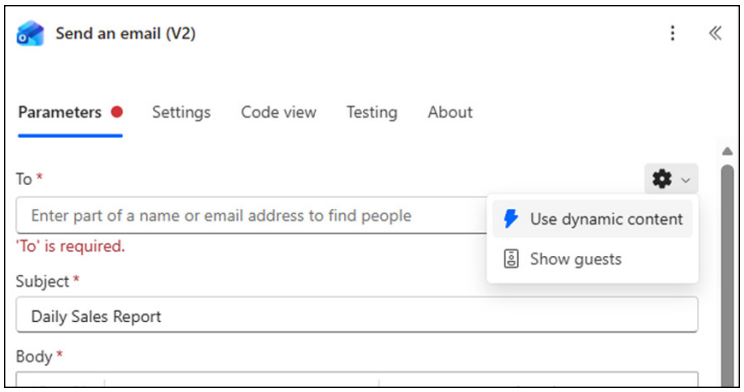


FIGURE 11-12: Choose the dynamic contents option in the Send an Email configuration pane.

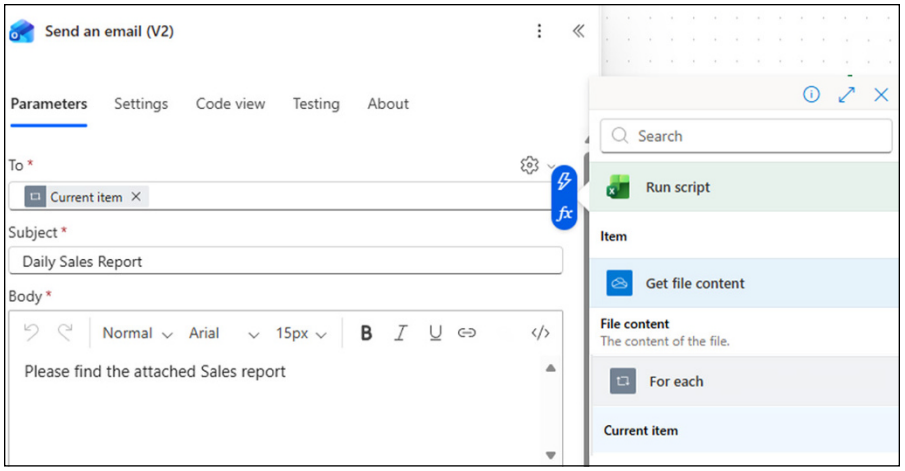


FIGURE 11-13: For the To input box, select the current item coming from the array provided by the Run Script action.

At this point, the flow designer will look like Figure 11-14, with the Send an Email action automatically placed inside a For Each control.

When a script returns an array (like the `E1_GetEmailList` script), Power Automate automatically recognizes that all subsequent actions need to be executed once for each element in that array. In this case, the array of email addresses returned by the `E1_GetEmailList` script (`emailList`) causes Power Automate to create a For Each control, making the flow loop through each address and send an email to each recipient individually.

All that's left to do is to save and test the flow to make sure it runs without issue.

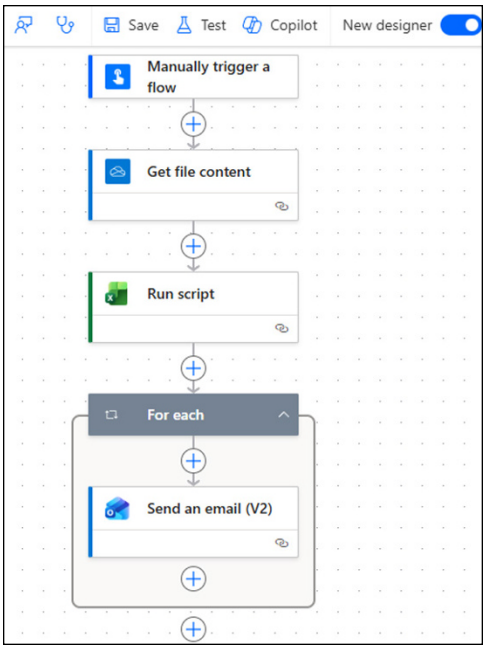


FIGURE 11-14: Power Automate places the Send an Email action in a For Each control.

Embedding images in emails

You can make your email process better by embedding in the email body an image that serves as a preview of the attached report. An embedded image can provide a quick glance at the attached report's contents, allowing recipients to quickly review the information and open the attachment only if more detailed analysis is needed.

The `SalesData.xlsx` file includes a worksheet named `Charts`. On that sheet, several charts are grouped into a single image called `ChartGroup`. The `E2_Email Images` script shown in Figure 11-15 captures the `ChartGroup` image and returns it as a Base64 HTML-encoded image.

Reviewing the script

Let's take a moment to review the `E2_EmailImages` script shown in Figure 11-15:

```
function main(workbook: ExcelScript.Workbook){

//Step 1: Capture the ChartGroup shape
 const mySheet = workbook.getWorksheet('Charts')
 const myChart = mySheet.getShape('ChartGroup')
```

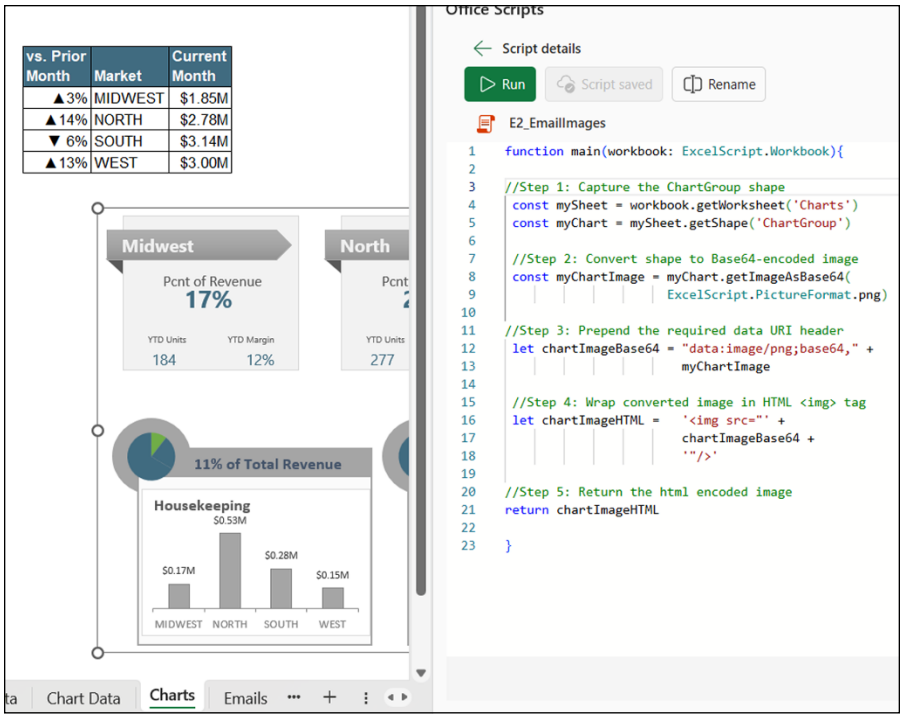


FIGURE 11-15: The `E2_EmailImages` script captures the image on the `Charts` worksheet.

```
 //Step 2: Convert shape to a Base64-encoded image
 const myChartImage = myChart.getImageAsBase64(
                      ExcelScript.PictureFormat.png);

//Step 3: Prepend the required data URI header
 let chartImageBase64 = "" +
                        myChartImage;

 //Step 4: Wrap the converted image in the HTML <img> tag
 let chartImageHTML =   '<img src="' +
                        chartImageBase64 +
                        '"/>';

//Step 5: Return the HTML encoded image
return chartImageHTML;

}
```

In Step 1, you create a reference to the `Charts` worksheet. Then you initialize the `myChart` variable. You fill `myChart` with the shape named `ChartGroup`. Every chart and shape has a name and can be loaded into a variable in this way.

Step 2 converts the chart shape into a Base64-encoded image string. Encoding the image as Base64 allows it to be safely passed to external systems, such as Power Automate, and embedded in emails or HTML content.

Step 3 prepends the required data URI header to the Base64 string, which tells HTML that the content is a PNG image encoded in Base64. Without this prefix, the image would not render correctly in an email or a browser.

Step 4 creates the `chartImageHTML` variable, containing the Base64 image string wrapped in an HTML `<img>` tag. The result is a self-contained HTML image that can be embedded directly in an email body.

Step 5 returns the HTML in `chartImageHTML` as a string that Power Automate uses to insert the image directly into an email action.

Updating the flow

The following steps will update the flow you built in the last section (refer to Figure 11-14) to embed an image in the body of the email being sent:

1. **In the flow designer, click the plus symbol below the Run Script action.**
2. **In the Add an Action task pane, select the Run Script action, as shown in Figure 11-16.**
3. **Configure the Run Script action to run the `E2_EmailImages` script on the `SalesData.xlsx` file (see Figure 11-17).**

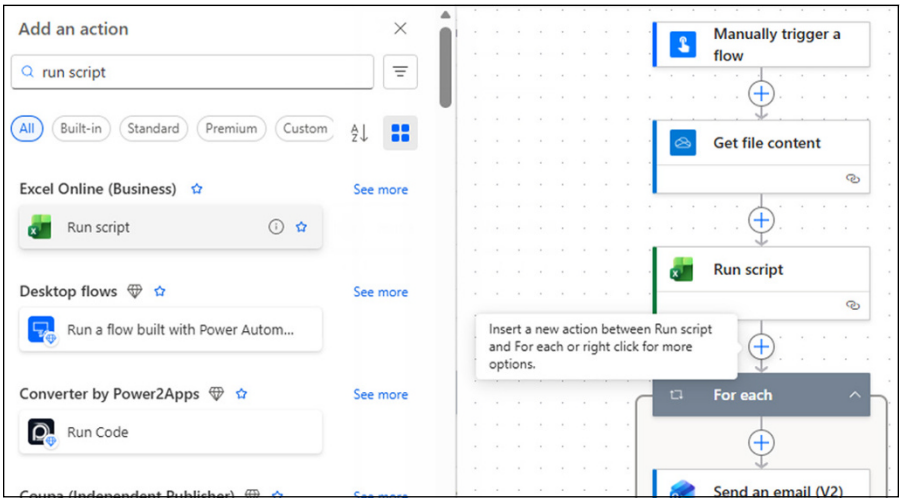


FIGURE 11-16: Add a new Run Script action.

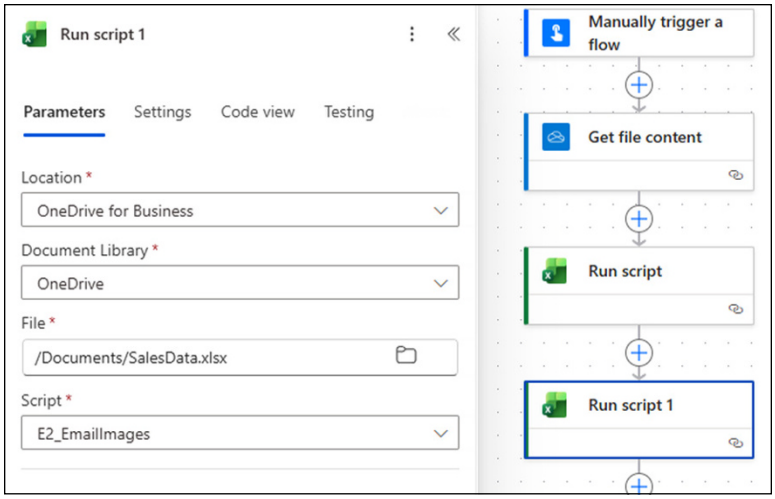


FIGURE 11-17: In the Script input box, specify the script you want to run.

4. **In the flow designer, select the Send an Email action.**

   The screen shown in Figure 11-18 appears. Next, you update the Body input to introduce the image, and then add the metadata coming from the Run Script action.

5. **In the Body input box, add some text introducing the image. Then click the lightning bolt icon, and select Body/Result from the new Run Script 1 action.**

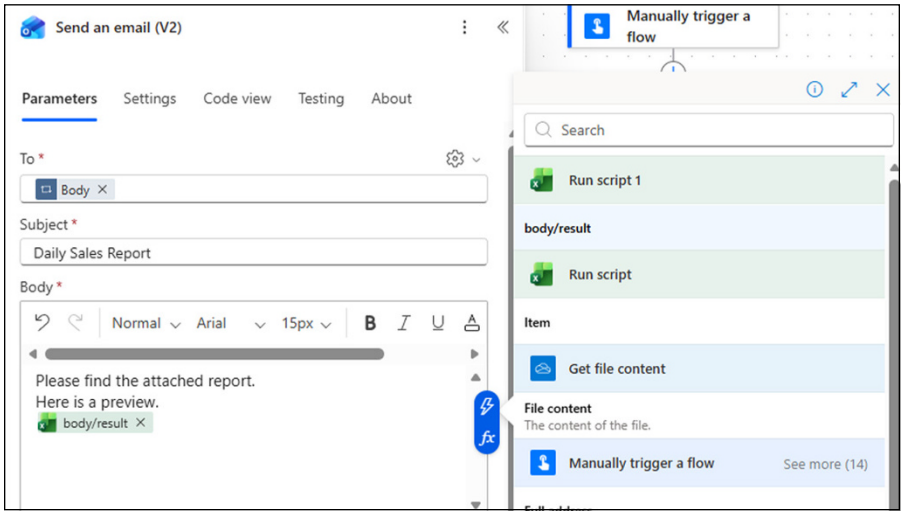


FIGURE 11-18: Update the email body to reference the metadata from the new Run Script action.

The email body now includes the HTML-encoded image being passed by `E2_EmailImages` script.

6. **Save and test the flow.**

Figure 11-19 is the final email sent when the flow runs successfully.

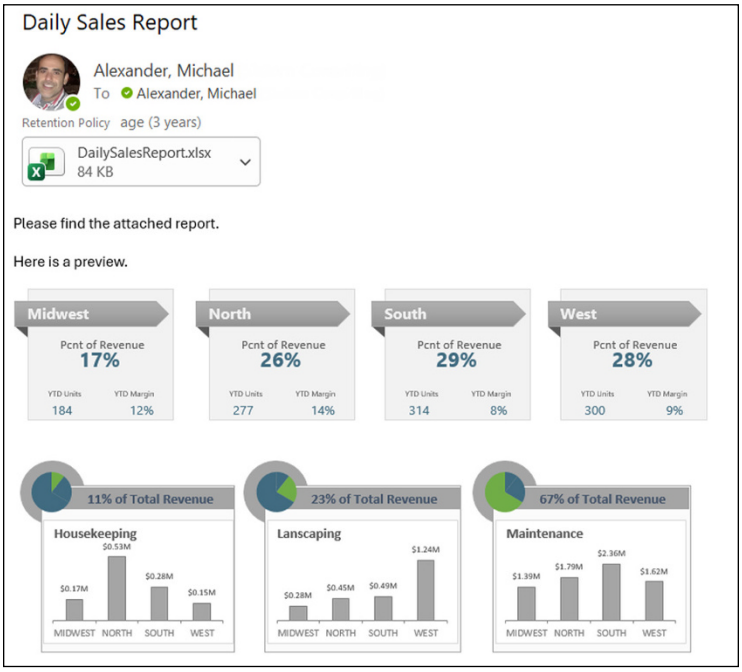


FIGURE 11-19: The final email sent from the flow.

Distributing Excel Data as PDF Files

Converting Excel data to PDF is a common practice across many organizations because it helps control both which information is shared and how it's presented.

PDFs provide a read-only format by default, preventing unintended changes and clearly signaling that the document is final. This makes PDFs ideal for distributing completed reports and finalized deliverables. In addition, PDF files ensure consistent presentation, guaranteeing that data appears the same regardless of the viewer's device, operating system, or screen resolution.

Introducing OfficeScript.convertToPdf

Microsoft provides support for PDF via the `convertToPdf()` method of the Office Scripts namespace.

The following code demonstrates the basic usage of this method. In this example, the workbook is being converted to PDF, saved as `MyReport.pdf`, and then downloaded.

```
function main(workbook: ExcelScript.Workbook) {

    const pdfObject = OfficeScript.convertToPdf();
    const pdfFile = {name: 'MyReport.pdf',
                     content: pdfObject};

    OfficeScript.downloadFile(pdfFile);

}
```

It's important to understand a few things about the `convertToPdf()` method:

- **You can't target specific ranges.** The `convertToPdf()` method converts all visible worksheets in the workbook into a single PDF document. You can't specify individual ranges, charts, or objects for inclusion. To control what appears in the PDF, worksheets must be shown or hidden before the conversion. Hidden worksheets are not included in the final PDF.
- **Page setup matters.** The PDF output respects each worksheet's page layout, print settings, and manual page breaks. The final PDF will look the same as the workbook does in print preview. For best results, make sure you set each worksheet's page size, orientation, margins, and print areas before converting to PDF.
- **Downloads are limited to the user's local `Downloads` folder.** The `OfficeScript.downloadFile()` method is not as useful as it seems. This method saves files only to the local `Downloads` folder of the user running the script. There is no option to save files to SharePoint, OneDrive, network drives, or any other custom location.
- **Empty and protected workbooks can't be converted.** The `convertToPdf()` method will fail if the workbook is protected. In addition, if none of the visible worksheets contain data, the conversion will fail with an error indicating that there is no content to output. At least one visible worksheet must contain data for the PDF to be generated successfully.

Leveraging OfficeScript.sendMail

The OfficeScript namespace also includes a `sendMail()` method that enables you to send an email directly from a script without the need for a Power Automate flow. The email is sent as the user running the script.

The minimum syntax needed for `sendMail()` is at least one `to`, `cc` or `bcc` email address:

```
//Valid
OfficeScript.sendMail({to: email@address.com});


//Valid
OfficeScript.sendMail({cc: email@address.com,
                       cc: email@address.com});

//Invalid
OfficeScript.sendMail({subject: This is the subject'});
```

To enhance emails you are sending, you can use several email properties beyond a simple To address. Table 11-1 highlights each property along with an example syntax.

TABLE 11-1 Mail Properties of sendEmail()

Property	Description	Example Syntax
`to`	Direct recipient(s)	`to: ["email1@address1.com, email2@address.com"]`
`cc`	Carbon copy recipient(s)	`cc: ["email1@address1.com, email2@address.com"]`
`bcc`	Blind carbon copy recipient(s)	`bcc: ["email1@address1.com, email2@address.com"]`
`subject`	Email subject line	`subject: "This is the subject line"`
`content`	Body of the email	`content: "this is text content"` `content: "<p>this is html content</p>"`
`contentType`	Structural format of the email (text or html)	`contentType: OfficeScript.EmailContentType.text` `contentType: OfficeScript.EmailContentType.html`

(continued)

TABLE 11-1 *(continued)*

Property	Description	Example Syntax

`importance`	Tag that indicates the priority or urgency of a message	`importance: OfficeScript.EmailImportance.high`
		`importance: OfficeScript.EmailImportance.normal`
		`importance: OfficeScript.EmailImportance.low`

`attachments`	File to attach	`attachments: [pdfFile]`

By combining the `convertToPdf()` method with `sendMail()`, you can generate a PDF directly from your workbook and immediately email it to recipients. The next script demonstrates an example scenario where you create a PDF file and then send that file as an attachment:

```
function main(workbook: ExcelScript.Workbook) {

//Step 1: Hide sheets we don't want included in the PDF
  workbook.getWorksheet('Chart Data').setVisibility
              (ExcelScript.SheetVisibility.hidden)

//Step 2: Create the PDF file
  const pdfObject = OfficeScript.convertToPdf();
  const pdfFile = { name: "report.pdf",
                    content: pdfObject };

//Step 3: Create an image you can send with the email
  const mySheet = workbook.getWorksheet('Charts')
  const myChart = mySheet.getShape('ChartGroup')
  const myChartImage = myChart.getImageAsBase64(
                 ExcelScript.PictureFormat.png);
  const chartImageBase64 = "" +
                           myChartImage;

//Step 4: Send email with the pdfFile attachment
  OfficeScript.sendMail({
    to: "susanpinkerton@contoso.com",
    cc: "billjanice@contoso.com",
    subject: "Monthly Sales Report",
    attachments: [pdfFile],
    contentType: OfficeScript.EmailContentType.html,
    content: 'Here is the Monthly Sales Report.' +
             '<br/><br/>' +
             '<img src="' + chartImageBase64 + '"/>' +
             '<br/><br/>' +
```

```
            '<strong> See attached PDF </strong>' +
            '<br/><br/>' +
            '-Mike Alexander'
})

//Step 5: Make sheet visible again
  workbook.getWorksheet('Chart Data').setVisibility
             (ExcelScript.SheetVisibility.visible)

}
```

TIP

You can test this script yourself by opening the `SalesReport.xlsx` file and running the `E5_EmailPDF` script. You can find both in the sample files for this chapter at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Step 1 starts the script by hiding the `Chart Data` worksheet. Because you're converting the workbook to a PDF, you want to exclude any worksheets that aren't presentation-ready. Temporarily hiding supporting or raw data sheets produces a cleaner, more professional PDF output.

In Step 2, you call the `convertToPdf()` method to generate a PDF of the current workbook state. The resulting PDF object is given a file name and stored in the `pdfFile` variable. At this point, `pdfFile` represents a fully self-contained PDF that can be attached directly to an email.

In Step 3, you create an HTML-ready image that can be embedded in the email body (similar to the technique covered in the previous section, "Embedding images in emails"). This step is optional, but it adds a nice visual flourish to the message. Here, you generate a Base64-encoded PNG from the grouped set of charts named `ChartGroup`, and then prepend the required data URI header (`data:image/png;base64`) to the Base64 string. The Base64 string enables the image to be used use inside an HTML `<img>` tag.

Step 4 initiates the `sendMail()` method to deliver the email. The message includes the `to`, `cc`, `subject` and `attachment` email properties. The `pdfFile` object is set as the attachment. The `contentType` of the email is set to `HTML`, allowing you to enhance `content` (the body of the mail) with formatting and an embedded image.

Step 5 rounds out the script by making the `Chart Data` worksheet visible again, returning the workbook to its original state once the email has been sent.

Running the script produces the email shown in Figure 11-20. As you can see, the resulting email delivers at-a-glance information while providing the full report as an attachment.

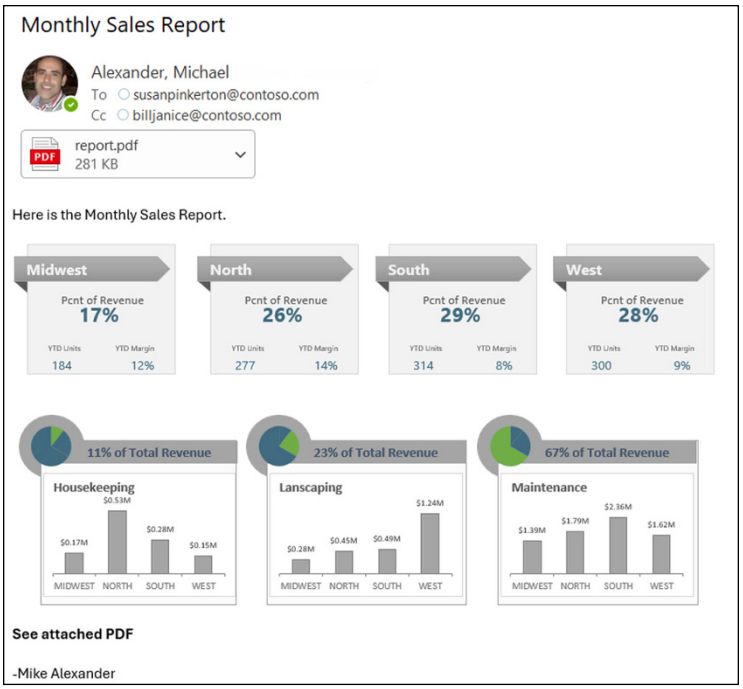


FIGURE 11-20: The final email includes an embedded image and a PDF attachment.

Power Automate remains the premier choice for advanced email-driven workflows. With its rich set of connectors, scheduling, and branching actions, Power Automate offers a level of orchestration that simply can't be replicated with a single method call.

That said, Power Automate isn't always necessary. For straightforward scenarios such as emailing a snapshot of a workbook on demand, the pairing of `convertToPdf()` and `sendMail()` provides a viable alternative that delivers the same result (a polished PDF delivered via email) without the overhead of a cloud flow.

IN THIS CHAPTER

- Using SharePoint actions
- Reading files in SharePoint
- Writing new files to SharePoint

Chapter 12

Integrating Excel with SharePoint

Excel and SharePoint are two of the most widely used tools in Microsoft 365. While Excel serves as the premier tool for data analysis and structured reporting, SharePoint provides centralized storage and a secure collaboration platform. Power Automate can act as the bridge between them, enabling automated data processing that reduces manual work.

Countless scenarios exist where integrating SharePoint and Excel can be instrumental in data-automation processes. In this chapter, you walk through one common scenario: parsing the data in a workbook on SharePoint into multiple workbooks. Through this case study, you'll come to understand the general steps for using some of the SharePoint actions found in Power Automate.

REMEMBER

The code sample files used in this chapter are available in the `Chapter 12 Code` folder, which you can download with the sample files for this book at `www.dummies.com/go/microsoft365excelofficescriptsfd`.

Preparing the Files

It's not uncommon for reporting systems to generate a single master file that contains data for, say, all regions or all employees. Often, these large files are too big to share or contain sensitive data that shouldn't be seen by all parties. In these cases, it's beneficial to split an existing workbook into separate workbooks for each unique value in a column.

For example, Figure 12-1 shows the data stored in the `AllRegionSales.xlsx` workbook, which contains business segment sales data for all the regions in the organization.

	A	B	C	D	E	F
1	Region	Market	Last_Name	First_Name	Business_Segment	Sales Amount
2	MIDWEST	DENVER	AAEMS	JOSEPH	Housekeeping and Organization	$465.33
3	MIDWEST	DENVER	AAEMS	JOSEPH	Landscaping and Area Beautification	$411.60
4	MIDWEST	DENVER	AAEMS	JOSEPH	Maintenance and Repair	$760.31
5	MIDWEST	DENVER	BEALIY	CHRISTOPH	Maintenance and Repair	$2,125.38
6	MIDWEST	DENVER	BEALIY	CHRISTOPH	Landscaping and Area Beautification	$5,909.14
7	MIDWEST	DENVER	BEALIY	CHRISTOPH	Maintenance and Repair	$39,829.79
8	MIDWEST	DENVER	BEWMAN	DIRK	Landscaping and Area Beautification	$319.18
9	MIDWEST	DENVER	BEWMAN	DIRK	Maintenance and Repair	$119.38
10	MIDWEST	DENVER	BIHRINS	KURT	Landscaping and Area Beautification	$914.20
11	MIDWEST	DENVER	BIHRINS	KURT	Maintenance and Repair	$17,645.38
12	MIDWEST	DENVER	BREWN	SCOTT	Maintenance and Repair	$112.01

FIGURE 12-1: You want to split sales data into a new workbook for each business segment.

Imagine that you want to create a workbook for each value in the Business_Segment column: Housekeeping and Organization, Landscaping and Area Beautification, and Maintenance and Repair. You can accomplish this with a Power Automate flow that leverages two Office Scripts to parse the workbook into separate files on SharePoint!

By automating the distribution of this data into separate workbooks on SharePoint, you transform a static file into a dynamic, multichannel reporting system.

To follow along with the example in this chapter, you need access to a SharePoint site and permission to create new files in the `/Documents` folder on that site. Be sure to save the `AllRegionSales.xlsx` file in the `/Documents` folder in SharePoint.

You'll also need access to OneDrive for Business and the capability to save the sample script files to the `/Documents/Office Scripts` folder. Place the `E3_SplitWorkbookByColumn.osts` and `E4_LoadDataToWorksheet.osts` files in the `/Documents/Office Scripts` folder.

Finally, Power Automate needs a blank Excel file as a starter workbook to create the files for each business segment, so create an empty Excel workbook and save it as `BlankWorkbook.xlsx` in your `Documents/Office Scripts` folder on OneDrive. You can use this file as the starter template that your flows can reference whenever they require file content.

Reviewing the Scripts

For this exercise, you need two scripts. The first script (`E3_SplitWorkbookByColumn.osts`) parses a given data table by a specified column to create an array of smaller datasets that can be passed to the flow. It's a bit confusing, so Figure 12-2 shows you what the `E3_SplitWorkbookByColumn.osts` script is doing.

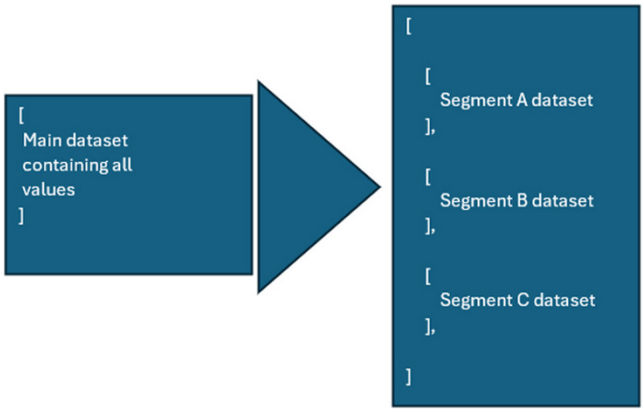


FIGURE 12-2: Split the full sales table into an array of smaller datasets.

The second script (`E4_LoadDataToWorksheet.osts`) simply writes a data array received from Power Automate to a new worksheet.

Since `E3_SplitWorkbookByColumn.osts` is such a big script, it's better to review each step of the script separately. Feel free to bring up the script in Excel as you follow along here.

The script begins with several parameters that make the script flexible and reusable: `DataSheetName`, `DataTableName`, and `SplitColumn`. These parameters are passed in at runtime and allow the script to work with different worksheets, tables, and split criteria, respectively, without changing the code. These

parameters are supplied by Power Automate, enabling dynamic behavior based on flow inputs:

```
function main(workbook: ExcelScript.Workbook,
    DataSheetName: string,
    DataTableName: string,
    SplitColumn: string,
) {
```

In Step 1, the script creates an empty array named `dataOutput`. This array is typed as `WorksheetData[]`, meaning it will store datasets as an array of objects. This array serves as the container for the final grouped datasets that are returned to Power Automate at the end of the script:

```
let dataOutput: WorksheetData[] = [];
```

Step 2 creates a reference to the worksheet whose name was passed into the script using the `DataSheetName` parameter. By accepting a worksheet name via a parameter, you avoid hard-coding sheet names and can reuse the script with different workbooks or sheet names:

```
let dataSheet =
        workbook.getWorksheet(DataSheetName);
```

In Step 3, the script checks whether a table with the specified `DataTableName` already exists in the worksheet. If the table does not exist, the script creates a table using the worksheet's used range and assigns it the provided name. This step ensures that the table exists before the rest of the script attempts to work with it:

```
if (!dataSheet.getTable(DataTableName)) {
    let newTable = workbook.addTable(dataSheet.getUsedRange(),
  true)
    newTable.setName(DataTableName)
};
```

Step 4 retrieves a reference to the table and determines the index of the column specified by the `SplitColumn` parameter. The column index is important because you will use it later to point to the column that contains the data you want to split by:

```
let dataTable = dataSheet.getTable(DataTableName)
const splitColumnNumber = dataTable
.getColumnByName(SplitColumn).getIndex();
```

In Step 5, the script loads all table values into the `dataValues` array. It then creates a second array called `splitValues` by extracting only the values from the split column. Empty values and the column header are filtered out, leaving a clean list of values that will be used to divide the dataset:

```
let dataValues = dataTable.getRange().getValues()
const splitValues = dataValues
.map(value => value[splitColumnNumber] as string)
.filter(s => s.length > 0 && s != SplitColumn);
```

Step 6 removes duplicate values from the `splitValues` array by using JavaScript's `Set` object. A `Set` in JavaScript is an array-like collection where duplicates are not allowed. So, forcing the `splitValues` array into a `Set` allows you to create an array called `uniqueSplits` that contains only distinct values from the split column:

```
const uniqueSplits = Array.from(
                     new Set(splitValues));
```

In Step 7, the script loops through each value in the `uniqueSplits` array. For every unique value, it filters the original `dataValues` array to find rows where the split column matches the current value. These matching rows are stored in a temporary array named `outputData`:

```
uniqueSplits.forEach(splitValue => {
let outputData = dataValues.filter(
    row => row[splitColumnNumber] == splitValue);
```

Step 8 pushes the filtered results into the `dataOutput` array. Each entry includes the split value (`name`) and the corresponding rows (`data`). This structured approach returns multiple grouped datasets as a single array, making the output easy for Power Automate to process:

```
dataOutput.push({
    name: splitValue,
    data: outputData as string[][]
});
```

In Step 9, the script returns the `dataOutput` array. Returning this array makes the grouped data available to Power Automate, where it can be used to write data into new Excel workbooks:

```
return dataOutput;
```

At the bottom of the script, you use `interface` to define the structure of each object in the `dataOutput` array. `interface` tells Power Automate that every object will include a `name` property (the split value) and a `data` property containing a two-dimensional array of strings. `interface` ensures that Power Automate understands the returned data:

```
interface WorksheetData {
    name: string
    data: string[][]
}
```

So, you've just been hit with a new term, `interface`, which is a bit difficult to conceptualize because it's not a method, not an object, and not an array. It's a TypeScript feature that describes the shape of data. Think of `interface` as a way to clearly tell Power Automate what values will be passed and what data type each value will be. The `interface` TypeScript feature is especially useful when you're passing complex structured data (array objects, rows for a table, image objects, or group values).

The second script, `E4_LoadDataToWorksheet.osts`, writes a data array to a specified worksheet and, if desired, formats the data as a table object. The full script follows:

```
function main(workbook: ExcelScript.Workbook,
Formatastable: boolean = false,
Sheetname: string = "Sheet1",
Data: string[][]
) {

//Step 1: Exit function if Data is empty
  if (Data.length == 0 && Data[0].length == 0) {
    return;
  }

//Step 2: Define the landing sheet
  let landingSheet = workbook.getWorksheet(Sheetname);

//Step 3: Get Data row and column count
  let rowCount = Data.length;
  let colCount = Data[0].length;

//Step 4: Define the landingRange and then write values
  let landingRange = landingSheet.getRangeByIndexes
      (0, 0, rowCount, colCount);
```

```
  landingRange.setValues(Data);

//Step 5: If Formatastable (true), format as table
if (Formatastable){
workbook.addTable(landingRange, true);
landingRange.getFormat().autofitColumns();
}
}
```

The script starts with three parameters in the function signature:

- **`workbook`:** This parameter is present in all Office Scripts and provides access to all the objects in the current workbook.
- **`Formatastable`:** This optional Boolean parameter controls whether the written data should be formatted as an Excel table. It defaults to `false`, meaning the data is written as a normal range unless explicitly set to `true`.
- **`Sheetname`:** This parameter specifies the name of the worksheet where the data will be written. It defaults to `Sheet1` if no value is supplied. This parameter is here for flexibility, allowing the script to reference a different starting worksheet without modifying the code.

Step 1 uses the `length` property to check the size of the `Data` array. `Data.length` returns the number of rows, and `Data[0].length` returns the number or columns. The `if` statement checks whether both the number of rows and the number of columns are 0 (that is, the `Data` array is empty). If `true`, the `return` method exits the code.

Step 2 creates a variable named `landingSheet` that references the worksheet specified by the `Sheetname` parameter. Using a dynamic worksheet name allows the same script to target different worksheets depending on how the script is called, making it flexible and reusable.

Step 3 uses the `length` property to check the size of the `Data` array. `Data.length` returns the number of rows, storing it in the `rowCount` variable. `Data[0].length` returns the number of column, storing it in the `colCount` variable. You use these variables later to determine the size of the range where the data will be written.

Step 4 uses the `getRangeByIndexes` method to create a range variable named `landingRange`. The `getRangeByIndexes` method defines a range of cells in a worksheet by specifying a starting row index, a starting column index, the number of rows, and the number of columns. Think of the method as telling Excel to "Give me a range starting here, with this many rows and this many columns." In this case, you tell Excel that `landingRange` starts in row 0 and column 0 (cell A1)

and extends to the number of rows and columns as defined by the `rowCount` and `colCount` variables. After establishing the `landingRange` variable, you use the `setValues()` method to write the values from the `Data` array.

Finally, Step `5` checks the `Formatastable` parameter and, if `true`, converts `landingRange` to an Excel table object with headers. The last line of the script simply autofits the columns to the data.

Building the Flow

Once your scripts are staged and ready, it's time to build the Power Automate flow. The underlying mechanics of this integration mirror the logic explored in the preceding chapter; the primary distinction here is the shift toward using dedicated SharePoint actions to manage your cloud-hosted workbooks.

Reading data arrays from the SharePoint file

The steps outlined here will create the flow to split the `AllRegionSales.xlsx` file by the `Business_Segment` column, creating a new workbook for each business segment:

1. **Sign in to `https://make.powerautomate.com/`.**
2. **Select the Create menu option, and then choose Instant Cloud Flow.**

   The Build an Instant Cloud Flow dialog box appears, as shown in Figure 12-3.
3. **Name the flow** Split Sales Data**, select the Manually Trigger a Flow option, and then click the Create button.**
4. **In the flow designer, click the plus symbol below the Manually Trigger a Flow action. In the Add an action dialog box that appears, enter** get file content **in the search bar. Select the Get File Content action.**
5. **In the Get File Content task pane, click the folder icon to browse for and select the `/Documents/Office Scripts/BlankWorkbook.xlsx` file.**

   The Get File Content action provides Power Automate with the blank Excel file it requires as a starter workbook to create the files for each business segment.
6. **In the flow designer, click the plus symbol below the Get File Content action. In the Add an Action dialog box that appears, enter** Run Script **in the search bar. Select the Run Script action.**

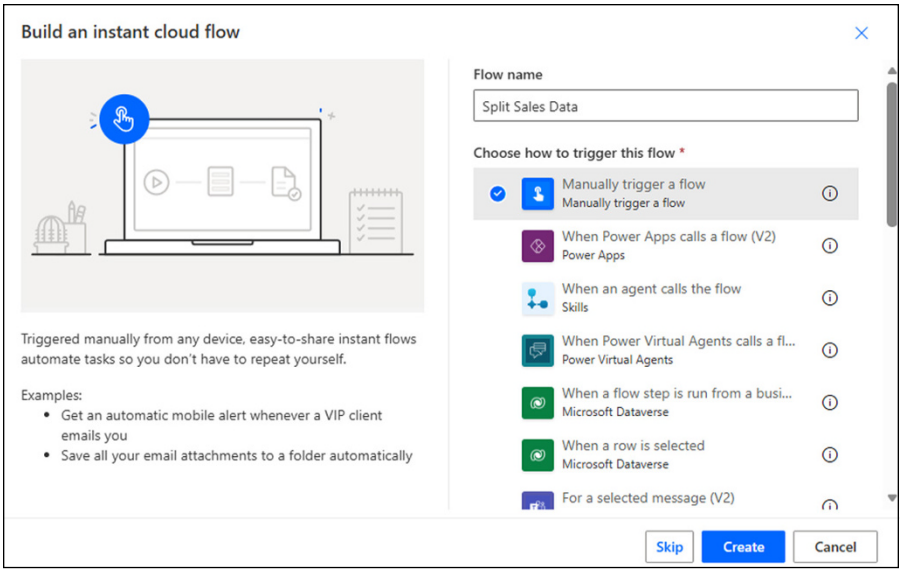


FIGURE 12-3: Create an instant cloud flow.

The Run Script configuration pane shown in Figure 12-4 appears. Here, you want to point to the `AllRegionSales.xlsx` file on SharePoint, and then you want to tell Power Automate to run the `E3_SplitWorkbookByColumn` script. Note that the Location input will be different for you. You will want to enter the URL of your SharePoint site.

7. **Adjust the Location, Document Library, and File input boxes to point to the SharePoint path where you saved the `AllRegionSales.xlsx` file.**

8. **Under Script, use the drop-down menu to select the `E3_SplitWorkbookByColumn` script.**

   When you select the script, Power Automate asks for the script parameters: DataSheetName, DataTableName and SplitColumn. You want to split the full dataset on the `MainData` worksheet by the `Business_Segment` column. If the data was in a table object, you would enter the table name as the `DataTableName`. But in this case, the data is not in a table object, so you can use a default name. `Table1` will do.

9. **Enter** MainData **for DataSheetName,** Table1 **for DataTableName, and** Business_Segment **for SplitColumn.**

10. **Save and test the flow to make sure it returns the expected data.**

After testing a flow, Power Automate automatically generates a Run Results report, like the one shown in Figure 12-5. This report provides a detailed, step-by-step record of how the flow was executed and what information was returned from the flow.

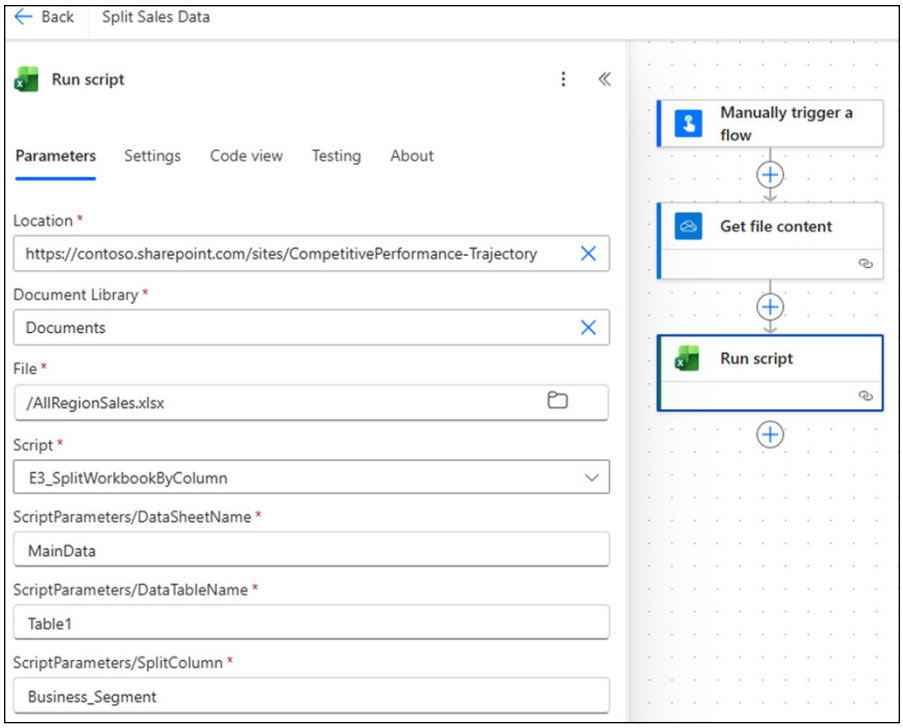


FIGURE 12-4: Configure the Run Script action.

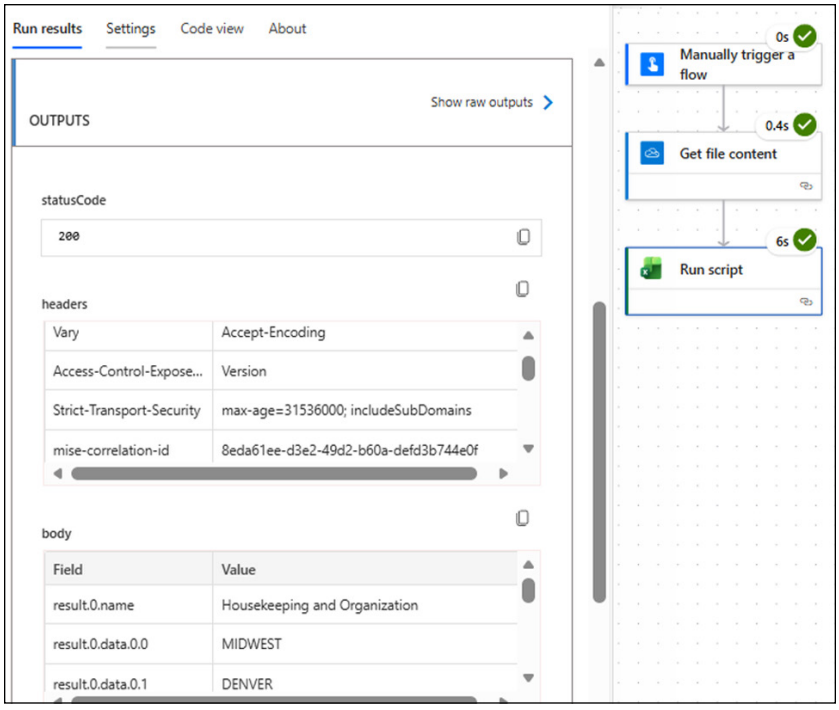


FIGURE 12-5: The Run Results report lets you analyze the data being returned from the flow.

To inspect the returned data in more detail, click the specific action you want to examine (for example, Run Script). Then scroll down to the Outputs section and review the output data.

For even greater visibility, select the Show Raw Outputs option in the upper right to reveal the complete, unformatted response returned by the action.

Writing data arrays to new workbooks

Now that you've tested the first Run Script action and confirmed that it's outputting the data arrays as expected, you can add the actions that incorporate the `E4_LoadDataToWorksheet` script to write each data array to new workbook:

1. **In the flow designer, click the plus symbol below the Run Script action. In the Add an Action dialog box that appears, enter** Compose **in the search bar. Select the Compose action.**

   A Compose action enables you to create a kind of variable that stores information for use later in the flow. Here, you use a Compose action to store an array of column headers.

TIP

   The `E3_SplitWorkbookByColumn` script returns the data for each `Business_Segment`, but it won't return column headers. The Compose Action creates a header row that can be used as column headers for the data.

2. **At the top of the Compose action configuration pane shown in Figure 12-6, Rename the action** Column Headers**. Enter the array of column headers.**

   You can create the array of column headers by entering two opening brackets ([[), two closing brackets (]]) and the column names wrapped in quotes between the brackets:

```
[[
    "Region",
    "Market",
    "Last_Name",
    "First_Name",
    "Business_Segment",
    "Sales Amount"
]]
```

3. **In the flow designer, click the plus symbol below the Column Headers action. In the Add an Action dialog box, enter** Create file **in the search bar. Select the Create File action under the SharePoint connector.**

   The Create File action configuration pane shown in Figure 12-7 appears.

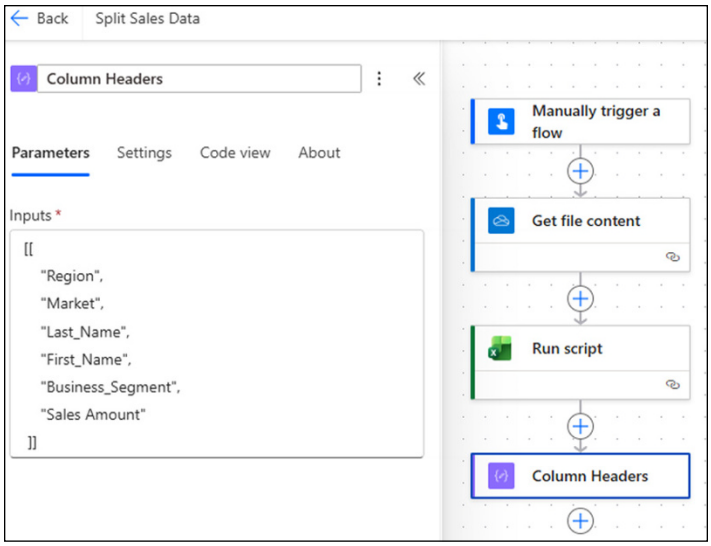


FIGURE 12-6: Add a Compose action that holds an array of column headers.

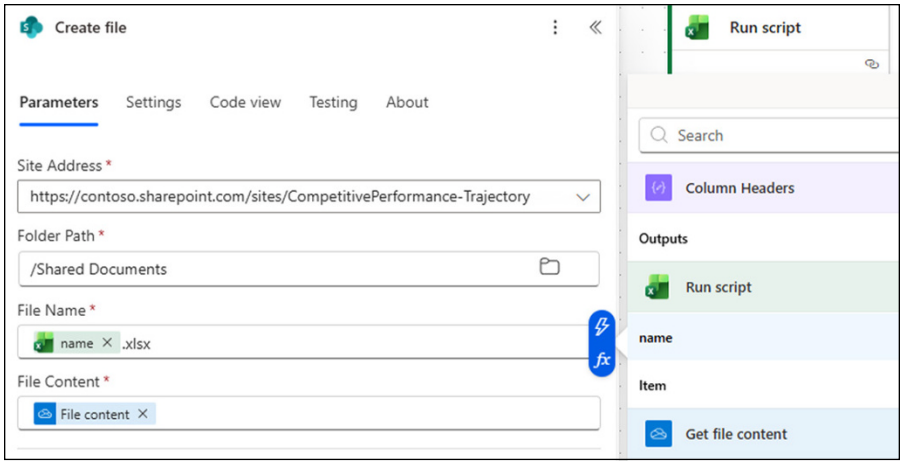


FIGURE 12-7: Configure the Create File action.

4. **Select the SharePoint site address and folder path where you want to place the created files.**

5. **Under File Name, click the lightning bolt icon and select `name` from the Run Script output. Then type** .xlsx **after `name`.**

   The File Name field determines what each created file will be called. Here, you use the `name` output from the Run Script action. In this case, the `name` outputs will be each unique `Business_Segment` name because you specified `Business_Segment` as the split column (see Figure 12-5). Adding .xlsx after the name ensures that the created file is a valid `xlsx` file.

6. **Under File Content, click the lightning bolt icon and select File Content from the Get File Content action.**

   The File Content field determines the content of each file upon creation. In this scenario, you want to create an empty file so you can write the data to it in another step.

7. **In the flow designer, click the plus symbol below the Create File action. In the Add an Action dialog box, enter** Run Script **in the search bar. Select the Run Script action.**

   The Run script configuration pane shown in Figure 12-8 appears. Here, you want to tell Power Automate to run the `E4_LoadDataToWorksheet` script on the newly created file on the SharePoint site. You point to the newly created file by referencing the body/Path metadata for the Create File action.

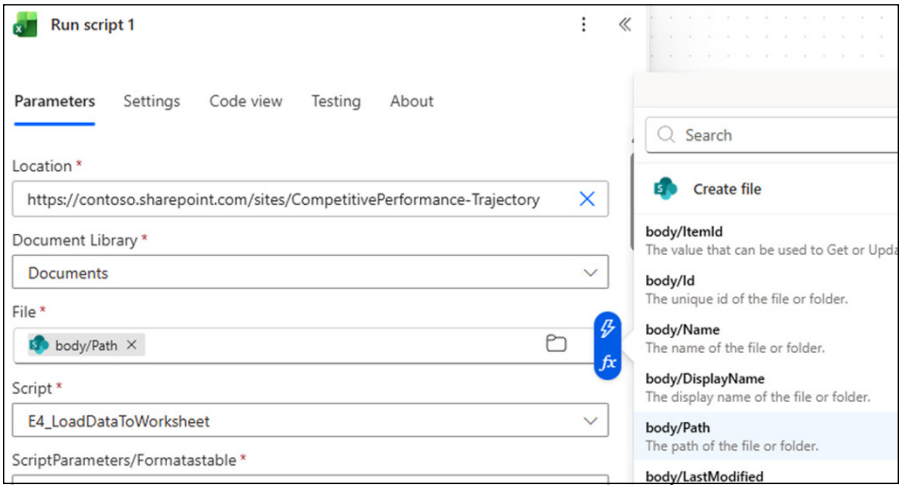


FIGURE 12-8: Configure the Run Script action to run the `E4_LoadDataToWorksheet.osts` script on the newly created SharePoint file.

8. **Under Location, enter the URL of your SharePoint site. Under Document Library, use the drop-down menu and select Documents.**

   This step indicates the location of the newly created file.

9. **Under the File input, click the lightning bolt icon and select body/Path from the Create File output.**

   This step defines the file to run the script on.

10. **Under Script, select the `E4_LoadDataToWorksheet` script.**

    Next you provide the `E4_LoadDataToWorksheet` script with the required `Data` parameter. For this, you want to feed it the results from the first Run Script action (`E3_SplitWorkbookByColumn`). But, because the results don't come

with column headers, you want to prepend the `Column Headers` array created in the Compose action (refer to Figure 12-6). You do this by using an expression.

11. **Click the T icon in the ScriptParameters/worksheetData input box (shown in Figure 12-9).**

    This switches the input box to expression mode.

12. **Click the fx icon to activate the expression builder shown in Figure 12-10.**

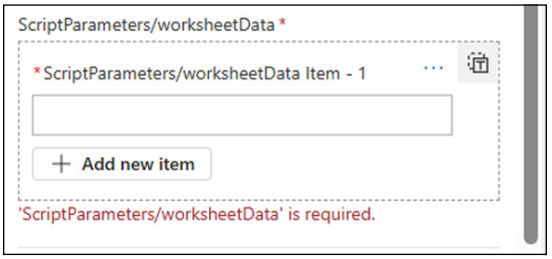


FIGURE 12-9: Click the T icon to switch to expression mode.

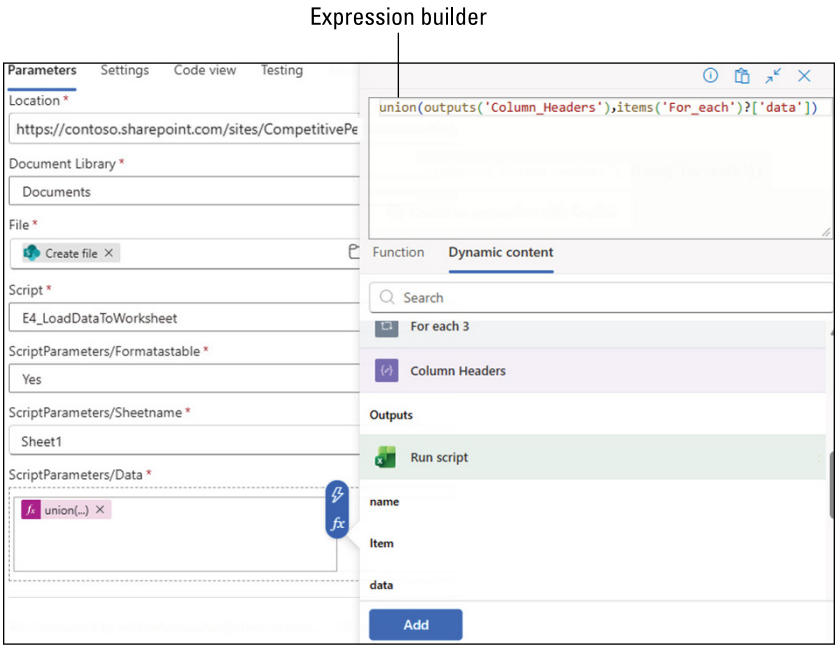


FIGURE 12-10: Use the expression builder to create a union of the column headers and the data.

13. **In the expression input box, enter** union() **and then click the Dynamic Content tab below the expression input box. Under Column Headers, select** `Outputs`**, type a comma, and then under Run Script select Data. Click the Add button.**

14. **Save and test the script.**

A successful run generates new files for each business segment, as shown in Figure 12-11.

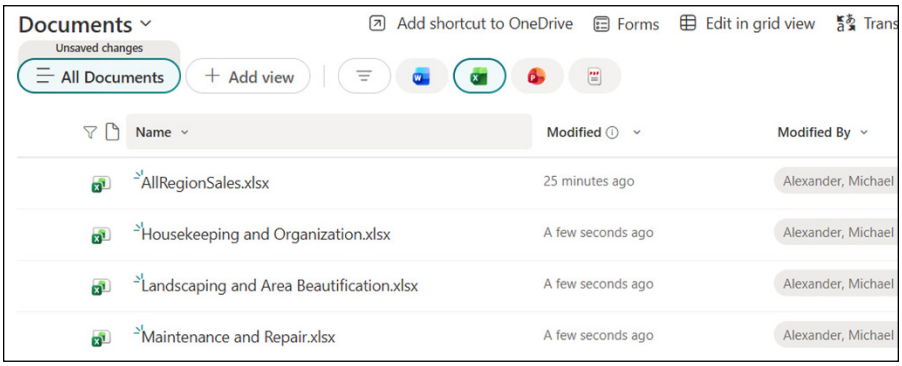


FIGURE 12-11: The final flow will output a new file for each business segment in the `AllRegion Sales.xlsx` file.

5 The Part of Tens

IN THIS PART . . .

Add clarity to your automation processes with ten tips for documenting and organizing your Office Scripts.

Improve script performance with ten tips for writing more efficient Office Scripts.

IN THIS CHAPTER

- Understanding effective naming standards
- Leveraging comments effectively
- Building logical partitioned scripts
- Organizing script files effectively

Chapter 13

Ten Best Practices When Writing Scripts

There's a saying in programming: Always code as if the person who ends up maintaining your script knows where you live. That might be extreme, and the future you is often the one who suffers most from poorly documented code. However, in the world of Excel automation, an undocumented script becomes a puzzle to solve months after you wrote it.

This chapter outlines ten best practices for writing Office Scripts that are readable, easy to maintain, and easy to share with others across project teams.

Apply Descriptive Variable and Function Names

Name every variable, function, parameter, and constant so that anyone reading the script can immediately ascertain its purpose without needing to scan multiple lines of code.

In real-world scenarios, you want to name your variables to reflect the work being done. For instance, this first example uses abbreviated names:

```
function main(workbook: ExcelScript.Workbook) {

  let ws = workbook.getActiveWorksheet();
  let rng = ws.getUsedRange();
  let val = r.getValues();

}
```

Using abbreviations like `ws` and `rng` might seem harmless, but what happens when you need to reference multiple objects of the same type? Do you use `rng2` and `rng3`? Or `ws1` and `ws2`? You can see how such a vague naming convention could become a problem with more complex scripts that reference multiple objects.

A better convention is to use full words over abbreviations. There's nothing wrong with spelling out what your variables represent. The following is the same code but with descriptive variable names:

```
function main(workbook: ExcelScript.Workbook) {

  let salesSheet = workbook.getActiveWorksheet();
  let salesDataRange = salesSheet.getUsedRange();
  let salesValues = salesDataRange.getValues();

}
```

In general, keep the following in mind when naming your scripting objects:

- **Variables should start with nouns.** Instead of `ws`, use `salesSheet`. Instead of `rng`, use `salesValues`.
- **Function names should start with a verb.** For example, use `formatSalesReport()` or `validateEmailAddresses()`.
- **Boolean variables should read like questions starting with `is`, `has` or `should`.** For example, use `isValidDate`, `hasEmptyRows`, `shouldSkipHeader`.

Use Established JavaScript Conventions

In TypeScript, it's considered best practice to use established JavaScript naming conventions (see Table 13-1). Adopting these conventions makes your code self-documenting, helping other developers understand the role of each identifier without additional context.

TABLE 13-1 JavaScript Naming Conventions

Category	Convention	Examples
Local variables	camel case	`invoiceDate`, `totalRevenue`
Functions	camel case with verb prefix	`applyFormat()`, `clearSheet()`
Boolean variables	camel case with `is`/`has` prefix	`isValidRow`, `hasEmptyFields`
Constants (fixed values)	upper snake case	`MAX_ROWS`, `DEFAULT_DATE`
Type aliases/interfaces	Pascal case	`OrderRecord`, `SummaryResult`
Parameters	camel case	`worksheetName`, `columnIndex`

You don't have to adhere to all the conventions listed in Table 13-1. For example, I don't like to have my constants in upper snake case, so I ignore that convention and use my preferred camel case style. What's important is that you select a set of conventions that works for your organization and apply it uniformly. Consistency allows readers to become familiar with your style, reducing the effort required to understand and maintain the code.

For example, the following function uses inconsistent naming conventions, leaving the reader to guess what each identifier is referring to:

```
function Proc(workbook: ExcelScript.Workbook) {

  let WS = workbook.getActiveWorksheet()
  let DoneFlag = false
  const max_limit = 500

}
```

The next function uses the proper naming conventions:

```
function processSales(workbook: ExcelScript.Workbook) {

  const processSheet = workbook.getActiveWorksheet()
  let   isProcessed = false
  const maxIterations = 500
}
```

The function uses the more descriptive name `processSales`. It's also easier to figure the role of each variable in the process. Even without the benefit of the rest of the code, the purpose of each variable is clear.

Apply Indentation Standards

Indentation in coding helps readability by providing a visual representation of the structural relationship between statements (which code is inside which block). Properly indented code is much easier to read and understand, especially when reading nested logic like loops and `if` statements.

In the following example, I'm unlocking cell A1 on the active sheet if it's not already unlocked. Without the indentation, it takes a bit of effort to determine the structure of the code:

```
const activeSheet = workbook.getActiveWorksheet();
const isLocked = activeSheet.getRange('A1')
.getFormat()
.getProtection()
.getLocked();

if (!isLocked) {
console.log('Already unlocked');
} else {
activeSheet.getRange('A1')
.getFormat()
.getProtection()
.setLocked(false);
}
}
```

In contrast, the indented version improves readability, effectively shifting the processing effort from cognitive to visual pattern recognition. It's much easier to determine which blocks of syntax go together:

```
function main(workbook: ExcelScript.Workbook) {
    const activeSheet = workbook.getActiveWorksheet();
    const isLocked = activeSheet.getRange('A1')
                                .getFormat()
                                .getProtection()
                                .getLocked();

    if (!isLocked) {
        console.log('Already unlocked');
    } else {
        activeSheet.getRange('A1')
          .getFormat()
          .getProtection()
          .setLocked(false);
    }
}
```

The general guidance for indentation is to indent four spaces per indent level. Every opening curly bracket ({) starts a new indented block and every closing curly bracket (}) returns to the previous level of indentation.

Align each continuation line with the opening delimiter of their expression. For example, when method chaining, you want to align the dot operators to visually signal that each line is part of the full method chain:

```
activeSheet.getRange('A1')
           .getFormat()
           .getProtection()
           .getLocked();
```

Write Effective Comments

A common mistake developers make is writing comments that simply restate the code in plain English. Instead of describing what the code does, good comments explain why the code was written that way. Effective comments provide context that isn't immediately obvious from reading the syntax alone, such as clarifying underlying business logic, documenting edge cases, or explaining necessary workarounds and constraints.

The following comments may be useful while you're writing the code, but once the code is working, they become useless to others because they only restate what is obvious by reading the code:

```
// Set the row count to the length of values array
    const rowCount = allValues.length;

// Loop through each row
    for (let i = dataStartRow; i < rowCount; i++){...}
```

The comments shown below provide valuable context, helping the reader better understand the logic behind why the associated values were used:

```
// First two rows are always header information
// Data always begins on row 2
    const dataStartRow = 2

// All orders must have a minimum $50 surcharge appliedPer
  finance external purchasing policy.
    const miniumSurcharge = 50
```

Leverage Comments to Partition Code

As scripts grow beyond a few dozen lines, they can become difficult to navigate. Comments can provide structural signposts that allow for quick visual navigation to key parts of your script. Use comments to create logical section dividers that enable the reader to locate specific blocks of code instantly.

The following snippet is a basic example of using section comments to make it easier to scan the script and quickly locate specific logical parts:

```
function main(workbook: ExcelScript.Workbook) {

    /* ============================
    1. Configuration
    ============================*/
    const sourceSheet = 'RawData'
    const outputSheet = 'Summary'
    const ThresholdAmount = 10000
```

```
    /* ============================
    2. Initializing variables
    ============================*/
let sourceSheet = workbook.getWorksheet(sourceSheet)
const outputSheet = workbook.getWorksheet(outputSheet)
const sourceRange = sourceSheet.getUsedRange()
const sourceValues = sourceRange.getValues()

    /* ============================
    3. Processing data
    ============================*/
    const highValueRows = filterTransactions(
      sourceValues, ThresholdAmount
    );

    /* ============================
    4. Outputs
    ============================*/
    writeResultsToSheet(outputSheet, highValueRows);

}
```

Note the alternative syntax for the comments. Instead of the double forward slash (`//`), I created a comment block, starting it with `/*` and ending it with `*/`. TypeScript treats everything between those two symbol pairs as a comment.

Centralize Object References

It's common to reference objects using hard-coded strings, such as `workbook.getWorksheet('Sales Data')`, where the worksheet name is hard coded, or `getRange('A1:G1')`, where the range address is hard coded.

Avoid embedding specific strings like worksheet names or range addresses in the body of your script. Instead, centralize these references at the beginning of your script. By declaring your worksheet names and range addresses as variables at the top, you create a single control point to make updates.

To see this concept in action, look at the following script:

```
function main(workbook: ExcelScript.Workbook) {

    /* ============================
       1. Hard-coded references
       ============================*/
    const sourceSheet = 'Sales Data'
    const sourceTable = 'tblSales'
    const reportRange = 'A1:G10000'
    const reportSheet = 'Sales Summary'

    /* ============================
       2. Initialize variables
       ============================*/
    let salesValues = workbook.getWorksheet(sourceSheet)
                        .getTable(sourceTable)
                        .getRange();
    let outputSheet = workbook.getWorksheet(reportSheet);
    let outputRange = outputSheet.getRange(reportRange);

}
```

I centralized all the hard-coded values for the script at the top. Now if the workbook structure changes, I can confidently adjust the values in this one location and know the script is fully updated.

Break Long Scripts into Custom Functions

When all your logic lives in one giant block of code, reviewing and understanding the code is a challenge. There's just too much context to hold in your head at once. Ideally, your `main()` function should read like a high-level outline, while custom functions do the heavy lifting behind the scenes.

Remember that you can create custom functions to perform repeatable actions to reduce the amount of code you need in your `main()` function. For example, instead of repeating the syntax to color each new range, you can create a custom function like the one in the following script. Here, I color each specified range yellow by simply calling the `colorYellow` function:

```
function main(workbook: ExcelScript.Workbook) {
  let mySheet = workbook.getWorksheet('Sheet1')
```

```
  mySheet.getRange('B2').select(); colorYellow();
  mySheet.getRange('B4').select(); colorYellow();
  mySheet.getRange('B5').select(); colorYellow();

 /* ============================
     Custom function
    ============================*/
  function colorYellow() {
    workbook.getActiveCell().getFormat()
                            .getFill()
                            .setColor('#ffff00');
  }

}
```

Ideally, for complex processes, your `main()` function will simply provide the needed variables and then call the custom functions necessary to accomplish the task at hand. Figure 13-1 illustrates the general pattern.

```
1  function main(workbook: ExcelScript.Workbook) {
2
3    /* ============================
4       1. Hard Coded References
5       ============================*/
6    const ordersworksheet = "Orders";
7    const reportworksheet = "Monthly Report";
8
9    /* ============================
10      2. Initialize Variables
11      ============================*/
12   const ordersSheet = workbook.getWorksheet(ordersworksheet);
13   const reportSheet = workbook.getWorksheet(reportworksheet);
14
15   /* ============================
16      3. Call functions to process the data
17      ============================*/
18   filterInvalidOrders(ordersSheet);
19   writeMonthlyReport(reportSheet)
20   formatReportSheet(reportSheet);
21
22   /* ============================
23      Custom Functions
24      ============================*/
25   function filterInvalidOrders(sheet: ExcelScript.Worksheet):
       (string | number | boolean)[][] {
26     return sheet.getUsedRange().getValues().slice(1);
27 }
```

FIGURE 13-1: For complex reporting processes, consider simplifying things by leveraging custom functions.

Build Modular Scripts

It's tempting to create one script that does everything: fetches data, cleans the data, performs calculations, and creates a nicely formatted report. But monolithic scripts like this are a nightmare to understand and maintain.

For large or complex processes, it's a best practice to adopt modular architecture. In this context, *modularization* means transitioning from a single procedure to a suite of specialized `osts` files, each dedicated to a specific phase of the larger business process.

Imagine a reporting process divided into three distinct scripts: `GetExternalData.osts` to get external data, `Transformation.osts` to clean and transform the data, and `ProcessReport.osts` to perform calculations and format reports. This separation provides several critical advantages:

- **Integrated safety nets:** Modularization allows you to validate data at each stage. For example, if `Transformation.osts` detects a missing column or invalid data type, it can throw an error and halt the execution chain before `ProcessReport.osts` runs. This approach prevents corrupted data from reaching your final output.
- **Isolated maintenance:** You can update the logic in `GetExternalData.osts` (for instance, to accommodate a new data source) without touching or the risk of breaking the logic in the transformation or reporting scripts.
- **Power Automate orchestration:** When your logic is broken into separate `osts` files, you can use a Power Automate flow as an orchestrator to run different modules at different points of a larger process.
- **Collaborative reusability:** In a team environment, modular scripts become a shared library of assets. General-purpose scripts such as `CreateTableOfContents.osts` or `CreateWaffleCharts.osts` can be reused by multiple developers across different projects, saving development time.

Organize a Script Library

When you create an Office Scripts, the script is saved to your OneDrive or SharePoint account as an independent `osts` file in the `Documents/Office Scripts` folder (see Figure 13-2). Because Excel requires scripts to reside in this specific directory to run them, this folder can become a cluttered mess of automation.

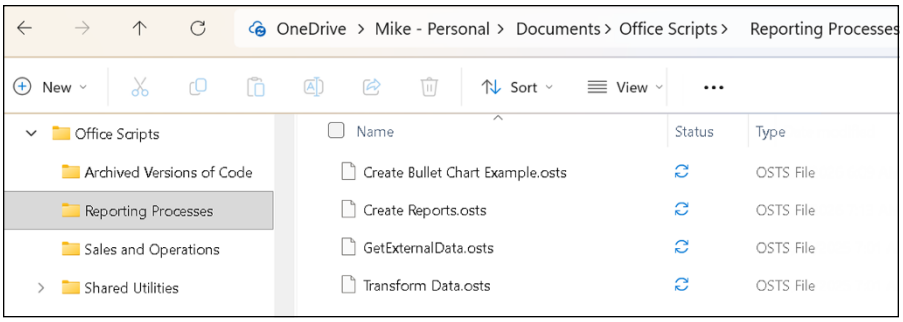


FIGURE 13-2: All scripts are in your OneDrive account under `Documents/Office Scripts`.

Although all scripts must be in the Office Scripts root, you can still house copies of each `osts` file in organized subdirectories under the `Documents/Office Scripts` folder.

Maintaining an organized file system allows you to curate a searchable repository of scripts, which is valuable in shared environments. Figure 13-3 illustrates a sample folder structure under the `Documents/Office Scripts` folder.

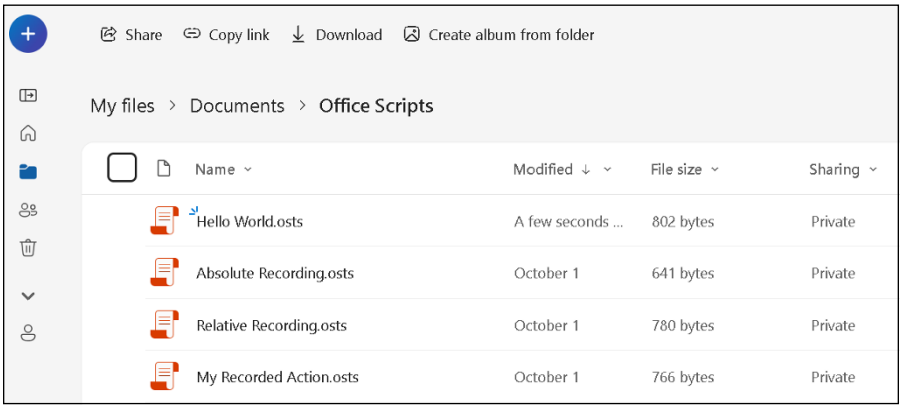


FIGURE 13-3: Curating scripts in organized folders makes it easier to share important scripts.

Remember that Excel will see only scripts located in the Office Scripts root folder, so you can't move the `osts` files to your organized subdirectories. You're simply storing copies. Think of your subfolders as your archive library, while the root folder acts as your active workspace.

Manage Script Versions

When you share an Office Scripts with a team or embed it in a Power Automate flow, it ceases to be a personal project file and becomes an important piece of the reporting infrastructure. Modifying the script without signaling a version change or without preserving the capability to roll back to a known stable state introduces a significant risk.

Since Office Scripts do not currently support native version control, the burden of versioning falls on the developer. A definitive best practice is to store every version of a script using a versioning naming convention. Most developers use what is known as *semantic versioning*, which uses a three-part number (representing major.minor.patch) to signal the magnitude of change from the preceding version. Table 13-2 provides some insight on how semantic versioning works.

TABLE 13-2 Versioning an Office Scripts

Action	Major.Minor.Patch Number	Version Name
We released the first version of a new script.	1.0.0	Myscript_1_0_0.ost
We patched a reported bug, but made no changes to inputs, outputs, or behavior.	1.0.1	Myscript_1_0_1.ost
We made a minor change to the script to add functionality. No changes in workbook or data structure.	1.1.0	Myscript_1_1_0.ost
We added another feature to the script but did not change the workbook or data structure.	1.2.0	Myscript_1_2_0.ost
We made a major change to the workbook. We deleted a worksheet and changed the columns in the main sales table, requiring a change to how the script references the objects.	2.0.0	Myscript_2_0_0.ost

In addition to saving each version of the script with a version number, it's a best practice to document every change in a header change log at the top of shared scripts. In this way, you create a clear audit trail and ensure that everyone knows when each feature and patch was applied. Figure 13-4 illustrates an example change log.

```
/*
 * SCRIPT:   Myscript.osts VERSION:  2.0.0
 *
 * CHANGE LOG:
 * 2.0.0  2026-06-16  We deleted Invoice worksheet and changed the
 *                    columns  in the main sales table.
 * 1.2.0  2026-06-01  Report now writes to Region Summary sheet
 * 1.1.0  2026-05-21  Added new function to export charts as images
 * 1.0.0  2024-05-01  Initial release.
 */
function main(workbook: ExcelScript.Workbook) {

  // ...

}

```

FIGURE 13-4: Be sure to include a change log for any shared scripts.

IN THIS CHAPTER

- Understanding batch processing
- Leveraging in memory processing
- Avoiding performance pitfalls

Chapter 14

Ten Tips for Improving Script Performance

When sharing your Office Scripts, you want to give your audience the best experience possible. A large part of that experience is ensuring that performance is not an issue. Performance (in terms of running a script) is typically synonymous with speed — speed in how quickly your script performs certain actions like writing, filtering, and formatting data.

Performance is heavily dependent on how you structure your code. Inefficient patterns such as reading cells in a loop, repeatedly resolving object references, or triggering unnecessary recalculations make your script slower than it needs to be.

This chapter offers ten tips to keep in your back pocket to help your scripts be as nimble as possible.

Read and Write Data in Batches

The primary performance cost in Office Scripts is the latency created between the script runtime and the Excel engine. Think of the script runtime and the Excel engine as being on two different islands. Every time your script calls an Excel API

model method such as `getRange` or `getValue`, the script runtime is like a tiny boat sent out to carry one piece of data across the ocean and return. So, if you read 1,000 cells individually, you're forcing that little boat to make 1,000 separate roundtrips.

Instead, load your entire dataset into an array at the start, perform all logic and transformations in memory, and write the results back to the workbook in one final operation. That process is like sending out a cargo ship that can carry 1,000 pieces of data in one trip.

Using the tiny boat

Examine the following code snippet:

```
for (let i = 1; i <= 1000; i++) {
 let val = sheet.getCell(i, 0).getValue();

  sheet.getCell(i, 1).setValue(val * 1.1);

}
```

In this snippet of code, I use a loop to traverse 1,000 cells. On every loop, I set the `val` variable by calling the `getcell(i.1).getValue()` method. Then I call a second method, `getCell(i, 2).setValue(val * 1.1),` which multiplies the `val` variable by 1.1 and writes that value back to Excel.

In this small snippet of code, I made that tiny boat travel across the ocean 2,000 times! One time to get the value in a cell and another time to write back to the cell, for 1,000 loops. Who knew that such a harmless-looking snippet of code could be so much trouble?

Using the cargo ship

Now let's examine a scenario where I process data in one batch. This code uses Excel's `getvalues()` method:

```
const dataRange = sheet.getRange('A1:A1000');

const vals = dataRange.getValues();

const updatedVals = vals.map(row =>
                   [row[0] as number * 1.1]);

sheet.getRange('B1:B1000').setValues(updatedVals);
```

The `getvalues()` method stores all the values in the specified range into an array, which is kept in memory. I just loaded all 1,000 values onto the cargo ship.

Now I use the `map()` method, which alters the values in a given array, outputting the transformed values into a new array. In the example, I use the `map()` method to multiply all the values in the `vals` array by 1.1, and then store the answers in a new array called `updatedVals`.

Finally, I use the `setValues()` method to write all 1,000 cells back to Excel from the `updatedVals` array. The cargo ship came back in one trip!

Avoid Repeated Lookups

You should never declare worksheet or range variables inside a loop. Every time you call `getWorksheet()` or `getRange()`, the script runtime must navigate the Excel object hierarchy and perform a lookup. By storing references in a variable before the loop starts, you eliminate redundant lookups and significantly reduce execution time.

To understand this concept, take a look at the following poorly written script:

```
function main(workbook: ExcelScript.Workbook) {

  for (let i = 0; i < 500; i++) {

     let targetRange = workbook.getWorksheet('Sales')
       .getTable('OrdersTable')
       .getColumnByName('Revenue')
       .getRangeBetweenHeaderAndTotal()

  targetRange.getCell(i, 0).setValue(i * 100);
  }
}
```

There's no reason to have the scripting runtime continuously look up the `Sales` worksheet or `OrdersTable`. The `targetRange` variable could be declared before the loop starts. This is not to say that declaring variables inside a loop is always a problem. Often, declaring a variable is ideal for the situation.

For example, declaring `name` and `price` inside the loop in the following script does not cause a performance problem. A simple value assignment is necessary because the values will change with each loop. Also, it's not a resource-intensive

operation because the lookup is being done in the context of the stored array of values:

```
for (const row of values) {
  const name = row[0];
  const price = row[1];
  console.log(name, price);
}
```

The real rule to keep in mind is to avoid redundant operations inside a loop. Don't declare variables that return the same result every iteration.

Sort and Filter in Arrays, Not Cells

If you use Action Recorder to capture sorting and filtering, it will happily write code using the `getSort().apply` method. But if you're doing extensive sorting and filtering in your script, you should be doing it in arrays, not against cells.

Sorting and filtering data through Excel API calls such as `.getSort().apply` triggers Excel to recalculate and redraw the screen. These operations are expensive in terms of speed when you need only a transformed copy of the data.

An alternative is to use TypeScript's `filter` and `sort` methods. You can use these two methods on any array in memory, resulting in a more efficient script. For example:

```
function main(workbook: ExcelScript.Workbook) {

  let mySheet = workbook.getActiveWorksheet()
  let myRange = mySheet.getRange('A2:B12')
  let valuesArray = myRange.getValues()

  let sortedArray = valuesArray
   //Filter out rows where first column is blank
    .filter(row => row[0] !== '')

   //Sort by second column desc
    .sort((a, b) => b[1] - a[1]);

  //Write new sorted array to the Report worksheet
  let outputRange = workbook.getWorksheet('Report')
    .getRange('A2:B' + (sortedArray.length + 1));
```

```
  outputRange.setValues(sortedArray);

}
```

The `filter` method enables you to efficiently search through each row in a given array and filter only the values that meet a given condition. In this case, I tell the script to filter only values in the first column that are not blank:

```
.filter(row => row[0] !== ''
```

The `sort` method sorts the values in the array by the column you specify. In this scenario, I am sorting column two descending:

```
b[1] - a[1]
```

To sort ascending, you would sort `a` to `b`, like this:

```
a[1] - b[1]
```

Sorting and filtering values in memory is much more efficient than calling the sorting and filtering features in Excel via Office Scripts.

Apply Formatting to Ranges, Not Cells

Formatting is one of the most overlooked performance bottlenecks in Office Scripts. It seems like an innocent action. After all, you're just applying formatting to a few cells. But the issue here, again, is that each time you format a cell, you need to call the `.getFormat()` method, which forces a roundtrip between the scripting runtime and the Excel engine. Formatting one or two cells is not a big deal, but if your script is littered with instances of formatting disparate cells, it can cause a performance slowdown.

Wherever possible, apply formatting to ranges instead of individual cells to limit the number of trips the scripting runtime needs to make. For instance:

```
let mySheet = workbook.getActiveWorksheet()
let myRange = mySheet.getRange('A2:B12')
let myRangeFormat = myRange.getFormat()

myRangeFormat.getFont().setBold(true);
myRangeFormat.getFont().setColor('#FF0000')
```

A general rule is to process your data first, and then apply your look-and-feel to the entire range as a final step in your script.

Use Table Objects Whenever Possible

Table objects are more efficient by far to work with than ranges. Table objects have structured and named references that internally optimize how the scripting runtime reads and writes data in them.

Table objects offer methods like `getRangeBetweenHeaderAndTotal()` or `getColumnByName()`, which provide a direct path to your data and bypass the expensive discovery phase required to find the last row in a standard range.

Also, by using table objects, you ensure that your script processes only rows that contain data, preventing you from storing large ranges that do not contain data. For example:

```
function main(workbook: ExcelScript.Workbook) {

//Inefficient: Possibly storing many empty rows
  let mySheet = workbook.getActiveWorksheet()
  let myRange = mySheet.getRange('A1:G50000')

  //Better: Only the table rows are loaded
  let table = workbook.getTable('SalesTable');
  let data = table.getRangeBetweenHeaderAndTotal()
             .getValues();

}
```

TIP

For a refresher on scripting table objects, revisit Chapter 7.

Leverage the getUsedRange Method

Moving from static ranges to dynamic ranges can give you a performance win. Hardcoding a massive range like A1:A10000 is essentially telling your cargo ship to pick up 100,000 crates even if only 200 of them have anything inside.

If you can't use a table object, you can leverage the `usedRange()` method to reference the range of cells that have values, formulas, or formatting. This method offers a shortcut to referencing all the cells that have been touched.

The following sample script demonstrates one way of leveraging the `usedRange()` method to capture a targeted range containing the data you're interested in:

```
function main(workbook: ExcelScript.Workbook) {

  //Get the last row index for the used range
  let mySheet = workbook.getActiveWorksheet()
  let usedRange = mySheet.getUsedRange()
  let lastRow = usedRange.getLastRow().getRowIndex()

  // Now build a targeted range
  let myRange = mySheet.getRange('A2:D${lastRow + 1}');
  let myValues = myRange.getValues();

}
```

It should be noted that the `usedRange()` method is not without its drawbacks. It returns all cells with values, formulas, or formatting. So, if your worksheet has a stray border on cell 100,000 — oops — your used range is now A1:A100000. For this reason, you should consider moving to table objects, as mentioned in the preceding section.

Postpone Calculations Until the End

Calling the `calculate()` method forces a full recalculation of every formula across the workbook. In a workbook with many formulas, this process can take several seconds per call. Calling these methods inside a loop can slow your script to a crawl.

Excel's engine handles recalculations automatically when you set values. The only scenario that requires an explicit `calculate()` call is when your script needs to read a formula's result immediately after writing data to a cell that the formula depends on. Even then, you should batch your writes to the workbook first and then call `calculate()` once.

Following is an example of a script with a critical performance error. Note that the `calculate()` method is inside the `for` loop. If there are 1,000 rows, this script

would force a full calculation 1,000 times, turning a script that should run in 1 second to one that takes 2 minutes:

```
for (let i = 0; i < rows; i++) {
  mySheet.getCell(i, 0).setValue(newData[i]);

  workbook.getApplication().calculate(
           ExcelScript.CalculationType.recalculate);
}
```

The key is to calculate only one time after you've completed writing values that will require a recalculation of formulas:

```
// Batch all your writes first
mySheet.getRange('A1:A1000').setValues(newData);

//Calculate once if you need to
workbook.getApplication().calculate(
         ExcelScript.CalculationType.recalculate);

//Now you can read formula results
const formulaResults = mySheet.getRange('B1:B1000')
                              .getValues();
```

Minimize Formula Usage in Scripts

Stop including formulas in your scripts if the user doesn't need to see the math behind the result. Every time your script uses the `setFormulas()` method, it adds a formula that increases the overhead of the workbook. When a script writes 1,000 formulas into a worksheet, those formulas stay there after the script ends. Now, every time a user changes anything in the workbook, Excel may try to recalculate those 1,000 cells, even if the user doesn't need them.

For example, the following script uses the `setFormulas()` method, leaving 1,000 active calculations in the worksheet:

```
const math = data.map(row => ['=A${row.index} * 1.1']);
sheet.getRange("B1:B1000").setFormulas(math);
```

A better alternative is to use the `setValues()` method to write static values to the worksheet:

```
const values = data.map(row => [Number(row[0]) * 1.1]);
sheet.getRange("B1:B1000").setValues(values);
```

Minimize Actions that Change Display

Every time your script adjusts a visual element in the workbook, such as autofitting columns, resizing rows, or freezing panes, it triggers the Excel layout engine to repaint the display layer. The engine recalculates the width, height, and alignment of the entire display layer to accommodate your change. If these calls are made in a loop, you force Excel to repaint the screen repeatedly, creating a massive performance lag.

A best practice is to treat any kind of visual formatting as a final step in your script. Complete all the data work and calculations first, and then apply your layout updates to entire columns or rows in a single batch at the end of execution.

The following snippet demonstrates this concept. Note how formatting is applied to entire columns and entire rows where needed. This approach limits the interaction with the layout engine:

```
//Write all data first
  mySheet.getRange('A1:A100').setValues(newData);

//Apply layout updates once at the very end
 mySheet.getRange('A:E').getFormat().autofitColumns();
 mySheet.getRange('1:50').getFormat().setRowHeight(15);
 mySheet.getRange('F:F').getFormat().setColumnWidth(20);
 mySheet.setFreezePanes(mySheet.getRange('B2'));
```

TIP

While autofitting entire columns is fast, try to avoid blind autofitting (that is, `getUsedRange().getFormat().autofitColumns()`). If your used range is massive, Excel has to calculate the width of every populated cell. Have targeted columns and rows in mind when using the `autofitColumns()` and `setRowHeight()` methods.

Create Modular Scripts

The final tip goes back to something I talked about in the last chapter — building modular scripts. It's tempting to create one script that does everything you need, but huge scripts are inherently harder to understand, maintain, and debug for performance issues.

For large or complex processes, adopt the best practice of creating smaller modular scripts. Consider transitioning from a single procedure to a suite of specialized `osts` files, each dedicated to a specific portion of the larger process.

Imagine a process divided into three distinct scripts: `DataSourcing.osts` to get external data, `DataTranformation.osts` to clean and transform the data, and `DataFormatting.osts` to perform calculations and format reports. By splitting the processing load into three separate scripts, you'll have an easier time optimizing performance while avoiding potential timeout issues from long-running scripts.

Index

A

absolute mode vs relative mode recording, 30–32
Action Recorder, 23, 207
  capturing conditional formatting code with, 192–194
  deleting scripts, 27
  edit a script, 26–27
  limitations of, 28
  limited recording of UI actions, 28
  no programming logic, 28
  one worksheet at a time, 28
  Record Actions task pane, 24–25
  rename the script, 25–26
  run a script, 26
  simple script creation, 24–25
  superfluous code, 28
`addColumnHierarchy` method, 172
`addComment()` method, 79
`addImage` method, 208
Add in Workbook option, 23
`addPivotTable` method, 165
`addRowHierarchy` method, 168
`addTable()` method, 157
AI coding assistants, 207
AND operator (&&), 89
application programming interface (API), 9
`applyCustomFilter()` method, 159–160
array data types, 48–51
assignment operator (=), 83
automated (event-based) trigger, 230
Automate tab, 12
  Office Scripts Environment, 15–17
automating charts, 199
  chart object model, 199–202
  working the chart elements, 202–204
    formatting the chart title, 204–205
    formatting the x-axis and y-axis, 205–206
    setting series formats, 206
    setting the chart position, 204
    setting the legend, 205

B

Boolean data type, 47–48
bounded loops, 94
bullet graph creation, 211–215

C

`calculate()` method, 305
`CalculateTax` function, 57–58
`CategoryAxis` object, 205
cellValue, 191
chaining range methods, 140

chart elements, 202–204
formatting
chart title, 204–205
x-axis and y-axis, 205–206
setting
chart position, 204
legend, 205
series formats, 206
chart object model, 199–202
automating charts, 199–202
chart recording challenges, 207
Choosing View Scripts, 17
clarity, adding, 52–53
classic flow designer, 244–246
`clear()` method, 151, 152
cloud automation, Power Automate, 225
cloud-based reporting system, 223
cloud-compatible JavaScript, 10
Code Editor task pane, 39
code storage, Office Scripts and VBA, 11
collection objects, 36
colorScale, 191
color scales, starter script for, 196–197
ColumnHierarchy, 165
comments, adding clarity with, 52–53
community forums, 207
conditional analyses, logical operators in, 88–89
NOT operator (!), 90–91
nullish coalescing operator (??), 91–92
AND operator (&&), 89
OR operator (||), 90
ternary operator, 93
conditional formatting
cellValue, 191
code with action recorder, 192–194
colorScale, 191
containsText, 191
dataBar, 191
data with, 190–191
fails to record properly, 194–195
starter script for color scales, 196–197
starter script for data bars, 195–196
starter script for icon sets, 198–199
iconSet, 191
presetCriteria, 191
syntax for automating, 190
topBottom, 191
waffle chart with, 215–220
conditions
`else if` statement, 85–86
equality operators, 83–84
assignment operator (=), 83
loose equality operator (==), 83
strict equality operator (===), 83–84
`if ... else` statement, 84–85
`if` statement, 82–83
logical with, 82
`switch` statement, 87–88
`console.log()` method, 64–68, 95
`convertToPdf()` method, 262, 264, 265
`copyFrom` method, 147, 148
Copy To command, 21–22

createRegionWorksheets parameter, 240
custom number formatting, 186–187
capturing conditional formatting code with action recorder, 192–194
conditional formatting that fails to record properly, 194–195
starter script for color scales, 196–197
starter script for data bars, 195–196
starter script for icon sets, 198–199
data with conditional formatting, 190–191
formatting dates and times, 189–190
formatting numbers in thousands and millions, 188–189
hiding and suppressing zeroes, 188
custom visuals creation, 211
bullet graph creation, 211–215

D

data arrays
to new workbooks, writing, 277–281
from SharePoint, reading, 274–277
dataBar, 191
data bars, starter script for, 195–196
data field items, pivot table, 169–170
setName method, 171–172
setNumberFormat method, 171
setSummarizeBy method, 170–171
DataHierarchy, 165
data in ranges, manipulating, 147
clearing cell contents, 151–152
converting all formulas in a range to values, 149–151
copying data between worksheets, 148–149
copying data from one range to another, 147
deleting rows and columns, 152–155
simulating paste-special functionality, 148
data types, 45
array, 48–51
Boolean, 47–48
number, 46–47
string, 45–46
debugging tools, Office Script, 64
console.log method, 64–68
finally block, 69–71
try ... catch statements, 67–69, 71
dot operator (.)., 37

E

Else statement, 183
Email, Excel with, 247–248
emailing an attachment, 248–252
embedding images in emails, 257
reviewing the script, 257–259
updating the flow, 259–261
sending emails to a dynamic list of recipients, 252–253
reviewing the script, 253–254
updating the flow, 254–257
error-handling patterns, 72
fallback pattern (recovery pattern), 74–75
guard clause pattern, 74
try ... catch ... finally pattern, 72
validation-first pattern, 72–73

errors in Office Scripts, 75
invalid arguments, 79–80
invalid range and out-of-bounds errors, 78–79
object not found errors, 78
reference errors, 77–78
syntax errors, 76–77
event-driven flows, 224
events, Office Scripts and VBA, 10
Excel and Outlook, 247–248
emailing an attachment, 248–252
embedding images in emails, 257
reviewing the script, 257–259
updating the flow, 259–261
sending emails to a dynamic list of recipients, 252–253
reviewing the script, 253–254
updating the flow, 254–257
Excel data as PDF files, 261
`OfficeScript.convertToPdf()` method, 262, 264
`OfficeScript.sendMail()` method, 263–266
Excel Scripts, 9, 40
Excel with SharePoint, 267
building the flow, 274
preparing the files, 268–269
reading data arrays from SharePoint, 274–277
reviewing the scripts, 269–274
writing data arrays to new workbooks, 277–281

F

fallback pattern, error-handling, 74–75
File Content, 236
FilterHierarchy, 165
filters
advance text filtering, 160–161
clearing, 161–162
filtering numeric values, 159–160
table, 158–159
`finally` block, 69–71
`for...of` loops, 98–100
`forEach` statement, 176
`for` loops, 94–97
`for` statement, 94–95
functions, adding flexibility with, 53–54
default parameter values, 58–59
defining a function, 54–56
drop-down selector for a parameter, 61–62
function parameters, 57
function return values, 57
optional parameters, 60–61
scripts with input parameters, 59–60

G

`getActiveCell()` method, 135
`getAddress()`method, 134
`getAxes()`method, 205
`getCellCount()`method, 134
`getCell()` method, 44, 141–144, 153
`getChart` method, 200
`getColumnByName()` method, 304

getColumnIndex()method, 134
getColumn() method, 136, 162
getEntireColumn() method, 153
getEntireRow() method, 153
getExtendedRange() method, 137, 143
Get File Content action, 236–237
getHierarchy() method, 168
getImage method, 207–209
getLastCell() method, 136
getLastColumn() method, 136
getLastRow() method, 136–137
getLegend method, 205
getName method, 39, 40
getOffsetRange() method, 136–137
getRangeBetweenHeaderAndTotal()method, 304
getRange method, 79, 132
getRowCount()method, 134
getRowIndex()method, 134, 135
getRow() method, 136
getSelectedRange() method, 135
getSeries method, 206
getSlicerItems method, 183
getSort().apply method, 302
getTable() method, 155–156
getTitle method, 204
getUsedRange() method, 135
getvalues() method, 300–301
getWorksheet, 39
guard clause pattern, error-handling, 74

H

hexadecimal (hex) code, 42

I

iconSet, 191
icon sets, starter script for, 198–199
if ... else statement, 84–85, 93
if statement, 82–83
Insert Action command, 28–30
instant (manual) trigger, 230
IntelliSense, 39
IntelliSense of Copy Editor, 80
invalid arguments, errors in Office Scripts, 79–80
invalid range errors, 78–79

J

JavaScript
  cloud-compatible, 10
  open-source superset of, 7–8
JSON (JavaScript Object Notation) script, 21

L

language, Office Scripts and VBA, 10
let keyword vs const keyword, 42–44
logical operators, in conditional analyses, 88–89
  NOT operator (!), 90–91
  nullish coalescing operator (??), 91–92
  AND operator (&&), 89
  OR operator (||), 90
  ternary operator, 93

loops, 94
bounded, 94
decrement operator (--), 96
`for...of` loops, 98–100
`for` loops, 94–97
prefixing vs postfixing, 97–98
unbounded, 94
`while` loops, 100–102
loose equality operator (==), 83

M

Macros dialog box, 11
`map()` method, 124, 301
meager documentation, 13
Microsoft 365 subscription, 12
Microsoft cloud-based platforms, 7
Microsoft Office Scripts documentation, 207
Microsoft's built-in Action Recorder, 13
modern flow designer, 244
Move To command, 21–22
moving worksheets, 114
before another sheet, 115–116
duplicating worksheets, 116–118
to first position, 114
to last position, 114–115

N

navigating ranges, 132–133
custom range functions creation, 144–146
dynamically adding a total column, 138–141
`getOffsetRange` method, 136–137
getting flexible with the `getCell` method, 141–144
leveraging range shortcut methods, 135–136
using methods that return metadata, 133–135
NOT operator (!), 90–91
nullish coalescing operator (??), 91–92
number data types, 46–47

O

object model, scripting fundamentals, 35–38
object not found errors, 78
Office Script debugging tools, 64
`console.log` method, 64–68
`finally` block, 69–71
`try ... catch` statements, 67–69, 71
Office Script editor
New Script, 17
Office Scripts Gallery, 17
View Scripts, 17
Office Scripts, 9
API, 39
benefits of, 13–14
automate a recurring task, 13–14
automate a repetitive task, 13
interact with other applications, 14
optimize work with functions, 13
simplify the user experience, 13
data limits, 12–13
editor, 27

meager documentation, 13
online development with, 8
requirements and limitations of, 11–13
subscription requirement, 11–13
technical and organizational limitations, 12
Office Scripts Environment
absolute mode vs relative mode recording, 30–32
Action Recorder, 23–28
diving into Automate tab, 15–17
first script writing, 17–21
Insert Action command, 28–30
sharing scripts, 21–23
Office Scripts vs VBA
code storage, 11
events, 10
language, 10
offline vs online, 8–9
procedure types, 10–11
script triggers, 11
security, 9
offline vs online, Office Scripts and VBA, 8–9
OR operator (||), 90
.osts file
emailing, 21
moving or copying, 21–22
out-of-bounds errors, 78–79

P

PivotHierarchy, 164, 165
pivot table
object model, 177
report layouts, 177
adjusting grand totals and subtotals, 178–179
setting the report layout, 178
in a workbook, 176–177
PivotTable Fields task pane, 164
`pivotTable.getlayout()`, 178
pivot table hierarchy, 164–165, 173–176
adding data items to the pivot table, 168–169
avoiding errors when adding data items, 172–173
customizing data field items, 169–170
`setName` method, 171–172
`setNumberFormat` method, 171
`setSummarizeBy` method, 170–171
starting a pivot table, 165–167
`pivotTable.refresh` method, 176
postfixing the operator, 97
Power Automate, 7, 223
avoid Office Script failures in flows, 243–244
cloud automation, 225
email-driven workflows, 266
excel data and API limitations, 242–243
first flow creation, 229
building the flow, 231–240
gathering thoughts and files, 230–231
triggers, 229–230
flow, 11, 296

Power Automate *(continued)*
going with the flow, 223
exploring the benefits of flows, 224
interface, 226
Create option, 226–227
My Flows option, 228–229
Templates option, 227–228
limitations of, 242–244
passing parameters to a script from, 240–242
platform limits, 243
Premium licenses, 225, 229
requirements, 224–225
prefix the operator, 97
Premium licenses, Power Automate, 225, 229
presetCriteria, 191
Problems tab, 76
protection, worksheets, 118–120
checking if a worksheet as protect, 119
protecting all worksheets in a workbook, 120–122
unprotecting all worksheets in a workbook, 123–124
unprotecting a worksheet, 120
`protect` method, 119
`Protect Sheet` command, 120, 121

R

ranges
manipulating data in, 147
clearing cell contents, 151–152
converting all formulas in a range to values, 149–151
copying data between worksheets, 148–149
copying data from one range to another, 147
deleting rows and columns, 152–155
simulating paste-special functionality, 148
navigation, 132–133
custom range functions creation, 144–146
dynamically adding a total column, 138–141
`getOffsetRange` method, 136–137
getting flexible with the `getCell` method, 141–144
leveraging range shortcut methods, 135–136
using methods that return metadata, 133–135
recent scripts, 17
Record Actions task pane, 17, 24–25, 30–31
recording, absolute mode vs relative mode, 30–32
recovery pattern, error-handling, 74–75
reference errors, 77–78
relative mode vs absolute mode recording, 30–32
report layouts, pivot table, 177
adjusting grand totals and subtotals, 178–179
setting the report layout, 178
RowHierarchy, 164, 165

S

scheduled flows, 224
scheduled trigger, 230

Script Details task pane, 24–25
scripting fundamentals
adding clarity with comments, 52–53
adding flexibility with functions, 53–62
data types, 45
array, 48–51
Boolean, 47–48
number, 46–47
string, 45–46
enhancing code with variables, 40–42
function parameters, 57–58
default parameter values, 58–59
drop-down selector for a parameter, 61–62
enhancing scripts with input parameters, 59–60
optional parameters, 60–61
function return values, 57
`let` keyword vs `const` keyword, 42–44
methods, 38–40
object model, 35–38
script performance improvement, 299
apply formatting to ranges, not cells, 303–304
avoid repeated lookups, 301–302
leverage the `getUsedRange` method, 304–305
minimize actions that change display, 307
minimize formula usage in scripts, 306–307
modular scripts creation, 308
postpone calculations until the end, 305–306
read and write data in batches, 299–301
using the cargo ship, 300–301
using the tiny boat, 300
sort and filter in arrays not cells, 302–303
use table objects whenever possible, 304
scripts writing, 285
apply descriptive variable and function names, 285–286
apply indentation standards, 288–289
break long scripts into custom functions, 292–293
build modular scripts, 294
centralize object references, 291–292
first
code in the block, 19
JSON format, 21
main function block, 18
new script in OneDrive account, 20
rename the script, 20
script test, 19
selecting New Script, 17–18
leverage comments to partition code, 290–291
manage script versions, 296–297
organize a script library, 294–295
use established JavaScript conventions, 287–288
write effective comments, 289–290
script triggers, Office Scripts and VBA, 11
security, Office Scripts and VBA, 9
semantic versioning, 296

sendMail() method, 263–266
  mail properties of, 263
setFormulas() method, 306
setLayoutType method, 178
setName method, 171–172
setNumberFormat method, 171, 186, 187, 189
setPosition method, 114, 205
setShowColumnGrandTotals, 178
setShowRowGrandTotals, 178
setSubtotalLocation, 178
setSummarizeBy method, 170–171
setValues() method, 301
Shared with Me tab, 228
sharing scripts
  attaching scripts to workbook and adding a button, 22–23
  detaching scripts from your workbook, 23
  .osts file
    emailing, 21
    moving or copying, 21–22
sheetName parameter, 240
short-circuit evaluation, 107
slicerItems, 183
slicers
  creating, 179–181
  filtering, 181–183
  working with, 179
strict equality operator (===), 83–84
string data types, 45–46
switch statement, 87–88
syntax errors, 76–77

T

table filters, 158–159
template literal, 99–100
ternary operator, 93
throw new Error() statement, 75
try ... catch statements, 67–69, 71
try...catch error handler, 123
two-dimensional array, 50–51
TypeScript, 7–8, 10, 39, 43–46
  number storage in, 47

U

unbounded loops, 94

V

validation-first pattern, error-handling, 72–73
ValueAxis object, 205
variables
  enhancing code with, 40–42
  naming, 43
VBA (Visual Basic for Applications), 1
  code written in, 11
  data types, 46
  debug.print statement in, 65
VBA vs Office Scripts
  code storage, 11
  events, 10
  language, 10
  offline vs online, 8–9
  procedure types, 10–11

script triggers, 11
security, 9
versioning an Office Script, 296
Visual Basic for Applications. *see* VBA (Visual Basic for Applications)
Visual Basic Green, 187
visualization, 185
converting visualizations into images, 207–211

W

waffle chart, with conditional formatting, 215–220
Walkenbach, John, 63
`while` loops, 100–102
Windows API calls, 10
workbook, pivot tables in, 176–177
`workbook` object, 36, 37, 40
workflow, 224
working with Excel tables, 155–156
applying table filters, 158–159
clearing filters, 161–162
creating a table from a range, 157–158
filtering numeric values, 159–160
getting range objects, 156
performing advance text filtering, 160–161
referencing parts of a table, 156–157
working with slicers, 179
creating a slicer, 179–181
filtering a slicer, 181–183
working with worksheets, 106
adding a new worksheet, 106–107
for each month, 107–108
deleting all but a specified sheet, 108–110
hiding all but a specified worksheet, 112–115
unhiding all worksheets in a workbook, 110–112
`worksheet` object, 36, 41
worksheets
moving, 114
before another sheet, 115–116
duplicating worksheets, 116–118
to first position, 114
to last position, 114–115
protection, 118–120
checking if a worksheet as protect, 119
unprotecting all worksheets in a workbook, 123–124
unprotecting a worksheet, 120
in a workbook, 120–122
working with, 106
adding a new worksheet, 106–108
deleting all but a specified sheet, 108–110
hiding all but a specified worksheet, 112–115
unhiding all worksheets in a workbook, 110–112

worksheets collection, 124–125
creating a table of contents for your workbook, 127–129
sorting worksheets alphabetically, 125–126
writing first script
code in the block, 19
JSON format, 21
main function block, 18
new script in OneDrive account, 20
rename the script, 20
script test, 19
selecting New Script, 17–18
writing scripts, 285
apply descriptive variable and function names, 285–286
apply indentation standards, 288–289
break long scripts into custom functions, 292–293
build modular scripts, 294
centralize object references, 291–292
leverage comments to partition code, 290–291
manage script versions, 296–297
organize a script library, 294–295
use established JavaScript conventions, 287–288
write effective comments, 289–290

About the Author

Michael Alexander is a senior consultant at Slalom Consulting and a former Microsoft MVP. He is a recognized Excel trainer and the author of multiple Excel books, including *Microsoft Excel 365 Bible* and Excel *Dashboards & Reports For Dummies.* You can find Michael at `Exceltopia.com`, where he shares articles and training on Excel development.

Dedication

For all the Excel analysts out there striving to enhance their skillset.

Author's Acknowledgments

My deepest thanks go to Susan Pink, whose meticulous attention to detail and dedication to clarity transformed this manuscript into its best possible version. I am equally indebted to Guy Hart-Davis for his brilliant technical guidance and for ensuring the rigor of every concept presented here. Finally, a heartfelt thank-you to my wife, Michele, for putting up with the long hours this project required.

Publisher's Acknowledgments

Associate Editor: Elizabeth Stilwell
Senior Editorial Assistant: Hanna Sytsma
Project and Copy Editor: Susan Pink
Production Editor: Magesh Elangovan
Managing Editor: Ajith Kumar
Technical Editor: Guy Hart-Davis
Cover Image: © NARONGRIT/Getty Images

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley’s ebook EULA.